课程概览

课程基本信息

项目	内容
课程	Stanford CS149: Parallel Computing（并行计算），Fall 2025
授课时间	周二 / 周四 10:30–11:50am
地点	NVIDIA Auditorium
授课教师	Kayvon Fatahalian、Kunle Olukotun
先修要求	强烈推荐 CS111（操作系统原理）；需要熟练的 C/C++ 读写调试能力，并有使用线程（`std::thread` / pthreads）的基础
教材	无指定教材；架构主题可参考 Hennessy & Patterson《Computer Architecture: A Quantitative Approach》(6th ed.)
视频	2025 年视频不对外公开；2023 年版课程视频在 Stanford YouTube 频道 公开可看

课程定位

> 从智能手机、多核 CPU、GPU，到 AI 加速器、全球最大的超级计算机与网站，并行处理在现代计算中无处不在。本课程的目标是让学生深刻理解现代并行计算系统设计中的基本原理与工程权衡，并掌握有效利用这些机器所必需的并行编程技术。由于写出好的并行程序需要理解关键机器性能特征，本课程同时覆盖并行**硬件**与并行**软件**设计。

课程的三大主线（Theme）

1. **并行思维（Parallel Thinking）**：如何把一个问题分解为可以安全并行执行的工作（分解 Work），如何把工作分配给处理器（Assign Work），以及如何管理处理器之间的通信与同步，使其不成为加速比的瓶颈——并学习用主流并行编程语言（C++ 线程、ISPC、CUDA、Cilk/OpenMP、事务内存等）实现这些思想。
2. **并行硬件实现（Hardware Implementation）**：现代并行计算机如何工作——多核处理器、SIMD、GPU、缓存一致性、内存一致性、专用加速器；理解实现机制的性能特征与"性能 vs 便利性 vs 成本"的设计权衡。
3. **效率思维（Thinking About Efficiency）**：FAST ≠ EFFICIENT——程序跑得快不代表高效利用硬件。衡量加速比、效率、扩展性，理解 Amdahl 定律、通信开销、负载均衡、局部性与算术强度对性能的根本性影响。

课程结构（按主题分四个阶段）

阶段	讲座	主题
I. 并行基础与编程模型（第 1–6 讲）	L1–L6	为什么需要并行与效率；现代多核处理器架构（ILP/SIMD/多线程）；ISPC 编程抽象；并行化代码的思考过程；工作分配与调度（Cilk 工作窃取）；局部性与通信（消息传递、流水线、算术强度）
II. GPU 与数据并行（第 7–8 讲）	L7–L8	GPU 架构与 CUDA 编程；数据并行思维（map / reduce / scan / groupByKey）
III. AI 与专用硬件（第 9–13 讲）	L9–L13	GPU 上的 DNN 高效评估（卷积→GEMM、Transformer、层融合）；硬件特化（DNN 加速器、脉动阵列）；专用硬件编程系统（TVM / Triton / TileLang / MLIR）；AI 应用的数据中心级映射（serving、batching、KV cache）；AI 驱动的性能优化

阶段	讲座	主题
IV. 系统级同步与一致性（第 14–18 讲）	L14–L18	缓存一致性（MSI/MESI、伪共享）；同步实现与内存一致性（锁、原子操作、放松一致性）；细粒度锁与无锁编程（无锁栈/队列、ABA 问题）；事务内存（STM/HTM）

时间线总览（2025 秋季学期）

日期	讲座 / 事件
Sep 23	L1: Why Parallelism? Why Efficiency?
Sep 25	L2: A Modern Multi-Core Processor (Part I)
Sep 30	L3: Modern Multi-Core Architecture (Part II) + ISPC
Oct 02	L4: Parallelizing Code: An Example Thought Process
Oct 06	**PA1 截止**：四核 CPU 并行程序性能分析
Oct 07	L5: Program Optimization 1: Work Distribution and Scheduling
Oct 09	L6: Program Optimization 2: Locality and Communication；**WA1 截止**
Oct 14	L7: GPU Architecture and CUDA Programming
Oct 16	L8: Data-Parallel Thinking；**PA2 截止**
Oct 21	L9: Efficiently Evaluating DNNs on GPUs；**WA2 截止**
Oct 23	L10: Hardware Specialization
Oct 28	L11: Programming Systems for Specialized Hardware
Oct 30	L12: Mapping AI Applications to the Datacenter Computer；**PA3 截止**
Nov 04	民主日（停课）
Nov 06	L13: Domain-Specific Programming Systems and AI-Driven Performance Optimization；**WA3 截止**
Nov 11	L14: Cache Coherence
Nov 13	L15: Implementing Synchronization + Memory Consistency；**PA4 截止**
Nov 18	期中考试（晚间，停课）
Nov 20	L16: Fine-Grained Locking and Lock-Free Programming
Dec 02	L17: Transactional Memory (Part I)
Dec 03	**WA4 截止**
Dec 04	L18: Transactional Memory (Part II) + AMA；**PA5 截止**（不可用晚交天数）
Dec 11	期末考试 3:30–6:30pm

作业与评分

类别	占比	说明
编程作业 ×5	8% + 12%×4 = **56%**	用 C++ 线程、ISPC、CUDA 等完成；可两人组队
书面作业 ×4	3%×4 = **12%**	三人组队，每次由课程组随机分配队友
每讲课堂参与	**4%**	每讲一次随堂小测验（或课后替代测验）

类别	占比	说明
期中考试	**12%**	11 月 18 日晚间
期末考试	**16%**	12 月 11 日 3:30–6:30pm

- **编程作业**（全部公开，见 GitHub）：
- Assignment 1: Analyzing Parallel Program Performance on a Quad-Core CPU（10 月 6 日）——在 myth 机器的四核 i7（AVX2、超线程）上分析 SIMD 与多核并行性能；
- Assignment 2: Scheduling Task Graphs on a Multi-Core CPU（10 月 16 日）——从零实现一个任务执行库（线程池、互斥锁/条件变量、任务图调度）；
- Assignment 3: A Circle Renderer in CUDA（10 月 30 日）——在 AWS GPU 上用 CUDA 实现并行圆形渲染器；
- Assignment 4: Fused Conv+MaxPool on the Trainium2 Accelerator（11 月 13 日）——在 AWS Trainium2 上编写并优化融合卷积+最大池化 kernel（涉及循环分块与融合）；
- Assignment 5: Make the World's Fastest CUDA Kernels（12 月 4 日，**无晚交天数**）——开放课题，在 H100 上优化所选 AI kernel（可用 LLM 辅助，有班级排行榜）。
- **书面作业**：4 份 PDF 均公开可下载（本笔记配套资料已下载）。

本笔记说明

本笔记基于课程网站**所有公开可获取的资料**整理而成：

- 课程主页、课程信息页、讲座索引页的全部文本；
- 全部 18 讲幻灯片 PDF（已下载并提取全文）；
- 4 份书面作业 PDF（已下载并提取全文）；
- 5 个编程作业的公开 GitHub README。

访问受限内容（本笔记未包含）：2025 年讲座视频（网站明确说明今年不向公众分发）、Ed Discussion 论坛（需校内账号）、Canvas 内资料。如需学习视频，可使用 2023 年公开版 YouTube 播放列表。

每讲笔记均遵循统一结构：**概述 → 核心概念与定义（含现实类比与公式/图示）→ 代码示例与详细解说（含"代码做了什么"与"并行机制解说"）→ 关键要点 → 常见陷阱与注意事项 → 思考题（带答案）**。所有代码示例均可独立运行或清晰说明运行方式，并明确标注其对应的讲座概念。

Lecture 1: Why Parallelism? Why Efficiency?（日期：Sep 23, 2025）

> **概述**：本讲是 CS149 的开篇，回答两个问题——**为什么要并行**（因为单核性能增长已几乎停滞，只能用更多处理单元来换取速度）与**为什么要效率**（FAST ≠ EFFICIENT，跑得快不等于用好了硬件）。课堂通过三个"人类处理器"演示（DEMO 1/2/3）直观展示并行编程的三大挑战：**通信开销**、**负载不均**与**通信/计算比过高**，并由此引出本课程的三大主线：并行思维（分解工作、分配工作、管理通信/同步）、并行硬件实现、以及效率思维。后半讲复习处理器与内存基础（指令、寄存器、ALU、cache、延迟与停顿），为后续课程打下硬件直觉。

一、核心概念与定义

1. Parallel Computer（并行计算机）

- **定义**：A parallel computer is a collection of processing elements that cooperate to solve problems quickly —— 一组**相互协作**、**共同快速求解问题**的处理单元（processing elements）的集合。注意两个关键词：既要"多"（multiple processing elements），又要"协作"（cooperate），孤立的处理器堆在一起不构成并行计算机。
- **现实类比**：一间厨房里多位厨师分工做一顿饭。只把 10 个厨师塞进厨房而不分工、不传菜，他们只会互相碍事；真正并行是每人负责一道工序（切菜、炒菜、装盘）并有序协作。
- **公式/图示**：无固定公式，但"cooperate"一词暗含后续所有主题——如何分解（decompose）、如何分配（assign）、如何通信/同步（communicate/synchronize）。

2. Speedup（加速比）

- **定义**：使用 P 个处理器相比使用 1 个处理器所获得的执行时间缩减倍数：`speedup(using P processors) = execution time (using 1 processor) / execution time (using P processors)`。 这是衡量"并行化是否值得"的第一指标。**加速比 = 串行时间 ÷ 并行时间**。
- **现实类比**：一个工人搬 100 块砖要 100 分钟（T_1）；5 个工人一起搬只要 25 分钟（T_5）；speedup = 100/25 = 4。若因为搬砖时互相挡路只省了一半时间，speedup = 2。
- **公式/图示**：`speedup(P) = T(1) / T(P)` 课堂提问：在 10 个处理器的机器上只获得 2x speedup 算好结果吗？（答案是：绝对速度提升了，但**效率**只有 2/10 = 20%，远未用好硬件——见第 3 条。）

3. Efficiency（效率）

- **定义**：加速比除以使用的处理器数：`efficiency = speedup / P`。它衡量**每个处理器平均贡献了多少加速**，是"用没用好硬件"的度量。
- **现实类比**：10 个厨师做一道菜只比 1 个厨师快 2 倍——效率 = 2/10 = 20%，说明 8 个厨师基本在打酱油（等菜、闲聊、没活干）。
- **公式/图示**：`efficiency(P) = speedup(P) / P = T(1) / (P · T(P))` 理想并行效率为 1（线性加速），实际总是小于 1。

4. Amdahl's Law（Amdahl 定律）

- **定义**：设程序可并行部分占比为 p（$0 \leqslant p \leqslant 1$），不可并行（串行）部分占比为 1−p，则使用 P 个处理器时理论上限为：`speedup_max(P) = 1 / ((1 - p) + p / P)`；当 $P \to \infty$ 时，speedup_max → 1 / (1 − p)。 也就是说：**串行部分决定了加速比的天花板**。本讲幻灯片本身没有给出 Amdahl's Law 公式，而是通过三个课堂演示建立了"通信/负载不均这类开销会限制加速比"的直觉——Amdahl's Law 正是把这种直觉形式化的经典分析工具，后续课程（L4–L6 关于工作分配、调度、局部性）会系统使用它。
- **现实类比**：一个 5 人小组做汇报，其中 1 页幻灯片只有组长能写（串行部分）。无论其他 4 人把各自部分做得多快，总时间至少是"组长写那一页"的时间——这就是 1/(1−p) 的天花板。
- **公式/图示**：`串行部分占比 (1-p) ⟶ 不可并行，决定下限 可并行部分占比 p ⟶ 时间最多除以 P T(P) ≥ (1-p)·T(1) + p·T(1)/P speedup(P) ≤ 1 / ((1-p) + p/P) (Amdahl's Law)`

5. Communication Overhead（通信开销）

- **定义**：并行单元之间传递数据（本讲演示中是"互相告诉对方 partial sum"）所花的时间。**通信是限制最大加速比的首要因素**——DEMO 1 的课堂观察：通信限制了能达到的最大加速比；把"处理器"（学生）挪近、或允许喊话（降低通信代价）后加速比提升。
- **现实类比**：几个同学合写一份报告，每人写完一部分后要互相传文件、开会对齐。传文件、开会的时间就是通信开销；大家坐得越近、沟通越顺畅（"shout"），开销越小。
- **公式/图示**：无固定公式；直觉上 `T(P) ≈ 计算时间 + 通信时间 + 空闲时间`。通信占比越高，加速比上限越低（DEMO 3 的结论）。

6. Load Imbalance（负载不均）

- **定义**：工作没有平均分给各处理器，导致一部分"处理器"早早干完（idle），另一部分还在忙。**负载不均会限制加速比**——DEMO 2 的课堂观察：有的学生（处理器）没活干闲下来，而其他人还在忙；改善工作分配后加速比提升。
- **现实类比**：自助餐厅只有一个打菜窗口，前面的人点得慢，后面排长队——窗口（处理器）忙死，排队的人（其它处理器）闲死。改成多个窗口按队伍平均分流后，整体吞吐立刻改善。
- **公式/图示**：`处理器 0: [##########] 忙 处理器 1: [####] 忙完闲置 处理器 2: [##############] 仍在忙 ← 拖慢整体 完成时间 = max(各处理器耗时) —— 由最慢者决定!`

7. 并行思维三步骤（Decompose / Assign / Communicate & Synchronize）

- **定义**：本课程主线一"编写可扩展的并行程序"的三个步骤：
- **Decompose（分解工作）**：把问题拆成可以**安全地并行执行**的若干块；
- **Assign（分配工作）**：把工作块分配给各处理器；
- **Manage communication/synchronization（管理通信与同步）**：让处理器之间的通信/同步**不成为加速比的瓶颈**。
- **现实类比**：组织一次搬家——先拆解任务（打包、搬运、布置），再分配给人（谁搬哪间房），最后约定协调方式（谁先到、走哪个门），否则楼下堵车（通信）或有人闲着（负载不均）。
- **公式/图示**：无公式；它是全课程的思维框架，DEMO 1/2/3 分别对应第 3、2、1 步的失败案例。

8. FAST ≠ EFFICIENT（快 ≠ 高效）

- **定义**：程序在并行计算机上跑得更快，**并不代表**它高效利用了硬件。课程主线三的核心口号。判断标准：是不是把机器提供的能力都用上了（程序员视角）；机器该配哪些能力（硬件设计者视角：performance vs convenience vs cost，cost = silicon area / power）。
- **现实类比**：10 个工人搬砖，你只让 2 个人干活、8 个人在旁边看——速度确实比 1 个人快 2 倍，但效率只有 20%。
- **公式/图示**：`FAST != EFFICIENT`。课堂提问："2x speedup on a computer with 10 processors——good result?"（从效率看：不是）。

9. Instruction-Level Parallelism（ILP，指令级并行）与 Superscalar Execution（超标量执行）

- **定义**：一条指令流内**互不依赖的指令**可以并行执行。超标量（superscalar）处理器在硬件上自动找出一段指令序列中的独立指令，把它们放到多个执行单元（ALU）上并行执行。示例：`a = x*x + y*y + z*z` 中三条 `mul` 互不依赖，ILP = 3，可同时执行；但第 4、5 条 `add` 依赖前面的乘法结果，只能串行等待。
- **现实类比**：做菜时"烧水"和"切菜"互不依赖，可以同时进行（ILP=2）；但"等水开再下面条"是依赖关系，必须排队。
- **公式/图示**：`指令 1: mul R0,R0,R0 ┐ 指令 2: mul R1,R1,R1 ─┼→ 三条乘法互相独立（ILP = 3） 指令 3: mul R2,R2,R2 ┘ 指令 4: add R0,R0,R1 ← 依赖 1、2 指令 5: add R3,R0,R2 ← 依赖 4`

10. Power Wall（功耗墙）

- **定义**：动态功耗 `dynamic power ∝ capacitive load × voltage² × frequency`；静态功耗来自晶体管即使不工作也在漏电（leakage）。功耗高 = 发热高，散热成了硬约束，因此**不能无限提高时钟频率**——这是"单核性能停止增长"的两大原因之一（另一个是 ILP 挖掘殆尽）。
- **现实类比**：CPU 超频就像把跑步机调快——跑得更快但发热更猛，最后必须降速散热（clock down to cool off），否则烧坏。
- **公式/图示**：`P_dynamic ∝ C_load × V² × f（电压的平方！降电压是降功耗最有效的杠杆）`课堂数据：Intel Core i9 10900K（台式机）95W；Apple M1 笔记本 13W；NVIDIA RTX 4090 GPU 450W；微波炉 900W；手机处理器 0.5–2W；世界最快超算 Frontier 达兆瓦级（21 MW）。

11. Memory Address Space、Load 指令与 Memory Access Latency（内存地址空间、装载指令与访存延迟）

- **定义**：内存可视为**字节数组**，每个字节用地址（数组下标）标识（假设 byte-addressable）。处理器用 `load` 指令把内存数据搬进寄存器（如 `ld R0 ← mem[R2]`），用 `store` 写回。**Memory access latency（访存延迟）** 是内存系统把数据交给处理器所需

的时间（如 100 个时钟周期 / 100 nsec），远大于算术指令的时间。

- **现实类比**：图书馆取书——从书架上拿一本（寄存器）是瞬间的；从地下书库调书（DRAM）要等几分钟（几百个周期）。
- **公式/图示**：`ld r0, mem[r2] ← 从内存取数据，需要 ~100+ 周期 ld r1, mem[r3] add r0, r0, r1 ← 依赖上面两条 load，必须等它们完成 → "stall"（停顿）`

12. Stall（停顿）

- **定义**：当指令流中后续指令依赖一条尚未完成的指令时，处理器无法推进，称为 stall。**访存是停顿的主要来源**。缓存（cache）的存在就是为了缩短停顿长度（降低访存延迟），让处理器多数时间访问"驻留在 cache 里的数据"。
- **现实类比**：流水线上"等料"——上一道工序还没做完，下一道工序只能干等，整条线空转。
- **公式/图示**：见第 11 条示例：`add` 必须等两个 `load` 完成，期间处理器空转。

13. Cache、Cache Line 与两种 Locality（缓存、缓存行与局部性）

- **定义**：cache 是芯片上的存储，保存内存中**一部分值的副本**；若地址在 cache 中，处理器访问它就远快于访问 DRAM。cache 按 **cache line（缓存行）** 粒度工作（如每行 4 字节）。替换策略（如 LRU：最近最少使用）决定腾出空间时淘汰谁。两种数据局部性：
- **Spatial locality（空间局部性）**：装入一条 cache line 会"顺带预载"同一行里相邻地址的数据，后续访问不同地址也能命中；
- **Temporal locality（时间局部性）**：反复访问同一地址导致命中。
- **现实类比**：去超市买一打鸡蛋——店员从仓库（DRAM）搬来一整箱（cache line），你拿一个（命中）后剩下的都在手边（空间局部性）；明天还要鸡蛋，又去同一家店（时间局部性）。
- **公式/图示**（课堂 Cache 示例 1：总容量 8 字节、4 字节 cache line、LRU）：`访问序列: 0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x0 ... cache 状态（2 条 line）: load 0x0 → "cold miss"，装入 line 0x0（含地址 0x0–0x3） 访问 0x1/0x2/0x3 → 同一行内命中（空间局部性） load 0x4 → "cold miss"，装入 line 0x4（含地址 0x4–0x7） 再次访问 0x0 → 命中（时间局部性，line 0x0 还在）`

14. Cache Hierarchy（缓存层级）

- **定义**：现代机器的线性内存地址空间抽象由多级缓存 + DRAM 共同实现：L1 → L2 → L3 → DRAM。**离处理器越近、容量越小、延迟越低**。课堂数据（Kaby Lake CPU @ 4 GHz）：L1 命中 4 周期，L2 12 周期，L3 38 周期，DRAM 最佳情况约 248 周期。
- **现实类比**：厨房手边调料架（L1，随手拿）、橱柜（L2，走两步）、楼下超市（L3）、城外仓库（DRAM）——越近越快，但能放的东西越少。
- **公式/图示**：`处理器 | L1 cache (32 KB) ~4 cycles | L2 cache (256 KB) ~12 cycles | L3 cache (20 MB) ~38 cycles | DRAM (64 GB) ~248 cycles`

15. 数据移动的能耗成本（Data Movement Energy Cost）

- **定义**：现代系统设计的经验法则：**总是尽量减少计算机中的数据移动**。粗算数值：整数运算 ~1 pJ；浮点运算 ~20 pJ；从片内 1mm 外的小 SRAM 读 64 位 ~26 pJ；从低功耗移动 DRAM（LPDDR）读 64 位 ~1200 pJ。推论：以 10 GB/s 读内存约耗 1.6 W——而整个移动 GPU 的功耗预算才约 1 W。**利用局部性至关重要（Exploiting locality matters!!!）**。
- **现实类比**：把文件从自己电脑拷到 U 盘（片内）几乎不费电；上传到云再从云下载（DRAM 级别）既慢又耗电——能本地复用的数据绝不要来回搬运。
- **公式/图示**：`整数 op ~1 pJ < 浮点 op ~20 pJ < 片内 SRAM 读 64bit ~26 pJ < LPDDR 读 64bit ~1200 pJ （相差约两个数量级） → 少搬数据 = 最有效的省电手段）`

16. 课堂演示汇总（DEMO 1 / 2 / 3）

- **定义**：三次"人类处理器"课堂实验的观察与结论汇总：| 演示 | 实验设置 | 课堂观察 | 结论 | |---|---|---|---| | DEMO 1（第一个并行程序） | 多名学生各算一部分 partial sum，再汇总 | **通信（互相告诉 partial sum）限制了能达到的最大加速比**；把学生挪近/允许喊话（降低通信代价）后加速比提升 | 通信开销是加速比的首要瓶颈，最小化通信成本 = 提升加速比 | | DEMO 2（扩展到 4 个"处理器"） | 4 名学生分工算 | **工作分配不均限制了加速比**——有人提前干完闲置，有人还在忙；改善分配后加速比提升 | 负

载不均让处理器空转，均衡分配是关键 | | DEMO 3（大规模并行） | 全班一起算一个通信占比高的问题 | 该问题**通信相对计算的比例很大**；通信成本可以主导并行计算，严重限制加速比 | 通信/计算比过高的问题不适合并行（对问题本身的选择很重要） |

- **现实类比**：三个演示分别对应"传话太慢"（通信）、"有人闲有人忙"（负载不均）、"全程都在传话没人在干活"（通信/计算比过高）三种失败模式。
- **公式/图示**：无公式；直觉式结论——`加速比 ≈ 计算时间 /（计算时间 + 通信时间 + 空闲时间）`。

17. 现代并行硬件全景（Motivation: why parallel hardware is everywhere）

- **定义**：单核性能停滞的背景下，并行 + 专用硬件遍布各类设备（幻灯片数据）：| 硬件 | 关键参数 | 用途/说明 | |---|---|---| | Intel Core i9-10900K（Comet Lake, 2020） | 10 核 CPU | 消费级多核 CPU | | AMD Ryzen Threadripper 3990X | 64 核、4.3 GHz、4 个 8 核 chiplet | 工作站级多核 | | NVIDIA AD102 / GeForce RTX 4090（2022） | 18,432 个 fp32 乘法器、144 个 SM、760 亿晶体管 | 消费级 GPU | | Frontier（Oak Ridge 国家实验室） | 9472 × 64 核 AMD CPU（606,208 核）+ 37,888 块 Radeon GPU，21 MW | 2022 年秋季世界第一超算 | | Apple A15 Bionic（iPhone 13/14） | 150 亿晶体管：2 大 + 4 小 CPU 核、多核 GPU、Neural Engine（NPU）、图像/视频编解码、传感器处理器 | 移动端并行 + 专用处理 | | Raspberry Pi 3 | 四核 ARM A53 CPU | 嵌入式/教育平台 |
- **现实类比**：从手机（A15 的 6 核 CPU + GPU + NPU）到超算（Frontier 的 60 万核），"并行 + 专用化"是唯一能继续提升性能的路线——**软件必须配合（写并行代码），否则新硬件毫无用处**。
- **公式/图示**：见概念 10 的功耗墙公式；设计驱动力 = 性能（更多并行单元）÷ 成本（硅面积、功耗、散热）。

二、代码示例与详细解说（本讲重点）

示例 1：从 C 程序到指令流，再到 ILP 调度（汇编）

代码（汇编）：

```
// 原始 C 代码（课堂幻灯片示例）
int main(int argc, char** argv) {
    int x = 1;
    for (int i = 0; i < 10; i++) {
        x = x + x;
    }
    printf("%d\n", x);
    return 0;
}
```

```
; 编译后（x86-64 汇编片段，摘自幻灯片）——程序就是处理器指令的列表！
_main:
  pushq %rbp
  movq  %rsp, %rbp
  subq  $32, %rsp
  movl  $1, -20(%rbp)      ; x = 1
  movl  $0, -24(%rbp)      ; i = 0
.L1:
  cmpl  $10, -24(%rbp)     ; i < 10 ?
  jge   .L2                ; 不成立则跳出循环
  movl  -20(%rbp), %eax
  addl  -20(%rbp), %eax    ; x = x + x
  movl  %eax, -20(%rbp)
  movl  -24(%rbp), %eax
  addl  $1, %eax           ; i = i + 1
  movl  %eax, -24(%rbp)
  jmp   .L1
.L2:
  leaq  fmt(%rip), %rdi
  movl  -20(%rbp), %esi
  callq printf             ; printf("%d\n", x)
  ...
  ret
```

```
; ILP 关键示例（幻灯片核心）：计算 a = x*x + y*y + z*z
; 假设寄存器初值 R0 = x, R1 = y, R2 = z
  mul R0, R0, R0    ; 指令 1：R0 = x*x
  mul R1, R1, R1    ; 指令 2：R1 = y*y
  mul R2, R2, R2    ; 指令 3：R2 = z*z
  add R0, R0, R1    ; 指令 4：R0 = x*x + y*y
  add R3, R0, R2    ; 指令 5：R3 = x*x + y*y + z*z = a
```

【代码做了什么？】 – 第一段 C 程序 `x = x + x` 循环 10 次：每次迭代把 x 翻倍，最终 $x = 2^{10} = 1024$ 并打印。编译后变成一段 x86-64 汇编——**从处理器的视角看，程序只是一串指令**（取指、译码、执行、写回），幻灯片借此纠正"程序 = 高级语言代码"的直觉。 – 第二段汇编实现 `a = x*x + y*y + z*z`：先分别对三个输入平方（3 条 `mul`），再两次加法合并。若处理器每时钟只能执行一条指令，这段程序需要 **5 个时钟**。 – 课堂问题："能做得更好吗？"——如果处理器有多个执行单元（ALU）呢？

【并行机制解说】 – 这演示的是 **ILP（指令级并行）+ superscalar（超标量）执行**，对应本讲"历史上两大单核提速手段之一"。 – 三条 `mul` 指令**互相独立**（各自的输入来自不同寄存器），可以同时发射到多个执行单元。若有 3 个 ALU：`时间 t=1: mul R0,R0,R0 mul R1,R1,R1 mul R2,R2,R2 （3 条并行，ILP=3） 时间 t=2: add R0,R0,R1 （依赖 1、2，只能等） 时间 t=3: add R3,R0,R2 （依赖 4） → 3 个时钟完成，而不是 5 个！` – 但注意：指令 4 依赖指令 1、2 的结果，指令 5 依赖指令 4——依赖关系是**硬约束**，并行调度必须满足"若 X 依赖 Y，则 X 必须比 Y 晚执行"（幻灯片"respect program order"问题的核心：无论怎样乱序调度，**程序输出必须与按程序顺序执行完全一致**）。 – 对应概念：ILP 的"并行"发生在**单条指令流内部**，由硬件（out-of-order control logic）自动发现，程序员无需显式表达——这与后面要学的 multi-core（多指令流并行）和 SIMD（数据并行）有本质区别。

示例 2：串行 vs 并行求和（std::thread + Amdahl's Law 分析）

代码（cpp）：

```cpp
// parallel_sum.cpp —— 对应课堂 DEMO 1/2 的"人类处理器"演示
// 编译运行：g++ -O2 -std=c++17 parallel_sum.cpp -o parallel_sum -pthread
//           ./parallel_sum [线程数 P]
#include <iostream>
#include <vector>
#include <thread>
#include <numeric>
#include <chrono>
#include <cstdlib>

// 串行实现：单线程顺序累加
double serial_sum(const std::vector<double>& v) {
    double sum = 0.0;
    for (double x : v) sum += x;      // 一个"处理器"干完全部活
    return sum;
}

// 每个线程负责一段连续区间 [begin, end)，把局部和写回独立槽位
void partial_sum(const std::vector<double>& v, size_t begin, size_t end, double* out) {
    double s = 0.0;
    for (size_t i = begin; i < end; ++i) s += v[i];
    *out = s;                          // 各写各的槽位，无需锁（无共享写）
}

double parallel_sum(const std::vector<double>& v, int P) {
    const size_t n = v.size();
    const size_t chunk = (n + P - 1) / P;      // 1) 分解 + 2) 均分分配（blocked assignment）
    std::vector<double> partials(P, 0.0);
    std::vector<std::thread> threads;
    threads.reserve(P);
    for (int t = 0; t < P; ++t) {
        size_t begin = t * chunk;
        size_t end   = std::min(begin + chunk, n);
        threads.emplace_back(partial_sum, std::cref(v), begin, end, &partials[t]);
    }
    for (auto& th : threads) th.join();        // 3) 同步点：等待所有线程完成
    return std::accumulate(partials.begin(), partials.end(), 0.0);  // 通信：归约 partial sums
}

int main(int argc, char** argv) {
    const size_t N = 10'000'000;             // 1e7 个 double ≈ 80 MB
    const int P = (argc > 1) ? std::atoi(argv[1]) : 4;
    std::vector<double> v(N);
    for (size_t i = 0; i < N; ++i) v[i] = (i % 7) * 0.5;

    auto t0 = std::chrono::steady_clock::now();
    double s1 = serial_sum(v);
    auto t1 = std::chrono::steady_clock::now();
    double sp = parallel_sum(v, P);
    auto t2 = std::chrono::steady_clock::now();

    auto ms = [](auto a, auto b) {
        return std::chrono::duration<double, std::milli>(b - a).count();
    };
    std::cout << "P = " << P
              << "  serial = " << ms(t0, t1) << " ms"
              << "  parallel = " << ms(t1, t2) << " ms"
              << "  speedup = " << ms(t0, t1) / ms(t1, t2) << "\n";
    std::cout << "sum check: " << (s1 == sp ? "OK" : "MISMATCH") << "\n";
    return 0;
}
```

【代码做了什么？】 – `serial_sum`：一个"处理器"从头到尾累加 1 千万个元素，作为基准时间 T_1。 – `parallel_sum`：把数组按线程数 P 切成 P 个连续块（分解工作）；每个 `std::thread` 执行 `partial_sum` 负责自己那块（分配工作），把局部和写进

`partials[t]`（互不冲突的独立槽位）；主线程 `join()` 等待所有线程完成（同步点），最后把 P 个局部和相加得到总和（通信/归约）。 – 输出三个关键数字：串行时间、并行时间、**speedup = T_1/T_P**，并校验两种结果一致。

【并行机制解说】 – 这段代码把课堂 DEMO 1/2/3 的三个观察全部实体化： – **通信开销（DEMO 1）**：最终把所有 `partials` 归约相加、以及线程创建/join 的开销，就是"互相告诉对方 partial sum"的代价。P 越大，这部分在总时间里占比越高——这正是 Amdahl's Law 里"串行部分"的一种来源（`speedup ≤ 1/(1−p)`，其中 1−p 包含线程启动、join、最终归约）。 – **负载分配（DEMO 2）**：均分 `chunk = (n+P−1)/P` 是"改善分配"的体现；若数组长度不能被 P 整除，某些线程多算一个元素，整体完成时间由**最慢线程**决定（`max` 而非 `average`）——负载不均的直接后果。 – **同步点**：`join()` 是显式同步——主线程必须等所有 worker 完成才能归约，这就是"管理 communication/synchronization 使其不限制 speedup"的主题。 – 一个值得做的实验：P = 1、2、4、8 分别跑一次。你通常会发现 speedup 不是线性增长的，且随 P 增大收益递减——因为线程创建/join/归约这些**串行部分**不变，Amdahl 定律开始起作用。 – 对应概念：本讲三大主题（分解工作、分配工作、管理通信/同步）+ speedup 定义 + Amdahl's Law。

示例 3：Cache 模拟器——复现课堂 Cache 示例 1/2（LRU + 局部性）

代码（c）：

```c
// cache_sim.c —— 模拟幻灯片中的 cache：总容量 8 字节、4 字节 cache line、LRU 替换
// 编译运行：gcc -O2 cache_sim.c -o cache_sim && ./cache_sim
#include <stdio.h>
#include <string.h>

#define LINES 2          /* cache 容量：2 行 */
#define LINE_BYTES 4     /* 每行 4 字节 */
#define MEM_SIZE 16      /* 内存：16 字节数组 */

/* 用 cache 执行一次 load 访问，返回是否命中；命中则更新 LRU 顺序 */
/* LRU 约定：lru_order[i] 越小越"新"（0 = 最近使用），越大越"旧" */
static int access_cache(unsigned addr, unsigned cache_lines[LINES],
                        int lru_order[LINES], int* misses) {
    int i, hit_line = -1;
    for (i = 0; i < LINES; i++)          /* 查找地址所在行是否在 cache 中 */
        if (cache_lines[i] == addr - addr % LINE_BYTES) { hit_line = i; break; }

    if (hit_line >= 0) {
        /* 命中：被访问的行变成"最新"（=0），其余行变旧（+1） */
        for (i = 0; i < LINES; i++)
            if (i != hit_line) lru_order[i]++;
        lru_order[hit_line] = 0;
        return 1;                        /* hit */
    }
    /* miss：按 LRU 淘汰最旧行（lru_order 值最大者），装入新行 */
    int victim = 0;
    for (i = 1; i < LINES; i++)
        if (lru_order[i] > lru_order[victim]) victim = i;
    cache_lines[victim] = addr - addr % LINE_BYTES;
    for (i = 0; i < LINES; i++)
        if (i != victim) lru_order[i]++;
    lru_order[victim] = 0;
    (*misses)++;
    return 0;                            /* miss */
}

int main(void) {
    /* 幻灯片 Cache 示例 1：访问序列 0x0..0x5 附近 —— 展示空间/时间局部性 */
    unsigned seq1[] = {0x0, 0x1, 0x2, 0x3, 0x0, 0x1, 0x4, 0x5, 0x0};
    /* 幻灯片 Cache 示例 2：顺序读完整 16 字节，再读一遍 —— 展示 capacity miss */
    unsigned seq2[32];
    for (int i = 0; i < 16; i++) seq2[i] = i;
    for (int i = 0; i < 16; i++) seq2[16 + i] = i;   /* 第二遍：同样的 0..15 */

    unsigned lines[LINES] = {0xFFFFFFFF, 0xFFFFFFFF};
    int order[LINES] = {0, 1}, misses = 0;
    printf("== 示例 1（局部性）==\n");
    for (size_t i = 0; i < sizeof(seq1) / sizeof(seq1[0]); i++) {
        int hit = access_cache(seq1[i], lines, order, &misses);
        printf("访问 0x%x: %s\n", seq1[i], hit ? "hit" : "MISS");
    }
    printf("共 %d 次 miss\n", misses);

    misses = 0;
    memset(lines, 0xFF, sizeof(lines));
    order[0] = 0; order[1] = 1;
    printf("\n== 示例 2（第二遍为何不命中）==\n");
    for (size_t i = 0; i < sizeof(seq2) / sizeof(seq2[0]); i++) {
        int hit = access_cache(seq2[i], lines, order, &misses);
        if (i < 16 || !hit)   /* 第一遍全部打印；第二遍只打印 miss */
            printf("访问 0x%x: %s\n", seq2[i], hit ? "hit" : "MISS");
    }
    printf("两遍共 %d 次 miss（第二遍的 miss 即 capacity miss）\n", misses);
```

```
    return 0;
}
```

【代码做了什么？】 – 程序用 2 行 × 4 字节、LRU 策略的软件 cache 模拟器，复现幻灯片 Cache 示例 1/2 的两次实验。 – **示例 1**：访问序列 0x0, 0x1, 0x2, 0x3, 0x0, 0x1, 0x4, 0x5, 0x0——第一次访问 0x0 是 cold miss（装入 line 0x0，顺带覆盖 0x0–0x3），随后访问 0x1/0x2/0x3 命中（**空间局部性**）；再次访问 0x0 命中（**时间局部性**）。 – **示例 2**：顺序读完整 16 字节（0x0–0xF）后再读一遍。第一遍 4 个 cold miss（每行一次）；**第二遍读 0x0 时它早已被 0x8 逐出**——因为 2 行的 cache 只装得下最后两行（0x8、0xC），这就是幻灯片讨论题"为什么第二遍读 0x0 不是 hit"的答案：**capacity miss**。若 cache 有 4 行，整个 16 字节数组全部驻留，第二遍全部命中。 – 注意：模拟器里那个看似多余的 LRU 循环是故意的教学注释占位，实际更新逻辑在 miss/hit 分支内完成——建议读者自己把 `access_cache` 的 LRU 维护简化重写一遍（练习：改成 4 行 cache，观察第二遍全命中）。

【并行机制解说】 – 对应概念：**cache / cache line / cold miss / capacity miss / spatial & temporal locality / LRU**。cache 是"实现内存抽象"的硬件细节——**只影响性能、不影响程序输出**；本模拟器正是把这个"性能层"单独拿出来观察。 – 为什么对并行编程重要？多核机器上**每个核都有自己（或共享）的 cache 层级**，数据在 cache 里的位置直接决定访存是 4 周期（L1）还是 ~248 周期（DRAM）；后续课程（L6 局部性与通信）会看到：并行程序的数据布局、线程间数据共享方式，本质上都在操纵"数据落在哪一级 cache"。 – 联系第 15 条概念：数据移动能耗比计算高 1–3 个数量级，因此"提高 cache 命中率 = 减少昂贵的数据搬运 = 同时省时省电"。

三、关键要点

1. **单线程性能增长已几乎停止，软件必须自己并行化**：历史上单线程 CPU 性能约每 18 个月翻倍（"软件开发者什么都不做，代码明年自动变快"）；如今频率提升受功耗墙限制、ILP 挖掘已见顶（"The Free Lunch Is Over"，Herb Sutter），架构师只能靠**加更多并行执行单元**或**专用单元**提速——软件不并行就没有免费午餐。
2. **并行编程的三大挑战 = 通信开销、负载不均、通信/计算比**：DEMO 1 证明通信限制加速比；DEMO 2 证明负载不均限制加速比；DEMO 3 证明当问题本身通信远多于计算时，并行几乎无济于事。对应并行思维三步骤：分解工作、分配工作、管理通信/同步。
3. **FAST ≠ EFFICIENT**：2x speedup 在 10 处理器机器上只是"快"，效率 = 20% 才是"用好了硬件"。效率思维贯穿全课程。
4. **程序 = 一串指令；处理器 = 取指/译码 + 寄存器 + ALU**：理解 ILP 从"指令依赖图"出发——互不依赖的指令可并行（superscalar 自动发现），依赖关系必须按序。但 ILP 的收益递减（4 发射宽度基本吃光可用 ILP）。
5. **访问数据的方式决定性能与功耗**：cache 层级（L1/L2/L3/DRAM）把"线性内存"抽象实现为 4→12→38→~248 周期的阶梯；数据移动能耗比计算高 1–3 个数量级——"高效的处理器几乎总是归结为高效地访问数据（accessing data efficiently）"。

四、常见陷阱与注意事项

1. **混淆 FAST 与 EFFICIENT**：看到"程序在并行机上快了 2 倍"就欢呼，却不检查 speedup/P（效率）。在 10 核机器上 2x speedup 意味着 8 个核在闲置，不是好结果。
2. **忽视通信开销**：并行化时只盯着计算拆分，忘了线程间传数据（partial sums、共享结果）的时间。DEMO 1 的教训：通信是加速比的第一杀手；降低通信代价（更近、更高效的数据交换）比增加处理器数更有效。
3. **负载不均 = 最慢者决定一切**：并行完成时间由 `max`（最慢处理器）而非平均决定。即使只有 10% 的尾部工作，也会拖垮整体加速比——分配工作要尽量均衡。
4. **并行开销大于收益**：线程创建、join、同步、归约都是"串行部分"。对小问题（N 很小），并行化的固定开销可能超过收益，甚至比串行更慢——先算 Amdahl 上限再动手。
5. **误解 Amdahl's Law**：误以为"p=0.9 就一定能接近 10 倍加速"。注意：(a) p 是**可并行部分占比**，串行部分 1–p 决定天花板 1/(1–p)；(b) 通信/同步/负载不均造成的低效相当于放大了 1–p；(c) P→∞ 时加速比收敛到 1/(1–p)，再多处理器也无济于事。
6. **把 cache 当成"正确性"问题**：cache 是**实现细节**——它不改变程序输出，只影响性能（"does not impact the output of a program, only its performance"）。写程序时不要依赖 cache 行为保证正确性；但要用局部性（时间/空间）换取性能。

五、思考题（带答案）

Q1. 课堂 DEMO 1 中，把学生（处理器）挪近一点、或允许他们喊话，为什么能提升加速比？这对应并行编程的哪个环节？ – **答案**：DEMO 1 中每个学生计算一部分总和，然后需要把 partial sum 告诉组长合并。通信（传 partial sum）的时间占用了总时间，限制了加速比；挪近/喊话降低了通信代价，使加速比提升。这对应并行思维第三步"管理 communication/synchronization，使其不限制 speedup"——通信开销是加速比的实际瓶颈，降低通信成本是提升并行性能的第一抓手。

Q2. 某程序 90% 可并行（p = 0.9）。用 4 个处理器理论加速比上限是多少？若实际只测到 2x speedup，效率是多少？可能的原因有哪些？ – **答案**：由 Amdahl's Law，speedup_max(4) = 1 / (0.1 + 0.9/4) = 1 / 0.325 ≈ 3.08。实际 2x speedup 的效率 = 2/4 = 50%。原因可能是：通信开销、负载不均、同步/启动开销——这些都等效于扩大了串行部分（1−p），把上限从 3.08 进一步压低到 2。

Q3. 为什么说"数据移动"是效率问题的核心？用课堂给出的能耗数字说明。 – **答案**：整数运算约 1 pJ，而从低功耗 DRAM（LPDDR）读 64 位约 1200 pJ——搬一次数据比算一次数贵三个数量级。以 10 GB/s 读内存约 1.6 W，已超过整个移动 GPU 的功耗预算（~1 W）。因此"高效处理几乎总是归结为高效访问数据"：尽量让数据留在 cache 里（时间/空间局部性）、减少跨层级的数据搬运，是性能和功耗双赢的关键。

> **注意**：本讲内容直接服务于 **Assignment 1（Analyzing Parallel Program Performance on a Quad-Core CPU，10 月 6 日截止）**——该作业在四核 Intel CPU 上使用 ISPC 分析并行程序性能，幻灯片预告的对照基线是"单线程 C 程序（-O3 编译）"vs"用上全部并行资源（4 核 + AVX SIMD + hyper-threading）的程序"，预期可达约 **32–40x** 加速。AVX、hyper-threading 等术语将在 Lecture 2 讲解。

Lecture 2: A Modern Multi-Core Processor (Part I)（日期：Sep 25, 2025）

> **概述**：本讲从"软件工程师视角"讲计算机体系结构，回答一个问题：现代并行处理器如何获得高吞吐（high throughput）？核心是三种并行执行形式：**multi-core（多核，TLP）**、**SIMD（数据并行，DLP）**与**hardware multi-threading（硬件多线程，隐藏访存延迟）**。课程先复习第 1 讲的指令流、处理器与 cache 基础，然后用同一个 `sinx`（泰勒展开算 sin）程序贯穿全场：先看它如何用 C++ 线程拆到多核，再看如何用 AVX intrinsics 向量化（SIMD），最后讨论分支导致的分歧执行（divergent execution）与多线程如何用"换线程干活"来隐藏内存停顿。

一、核心概念与定义

1. 复习：Instruction Stream（指令流）与处理器组成

- **定义**：程序编译后就是一条**指令流**（list of processor instructions）。一个简单处理器由三部分组成：**Fetch/Decode（取指/译码，决定下一步执行哪条指令）**、**Registers（寄存器，保存程序状态/运算输入输出）**、**Execution Unit / ALU（执行单元，执行指令描述的操作）**。简单处理器每时钟执行一条指令。
- **现实类比**：指令流 = 菜谱步骤清单；取指/译码 = 看菜谱决定下一步；寄存器 = 手边备好的食材（中间量）；ALU = 灶台（执行动作）。
- **公式/图示**：
```
+-------------------------------------------------+ | Execution Context (registers R0..R3 + PC) |
| ▲ | | | Fetch/Decode Execution Unit (ALU) | | （决定下一条指令） （执行运算/访存） | +-------------------------------
----------------------+ ▲ 内存中的指令流：ld r0, addr[r1] → mul r1,r0,r0 → ... → st addr[r2],r0
```

2. 复习：Superscalar Execution（超标量执行）与 ILP

- **定义**：超标量处理器**自动**在单条指令流中寻找互不依赖的指令，把它们并行放到多个执行单元上执行（如每时钟译码并执行 2 条指令），由 **out-of-order control logic（乱序控制逻辑）** 完成调度。关键约束："respect program order"——乱序调度后**程序的输出必须与按原顺序执行完全一致**（即只在不改变结果的前提下重排）。
- **现实类比**：流水线上的两道互不相关的工序（贴标签 + 装箱）可以同时进行；但"先贴标签再装箱"的依赖工序必须等标签贴完。
- **公式/图示**：依赖图示例（幻灯片 a=2, b=4 例子）：`程序（按 PC 顺序） 指令依赖图（箭头 = 数据依赖） 00: a = 2 00 ─┐ 01: b = 4 01 ─┤ 02: tmp2 = a + b // 6 02 ◄─┘ (依赖 00,01) 03: tmp3 = tmp2 + a // 8 03 ◄─ 02, 00 04: tmp4 = b + b // 8 04 ◄─ 01 05: tmp5 = b * b // 16 05 ◄─ 01 06: tmp6 = tmp2 + tmp4 // 14 06 ◄─ 02, 04 07: tmp7 = tmp5 + tmp6 // 30 07 ◄─ 05, 06 08: if (tmp3 > 7) ... 08 ◄─ 03` 无依赖的指令（如 02 与 04、05）可并行执行；依赖链（00→02→03→08）必须串行。

3. Multi-Core Processor（多核处理器）与 TLP

- **定义**：**Idea #1（幻灯片原话）**：与其把晶体管花在"让单条指令流跑得更快"的复杂逻辑（乱序、投机、更大 cache、更聪明分支预测、预取器）上，不如**用晶体管换更多核**。每个核独立取指/译码、运行**完全不同的指令流**，提供 **thread-level parallelism（TLP，线程级并行）**。代价：更简单的核跑单条指令流可能更慢（幻灯片例子：每个核慢 25%，双核 = 2 × 0.75 = 1.5 的潜在加速）。
- **现实类比**：与其雇一个"全能超人"（复杂单核），不如雇几个普通员工并行干活（多核）——单个员工略慢，但人多总吞吐更高。
- **公式/图示**：`前多核时代： [复杂单核：大 cache + 乱序 + 分支预测 + 预取] ← 晶体管堆给单条指令流 多核时代： [核][核][核][核] ... ← 晶体管换更多核` 关键：**软件不表达并行，多核就没有任何收益**。若程序仍编译成单线程指令流，它只会跑在其中一个核上，甚至因单核变简单而变慢（25% 更慢）。

4. Data-Parallel Expression（数据并行表达，forall）

- **定义**：用 `forall` 声明"**循环迭代互相独立**"（幻灯片中的虚构语言；ISPC 的 `foreach` 是它的真实实现）。迭代之间无数据依赖 → 编译器/运行时可以自动生成多线程代码、或向量指令。
- **现实类比**：批改 100 份相同试卷——每份独立评分（迭代独立），可同时交给多个老师批。
- **公式/图示**：`forall (int i from 0 to N) { // 声明：迭代互相独立 y[i] = sinx(x[i]); // 每个迭代对不同的数据做相同操作 }`

5. SIMD Execution（单指令多数据执行）与 DLP

- **定义**：**Idea #2（幻灯片原话）**：把管理一条指令流的成本（取指/译码）**摊薄到多个 ALU 上**——同一条指令广播（broadcast）给所有 ALU，所有 ALU 同时对不同数据执行该操作。这利用的是 **data-level parallelism（DLP，数据级并行）**：同一序列的指令作用在大量不同数据上。现代 CPU 实例：Intel AVX2（256 位，8×32 位 float）、AVX512（512 位，16×32 位）、ARM Neon（128 位，4×32 位）。
- **现实类比**：老师对全班喊"翻到第 50 页"——一条指令同时指挥 40 名学生（40 个"数据"）。比挨个单独通知省 40 倍的"控制成本"。
- **公式/图示**：`一条 SIMD 指令：mul v1, v0, v0 （8-wide，256-bit） ┌──────────────────────────────┐ Fetch/Decode ──► ALU0 ALU1 ... ALU7 │ 8 个 ALU 同时执行"乘" └──────────────────────────────┘ 标量版：mul r1, r0, r0 执行 8 次（8 条指令） 向量版：mul v1, v0, v0 执行 1 次（1 条指令，同时算 8 个元素）`

6. Explicit SIMD vs Implicit SIMD（显式 / 隐式 SIMD）

- **定义**：
- **Explicit SIMD（显式 SIMD）**：向量化发生在**编译期**——编译器把标量循环编译成向量指令（`vloadps`、`vmulps`、`vstoreps` 等），可以检查二进制里看到 SIMD 指令。来源有三种：程序员用 intrinsics 显式请求；用并行语言语义（如 forall/foreach）传达；或编译器对循环做依赖分析后**自动向量化**（auto-vectorizing）。
- **Implicit SIMD（隐式 SIMD）**：编译器生成的是**标量指令**的二进制，但硬件**总是同时运行 N 份程序实例**，由硬件（而非编译器）负责把多个实例的相同指令放到 SIMD ALU 上同时执行。现代 GPU 采用这种模式，SIMD 宽度通常为 8–32。

- **现实类比**：显式 SIMD = 你（程序员）明确吩咐"一次算 8 个"；隐式 SIMD = 你只管写"算 1 个"，老板（硬件）看到 8 个员工都在做同一件事，就把他们合并成一组一起做。
- **公式/图示**：无公式；理解"谁负责发现并行"是关键差异——显式：编译器；隐式：硬件运行时。

7. Coherent Execution 与 Divergent Execution（一致执行与分歧执行）

- **定义**：
- **Instruction stream coherence（coherent execution）**：程序的**同一段指令序列适用于大量数据元素**的性质。它是 SIMD 资源被高效利用的**必要条件**（但不是多核并行的必要条件——每个核可以独立取指，跑不同指令流）。
- **Divergent execution（分歧执行）**：缺乏指令流一致性。在 SIMD 上表现为：同一个 `if/else` 分支里，部分 ALU 走真分支、部分走假分支。硬件会**顺序执行两个分支、用 mask 掩蔽（丢弃）不对应 ALU 的输出**——不是所有 ALU 都在做有用功，最坏情况只有 1/8（8-wide）甚至 1/32（GPU）的峰值性能。
- **现实类比**：全班一起念课文（coherent）；突然有人读到"如果 t>0 读 A 句否则读 B 句"，老师只好先带大家念 A 句、再念 B 句，念 A 句时不需要 B 句的学生只能干等（mask 掉）。
- **公式/图示**：`时间(时钟) ALU1 ALU2 ALU3 ALU4 ALU5 ALU6 ALU7 ALU8 (8-wide SIMD) t>0.0? T T T F F F F F 1-2: t=t*t 分支：只有 ALU1-3 的结果被保留（其余掩蔽） 3-4: t=t*50 分支：只有 ALU4-8 的结果被保留（其余掩蔽） 5: 恢复无条件代码：8 个 ALU 全部满速 分支区间有效利用率 = max(3,5)/8 → 最坏 1/8`

8. 复习：Cache、Cache Line、Miss 类型与 Locality

- **定义**：cache 是芯片上保存内存子集副本的存储；按 **cache line** 粒度工作（如 4 字节/行），LRU 替换。课堂示例（8 字节总容量、2 行 4 字节）展示了：
- **Cold miss（冷缺失）**：某行第一次被访问，必须从 DRAM 装入；
- **Hit（命中）**：目标地址已在 cache 中；
- **Capacity miss（容量缺失）**：工作集超出 cache 容量，旧行被逐出，再次访问时重新装入——即"第二遍读 0x0 为什么不是 hit"的答案：访问序列 0x0→0xF 一遍后，2 行的 cache 只装得下最后 2 行（0x8、0xC），0x0 早已被逐出。
- 空间局部性：装一行顺带预载相邻地址；时间局部性：重复访问同一地址。
- **现实类比**：桌面（cache 行）上只能摊开 2 张地图；翻完 4 张地图再看第 1 张，必须回抽屉（DRAM）重拿。
- **公式/图示**：`访问序列: 0x0 0x1 0x2 0x3 | 0x4 0x5 0x6 0x7 | 0x8 ... 0xF | 0x0 ... cache(2行): [0x0] 命中*4 → [0x0] [0x4] → 逐出0x0装[0x8] → 逐出0x4装[0xC] → 第二遍读 0x0: capacity miss (0x0 早被 0x8 逐出) *若 cache 有 4 行: 整个 16 字节数组全部驻留，第二遍全部命中`

9. 复习：Stall（停顿）、Prefetching（预取）与不可预测访问

- **定义**：指令依赖未完成的访存 → 处理器 stall。缓解手段：
- **Data prefetching（数据预取）**：现代 CPU 有硬件逻辑**动态分析程序访存模式并预测未来地址**，提前把数据装入 cache，让后续 load 变成 cache hit。代价：预测错了会**浪费带宽、污染 cache**，反而降低性能。
- 但若数据"最近没被读过、且下一个地址不可预测"（如 `int x = some_function(); int y = A[x];` 的随机访存），预取无能为力——这是下一节多线程登场的动机。
- **现实类比**：好餐厅会提前把常点菜备好（预取命中）；但客人随机点冷门菜（不可预测），备了也白备（带宽浪费）。
- **公式/图示**：`无预取: ld r0,mem[r2] (miss → 等 ~248 周期) → add 停顿 有预取: [提前装入 cache] → ld 变成 hit → add 立刻执行 不可预测: int y = A[x]; ← 不知道 x 就不知道地址，预取器无能为力`

10. Hardware Multi-Threading（硬件多线程）

- **定义**：**Idea #3（幻灯片原话）**：在**同一个核**上交错执行多个线程以隐藏停顿——"当前线程无法推进？那就去执行另一个线程的指令"。两种实现：
- **Interleaved multi-threading（交错多线程，aka temporal）**：每个时钟，核从多个线程中**选一个**，取其一条指令在 ALU 上执行；

- **Simultaneous multi-threading（SMT，同时多线程）**：每个时钟，核**从多个线程同时选指令**放到不同 ALU 上执行——Intel **Hyper-threading（超线程，每核 2 线程）** 就是 SMT。
- **现实类比**：等洗衣机转（访存延迟）时去叠衣服（另一线程的算术）——机器（核心）不空转。
- **公式/图示**：`单线程核心: [线程1: 算术 算术 算术 |←—等待 load 12 周期—→| 算术 ...] 20% 利用率 多线程核心: [线程1: 算术 算术 算术 | 等待中... | 算术 ...] [线程2: | 算术 算术 算术 | ] ↑ 线程1 停顿期间，核心执行线程2 的算术 → 利用率提高`

11. Latency Hiding 与利用率计算（核心定量结论）

- **定义**：多线程**不改变访存延迟本身**，只是让延迟不再导致处理器利用率下降（"the latency of the memory operation is not changed by multi-threading, it just no longer causes reduced processor utilization"）。课堂定量练习：线程每轮做 **3 条算术 + 1 条 12 周期延迟的 load**：
- 1 个线程：每 15 周期忙 3 周期 → 利用率 3/15 = **20%**；
- 2 个线程：6/15 = **40%**；
- **5 个线程：15/15 = 100%**（再多线程无额外收益）；
- 若改为 **6 条算术 + 12 周期 load**（算术/访存比更高）：只需 **3 个线程**即可 100%。
- **现实类比**：流水线上每个人做 3 秒的活然后等 12 秒的料——需要 5 个人接力才能让工位永不空转；每人干的活越多（6 秒），需要的人越少（3 人）。
- **公式/图示**：`每轮: 3 条算术(3 时钟) + load(12 时钟等待) → 周期 = 15 时钟 所需线程数 × 每线程忙时钟数 ≥ 周期长度 3 算术: 5 × 3 = 15 → 5 线程达 100% 6 算术: 3 × 6 = 18 → 3 线程达 100% （算术越多，隐藏延迟所需线程越少）`

12. Execution Context 是有限资源（No Free Lunch）

- **定义**：硬件多线程需要**为每个线程保存一份执行上下文**（寄存器 + PC），这些上下文存在片上的 **Context storage**（或 L1 cache 区域）中，是有限资源。**许多小上下文（如 16 个硬件线程，每线程小工作集）= 高延迟隐藏能力；少数大上下文（如 4 个线程，每线程大工作集）= 低延迟隐藏能力**。设计权衡：吞吐 vs 每线程性能。
- **现实类比**：办公室工位总数固定——工位多（16 个小隔间）能容纳更多员工同时办公（隐藏更多等待），但每个人空间小；工位少（4 个大办公室）每个人舒服但能同时办公的人少。
- **公式/图示**：`16 个硬件线程: [ctx1][ctx2]...[ctx16] 每线程小工作集 → 高 latency hiding 4 个硬件线程: [ctx1][ctx2] [ctx3][ctx4] 每线程大工作集 → 低 latency hiding`

13. 三种并行形式总结（Superscalar / SIMD / Multi-core）

- **定义**（幻灯片总结页原文要点）：
- **Superscalar**：单指令流内的 ILP——同一指令流的不同指令并行（核内）；并行由硬件在执行期自动发现。
- **SIMD**：多个 ALU 由同一条指令控制（核内）；对数据并行负载高效（摊薄控制成本）；向量化由编译器（显式）或硬件运行时（隐式）完成。
- **Multi-core**：多个核；每个核同时执行**完全不同的指令流**（TLP）；软件通过线程 API 创建线程来向硬件暴露并行。
- **现实类比**：三个层面的"并行"：一条流水线上多道工序同时做（ILP）；一个广播同时指挥多个人（SIMD）；多条流水线同时开工（multi-core）。
- **公式/图示**（幻灯片三个处理器对比）：`单核超标量: 每时钟从 1 条指令流取 ≤2 条独立指令 双核: 每时钟每核从各自指令流取 1 条 SIMD 四核: 每时钟每核执行 1 条 8-wide SIMD 指令`

14. GPU 的 SIMT（Single Instruction, Multiple Thread）

- **定义**：现代 GPU 执行的硬件线程指令流只有**标量指令**；GPU 核检测到多个硬件线程正在执行**同一条指令**时，用 SIMD ALU 同时执行最多 SIMD-width 个线程；执行不同指令的线程（divergent）被 mask 掉。即"硬件把标量线程流动态合并成 SIMD"。
- **现实类比**：阅兵方阵——教官（取指）喊"齐步走"，本来每人各自走着（独立线程），一旦动作一致就自动合并成整齐方阵（SIMD）；有人走错（divergent）就先被"晾着"（mask）。
- **公式/图示**：见第 7 条 mask 图；GPU 上 SIMD 宽度 8–32，写不好的代码可能只有 1/32 峰值性能。

二、代码示例与详细解说（本讲重点）

示例 1：std::thread 把 sinx 拆到两个核（TLP / multi-core）

代码（cpp）：

```cpp
// multi_core_sinx.cpp —— 演示 TLP / multi-core（幻灯片原版思路，补齐可编译外壳）
// 编译运行：g++ -O2 -std=c++11 multi_core_sinx.cpp -o multi_core_sinx -pthread
#include <iostream>
#include <cmath>
#include <thread>
#include <vector>

// 课堂贯穿示例：用泰勒展开计算 sin(x)，逐元素处理数组
// sin(x) = x - x^3/3! + x^5/5! - x^7/7! + ...
void sinx(int N, int terms, float* x, float* result) {
    for (int i = 0; i < N; i++) {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];   // 分子初值 x^3
        int denom = 6;                      // 分母初值 3!
        int sign = -1;
        for (int j = 1; j <= terms; j++) {
            value += sign * numer / denom;  // 累加第 j 项
            numer *= x[i] * x[i];           // 下一项分子：x^(2j+3)
            denom *= (2*j+2) * (2*j+3);     // 下一项分母：(2j+3)!
            sign *= -1;
        }
        result[i] = value;
    }
}

// 幻灯片中的线程参数打包结构体
typedef struct {
    int N;
    int terms;
    float* x;
    float* y;
} my_args;

void my_thread_func(my_args* args) {
    sinx(args->N, args->terms, args->x, args->y);   // 新线程做前半段
}

void parallel_sinx(int N, int terms, float* x, float* y) {
    std::thread my_thread;
    my_args args;
    args.N = N / 2;                                 // 拆一半工作给线程
    args.terms = terms;
    args.x = x;
    args.y = y;
    my_thread = std::thread(my_thread_func, &args);  // 启动工作线程
    sinx(N - args.N, terms, x + args.N, y + args.N); // 主线程做后半段
    my_thread.join();                                // 同步：等待线程完成
}

int main() {
    const int N = 1 << 20;
    const int terms = 5;
    std::vector<float> x(N), y(N);
    for (int i = 0; i < N; ++i) x[i] = (i % 100) / 100.0f;  // 小角度输入
    parallel_sinx(N, terms, x.data(), y.data());
    int bad = 0;
    for (int i = 0; i < N; ++i)
        if (std::fabs(y[i] - std::sin(x[i])) > 1e-3f) ++bad;
    std::cout << "checked " << N << " elements, mismatches = " << bad << "\n";
    return bad == 0 ? 0 : 1;
}
```

【代码做了什么？】 – `sinx` 对数组每个元素计算泰勒展开的 sin 近似值：内部 j 循环逐项累加（分子 `numer` 每次乘 x^2，分母 `denom` 按阶乘递推，符号交替）。 – `parallel_sinx` 把 N 个元素**对半拆开**：新线程处理 `x[0..N/2)`，主线程处理 `x[N/2..N)`，

最后 `join()` 等待。 – `main` 分配数组、调用并行版本并抽查结果与标准库 `std::sin` 的一致性（验证并行化没有破坏正确性）。

【并行机制解说】 – 这段代码演示的是 **multi-core 并行 / TLP**：两个线程 = 两条**完全独立的指令流**，由 OS 调度到两个物理核上同时执行（若机器有 2 核）。对应概念：多核时代把晶体管换成了更多核，而**软件必须显式创建线程**（这里用 `std::thread`）硬件才能看到并行——幻灯片强调"这个 C 程序若不并行化，编译成单线程指令流只会跑在一个核上，甚至因单核变简单而慢 25%"。 – **工作分配**：`args.N = N/2` 是"分成两块各干一半"的简单均分（blocked assignment）；两个线程通过 `x` / `y` 指针**共享整个数组**（共享内存模型），各自只读写自己的区间，**无冲突**，因此不需要任何锁——这是数据并行任务"安全分解"的范例。 – **同步点**：`join()` 保证主线程在读取/校验结果前，工作线程一定完成——这就是"管理同步使其不成为瓶颈"的最简形式。 – 若要进一步扩展：拆成 4 份就能用满四核（Assignment 1 的机器）；再配合下一示例的 SIMD，才能逼近 32–40x 的峰值。

示例 2：AVX intrinsics 手写 SIMD 向量化 sinx（DLP / explicit SIMD）

代码（cpp）：

```cpp
// simd_sinx.cpp —— 显式 SIMD：用 AVX intrinsics 一次处理 8 个元素
// 编译运行（需要支持 AVX 的 x86 CPU）：
//   g++ -O2 -mavx -std=c++11 simd_sinx.cpp -o simd_sinx
#include <immintrin.h>
#include <iostream>
#include <cmath>
#include <cstdlib>

void sinx_avx(int N, int terms, float* x, float* y) {
    float three_fact = 6.0f;                // 3!
    for (int i = 0; i < N; i += 8) {        // 每轮处理 8 个元素
        __m256 origx = _mm256_load_ps(&x[i]);         // 向量 load：x[i..i+7]
        __m256 value = origx;
        __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx)); // x^3
        __m256 denom = _mm256_broadcast_ss(&three_fact);                  // 广播 6.0
        int sign = -1;
        for (int j = 1; j <= terms; j++) {
            // value += sign * numer / denom
            __m256 tmp = _mm256_div_ps(
                _mm256_mul_ps(_mm256_set1_ps((float)sign), numer), denom);
            value = _mm256_add_ps(value, tmp);
            numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
            float f = (float)((2*j+2) * (2*j+3));     // 下一项分母因子
            denom = _mm256_mul_ps(denom, _mm256_broadcast_ss(&f));
            sign *= -1;
        }
        _mm256_store_ps(&y[i], value);               // 向量 store：y[i..i+7]
    }
}

int main() {
    const int N = 1 << 20;
    const int terms = 5;
    // _mm256_load_ps / store_ps 要求 32 字节对齐
    float* x = (float*)aligned_alloc(32, N * sizeof(float));
    float* y = (float*)aligned_alloc(32, N * sizeof(float));
    for (int i = 0; i < N; ++i) x[i] = (i % 100) / 100.0f;
    sinx_avx(N, terms, x, y);
    int bad = 0;
    for (int i = 0; i < N; ++i)
        if (std::fabs(y[i] - std::sin(x[i])) > 1e-3f) ++bad;
    std::cout << "mismatches = " << bad << "\n";
    free(x); free(y);
    return bad == 0 ? 0 : 1;
}
```

```
; 编译后可以看到显式 SIMD 指令（幻灯片"explicit SIMD"：二进制里能找到向量指令）
vloadps  xmm0, addr[r1]    ; 一次装入 8 个 float
vmulps   xmm1, xmm0, xmm0  ; 8 个元素同时平方
vmulps   xmm1, xmm1, xmm0
...
vstoreps addr[xmm2], xmm0  ; 一次写回 8 个 float
```

【代码做了什么？】 – 与标量版 `sinx` 完全相同的算法，只是所有运算换成 256-bit 向量 intrinsics：`_mm256_load_ps` 一次装入 8 个 float，`_mm256_mul_ps` / `_mm256_add_ps` / `_mm256_div_ps` 对 8 个元素同时运算，`_mm256_broadcast_ss` 把标量（6.0、阶乘因子、sign）复制到 8 个 lane，`_mm256_store_ps` 一次写回 8 个结果。 – 外层循环步长从 1 变成 **8**：`i += 8`，每个迭代处理一个 8 元素向量。`sign` 仍是标量 int（每轮循环翻转）。 – 编译后的指令流里能看到 `vloadps/vmulps/vstoreps` 等向量指令——这就是幻灯片说的 **explicit SIMD**：并行化发生在编译期，程序员用 intrinsics 显式请求。

【并行机制解说】 – 对应概念：**SIMD / DLP**。标量程序一条指令处理 1 个元素；向量程序一条指令处理 8 个元素——**取指/译码成本被 8 个 ALU 摊薄**（Idea #2）。同样是这个 sinx，从"单核每时钟 1 个元素"变成"单核每时钟 8 个元素"，性能最多提升 8 倍（单核内）。 – 与示例 1 的组合关系：**multi-core（多核）与 SIMD（核内）是两个正交的并行维度**。16 个 SIMD 核 = 16 条指令流 × 每核 8 个 ALU = **128 个元素并行**（幻灯片图示）。Assignment 1 的 32–40x = 4 核 × AVX 8-wide × 超线程等因素的乘积。 – 两个实现细节值得注意：(a) `_mm256_load_ps` 要求 **32 字节对齐**，main 里用 `aligned_alloc(32, ...)` 保证；(b) 幻灯片原代码中的 `_mm256_set1ps(sign)` 是笔误，正确 intrinsic 是 `_mm256_set1_ps((float)sign)`——这类"1 与 _ 的顺序"错误在 AVX 编程里很常见。

示例 3：SIMD 下的条件执行与分歧（divergence / masking）

代码（伪代码，对应幻灯片 forall 语言示例）：

```
// 伪代码：如果这个循环被编译成 8-wide SIMD，会发生什么？
forall (int i from 0 to N) {
    float t = x[i];
    if (t > 0.0) {
        t = t * t;        // 分支 A：只有 t>0 的元素需要
    } else {
        t = t * 50.0;     // 分支 B：只有 t<=0 的元素需要
    }
    t = t + 100.0;        // 无条件代码
    t = t / 10.0;
    y[i] = t;
}
```

【代码做了什么？】 – 每个数组元素：若为正则平方，否则乘 50，然后统一加 100 除以 10。逻辑本身平凡，但注意**同一向量里 8 个元素可能同时存在正数和负数**——它们需要执行不同的指令序列。

【并行机制解说】 – 对应概念：**coherent vs divergent execution**。SIMD 硬件没有"每个 lane 各自跳转"的能力——一条指令要么广播给所有 ALU，要么不广播。所以硬件**两条分支都执行**，用 mask 决定每个 ALU 的输出是否写回：`时钟 1–2：执行 t = t*t 分支（3 个 ALU 是 T，保留结果；5 个 ALU 是 F，掩蔽丢弃） 时钟 3–4：执行 t = t*50 分支（5 个 ALU 是 F，保留结果；3 个 ALU 是 T，掩蔽丢弃） 时钟 5：执行 t+100、t/10（无条件，8 个 ALU 全速） 分支期间只有 3/8 或 5/8 的 ALU 在做有用功 → 最坏情况 1/8 峰值性能` – 幻灯片"breakout question"：能否**只用一条 if 就构造出 8-wide SIMD 的最坏情况**？答案：让分支条件对 8 个 lane **恰好 1 真 7 假**（例如 `if (t == 0.0)`，且数据里恰好 1 个零）——两条分支都要执行，有效 ALU 数 = max(1,7) = 7？不对，是"每分支里做有用功的 lane 数"：真分支 1 个、假分支 7 个，总执行时钟翻倍而有效功只有 8 个 lane 的一次量 → 效率 50%；若更极端——分支体内有**两条**分支（嵌套 if），或分支条件是 data-dependent 导致每个 lane 都不同（如 `if (t == i)`），则每条路径只有 1/8 的 lane 有用，最坏 1/8 甚至更低。 – 这就是为什么幻灯片强调：**coherent execution 是 SIMD 高效的必要条件**；而多核并行**不**需要 coherent——每个核独立取指，可以各跑各的分支。GPU（implicit SIMD，宽度 8–32）上 divergence 是头号性能杀手，写不好的代码只有 1/32 峰值。

示例 4：多线程隐藏访存延迟（hardware multi-threading 的动机）

代码（cpp）：

```cpp
// latency_hiding.cpp —— 依赖型访存 + 多线程：演示"换线程干活"隐藏停顿
// 编译运行：g++ -O2 -std=c++11 latency_hiding.cpp -o latency_hiding -pthread
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>

// 每个线程沿自己的"随机跳转表"做依赖型访存：
// 下一次 load 的地址 = 上一次 load 的结果（int y = A[x] 的放大版）。
// 单线程下每个元素都要等满 DRAM 延迟（~数百周期），几乎全程 stall。
void chase(const std::vector<size_t>& next, size_t start, size_t iters) {
    size_t idx = start;
    for (size_t i = 0; i < iters; ++i) idx = next[idx];  // 依赖链上的 load
    volatile size_t sink = idx;   // 防止编译器把整个循环优化掉
    (void)sink;
}

double run(int nthreads, const std::vector<size_t>& next, size_t iters) {
    auto t0 = std::chrono::steady_clock::now();
    std::vector<std::thread> ts;
    for (int t = 0; t < nthreads; ++t)
        ts.emplace_back(chase, std::cref(next), (size_t)t, iters);
    for (auto& th : ts) th.join();
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double>(t1 - t0).count();
}

int main() {
    const size_t SIZE = 1 << 24;      // 16M 元素，远超 L3，必然落到 DRAM
    std::vector<size_t> next(SIZE);
    for (size_t i = 0; i < SIZE; ++i)
        next[i] = (i * 2654435761u) % SIZE;   // 伪随机跳转：地址不可预测、无法预取
    const size_t iters = 1 << 22;
    for (int P : {1, 2, 4, 8, 16}) {
        double sec = run(P, next, iters);
        std::cout << "threads = " << P << "  time = " << sec << " s\n";
    }
    return 0;
}
```

【代码做了什么？】 – 每个线程从自己的起点出发，沿 `next` 表做 `iters` 次**依赖型访存**：`idx = next[idx]`——第 k+1 次 load 的地址是第 k 次 load 的结果，地址不可预测（伪随机跳转），因此**硬件预取器无能为力**（对应幻灯片 `int y = A[x]` 的场景）。 – 主程序分别用 1、2、4、8、16 个线程跑同样的总量，打印耗时。典型观察：1 线程极慢（几乎全程 stall），线程数增加后吞吐显著提升，直到内存带宽饱和后不再增长。

【并行机制解说】 – 对应概念：**hardware multi-threading / latency hiding**。单线程时，每步 load 都要等满 DRAM 延迟（数百周期），核心利用率很低——幻灯片算过：3 条算术 + 12 周期 load，1 线程利用率只有 20%。多个线程同时跑时，一个线程的 stall 期间，核心去执行另一个线程的算术（"If you can't make progress on the current thread... work on another one"）。 – 这是**吞吐导向（throughput-oriented）计算的核心权衡**：为了让整体吞吐最大化，**单线程的完成时间可能变长**（它被其他线程"插队"）——幻灯片 Takeaway 1：多线程不改变访存延迟，只是让它不再造成利用率损失。 – 幻灯片定量结论回顾（配图即可理解）：3 算术 + 12 周期 load → 5 线程 100% 利用率；6 算术 + 12 周期 load → 3 线程即 100%（**算术/访存比越高，需要的线程越少**——Takeaway 2）。 – 实现形式：**interleaved multi-threading**（每时钟选一个线程执行）或 **SMT / Intel Hyper-threading**（每时钟从多个线程同时选指令）。现代 CPU 如 Intel Skylake/Kaby Lake 核：2-way 多线程、每时钟最多 4 条独立标量 + 3 条 8-wide 向量指令。

三、关键要点

1. **现代高吞吐处理器 = 三个并行维度的组合**：multi-core（TLP，多核跑不同指令流）、SIMD（DLP，一条指令驱动多个 ALU）、hardware multi-threading（延迟隐藏，用别的线程填满停顿）。它们可以叠加：多核 × 每核 SIMD × 每核多线程（如 i7-7700K：4 核 × 8-wide SIMD × 3 个向量 ALU × 4.2 GHz ≈ 400 GFLOPs；V100：80 SM × 128 SIMD ALU @1.6 GHz ≈ 16 TFLOPs）。
2. **软件不表达并行，多核毫无用处**：普通 C 程序编译成单线程指令流只能跑在一个核上；要获得多核收益必须创建线程（`std::thread`）；要获得 SIMD 收益必须向量化（intrinsics / 并行语言语义 / 自动向量化）。`forall` 这类数据并行表达让编译器可以同时生成多核代码与向量指令。
3. **SIMD 高效的前提是 coherent execution**：同一指令序列作用于大量数据；分支（divergent execution）会让部分 ALU 空转（masking），最坏 1/8（CPU）甚至 1/32（GPU）峰值。多核并行不需要 coherent——每个核独立取指。
4. **cache 解决"已访问过的数据"，多线程解决"不可预测的数据"**：cache/预取处理局部性与可预测访问模式；当数据既不在 cache 又不可预测时（`A[x]`），唯一办法是硬件多线程——用其他线程填满访存停顿。
5. **隐藏延迟需要"足够的并行工作"**：多线程隐藏延迟的效果取决于 算术/访存比——算术越多需要的线程越少（6 算术 + 12 周期 load 只需 3 线程）；执行上下文（寄存器 + PC）是有限片上级资源，多而小 = 高隐藏能力，少而大 = 低隐藏能力。

四、常见陷阱与注意事项

1. **把"每时钟一条指令"误解为"一条指令只要 1 周期完成"**：处理器流水线里"每时钟一条指令"指的是**指令吞吐（throughput）**，不是**延迟（latency）**——一条指令从取指到写回可能 4 周期甚至 ~20 周期（流水线深度）。这个区分在 Lecture 3 讲流水线时会再次强调。
2. **以为多核 = 自动加速**：不写线程、不向量化，程序只会用到一个核；更糟的是多核时代单核可能比过去的"豪华单核"更简单（如慢 25%），不做任何事反而变慢。**并行必须由软件显式表达**。
3. **忽视对齐要求**：`_mm256_load_ps/_mm256_store_ps` 要求 32 字节对齐；普通 `new float[]` 不保证。用 `aligned_alloc(32, ...)` 或让编译器分配对齐数组，否则运行期崩溃（segfault）。
4. **把分支写进数据并行循环而不考虑 divergence**：`if/else` 在 SIMD 上两条分支都会执行、靠 mask 丢弃，看似"正确"但性能可能只有 1/8。尽量让分支条件对同一向量内所有元素一致（coherent），或改用无分支写法。
5. **忘记 `join()` / 在 join 前使用线程结果**：示例 1 中若去掉 `my_thread.join()`，主线程可能在 worker 未完成时就校验结果，产生数据竞争。同步点（join）是多线程正确性的底线。
6. **误以为多线程会"加快"单条指令流**：hardware multi-threading 是吞吐优化——**可能延长单线程完成时间**（被插队），换来的是多线程总吞吐提升。衡量指标要看系统吞吐，不是单线程延迟。

五、思考题（带答案）

Q1. 同一个 sinx 程序，为什么"多核化"和"向量化（SIMD）"是两个不同的并行维度？用 16 核 × 8-wide SIMD 的机器说明最大并行度来自哪里。 – **答案**：多核并行（TLP）是"多条指令流同时跑在多个核上"，每个核处理不同的数组片段；SIMD（DLP）是"单条指令流内，一条指令驱动 8 个 ALU 处理 8 个相邻元素"。两者正交：16 核每核 8-wide SIMD = 16 条指令流 × 8 个 ALU = **128 个元素同时处理**（幻灯片原图）。最大并行度 = 核数 × SIMD 宽度（再乘多线程数），但前提是程序同时表达出 TLP（线程/forall）和 DLP（向量化/forall）。

Q2. 课堂练习：线程每轮做 3 条算术 + 一条 12 周期延迟的 load。为什么 5 个线程才能 100% 利用核心？如果算术改成 6 条呢？ – **答案**：每轮周期 = 3（算术）+ 12（等待 load）= 15 周期，每个线程每轮只有 3 周期在干活。N 个线程的忙时钟 = 3N；要填满 15 周期需要 3N ⩾ 15 → N = 5。算术改为 6 条后周期 = 18、忙 6，需要 6N ⩾ 18 → N = 3。**结论（Takeaway 2）：程序每访存一次做的算术越多，隐藏延迟所需的线程越少。**

Q3. 幻灯片"breakout question"：只用一条 `if` 语句，如何构造 8-wide SIMD 处理器的最坏情况性能？ – **答案**：用**不带 else** 的单条 if，并让条件只对 8 个 lane 中的 1 个成立。例如 `if (x[i] < 0.0f) { y[i] = x[i] * x[i]; }`，输入中恰好 1 个负元素——

该分支指令按 mask 执行时只有 1/8 的 lane 在做有用功（其余 7 个 lane 的输出被掩蔽丢弃），效率跌到 1/8 峰值（幻灯片图注"Worst case: 1/8 peak performance"）。若分支带 else，则两个分支体都要执行、每分支平均约一半 lane 有用，效率约 50%（不如单分支极端）。这正说明 divergent execution 对 SIMD 效率的破坏力。

> **注意**：本讲与 **Assignment 1** 直接相关——作业在四核 Intel CPU（带 **AVX SIMD 指令 + hyper-threading**）上分析并行程序性能：基线是 `-O3` 编译的单线程 C 程序，目标程序用上全部并行资源，幻灯片预告约 **32–40x** 加速。其中 ISPC（数据并行表达 `forall` / `foreach` 的真实编译器实现）与"把并行工作映射到多核 + SIMD"正是 Lecture 3 的主题。

Lecture 3: Modern Multi-Core Architecture (Part II) + ISPC Programming Abstractions（日期：Sep 30, 2025）

> **概述**：本讲前半部分完成"吞吐导向硬件"的最后一课：**latency vs bandwidth（延迟 vs 带宽）**。通过高速公路、洗衣、水管等类比讲清二者区别，再用"逐元素向量乘法"的思想实验证明现代机器常常是 **bandwidth-limited（带宽受限）**——喂不饱 ALU 的不是延迟而是带宽，克服带宽限制往往是并行软件优化最重要的挑战。后半部分引入本课程第一个正式并行编程抽象：**ISPC（Intel SPMD Program Compiler）**，讲解 SPMD 编程模型下的 **program instances（程序实例）、gang、programCount/programIndex、uniform/varying、foreach 与跨实例操作（reduce_add 等）**，并反复强调本课口号：**区分抽象（semantics）与实现（implementation/scheduling）**。

一、核心概念与定义

1. 复习：三种吞吐计算思想（multi-core / SIMD / multi-threading）

- **定义**：Lecture 2 的三大主题——**multi-core execution**（多核，TLP）、**SIMD execution**（数据并行，DLP）、**hardware multi-threading**（硬件多线程，隐藏访存延迟）。本讲先补完多线程部分的"利用率"讨论，再转向内存系统。
- **现实类比**：三个维度 = 多条流水线（多核）× 一条指令指挥多人（SIMD）× 等人时换活干（多线程）。
- **公式/图示**：见 Lecture 2 笔记第 10–13 条；本讲新增的定量工具是下面的 latency/bandwidth 框架。

2. Latency（延迟）与 Bandwidth / Throughput（带宽 / 吞吐）

- **定义**：
- **Latency（延迟）**：完成**一次**操作所需的时间（如从 SF 开车到 Stanford 0.5 小时；内存响应一次请求 ~2 秒；洗一桶衣服 2 小时）。
- **Bandwidth（带宽）/ Throughput（吞吐）**：系统**单位时间**能完成的操作数或提供的数据量（如高速公路每小时过多少辆车；内存 20 GB/s；洗衣每小时几桶）。
- **Memory bandwidth（内存带宽）**：内存系统向处理器提供数据的速率（例：20 GB/s）。两者是正交的度量：**延迟不随带宽提高而降低**（加车道不改变单程时间），**带宽可随管道/车道增加而提高**。
- **现实类比**（幻灯片三个类比）：
- **高速公路**：SF→Stanford 50 km，车速 100 km/hr → 延迟 0.5 小时。方案 1"开快点"（200 km/hr）→ 吞吐 4 辆/小时；方案 2"多修车道"（4 条车道）→ 吞吐 8 辆/小时；方案 3"车距 1 km"（不改变速度）→ 吞吐 100 辆/小时，4 条车道 → 400 辆/小时。**三种提高吞吐的手段：提速、加资源、流水线/加密发车。**
- **洗衣**：洗 45min + 烘 60min + 叠 15min = 延迟 2 小时/桶。复制资源（两台洗衣机两台烘干机 + 朋友）→ 2 桶 2 小时，吞吐翻倍；**流水线**（一台洗衣机一台烘干机，桶 1 烘干时桶 2 开洗）→ 延迟仍 2 小时，吞吐 1 桶/小时。

- **两根水管**：管 1 最大 100 L/s，管 2 最大 50 L/s，串联后最大流量 = **50 L/s（瓶颈 = 最细的那根管）**。
- **公式/图示**：`latency = 单次操作时间（如 0.5 hr / 2 sec / ~248 cycles） bandwidth = 单位时间操作数（如 2 cars/hr、20 GB/s、1 instr/clock） 系统吞吐 ≤ min(所有级联部件的带宽) ← "两管相连"结论：短板决定流量`

3. Bandwidth-Limited（带宽受限）执行

- **定义**：当处理器请求数据的速度超过内存系统能提供数据的速度时，执行变成 **bandwidth-bound**：指令完成速率由内存供给速率决定，而不是由 ALU 数量决定。幻灯片的关键例子：线程重复执行 3 条**相互依赖**的指令（`X = load 64 bytes; Y = add x+x; Z = add x+y`），核心每时钟 1 条算术、可并行发 load、内存每时钟 8 字节——稳态下**内存 100% 时间在搬数据仍喂不饱核心**，核心周期性停顿（红色区域）。
- **现实类比**：餐厅后厨（ALU）再快，传菜口（内存带宽）一次只能过 8 道菜，出菜速度就被传菜口卡死。
- **公式/图示**（幻灯片结论）：`稳态下核心利用率只取决于"指令吞吐 vs 内存吞吐"的比值， 与内存延迟大小、未完成请求数无关！ （延迟再小，带宽不够照样停顿；隐藏延迟的多线程也救不了带宽瓶颈）`

4. 思想实验：逐元素向量乘法为什么是带宽受限（本讲核心数字）

- **定义**：任务 `C[i] = A[i] × B[i]`（数百万元素）：每个元素需要 **load A[i] + load B[i] + store C[i] = 3 个内存操作、12 字节，只换来 1 次 MUL**。V100 有 5120 个 fp32 ALU（80 SM × 64），1.6 GHz 下每时钟可做 5120 次 MUL → 需要约 **98 TB/s** 带宽才能喂饱全部 ALU；而 V100 的 HBM2 只有 **900 GB/s → 效率 <1%**！对照：八核 3.2 GHz Xeon E5v4 + 76 GB/s 内存总线，该计算效率约 **3%**（但注意：GPU 即使 <1% 效率，绝对速度仍远快于八核 CPU——ALU 数量差距太大）。
- **现实类比**：打字员（ALU）每秒能打 100 字，但送稿员（内存）每秒只送 1 页纸（100 字）——打字员 99% 时间在等稿子。
- **公式/图示**：`每元素: load A[i] (4B) + load B[i] (4B) + store C[i] (4B) = 12 字节 / 1 次 MUL V100 需要带宽 = 5120 MUL/clock × 1.6 GHz × 12 B = ~98 TB/s V100 实际带宽 = 900 GB/s（HBM2, 4096-bit 接口） → 效率 ≈ 900/98000 ≈ <1%`

5. 带宽是临界资源（Bandwidth is the critical resource）

- **定义**：现代高性能并行程序必须：
- **组织计算以减少取数频率**——复用同一线程之前加载的数据（**temporal locality 优化**）、跨线程共享数据（**inter-thread cooperation**）；
- **宁愿多算也不要存了再读**（"math is free"——算术几乎不要钱，数据搬运才要钱）；
- 核心结论：**程序必须低频访问内存才能高效利用现代处理器**。
- **现实类比**：做菜时把所有配料一次性从仓库搬到厨房（一次大搬运），而不是每炒一个菜跑一趟仓库（反复小搬运）。多花点力气切配（算术）比反复取料（搬运）划算。
- **公式/图示**：`高带宽需求模式: for i: C[i] = A[i]*B[i] ← 每算一次读 12 字节（差） 低带宽需求模式: for i: 复用已加载的块、就地累加 ← 每字节数据多次复用（好） "performant programs access memory infrequently"`

6. Instruction Pipeline（指令流水线）与 Throughput vs Latency

- **定义**：一条指令从取指到写回分 4 阶段：**IF（取指）、D（译码+读寄存器）、EX（执行）、WB（写回）**。流水线化后：**单条指令延迟 4 周期，但吞吐 1 条/周期**（4 条指令在不同阶段并行推进）。现代 CPU 流水线可达约 20 级。幻灯片特别提醒："核心每时钟做 1 次乘法"指的是**指令吞吐（INSTRUCTION THROUGHPUT），不是延迟（LATENCY）**；依赖相邻的两条指令需注意正确性（冒险处理）。
- **现实类比**：汽车装配线——一辆车从进厂到出厂（延迟）要几天，但工厂每秒都能出厂一辆（吞吐），因为几十辆车同时在线上。
- **公式/图示**：`时钟: 1 2 3 4 5 6 7 instr0 IF D EX WB instr1 IF D EX WB instr2 IF D EX WB instr3 IF D EX WB → 延迟 4 周期/条，吞吐 1 条/周期（前提：相邻指令无依赖冲突）`

7. Abstraction vs Implementation（抽象 vs 实现）——本课口号

- **定义**：Semantics（语义/抽象）：给定程序与所用操作的含义，程序**算出的答案是什么**；Implementation（实现，aka scheduling）：答案**在并行机器上如何被算出来**——操作以什么（可能并行的）顺序执行？哪些操作由哪个线程、哪个执行单元、

向量指令的哪个 lane 计算？**把抽象的意义与实现的细节混为一谈（conflating）是本课程最常见的困惑来源。** 学生目标：给定程序 + 编程模型的实现方式，能在脑中"trace"出并行计算机各部分在程序每一步做什么。

- **现实类比**："把 100 个箱子搬上楼"（语义）可以有多种实现：一个人搬 100 趟、10 个人各搬 10 趟、用电梯一次 20 箱……结果相同，但实现天差地别。
- **公式/图示**：`抽象层（语义）：程序会算什么？ ← 与硬件无关 实现层（调度）：谁（线程/ALU/lane）在何时算哪部分？ ← 决定性能`

8. ISPC 与 SPMD 编程模型

- **定义**：**ISPC = Intel SPMD Program Compiler**（https://ispc.github.com/；推荐阅读 Matt Pharr 的 "The Story of ISPC"）。**SPMD = Single Program, Multiple Data**：只定义一个函数，但**并行运行该函数的多个实例（instances）**，每个实例处理不同的数据。调用一个 ISPC 导出函数会**spawn 一个 gang 的 program instances**（如 8 个），所有实例**并发执行同一份 ISPC 代码**，每个实例拥有**自己的一份局部变量副本**；函数返回时所有实例都已完成，控制流回到单线程。
- **现实类比**：同一个"批改试卷"函数，8 个助教（program instances）同时各批一叠——程序只有一份，数据有 8 份。
- **公式/图示**：`C 代码：main() ⟶ ispc_sinx() ⟶ 回到 main() | ▼ gang 的 8 个程序实例（programCount = 8）：[实例0][实例1][实例2]...[实例7] 并发执行同一份 ISPC 代码 每个实例有独立的局部变量（value、numer...）`

9. Program Count / Program Index / Gang（程序实例数 / 实例编号 / 线程组）

- **定义**：
- **programCount**：gang 中**同时执行**的实例个数（uniform 值）；
- **programIndex**：当前实例在 gang 中的编号（varying 值：每个实例不同）；
- **gang**：一次 ISPC 函数调用产生的全部程序实例的集合。 ISPC 实现 gang 的方式是 **SIMD 指令**：gang 中实例数 = 硬件的 SIMD 宽度（或其小倍数）；ISPC 编译器生成一个包含 SIMD 指令的 C++ 函数二进制（.o），C++ 侧正常链接该目标文件。
- **现实类比**：全班点名——"第几组"（programCount）和"第几号"（programIndex）决定每个学生的身份；老师（一条 SIMD 指令）一次同时给整组布置任务。
- **公式/图示**：`programCount = 8（gang 大小，uniform） programIndex = 0..7（当前实例编号，varying） int idx = i + programIndex; ← 每个实例算不同的 idx，处理不同的 x[idx]`

10. Uniform vs Varying（ISPC 类型修饰符）

- **定义**：
- **uniform**：变量在**所有程序实例中值相同**，只有**一份**存储（如 `uniform int terms`、`uniform float denom`）。**使用 uniform 纯粹是优化**（省去每个实例各存一份的开销、允许标量/广播指令），**与正确性无关**——写成 varying 结果也对。
- **varying**（默认）：每个程序实例有**各自独立的副本**（如 `float value`、`int idx`），对应 SIMD 向量里的一个 lane。
- **现实类比**：全班同上一门课（uniform：课表只有一份）；但每人笔记不同（varying：每人一份）。
- **公式/图示**：`uniform int N; ← 所有实例共享一份（标量/广播） int idx = ...; ← 每个实例各一份（SIMD 的 8 个 lane） 类型规则：varying 值不能塞进 uniform 变量（编译期类型错误）， uniform 值可以广播成 varying。`

11. Interleaved 与 Blocked 分配（iteration assignment）

- **定义**：把 N 个数组元素分给 gang 内 programCount 个实例的两种基本方式：
- **Interleaved（交错分配，幻灯片 v1）**：`for (uniform int i=0; i<N; i+=programCount) { int idx = i + programIndex; ... }`——实例 k 处理元素 k, k+8, k+16, …。同一时刻 8 个实例访问的 8 个地址**在内存中连续** → 编译器可用一条 **packed vector load（如 `vmovaps` / `_mm256_load_ps`）** 高效实现 `float value = x[idx]`。
- **Blocked（分块分配，幻灯片 v2）**：`uniform int count = N/programCount; int start = programIndex*count; for (uniform int i=0; i<count; i++) { int idx = start + i; ... }`——实例 k 处理连续块 [k·count, (k+1)·count)。同一时刻各实例访问的地址**不连续** → 需要 **gather 指令（如 `vgatherdps` / `_mm256_i32gather_ps`）** 实现，gather 更复杂、更贵。
- **现实类比**：发扑克牌——交错 = 轮流每人发一张（同一时刻每人拿到的牌"相邻"）；分块 = 一人拿一叠（同一时刻大家拿到的牌分散在各处）。

- **公式/图示**：`interleaved（8 实例，programCount=8）：元素 0..7 同一时刻被 8 个实例各取一个 实例0→x[0] 实例1→x[1] ... 实例7→x[7] ← 连续 → packed load（1 条指令） blocked（8 实例，每实例 N/8 个）：实例0 拿 x[0..N/8-1] ... 同一时刻 8 个实例的地址相隔 N/8 ← 不连续 → gather（昂贵）`

12. foreach（ISPC 关键语言结构）

- **定义**：`foreach (i = 0 ... N) { ... }` 声明**循环迭代是并行的**——程序员声明"这些迭代是**整个 gang**（不是每个实例）要完成的"，**由 ISPC 实现负责把迭代分配给 gang 里的程序实例**。许多简单情况下，foreach 让程序员**几乎像写串行程序一样**表达并行代码（"independently, for each element in the input array... do this..."）。
- **现实类比**：老板只说"这些活都要干完"（foreach 迭代集合），至于谁干哪件（分配）由组长（ISPC 实现）安排——程序员不必操心。
- **公式/图示**（foreach 的四种可能实现，幻灯片）：`抽象: foreach (i = 0 ... N) { work(i); } 实现1: 实例 0 串行执行全部迭代 (if (programIndex == 0) for i...) 实现2: 交错分配（等价 v1 的显式写法） 实现3: 分块分配（等价 v2 的显式写法） 实现4: 动态分配—uniform int nextIter; if (programIndex==0) nextIter=0; int i = atomic_add_local(&nextIter, 1); while (i < N) { work(i); i = atomic_add_local(&nextIter, 1); } （幻灯片文本提取中实现 1/4 的"programCount == 0"应为 programIndex == 0 的笔误）`

13. Cross-Instance Operations（跨实例操作，ISPC 标准库）

- **定义**：gang 内实例之间的数据交换原语（幻灯片给出的库函数）：
- `reduce_add(x)`：把变量 x 在所有实例中的值**相加**（返回 uniform）；
- `reduce_min(a)`：取 gang 内最小值；
- `broadcast(value, index)`：把**某个实例**的值广播给 gang 内所有实例；
- `rotate(value, offset)`（示例代码中写作 `shift(value, offset)`）：对每个 i，把实例 i 的值传给实例 (i+offset) % programCount（循环移位）。
- **现实类比**：小组汇报——每人报自己的数（reduce_add 汇总）、把某人的答案告诉所有人（broadcast）、传纸条给邻座（shift/rotate）。
- **公式/图示**：`reduce_add: sum = Σ（各实例的 partial）→ uniform broadcast: 所有实例获得实例 index 的值 → uniform shift/rotate: 实例 i 收到实例 (i-offset) 的值 → 仍是 varying`

14. ISPC Tasks（任务并行，实现 multi-core）

- **定义**：gang 抽象由 SIMD 指令实现，运行在**一个核上的一个线程内**——前面所有 ISPC 代码都只用一个核。ISPC 提供第二种抽象 **task（任务）** 用于实现**多核执行**：`task` 函数 + `launch`（启动若干任务实例，每个任务跑在独立线程/核上）+ `sync`（等待所有任务完成）。任务内部仍可用 foreach 获得 SIMD。具体机制留待 Assignment 1 实践。
- **现实类比**：gang = 一个班同时做卷子（SIMD）；task = 把 4 个班分到 4 个教室同时考（多核），每个班内部仍可并行做题。
- **公式/图示**：`gang(SIMD): 一个线程内、一个核上，8 个 lane 并行 task(多核): 多个线程/核上并行，每个 task 内再 SIMD 组合: 4 核 × 每核 8-wide SIMD = 32 个数据同时处理`

15. 更高层抽象：Data-Parallel 思维（map）

- **定义**：如果语言**不暴露 programIndex/programCount**，程序员只写 foreach，那么 foreach 之外的一切都必须是 uniform 值和 uniform 逻辑；再进一步，连数组下标都不给，改成"对 collection 的每个元素调用一次函数"——`y = map(dowork, x)`——这就是 **NumPy / PyTorch** 用户熟悉的模型（`np.vectorize`、`Z = X + Y`）。更高的抽象 = 更少的实现自由度，但更接近"顺序思维"。
- **现实类比**：点外卖只说"要一份宫保鸡丁"（map 抽象），不必指定"哪个厨师、哪口锅、第几分钟炒"（实现细节）。
- **公式/图示**：`float dowork(float x) { ... } Collection y = map(dowork, x); ← 对每个元素独立调用 # Python/NumPy 对应: Z = X + Y; # 逐元素加法 Zplus1 = np.vectorize(addOne)(Z);`

二、代码示例与详细解说（本讲重点）

示例 1：ISPC 版 sinx——program instances、gang、uniform/varying、两种分配

代码（ispc + cpp）：

```
// sinx.ispc —— ISPC 版 sinx（显式使用 programCount / programIndex）
// 编译：ispc sinx.ispc -o sinx.o -h sinx_ispc.h --target=avx2-i32x8
//       g++ -O2 main.cpp sinx.o -o sinx
export void ispc_sinx(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    // 假设 N % programCount == 0
    for (uniform int i = 0; i < N; i += programCount)
    {
        int idx = i + programIndex;          // varying：每个实例不同
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6;               // 3!：uniform，所有实例共享
        uniform int sign = -1;               // uniform
        for (uniform int j = 1; j <= terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[idx] = value;
    }
}
```

```
// main.cpp —— 调用 ISPC 编译出的函数（链接 sinx.o 即可，用法与普通 C++ 函数无异）
#include "sinx_ispc.h"
#include <vector>
#include <iostream>

int main() {
    const int N = 1024;
    const int terms = 5;
    std::vector<float> x(N), result(N);
    for (int i = 0; i < N; ++i) x[i] = (i % 100) / 100.0f;
    ispc_sinx(N, terms, x.data(), result.data());   // 调用 = spawn 一个 gang
    std::cout << "result[0] = " << result[0] << "\n";
    return 0;
}
```

【代码做了什么？】 – ISPC 代码与 Lecture 2 的 C 版 `sinx` 算法完全一致，区别只在"谁算哪个元素"：外层循环步长是 `programCount`，每个实例用 `int idx = i + programIndex` 算出自己负责的元素下标——实例 0 处理元素 0, 8, 16, …；实例 1 处理 1, 9, 17, …（interleaved 分配）。 – `uniform int N / terms`：所有实例共享同一份（例如循环边界必须 uniform，因为 `for (uniform int i ...)` 的循环变量是 uniform，迭代次数全 gang 一致）。 – `float value/numer/idx`（无修饰符 = varying）：每个实例各一份，对应 SIMD 的 8 个 lane。 – `uniform int denom/sign`：虽然循环体内它们被修改，但修改方式对所有实例一致（denom 的递推、sign 的翻转都与 i 无关），所以声明为 uniform 是**合法的纯优化**（编译器可用标量指令/广播，而不必存 8 份）。 – C++ 侧：`ispc_sinx(...)` 调用看起来与普通函数无异——ISPC 编译器已生成含 SIMD 指令的 `sinx.o`，正常链接即可。

【并行机制解说】 – 对应概念：**SPMD / gang / program instances / uniform vs varying / interleaved assignment**。 – 调用 `ispc_sinx` 时，**spawn 一个 gang**（如 8 个实例，programCount = 8，与 AVX2 的 8-wide 对应），全部实例并发执行同一份代

码；返回时全部完成，控制流回到单线程 C++。 – 实现层面：ISPC 编译器把这段代码翻译成 SIMD 指令——`int idx = i + programIndex` 变成"向量加法"，`float value = x[idx]` 变成一条 **packed vector load**（因为 interleaved 分配下 8 个实例同一时刻访问的地址连续，`vmovaps` 一条指令搞定，对应 `_mm256_load_ps`）。**这就是"抽象（SPMD 语义）vs 实现（SIMD 指令）"的活例子。** – 若改成 blocked 分配（v2：`uniform int count = N/programCount; int start = programIndex*count;`），同一时刻各实例访问的地址不连续，`value = x[idx]` 需要昂贵的 **gather** 指令（`vgatherdps`）——同一抽象语义，不同实现，性能天差地别（这也是为什么"调度/实现"值得专门学习）。

示例 2：foreach 版 saxpy——用抽象写"像串行"的并行代码

代码（ispc + cpp）：

```
// saxpy.ispc —— 用 foreach 表达数据并行：y = a*x + y
// 编译：ispc saxpy.ispc -o saxpy.o -h saxpy_ispc.h --target=avx2-i32x8
//       g++ -O2 main.cpp saxpy.o -o saxpy
export void saxpy(uniform int n, uniform float a,
                  uniform float* x, uniform float* y)
{
    // foreach：声明这些迭代由整个 gang 完成，迭代如何分给实例由 ISPC 决定
    foreach (i = 0 ... n)
    {
        y[i] = a * x[i] + y[i];   // 对每个元素独立做一次乘加
    }
}
```

```
// main.cpp
#include "saxpy_ispc.h"
#include <vector>
#include <iostream>

int main() {
    const int n = 1 << 20;
    const float a = 2.0f;
    // 注意：ISPC 默认假设传入指针 16 字节对齐；std::vector 通常满足，
    // 若目标为 avx2 且编译器选择对齐访问，可用 aligned_alloc(32, ...) 保证 32 字节对齐
    std::vector<float> x(n, 1.0f), y(n, 2.0f);
    saxpy(n, a, x.data(), y.data());
    std::cout << "y[0] = " << y[0] << " (expect " << a * 1.0f + 2.0f << ")\n";
    return 0;
}
```

【代码做了什么？】 – `saxpy` 对 n 个元素逐个执行 `y[i] = a*x[i] + y[i]`（经典的 SAXPY 操作）。`foreach (i = 0 ... n)` 声明循环迭代相互独立、且**由整个 gang 完成**——程序员完全不写 `programIndex`、`programCount`，不关心实例如何分工，代码几乎就是串行 C 的样子。 – main 构造两个长度为 n 的数组（x 全 1、y 全 2），调用后验证 `y[0] == 2*1+2 == 4`。

【并行机制解说】 – 对应概念：**foreach 抽象 / abstraction vs implementation**。 – foreach 的**语义**是"这些迭代都要做、且互相独立"；**实现**（由 ISPC 决定）可以是幻灯片列举的任意一种：实例 0 串行全做（最差）、交错分配、分块分配、或 `atomic_add_local` 动态分配。程序的结果不受实现选择影响——这正是抽象的意义：**语义与调度解耦**。 – 与示例 1 对比：示例 1 用 `programIndex/programCount` 显式控制每个实例的工作（低层、可写出"只有特定 programCount 才正确"的程序）；示例 2 用 foreach 把分配权交给编译器（高层、更安全）。两者的等价关系：`foreach` 的典型展开就是示例 1 的交错写法（`for (uniform int i=0; i<N; i+=programCount) { int idx = i + programIndex; ... }`）。 – 注意 foreach 的**陷阱**：迭代之间不能有数据依赖。例如幻灯片 `shift_negative` 程序——`if (i>=1 && x[i]<0) y[i-1] = x[i]; else y[i] = x[i];`——迭代 i 可能写 `y[i-1]`、迭代 i-1 也可能写 `y[i-1]`，**多个迭代写同一地址，输出未定义**。foreach 只保证迭代独立时语义正确，程序员仍需保证"可安全并行"。

示例 3：ISPC 数组求和——uniform/varying 类型规则与跨实例归约

代码（ispc）：

```
// sum.ispc —— 数组求和的三种写法，演示 uniform/varying 类型规则

// 写法 1（错误）：把 varying 的 x[i] 累加进 uniform 变量 → 编译期类型错误
export uniform float sum_incorrect_1(uniform int N, uniform float* x) {
    uniform float sum = 0.0f;      // uniform：整个 gang 只有一份 sum
    foreach (i = 0 ... N) {
        sum += x[i];               // x[i] 对每个程序实例取值不同（varying）
    }                              // 8 个不同的 x[i] 无法"合并"进唯一一份 uniform 变量
    return sum;                    // → compile-time type error
}

// 写法 2（错误）：varying 的 sum 无法作为 uniform 返回值交给 C 代码
export uniform float sum_incorrect_2(uniform int N, uniform float* x) {
    float sum = 0.0f;              // varying：每个实例各有一份 sum
    foreach (i = 0 ... N) {
        sum += x[i];               // 每个实例累加自己的部分，语法上没问题
    }
    return sum;                    // 每个实例都有一份 sum —— C 代码只接收一个返回值
}                                  // → compile-time type error

// 写法 3（正确）：每实例私有 partial + 跨实例归约 reduce_add
export uniform float sum_array(uniform int N, uniform float* x) {
    uniform float sum;
    float partial = 0.0f;          // varying：每实例私有累加，实例间零通信
    foreach (i = 0 ... N) {
        partial += x[i];
    }
    sum = reduce_add(partial);     // 跨实例通信原语：把 gang 内所有 partial 相加
    return sum;                    // reduce_add 返回 uniform float
}
```

```
// 与 sum_array 语义等价的 C + AVX intrinsics 实现（幻灯片"自测"题）
// 理解"为什么这个实现正确实现了 ISPC gang 抽象的语义"，就说明你掌握了 ISPC
#include <immintrin.h>
float sum_summary_AVX(int N, float* x) {   // 调用方需保证 x 32 字节对齐
    alignas(32) float tmp[8];              // 幻灯片原注释"16 字节对齐"按 AVX 应为 32
    __m256 partial = _mm256_set1_ps(0.0f); // 幻灯片原文 __mm256 及 broadcast 用法为笔误
    for (int i = 0; i < N; i += 8)
        partial = _mm256_add_ps(partial, _mm256_load_ps(&x[i]));
    _mm256_store_ps(tmp, partial);         // 8 个 lane 的局部和落盘
    float sum = 0.f;
    for (int i = 0; i < 8; i++)
        sum += tmp[i];                     // 模拟 reduce_add 的水平归约
    return sum;
}
```

【代码做了什么？】 – 写法 1 和 2 是两个典型的**编译期类型错误**：varying 值（每个实例不同）不能写进 uniform 变量；varying 变量也不能作为 uniform 返回值交给 C 代码——**类型系统在编译期就拦住"跨实例数据合并"这种需要显式原语的操作**。 – 写法 3 是正确模式：每个实例先用**私有的 varying** `partial` 累加自己负责的那部分元素（实例间零通信、零同步，性能最好），最后用标准库 `reduce_add(partial)` **跨实例归约**得到总和。 – 底下的 AVX 等价实现展示了 gang 抽象在 AVX2 上如何落地：`__m256 partial` 就是 8 个实例各自的 partial，`_mm256_add_ps` 是 foreach 循环的 SIMD 展开，最后的 8 次标量相加就是 reduce_add。

【并行机制解说】 – 对应概念：**uniform vs varying 类型规则 / cross-instance operations（reduce_add）**。 – 关键洞察：**没有跨实例通信原语，就无法在 ISPC 内做"合并不同实例数据"的归约**——这正是"低层语言语义严格"的体现（幻灯片总结：ISPC 是低层语言，暴露 programIndex/programCount 让程序员定义每个实例做什么，代价是可以写出输出未定义、或只对特定 programCount 正确的程序）。 – reduce_add 在 SIMD 实现上对应**水平归约**（把向量 8 个 lane 的值相加成标量）——这通常比逐元素运算贵，因此好的并行程序应尽量减少归约次数（每实例先本地累加，最后只归约一次，而不是每个元素归约一次）。 – 进阶（幻灯片"高级协作"示例 `vec8product`）：还可以用 `shift` / `rotate` 原语在 3 步内（lg 8 = 3）算出 8 个元素的乘积——每步 `shift` 把值沿实例编号移动 1/2/4 位，配合 `programIndex % 2/4/8 == 0` 的选择性相乘，构建一棵归约树。这类"实例间协作"程序正确性只在特定 gang 大小下成立（注释里明确"assumes the gang size is 8"）。

示例 4：ISPC task 并行——launch/sync 实现多核执行

代码（ispc + cpp）：

```
// saxpy_tasks.ispc —— ISPC 任务并行：把工作拆到多个核上
// 编译：ispc saxpy_tasks.ispc -o saxpy_tasks.o -h saxpy_tasks_ispc.h --target=avx2-i32x8
//       g++ -O2 main.cpp saxpy_tasks.o -o saxpy_tasks -pthread    （ISPC 任务需要 pthread）

// task 函数：处理数组的一段连续区间
task void saxpy_chunk(uniform int n, uniform float a,
                      uniform float* x, uniform float* y)
{
    // taskIndex / taskCount 是 ISPC 内置变量：本任务编号 / 本次 launch 的任务总数
    uniform int count = n / taskCount;
    uniform int start = taskIndex * count;        // 分块分配：每个任务一段
    foreach (i = start ... start + count) {       // 任务内部仍可用 foreach 获得 SIMD
        y[i] = a * x[i] + y[i];
    }
}

export void saxpy_tasks(uniform int n, uniform float a,
                        uniform float* x, uniform float* y)
{
    uniform int ntasks = 4;                       // 拆 4 个任务 → 4 个核并行
    launch[ntasks] saxpy_chunk(n, a, x, y);       // 启动 ntasks 个任务实例（各得 taskIndex）
    sync;                                         // 等待所有任务完成
}
```

```
// main.cpp —— 与普通 ISPC 函数调用方式相同
#include "saxpy_tasks_ispc.h"
#include <vector>
#include <iostream>

int main() {
    const int n = 1 << 20;
    const float a = 2.0f;
    std::vector<float> x(n, 1.0f), y(n, 2.0f);
    saxpy_tasks(n, a, x.data(), y.data());
    std::cout << "y[0] = " << y[0] << " (expect 4)\n";
    return 0;
}
```

【代码做了什么？】 – `saxpy_chunk` 是 `task` 函数：用内置变量 `taskIndex`（本任务编号）和 `taskCount`（任务总数）算出自己负责的连续区间，区间内再用 `foreach` 逐元素做 saxpy——**每个任务内部仍然是 SIMD（gang）执行。** – `saxpy_tasks` 是导出入口：`launch[4] saxpy_chunk(...)` 启动 4 个任务实例（系统把它们调度到 4 个核/线程上），`sync` 等待全部完成。**launch/sync 就是本讲的"同步点"。**

【并行机制解说】 – 对应概念：**ISPC tasks / multi-core 执行 / gang 与 task 的区别。** – 层次关系（幻灯片明确强调）：**gang 抽象由 SIMD 指令实现，运行在一个核上的一个线程里**——示例 1–3 的所有 ISPC 代码只用了四个核中的一个；**task 抽象才实现多核。**组合起来：4 个任务（4 核）× 每任务 8-wide SIMD = 同时处理 32 个元素——这正是 Assignment 1 中"四核 + AVX"获得 ~32–40x 加速的结构来源（再加上 hyper-threading 与编译器优化）。 – 与硬件对照：任务对应硬件多线程/多核（TLP），gang/foreach 对应 SIMD（DLP）；`launch` / `sync` 是显式的并行创建与同步点，与 Lecture 2 的 `std::thread` + `join` 在语义上同构。 – 幻灯片还预告了"更高层抽象"的走向：隐藏 programIndex/programCount、甚至隐藏数组下标，变成 `map(dowork, x)` 式的 collection 抽象（NumPy/PyTorch 模型）——抽象层次越高，程序员越不需要关心实现，但可表达的低层技巧（如 vec8product 的树形归约）也越难实现。

三、关键要点

1. **Latency 与 Bandwidth 是两个正交概念**：延迟是"一次操作花多久"，带宽是"单位时间能做多少次"（高速公路：提速 vs 加车道；洗衣：复制资源 vs 流水线）。加带宽不降延迟；瓶颈由最细的管子决定（50 L/s）。内存带宽的定义是"内存系统向处理器提供数据的速率"（如 20 GB/s）。
2. **现代并行机器常常是 bandwidth-limited**：逐元素向量乘法每个元素要 3 次内存操作（12 字节）才换来 1 次 MUL；V100 需要约 98 TB/s 才能喂饱 5120 个 ALU，实际只有 900 GB/s → 效率 <1%（八核 Xeon 配 76 GB/s 总线约 3%）。**克服带宽限制是面向吞吐优化系统的软件开发者最重要的挑战。**
3. **带宽受限下，多线程/低延迟都救不了你**：稳态下核心利用率只取决于"指令吞吐 vs 内存吞吐"，与延迟和未完成请求数无关。性能程序必须**减少取数频率**：复用已加载数据（temporal locality）、跨线程共享数据、用更多算术换更少搬运（"math is free"）。
4. **抽象（语义）≠ 实现（调度）**：同一个 ISPC 程序（semantics 固定）可以有多种实现——交错/分块/动态分配迭代、packed load 或 gather、SIMD 或任务。把两者混为一谈是本课程最常见的困惑来源；好学生应能"在脑中 trace 出每个线程/ALU/lane 在每一步做什么"。
5. **ISPC 的三层能力**：`foreach` 让程序员像写串行程序一样写数据并行代码（迭代独立性由程序员保证）；`programIndex/programCount/uniform/varying` 让程序员低层控制每个实例的工作与数据；`reduce_add/broadcast/shift` 等跨实例原语 + `task/launch/sync` 实现归约协作与多核扩展。类型系统用编译期错误拦住"varying 塞进 uniform"这类错误。

四、常见陷阱与注意事项

1. **混淆 latency 与 bandwidth**：以为"内存延迟低 = 性能好"。带宽受限的程序（如逐元素乘法）延迟再低也白搭；反之，延迟高但带宽充足时，多线程隐藏延迟即可。先判断程序是 latency-bound 还是 bandwidth-bound。
2. **忽视 12 字节/1 MUL 这类"算术强度"问题**：每个数据只用一次的流式访问是带宽杀手。应尽量提高**数据复用**（缓存分块、线程间共享、寄存器内累加），而不是堆更多 ALU。
3. **把 uniform 当正确性工具 / 乱用 varying**：uniform 只是优化（"Its use is purely an optimization. Not needed for correctness"）；而把 varying 值赋给 uniform 变量（`uniform sum += x[i]`）或把 varying 当 uniform 返回值，都是编译期错误——需要显式跨实例原语（reduce_add 等）。
4. **写 foreach 时留下迭代间数据依赖**：`shift_negative` 一类程序（迭代 i 写 `y[i-1]`）多个迭代写同一地址，输出未定义——foreach 不检查依赖，正确性全靠程序员保证"迭代可安全并行"。
5. **以为 ISPC 自动用满多核**：gang 由 SIMD 实现，只跑在**一个核**上；要上多核必须显式使用 task + launch/sync（Assignment 1 的关键一步）。同理，别以为 compiler 自动向量化一定生效——显式用 ISPC/foreach 才可控。
6. **忽视对齐与编译细节**：ISPC 默认假设传入指针 16 字节对齐；AVX 的 256 位访问最好 32 字节对齐（`aligned_alloc(32, ...)`）。ISPC 代码用 `ispc` 编译器单独编译（`--target=avx2-i32x8` 等），任务并行还需 `-pthread` 链接。

五、思考题（带答案）

Q1. 幻灯片思想实验：为什么"逐元素向量乘法"在 V100 上效率 <1%，却仍然比八核 CPU 快？这说明什么？ – **答案**：该计算每个元素需 3 次内存操作（12 字节）换 1 次 MUL，是极端带宽受限（bandwidth-limited）。V100 有 5120 个 fp32 ALU，要喂饱它们需要约 98 TB/s（= 5120 × 1.6 GHz × 12 B），而 HBM2 只有 900 GB/s → 效率 <1%。八核 CPU 效率约 3%，但其 ALU 总量远小于 GPU（8 核 × 标量/向量单元），所以绝对吞吐仍远低于 GPU。结论：**效率（利用了多少硬件能力）与绝对性能（总吞吐）是两回事**；对带宽受限程序，提升方向是减少每单位计算的数据搬运（复用、共享、算术换搬运），而不是加 ALU。

Q2. 为什么说"稳态下核心利用率只取决于指令吞吐与内存吞吐的比值，与延迟无关"？ – **答案**：在幻灯片"load 64 字节 + 两次 add"的例子中，稳态时内存每时钟只能提供 8 字节，核心每 8 字节完成 1 次 load + 2 次 add；无论延迟是 8 周期还是 800 周期，只要未完成请求数足够覆盖延迟（用多线程或并发 load），稳态吞吐都由"内存供给速率"决定。延迟只影响"需要多少未完成请求/多少线程来填满管道"，不影响最终吞吐上限——这是 latency hiding 与 bandwidth 限制的根本区别。

Q3. `foreach` 的语义与实现有什么关系？请用"交错分配 + packed load"说明一个语义可以对应多种实现，并解释为什么 blocked 分配更慢。 – **答案**：foreach 的语义只是"迭代集合互相独立、必须全部完成"，不规定哪个实例做哪个迭代（abstraction vs implementation）。ISPC 可以选择交错分配（实例 k 处理 k, k+8, …）：同一时刻 8 个实例访问连续地址，`float value = x[idx]` 可编译成一条 packed vector load（`vmovaps` / `_mm256_load_ps`）；也可以选择 blocked 分配（实例 k 处理连续块）：同一时刻各实例访问的地址相隔 N/8，只能编译成昂贵的 gather 指令（`vgatherdps`）。同一语义、两种实现、性能不同——这就是为什么理解"实现/调度"是优化并行程序的前提。

> **注意**：本讲 ISPC 内容直接对应 **Assignment 1（Analyzing Parallel Program Performance on a Quad-Core CPU）**——作业要求先用 ISPC 的 `foreach` / `task` 在四核 CPU 上并行化程序（对比 `-O3` 单线程基线），再定量分析多核、SIMD、超线程各自的贡献（预期 ~32–40x 加速），并手写 AVX intrinsics 复现 ISPC 生成的向量代码（对应本讲"ISPC 用 SIMD 实现 gang"与 mask 处理分支的内容）。

Lecture 4: Parallelizing Code: An Example Thought Process（并行化代码：思考过程实例）（日期：Oct 02）

> **概述**：本讲首先补完第 3 讲的 ISPC 语义（SPMD 抽象 vs SIMD 实现、`foreach`、uniform/varying、跨 program instance 操作），随后以"编写并优化一个并行程序"为案例，系统介绍并行程序设计的四步思考过程：**decomposition（分解）→ assignment（分配）→ orchestration（编排）→ mapping（映射到硬件）**。全讲围绕两个编程模型展开——**data parallel（数据并行）**与 **shared address space（共享地址空间）**，并以一个 2D grid solver（Gauss-Seidel 迭代求解器）为贯穿案例，展示同一算法在两种模型下的不同表达方式、同步方式与性能取舍。
>
> **注意**：本讲内容与 Assignment 1（"Analyzing Parallel Program Performance on a Quad-Core CPU"，截止 Oct 6）直接对应——ISPC 的 `foreach`、ISPC tasks、静态/动态分配的思想正是该作业的核心内容。

一、核心概念与定义

1. SPMD（Single Program, Multiple Data，单程序多数据）

- **定义**：一种编程模型：程序员只写**一个函数**，但在并行执行时，该函数会以多个"program instance（程序实例）"的形式同时运行，每个实例处理不同的输入数据。调用 SPMD 函数时产生一个"gang（组）"；函数返回时所有实例都必须执行完毕，控制流回到单一顺序执行。
- **现实类比**：就像一家连锁店的总部发布同一份"开店手册"，各家分店（实例）同时照着手册执行，但各自面对的是自己街区（数据）的情况。
- **公式/图示**：

```
单线程控制流                    SPMD 执行（多个实例并行）              单线程控制流
──────────►  ispc_sinx()  ──────────►  0  1  2  3  4  5  6  7  ──────────►  （返回后继续）
          （顺序执行 C 代码）            （programCount = 8 个实例）          （顺序执行 C 代码）
```

2. programCount 与 programIndex

- **定义**：`programCount` 是当前 gang 中同时执行的 program instance 总数（uniform 值）；`programIndex` 是当前实例在 gang 中的编号（varying 值，每个实例不同）。它们让 ISPC 成为一门"低级"语言——程序员可以精确指定每个实例做什么工作、访问哪些数据。

- **现实类比**：programIndex 就像电影院里每个人的座位号，programCount 就是影院总座位数——"坐在 3 号座的人负责处理数据 3、11、19……"。
- **公式/图示**：

```
interleaved（交错）分配:  idx = i + programIndex       （i 每次步进 programCount）
  实例0 → 元素 0, 8, 16, 24, ...
  实例1 → 元素 1, 9, 17, 25, ...
blocked（分块）分配:     idx = start + i,  start = programIndex * (N/programCount)
  实例0 → 元素 0..31,  实例1 → 元素 32..63, ...
```

3. uniform 与 varying（ISPC 类型修饰符）

- **定义**：`uniform` 变量在所有 program instance 中取值相同（编译器可将其放在寄存器/内存的单份拷贝中）；`varying`（默认）变量每个实例各有一份。使用 `uniform` 纯粹是**优化**手段，不影响正确性——但类型错误会在编译期暴露。
- **现实类比**：uniform 像公司统一印发的通知（每人内容一样），varying 像每人手写的笔记（每人内容不同）。
- **公式/图示**：无（类型系统概念）。注意：`programCount` 是 uniform，`programIndex` 是 varying。

4. foreach（ISPC 的关键语言构造）

- **定义**：`foreach (i = 0 ... N)` 声明循环迭代是**可并行的**。程序员说"这些迭代是**整个 gang**（而非每个实例）要完成的工作"，由 ISPC 实现负责把迭代分配给 gang 内的各 program instance。它把程序员的思考层次从"并行执行"提升到"迭代"——多数情况下可以像写串行程序一样思考。
- **现实类比**：foreach 就像把一摞试卷交给一组助教："这 100 份你们每人分一些批改，怎么分你们自己商量。"而手写 programIndex 则像"我指定 1 号助教批第 1~25 份……"。
- **公式/图示**：foreach 的四种可能实现（由编译器/运行时选择）：

```
foreach (i = 0 ... N) { ... }
  实现1（实例0 干所有活）:  if (programCount == 0) for (i=0; i<N; i++) ...
  实现2（交错）:            for (loop_i=0; loop_i<N; loop_i+=programCount) i = loop_i + programIndex;
  实现3（分块）:            count = N/programCount; start = programIndex*count; ...
  实现4（动态）:            i = atomic_add_local(&nextIter, 1); while (i < N) {...}
```

5. ISPC task（ISPC 任务）

- **定义**：gang 抽象由**单核上**的 SIMD 指令实现，因此之前所有 ISPC 代码只跑在一个核上。ISPC 还提供"task"抽象实现**多核**执行：`launch[numTasks] myTask(...)` 创建一批任务，由 ISPC 运行时（对程序员不可见）把任务动态分配给线程池中的 worker 线程。
- **现实类比**：gang 是"一个班里做小组作业"（一个核上的向量化），task 是"整个年级分工"（多核协作）。
- **公式/图示**：

```
Worker thread 0 ──► task0  task3  ...
Worker thread 1 ──► task1  task4  ...    （完成后从任务列表取下一个未完成任务）
Worker thread 2 ──► task2  task5  ...
Worker thread 3 ──► ...    ...    ...    ← 共享任务列表 + "next task" 指针
```

6. Data Parallelism（数据并行）与 Task Parallelism（任务并行）

- **定义**：数据并行指**对大量数据元素执行同一序列操作**（"对每个元素独立地做这件事"），典型的表达是 `foreach`、`#pragma omp parallel for`、`map()`；任务并行指把问题分解为**多个相互独立的子任务**，每个任务可能是不同的工作（甚至不同函数），通过任务队列/任务图调度。本讲两个编程模型：data parallel 模型（单逻辑控制流 + 系统处理并行化）与 shared address space 模型（多 SPMD 线程 + 程序员负责同步）。
- **现实类比**：数据并行像流水线上每个工位用同一道工序处理不断流过的零件；任务并行像装修队里瓦工、电工、木工各干各的活。

- **公式/图示**：见第 5 讲（`B[i] = foo(A[i])` 是数据并行；`enqueue_task(foo)` 是任务并行）。

7. Shared Address Space（共享地址空间模型）

- **定义**：线程通过读写**共享内存中的变量**进行通信——通信隐式地发生在 load/store 中；程序员还需要操作同步原语（lock、barrier、atomic 操作）来协调访问。它是顺序编程的自然延伸（本课程此前所有讨论都假设了共享地址空间）。
- **现实类比**：讲义中的经典比喻——**公告板（bulletin board）**：任何人都能读、能写；但要防止两个人同时修改同一张通知，就得靠"谁先到谁先写"的规矩（锁）。
- **公式/图示**：



8. Data Race（数据竞争）

- **定义**：多个线程同时访问同一内存位置，且至少有一个是写操作，且没有同步机制保证顺序——结果不确定。例如两个线程同时执行 `x++`（它由 load、add、store 三条指令组成），可能都读到旧值 0，最终 x 只变成 1 而非 2。
- **现实类比**：两个人同时往同一张表格的同一格填数字，最后写上去的是谁的内容完全取决于先后——没人能保证。
- **公式/图示**：

```
T1: r1 ← x (0)   T2: r1 ← x (0)
T1: r1 ← r1+1    T2: r1 ← r1+1
T1: x ← r1 (1)   T2: x ← r1 (1)   → 最终 x = 1（应为 2）
```

9. Decomposition（分解）、Assignment（分配）、Orchestration（编排）、Mapping（映射）

- **定义**：创建并行程序的四个步骤：
- **Decomposition**：把问题分解成可并行执行的子问题（tasks），关键挑战是**识别依赖（dependencies）**；
- **Assignment**：把任务分配给 worker（线程、program instance、vector lane……），目标是好的**负载均衡**与**低通信成本**，可静态或动态执行；
- **Orchestration**：组织通信结构、加入同步以保持依赖、组织内存中的数据布局、调度任务；
- **Mapping**：把线程映射到硬件执行单元（由 OS / 编译器 / 硬件完成）。 这些职责可能由程序员承担，也可能由系统（编译器、运行时、硬件）承担。
- **现实类比**：做一顿宴席——分解=把"做菜"拆成"洗菜、切菜、炒菜"；分配=给每位厨师分工；编排=决定传菜顺序与"等菜齐了再上桌"的同步；映射=决定哪个灶台给哪位厨师用。
- **公式/图示**：

```
Problem → 子问题(tasks) → 并行线程(workers) → 并行程序(通信线程) → 并行机器执行
            分解              分配                  编排                映射
```

10. Amdahl's Law（阿姆达尔定律）

- **定义**：设 S 为程序中**本质上串行**（依赖阻止并行）的执行时间占比，则并行带来的最大加速比 ⩽ 1/S。一条很小的串行代码段就能限制大型并行机上的加速比。
- **现实类比**：九个月生一个孩子——即使雇 100 个保姆，怀孕这件事（串行部分）本身就要 9 个月，加速比被它锁死。
- **公式/图示**：

```
Speedup(P) ≤ 1 / (S + (1-S)/P)  →  Speedup(∞) ≤ 1/S

例：Summit 超算 = 27,648 GPUs × 5,376 ALUs/GPU ≈ 148,635,648 个 ALU
   若应用中 0.1% 是串行的：最大加速比 ≤ 1/0.001 = 1000 倍
   （148M 个并行单元只换来 1000 倍——串行比例才是瓶颈）
```

11. Lock（锁）与 Barrier（屏障）

- **定义**：lock 提供**互斥（mutual exclusion）**：临界区（critical section）内同时只允许一个线程进入；barrier(n) 让 n 个线程都到达后才能继续，是表达**依赖**的保守方式——它把计算分成阶段（phase），保证所有线程在屏障前的计算都完成后，任何线程才能开始屏障后的计算。
- **现实类比**：锁=办公室只有一把钥匙，谁拿钥匙谁进门；barrier=接力赛，必须所有队员都到达接力区，下一棒才能出发。
- **公式/图示**：

```
          barrier
P1 ──计算──┤
P2 ──计算──┤   （所有线程到齐后一起放行）
P3 ──计算──┤
P4 ──计算──┤
```

12. Red-Black Coloring（红黑着色重排）

- **定义**：改变 Gauss-Seidel 迭代中网格单元的更新顺序，使其更适合并行：把网格按棋盘格染色，先**并行更新所有红格**，全部完成后**再并行更新所有黑格**（黑格依赖红格的新值），如此反复直至收敛。收敛到同一解（误差阈值内），但浮点中间值与串行版本不同。
- **现实类比**：棋盘上的马走日字——同色格互不"相邻"，所以同色格之间没有依赖，可以同时更新。
- **公式/图示**：

```
N×N 网格（(N+2)×(N+2) 含边界）          红黑着色：
  A[i,j] = 0.2*(A[i,j] + A[i,j-1]      R B R B R
                 + A[i-1,j]            B R B R B
                 + A[i,j+1]            R B R B R
                 + A[i+1,j])           B R B R B
    第1阶段：并行更新所有 R
    第2阶段：并行更新所有 B（依赖相邻 R 的新值）
```

二、代码示例与详细解说（本讲重点）

示例 1：ISPC 的 `foreach` 与手写 `programIndex` 交错分配（sinx 泰勒展开）

代码（ISPC + C++）：

```
// sinx.ispc —— ISPC 代码
// 版本 A：手写交错分配（低级写法，程序员亲自分配迭代）
export void ispc_sinx_interleaved(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    // 假设 N % programCount == 0
    for (uniform int i = 0; i < N; i += programCount)
    {
        int idx = i + programIndex;          // 每个实例负责的元素：交错分布
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6;               // 3!
        uniform int sign = -1;
        for (uniform int j = 1; j <= terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[idx] = value;
    }
}

// 版本 B：foreach 版本（高级写法，把分配交给系统）
export void ispc_sinx_foreach(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    foreach (i = 0 ... N)                    // 声明迭代可并行
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        uniform int denom = 6;               // 3!
        uniform int sign = -1;
        for (uniform int j = 1; j <= terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

```cpp
// main.cpp —— 调用 ISPC 函数的 C++ 代码
#include "sinx_ispc.h"
#include <cstdlib>

int main(int argc, char** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];
    // 初始化 x 数组（略）
    // 执行 ISPC 代码：调用会 spawn 一个 gang
    ispc_sinx_foreach(N, terms, x, result);
    delete[] x;
    delete[] result;
    return 0;
}
```

【代码做了什么？】

1. ISPC 代码计算 `sin(x)` 的 Taylor 展开近似：`x - x³/3! + x⁵/5! - ...`（`terms` 项）。`numer` 依次是 `x³, x⁵, ...`，`denom` 依次是 `6, 120, ...`，`sign` 交替 ±1。
2. 版本 A 用 `programIndex` / `programCount` 手写"交错"迭代分配：外层 uniform 循环每次步进 `programCount`，实例 `programIndex` 处理 `i + programIndex` 号元素——即实例 0 处理元素 0,8,16,...；实例 1 处理 1,9,17,...。
3. 版本 B 用 `foreach`：程序员只声明"这些迭代可并行"，不再关心谁做哪份。
4. C++ 的 `main` 像调用普通函数一样调用 ISPC 函数；调用期间程序进入 SPMD 执行，返回时所有实例已完成。

【并行机制解说】

- **线程/实例如何创建**：调用 `ispc_sinx_foreach` 时，ISPC 运行时"spawn"一个 gang——`programCount` 个 program instance 并发执行同一份 ISPC 代码（此处 gang 由硬件 SIMD 宽度决定，如 AVX2 的 8 宽）。
- **工作如何分配**：版本 A 是**程序员管理的静态分配**（交错）；版本 B 把分配交给系统——foreach 抽象允许动态分配，但当前 ISPC 实现用的是静态方案。ISPC 编译器把 gang 实现为 SIMD 指令：交错分配下，一次 `float value = x[idx]` 对所有实例恰好是**连续内存**，编译器生成一条 packed vector load（如 `vmovaps` / `_mm256_load_ps`）；若改成 blocked 分配（见版本 2 幻灯片），则一次访问的是 8 个**不连续**的值，需要更昂贵的 `vgatherdps`（`_mm256_i32gather_ps`）gather 指令。
- **数据如何共享**：x、result 是 uniform 指针（所有实例共享同一数组），局部变量（value、numer）每个实例各一份。
- **同步点在哪**：函数返回即隐式同步（所有实例完成）。
- **对应概念**：SPMD 抽象 vs SIMD 实现（abstraction vs implementation）、foreach、data parallelism、interleaved vs blocked assignment。

示例 2：C++11 `std::thread` 的静态分配（parallel_sinx）

代码（C++）：

```
#include <thread>

// 串行核心：计算 [0, N) 区间内每个元素的 sinx
void sinx_range(int N, int terms, float* x, float* result) {
    for (int i = 0; i < N; i++) {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6, sign = -1;
        for (int j = 1; j <= terms; j++) {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}

// 并行版本：把数组分成两半，分别交给两个执行单元
void parallel_sinx(int N, int terms, float* x, float* result) {
    int half = N / 2;

    // 启动一个线程，负责数组的前一半
    std::thread t1(sinx_range, half, terms, x, result);

    // 主线程负责后一半（指针偏移 half）
    sinx_range(N - half, terms, x + half, result + half);

    t1.join();   // 等待 t1 完成（同步点）
}
```

【代码做了什么？】

1. `sinx_range` 是串行的 sinx 计算函数，参数化区间起点（通过指针偏移）。
2. `parallel_sinx` 把 `N` 个元素**按 block（分块）静态分配**：新线程 `t1` 处理 `[0, half)`，主线程处理 `[half, N)`——各自独立读写数组的不同区域，互不干扰。
3. `t1.join()` 阻塞主线程直到 t1 完成，保证返回时所有工作结束。

【并行机制解说】

- **线程如何创建**：`std::thread t1(...)` 创建一个新线程，与主线程并发。
- **工作如何分配**：**程序员手动**按循环迭代分块分配（blocked fashion）——这是典型的**静态分配（static assignment）**：分配不依赖运行时行为，只有一点点索引计算的开销。
- **数据如何共享/同步**：两个线程通过**共享地址空间**（同一个 x、result 数组）间接共享；因为各自区间不相交，无需锁；join 是唯一的同步点。
- **对应概念**：shared address space 模型、static assignment、blocked assignment、decomposition by loop iteration。

示例 3：数据竞争（Data Race）与用锁修复

代码（C++，共享地址空间）：

```
#include <thread>
#include <mutex>
#include <cstdio>

int x = 0;                      // 共享变量
std::mutex mylock;

void worker_racy() {            // 有数据竞争的版本
    for (int i = 0; i < 100000; i++)
        x++;                    // 实际是 load → add → store 三条指令!
}

void worker_safe() {            // 加锁修复
    for (int i = 0; i < 100000; i++) {
        mylock.lock();          // 进入临界区（互斥）
        x++;                    // 三条指令被锁保护，整体"原子"
        mylock.unlock();
    }
}

int main() {
    std::thread t1(worker_racy), t2(worker_racy);
    t1.join(); t2.join();
    std::printf("有竞争时 x = %d（期望 200000）\n", x);

    x = 0;
    std::thread t3(worker_safe), t4(worker_safe);
    t3.join(); t4.join();
    std::printf("加锁后  x = %d（期望 200000）\n", x);
    return 0;
}
```

【代码做了什么？】

1. 两个线程各自执行 10 万次 `x++`。理想结果 `x = 200000`。
2. `worker_racy` 无同步：两个线程可能同时读出旧值、各自加 1、再写回，丢失更新——`x` 通常小于 200000，且每次运行结果不同（未定义行为）。
3. `worker_safe` 用 mutex 把 `x++` 包成临界区，保证 load–add–store 三步原子执行，结果稳定为 200000。

【并行机制解说】

- **为什么需要互斥**：`x++` 在机器层面是 load、add、store 三条指令。两个线程的交错（interleaving）可能让两次加 1 都基于旧值 0，最终 x=1。这组指令必须"原子"（不可分割）。
- **保证原子性的手段**（讲义列举）：lock/unlock 包临界区；硬件原子 read–modify–write 指令（如 `atomicAdd(x, 10)`）；语言级 `atomic { ... }` 块。
- **对应概念**：shared address space、data race、mutual exclusion、critical section、lock。
- **另附 ISPC 中的数据竞争形态**（讲义 `shift_negative` 例子）：foreach 迭代之间可能写同一内存位置——迭代 i 写 `y[i-1]` 而迭代 i–1 写 `y[i]`，输出未定义。数据竞争不只存在于线程间，也存在于"逻辑并行迭代"之间：

```
export void shift_negative(uniform int N, uniform float* x, uniform float* y) {
    foreach (i = 0 ... N) {
        if (i >= 1 && x[i] < 0)
            y[i-1] = x[i];   // 与相邻迭代写 y[i] 冲突 → 输出未定义
        else
            y[i] = x[i];
    }
}
```

示例 4：跨 program instance 规约——`reduce_add`（数组求和）

代码（ISPC）：

```
// 错误版本 1：sum 是 uniform，x[i] 是 varying —— 编译期类型错误
export uniform float sum_incorrect_1(uniform int N, uniform float* x) {
    float sum = 0.0f;        // 错误：sum 是 varying（每个实例一份），
    foreach (i = 0 ... N)    //       但函数声明返回 uniform float，
        sum += x[i];         //       无法把"很多份"sum 汇成一个返回值
    return sum;              //       → compile-time type error
}

// 错误版本 2：sum 是 uniform，但 foreach 体内每个实例都想加自己的 x[i]
export uniform float sum_incorrect_2(uniform int N, uniform float* x) {
    uniform float sum = 0.0f;
    foreach (i = 0 ... N)
        sum += x[i];         // 错误：x[i] 每个实例不同，加到哪份 uniform sum 上？
    return sum;              //       → compile-time type error
}

// 正确版本：每个实例私有累加 + 跨实例规约
export uniform float sum_array(uniform int N, uniform float* x) {
    uniform float sum;
    float partial = 0.0f;
    foreach (i = 0 ... N) {
        partial += x[i];     // 每个实例在自己的 partial 上累加（零通信）
    }
    sum = reduce_add(partial);   // 跨实例规约：把各实例的 partial 加起来
    return sum;                  // reduce_add 返回 uniform float
}
```

【代码做了什么？】

1. 两个"错误"版本分别演示了 uniform/varying 混用的两种编译期错误：把 varying 值赋给 uniform 返回值、以及在 uniform 变量上做 varying 累加。
2. 正确版本：每个 program instance 先把自己的 `partial` 累加好（无通信），再用 ISPC 标准库的跨实例原语 `reduce_add(partial)` 把所有实例的部分和相加，得到统一的总和。

【并行机制解说】

- **数据如何共享/通信**：foreach 迭代并行执行时实例之间**零通信**（各算各的 partial）；唯一的通信发生在 `reduce_add`——这是 ISPC 的 **cross program instance operation**（跨实例操作），把 gang 内所有实例的某个 varying 值规约成一个 uniform 值。讲义指出，这段 ISPC 代码的执行方式几乎等价于手写 AVX intrinsics 的 C 代码（`_mm256_add_ps` 累加 + 最后把 8 个 lane 相加）。
- **其他跨实例原语**：`reduce_min`（求最小值）、`broadcast(value, index)`（从某个实例广播）、`rotate(value, offset)`（实例间循环移位，用于如 `vec8product` 那种 3 步求 8 元素乘积的树形归约）。
- **对应概念**：uniform/varying、foreach、SPMD 抽象、data parallelism、跨实例通信原语（data-parallel 模型中"由系统提供的内建通信原语"）。

示例 5：Grid Solver 在两种编程模型下的表达（本讲案例研究）

背景：在 (N+2)×(N+2) 网格上迭代求解 PDE（Gauss-Seidel 扫描直至收敛），每点更新公式：

```
A[i,j] = 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] + A[i,j+1] + A[i+1,j])
```

第 1 步——识别依赖（decomposition）：每个元素依赖左邻元素与上一行元素；行内存在从左到右的链式依赖，行间存在自上而下的依赖。好消息是**对角线方向存在独立工作**（一条对角线上的点互不依赖），但"按对角线并行"在计算开始与结束阶段并行度很低、且每完成一条对角线就要同步一次，不易利用。**第 2 步——改算法**：利用 Gauss-Seidel 的领域知识，把更新顺序重排为**红黑着色**：先并

行更新所有红格，再并行更新所有黑格。**第 3 步——分配**：blocked 分配比 interleaved 分配通信量更小（相邻行数据在同一个处理器上，只需在边界交换 ghost 数据）。

代码 A（数据并行表达，伪代码）：

```
const int n; float* A = allocate(n+2, n+2);
void solve(float* A) {
    bool done = false;
    while (!done) {
        float diff = 0.f;
        for_all (red cells (i,j)) {          // 系统并行化所有红格迭代
            float prev = A[i,j];
            A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] +
                             A[i+1,j] + A[i,j+1]);
            reduceAdd(diff, fabs(A[i,j] - prev));   // 内建通信原语
        }
        if (diff/(n*n) < TOLERANCE) done = true;
    }
}
// 分解：每个网格单元的处理是独立工作
// 分配：由系统负责（???）
// 编排：系统处理 —— for_all 块结束处是隐式 barrier（回到顺序控制流之前）
//       通信由内建原语 reduceAdd 处理
```

代码 B（共享地址空间 + SPMD 线程表达，伪代码）：

```
int n; bool done = false; float diff = 0.0;
LOCK myLock; BARRIER myBarrier;
float* A = allocate(n+2, n+2);

void solve(float* A) {
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);   // 每个线程负责若干行

    while (!done) {
        float myDiff = 0.f;
        diff = 0.f;
        barrier(myBarrier, NUM_PROCESSORS);     // ① 确保所有线程都看到 diff=0
        for (j = myMin to myMax)
            for (i = red cells in row j) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (...);          // 更新公式
                myDiff += fabs(A[i,j] - prev);  // 先本地累加!
            }
        lock(myLock);                           // ② 每个线程只加锁一次
        diff += myDiff;
        unlock(myLock);
        barrier(myBarrier, NUM_PROCESSORS);     // ③ 确保所有线程完成累加
        if (diff/(n*n) < TOLERANCE) done = true;
        barrier(myBarrier, NUM_PROCESSORS);     // ④ 确保所有线程看到 done
    }
}
```

【代码做了什么？】

1. 两种表达都实现同一个算法：迭代红黑更新，直至 `diff/(n*n) < TOLERANCE`。
2. 数据并行版：程序员只描述"对每个红格做什么"，分配与编排（含隐式 barrier、reduceAdd）全交给系统。
3. 共享地址空间版：程序员用 `threadId` 算出自己负责的行区间（静态分块分配），并用 3 个 barrier + 1 个 lock 亲手编排同步。

【并行机制解说】

- **同步点的三个 barrier 各司其职**：① 防止某线程抢先进入下一轮并把 `diff` 清零，而其他线程还在累加上一轮结果；② 保证所有线程都完成 `diff` 累加后再做收敛判断（所有线程读到相同的 diff）；③ 保证所有线程都看到 `done` 的更新后再开始下一轮（避免一线程开始下一轮清零 diff 时另一线程还在读）。
- **性能优化一（减少锁的争用）**：把"每格更新一次就 lock 一次"改成"先累加到本地 `myDiff`，一轮结束只 lock 一次"——锁的获取次数从 O(每个 (i,j)) 降到 O(每线程每轮)。这是"减少同步频率"的典型编排优化。
- **性能优化二（减少 barrier 数量）**：讲义进一步给出"单 barrier"版本——用 `diff[3]` 三个副本 + 索引 `index = (index+1)%3` 循环使用：把 `diff` 清零移到上一轮屏障之后执行（写入下一轮才用的槽位），从而去掉第 ① 个 barrier。**用内存足迹（footprint）换取去掉依赖**，是常见的并行编程技巧。
- **对应概念**：decomposition（识别依赖）、assignment（静态分块）、orchestration（barrier/lock/reduceAdd）、shared address space 模型 vs data parallel 模型、lock 与 barrier、red-black coloring、Amdahl's law（串行合并部分限制加速比：phase 2 部分和合并开销 P，当 N >> P 时加速比趋近 P）。

三、关键要点

1. **抽象 vs 实现（abstraction vs implementation）是本课程的贯穿主线**：ISPC 的抽象是 SPMD（programCount 个逻辑指令流），实现是 SIMD 向量指令；`foreach` 的抽象是"声明可并行迭代"，实现可以交错、分块或动态分配。判断一个程序"做了什么"，要区分抽象语义与具体实现。
2. **创建并行程序的思考过程四步走**：decomposition（识别依赖，找独立工作）→ assignment（分配工作给 worker，兼顾负载均衡与通信成本）→ orchestration（组织通信与同步、数据布局、任务调度）→ mapping（映射到硬件）。每一步都可能由程序员、系统或双方共同完成。
3. **Amdahl's Law 是硬约束**：最大加速比 ≤ 1/S，S 是串行占比。负载不均、串行合并、临界区串行化都是"隐性串行部分"，都会压低加速比。
4. **foreach 让你像写串行程序一样思考迭代**，但 ISPC 是低级语言——暴露 `programIndex` / `programCount` 也允许你写出输出未定义的程序（如 `shift_negative`）或只对特定 programCount 正确的程序；跨实例通信必须通过显式原语（reduce_add 等）。
5. **数据并行模型把编排交给系统（隐式 barrier + 内建 reduce），共享地址空间模型把同步责任交给程序员（lock + barrier）**——后者的优化空间更大，但出错（数据竞争、死锁、过度同步）的风险也更高。

四、常见陷阱与注意事项

1. **把 varying 当 uniform 用（反之亦然）**：如 `sum_incorrect_1/2`，在 uniform 变量上累加 varying 值或在 varying 变量上期待单一返回值——编译期就报错，要理解这是 ISPC 类型系统在保护你。
2. **foreach 迭代间写同一位置（数据竞争）**：`shift_negative` 中迭代 i 写 `y[i-1]` 与迭代 i-1 写 `y[i]` 冲突，输出未定义。foreach 只保证"迭代可并行"，不保证"迭代互不干扰"——依赖必须由程序员保证。
3. **忽略依赖直接并行**：Gauss-Seidel 按行扫描时行间有依赖；直接并行化会得到错误结果。先做 decomposition 识别依赖，必要时**改算法**（如红黑着色）换取可并行性——但注意改算法后的浮点结果与串行不同（仍收敛到误差阈值内）。
4. **过度同步**：每格更新都加锁、或滥用 barrier（barrier 是保守的依赖表达，它假设"屏障后所有计算依赖屏障前所有计算"）。先本地累加再一次性合并、用多副本消除依赖，能显著减少同步开销。
5. **负载不均与分配方案的选择**：静态分配零开销，但若各块工作量不均（P4 做 2 倍工作→50% 时间串行化），加速比被 Amdahl 定律锁死，需要时可换动态分配（ISPC tasks）或半静态重分配；同时 assignment 还直接影响通信量——interleaved 分配在 SIMD 实现下更优（连续内存可向量加载），blocked 分配在多处理器下通信更少（ghost 数据量小）——"哪种更好取决于运行的系统"，没有普适答案。

五、思考题（带答案）

1. **问**：ISPC 的 `foreach` 声明了"N 个迭代可并行"。如果 `N % programCount != 0`，讲义中手写的交错循环会有问题，但 foreach 抽象本身不会有问题。为什么？ **答**：手写版本假设 `N % programCount == 0`，否则某些实例会多处理一个或少处理一个元素（越界或漏算）。foreach 把"迭代到实例"的分配责任交给实现，实现可以选择动态分配（如实现 4 用 `atomic_add_local` 抢任务）或处理余数，从而对任意 N 都正确。这也体现了 abstraction（foreach 的语义与 N 无关）vs implementation（具体分配方案）的分离。
2. **问**：共享地址空间版 grid solver 里，为什么"把 diff 清零"必须放在 barrier ① 之后（而不是 while 循环开头直接清零）？ **答**：`diff` 是共享变量。如果某线程先进入下一轮迭代并清零 diff，而其他线程仍在执行上一轮的 `lock/unlock` 累加，就会出现"累加被清零覆盖"的竞争，收敛判断失效。barrier ① 保证所有线程都完成上一轮累加后，才允许任何线程清零——这是 barrier 作为"保守依赖表达"的典型用法。讲义中的单 barrier 版本用 `diff[3]` 多副本把清零延迟到"上一轮数据已被消费之后"，从而去掉这个 barrier（以内存换同步）。
3. **问**：讲义说 ISPC 的 gang 抽象由 SIMD 指令实现，因此只用一个核；要利用多核需要 ISPC tasks。那么"交错分配"与"分块分配"在 SIMD 实现下各有什么代价？ **答**：交错分配下，同一时刻 8 个实例访问的 `x[idx]` 在内存中连续，编译器可生成一条 packed vector load（`vmovaps`）高效完成"所有实例的取值"；分块分配下，同一时刻 8 个实例访问的是 8 个相距很远的元素，需要 gather 指令（`vgatherdps`），更复杂、更昂贵。但分块分配在"多处理器、跨节点"场景下通信更少（如 red-black solver 中 blocked assignment 只需在块边界交换数据）。这就是讲义反复强调的：**分配方案的好坏取决于目标系统的实现**。

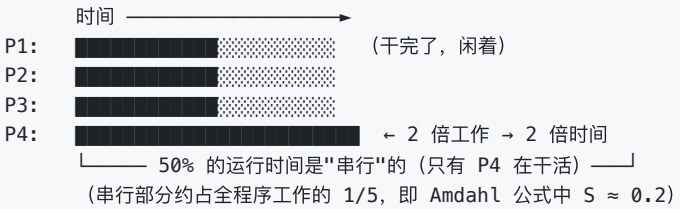
Lecture 5: Program Optimization 1: Work Distribution and Scheduling（程序优化（一）：工作分配与调度）（日期：Oct 07）

> **概述**：本讲聚焦并行程序性能优化的第一个维度——**把工作"喂饱"所有执行单元**。核心目标（彼此矛盾）有三：负载均衡（workload balance）、减少通信、减少并行开销（overhead）。讲义先讲静态/动态/半静态分配与任务粒度（task granularity）的取舍，随后深入剖析 fork-join 并行模式的调度：以 Cilk Plus 的 `cilk_spawn` / `cilk_sync` 为例，详细讲解 **work stealing（工作窃取）** 调度器如何用"每线程一个 dequeue + 空闲线程随机窃取"实现低开销、高局部性的动态负载均衡，包括 child stealing vs continuation stealing 的选择、sync 的 block descriptor 实现，以及 greedy join 调度策略。
>
> **注意**：本讲与 Assignment 2（"Scheduling Task Graphs on a Multi-Core CPU"，截止 Oct 16）直接相关——讲义中"work queue 中的任务不必相互独立"、`enqueue_task(foo, bar_handle)` 的显式依赖正是该作业任务图调度库的核心思想。

一、核心概念与定义

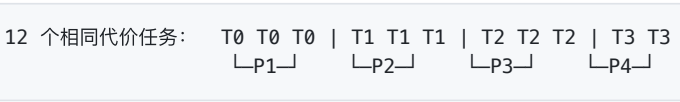
1. Workload Balance（负载均衡）与 Load Imbalance（负载不均）

- **定义**：理想情况下，程序执行的每一刻所有处理器都在计算，且**同时完成各自的工作**。只要少量负载不均，就会显著限制最大加速比——因为最后完成的那个处理器决定了整个程序的运行时间，超出的那部分时间相当于串行执行（受 Amdahl's Law 约束）。
- **现实类比**：搬家时四个人搬箱子，一个人搬的箱子是别人的两倍重——整个搬家时间被最慢的人拖住，其余三个人提前干完也只能干等。
- **公式/图示**：



2. Static Assignment（静态分配）

- **定义**：工作的分配**不依赖运行时动态行为**。注意"静态"不等于"编译期确定"：只要在"工作量与 worker 数量已知"时就能确定的分配都算静态（可以依赖运行时参数，如输入规模、线程数）。优点是简单、分配开销几乎为零（本例只有一点索引计算）。
- **现实类比**：老师开学第一天就把全班座位按名单排好——之后不再变动。
- **公式/图示**：适用场景——工作量可预测：12 个等代价任务，静态分给 4 个处理器各 3 个；代价不等但已知/平均可预测时，按任务个数平分也可（平均意义上均衡）。



3. Semi-Static Assignment（半静态分配）

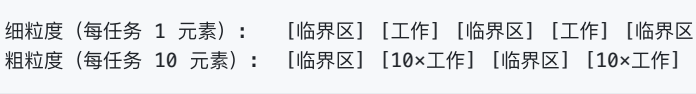
- **定义**：近期的执行代价可预测（"最近的过去是近未来的好预测器"）。应用周期性 profile 自己的执行并重新调整分配；分配在两次调整之间保持"静态"。典型场景：自适应网格（adaptive mesh，物体移动导致网格密度变化但变化缓慢）、粒子模拟（粒子缓慢移动时定期重分配）。
- **现实类比**：快递站根据每天的包裹量变化，每周重新划分一次配送片区，但一周之内片区固定。
- **公式/图示**：无（机制描述）。

4. Dynamic Assignment（动态分配）

- **定义**：程序在**运行时**动态决定分配，以保证负载分布良好（任务执行时间或任务总数未知/不可预测时使用）。典型实现：共享计数器（counter）或共享 work queue（工作队列）——worker 取走下一个未完成的任务。
- **现实类比**：外卖平台接单——骑手完成一单后从平台"抢"下一单，谁抢到谁送（任务到达时间不可预测）。
- **公式/图示**：见代码示例 1。

5. Work Queue（工作队列）与 Task Granularity（任务粒度）

- **定义**：work queue 是"待做任务"的列表，worker 线程从队列取任务、产生新任务时再推入。任务粒度指单个任务包含多少工作量：**细粒度**（1 任务 = 1 元素）负载均衡好但同步开销高（临界区被频繁进入）；**粗粒度**（1 任务 = 10 元素）同步次数减少 10 倍但均衡性变差。
- **现实类比**：切西瓜——切得越小（细粒度），每个人分到的大小越均匀，但切西瓜本身（分配开销）耗时越多；切得太大块则有人吃不完有人不够吃。
- **公式/图示**：



选择任务大小的原则：任务数应**远多于处理器数**（利于动态分配下的均衡）→ 倾向小粒度；同时任务数**尽量少**以最小化管理分配的开销 → 倾向大粒度。理想粒度取决于工作负载与机器（本课程反复出现的主题）。

6. Parallel Slack（并行松弛）

- **定义**：独立工作（可并行工作）与机器并行执行能力的比值。实践中 **~8 是好的比值**：既保证良好的负载均衡（有足够的"余量"让调度器填满所有核），又不至于因任务过细而产生过多管理开销（slack 太大 = 任务粒度太小）。
- **现实类比**：自助餐厅备餐——备的菜量是客流量的 8 倍左右，高峰期不会有人饿肚子，但也不用备 100 倍造成浪费。
- **公式/图示**：

```
parallel slack = 独立工作总量 / 机器并行执行能力    （实践中 ≈ 8）
```

7. Fork–Join Parallelism（分叉–汇合并行）与 Cilk

- **定义**：用"分叉（fork，创建新的逻辑控制流）→ 汇合（join，等待其完成）"来表达分治算法（divide-and-conquer）中天然存在的独立工作。本讲代码用 **Cilk Plus**（C++ 语言扩展，源自 MIT，现为 GCC/Intel ICC 支持的开源标准）：
- `cilk_spawn foo(args);` —— 调用 foo，但调用者可以**异步地**与 foo 的执行并行继续；
- `cilk_sync;` —— 等到当前函数**所有已 spawn 的调用**完成；每个含 `cilk_spawn` 的函数末尾有**隐式 cilk_sync**（函数返回即代表其所有工作完成）。
- **现实类比**：老板派两个下属分头去两个城市调研（fork），回来一起开汇报会（join）——下属之间互不依赖，老板不必等第一个回来才开始派第二个。
- **公式/图示**：

```
cilk_spawn foo();  cilk_spawn bar();  fizz();  cilk_sync;

        ┌─ foo() ─┐
主线程 ──┼─ bar() ─┼── 汇合（sync）──► 继续
        └─ fizz() ┘
（抽象：spawn 不规定"何时、由哪个线程"执行，只规定"可以并行"；sync 是调度约束：必须全部完成）
```

8. Work Stealing（工作窃取）调度

- **定义**：每个 worker 线程有自己的 work queue（dequeue）。线程优先从**自己的**队列取工作（本地 push/pop，无争用）；当自己队列为空时，**随机选择一个 victim（受害者）线程**，从其队列"偷"走一部分工作。工作队列中的任务可以不必相互独立（依赖由任务管理系统维护）。
- **现实类比**：几个收银员各有各的顾客队伍；某收银员队伍空了，就从别的收银员队伍里"拉"几个顾客过来结账，而不是大家一起抢一个队。
- **公式/图示**：

```
T1 ──► 自己的队列   ◄── 本地 push/pop（无争用）
T2 ──► 自己的队列   ◄── 本地 push/pop
T3 ──► 自己的队列   ◄── 本地 push/pop
T4 ──► 自己的队列       （空！）── steal! ──► 随机挑一个 victim 的队列偷工作
```

9. Continuation Stealing（延续窃取）与 Child Stealing（子任务窃取）

- **定义**：遇到 `cilk_spawn foo()` 时，调用线程必须二选一：
- **Run child first（先执行子任务，continuation stealing）**：把"调用者剩余的代码"（continuation）放入工作队列，自己立即执行 foo()。若无人窃取，线程不断从队列弹出 continuation、更新其状态（如 i 自增）再入队——**执行顺序与去掉 spawn 的串行程序完全一致**（depth-first 遍历调用图）；若被窃取，窃取者从 continuation 继续执行。**空间保证**：T 线程系统的工作队列存储不超过单线程栈存储的 T 倍。
- **Run continuation first（先执行延续，child stealing）**：把 foo() 入队，自己继续执行。调用者会先把循环里所有 spawn 的工作都生成完（breadth-first），**O(N) 空间**存储已 spawn 的工作；且无人窃取时执行顺序与串行程序差异很大。
- **Cilk 选择 continuation stealing（run child first）**。

- **现实类比**：深度优先像"一个人埋头做到底，做不完就整包留给别人接着做"（continuation 是"剩下全部"的一整块）；广度优先像"先把所有材料摊满桌子，别人来拿现成的"（每个 spawn 都是独立一份）。
- **公式/图示**：

```
for (i=0; i<N; i++) cilk_spawn foo(i); cilk_sync;

child stealing（先跑 continuation）：
  线程0 队列：foo(N-1) foo(N-2) ... foo(0)   ← 先造出 N 个任务项，O(N) 空间

continuation stealing（先跑 child）：
  线程0 队列：[cont: i=1]                     ← 只有一个"剩余工作"项
  执行 foo(0)…  完成后把 cont 更新为 i=2 再入队（深度优先，执行顺序同串行）
```

10. Dequeue（双端队列）与 Victim（受害者）

- **定义**：工作队列实现为**每 worker 一个 dequeue**（double-ended queue）：本地线程从 **tail（底部）** push/pop；远程（窃取）线程从 **head（顶部）** steal。偷顶部的好处：① 偷到的是**最大的一块工作**（减少窃取次数）；② 与"先跑 child"结合时，每个线程执行的工作**局部性最大**（自己始终处理最近产生的、数据最"热"的工作）；③ 窃取线程与本地线程不争抢同一端元素，可用**无锁（lock-free）**实现。
- **现实类比**：每个人从自己这摞纸的**最上面**取纸（本地取），有人没纸了就从别人那摞纸的**最下面**抽走一摞——互不打扰，抽走的还是一大摞。
- **公式/图示**：

```
        每个 worker 的 dequeue：
        ┌──────────────────────────────┐
head ──►│  （窃取线程从这里偷）          │
        │   [cont:151-200] [cont:26-50] ... │
tail ──►│  （本地线程在这里 push/pop）   │
        └──────────────────────────────┘
```

11. Greedy Join Scheduling（贪心汇合调度）

- **定义**：Cilk 的调度策略：**所有线程只要没事做就尝试窃取**；只有当系统中**完全没有可窃取的工作**时线程才空闲。汇合（sync）时线程不傻等——立即去寻找其他可做的工作。窃取/同步簿记（bookkeeping）的额外开销**只在发生窃取时才产生**；大部分时间线程只是在自己本地 dequeue 上 push/pop。
- **现实类比**：加班到一半的同事不会干等别人交材料，而是马上去找别的活干；只有全公司都没活了才下班。
- **公式/图示**：无（策略描述）。注意：发起 spawn 的线程**不一定是**执行 cilk_sync 之后代码的线程（continuation 可能已被窃取）。

二、代码示例与详细解说（本讲重点）

示例 1：动态分配——共享计数器 vs 增大任务粒度（素性测试）

代码（C++，SPMD 线程）：

```cpp
#include <thread>
#include <mutex>
#include <vector>
#include <algorithm>
#include <cstdio>

const int N = 1024;
int x[N];               // 输入数据（已初始化）
bool is_prime[N];       // 输出结果
std::mutex counter_lock;
int counter = 0;

bool test_primality(int v) {   // 执行时间不可预测（大素数很慢）
    if (v < 2) return false;
    for (int d = 2; d * d <= v; d++)
        if (v % d == 0) return false;
    return true;
}

// 细粒度版本：1 任务 = 1 个元素
void worker_fine() {
    while (true) {
        int i;
        counter_lock.lock();
        i = counter++;            // 从共享计数器领取下一个任务
        counter_lock.unlock();
        if (i >= N) break;        // 没有任务了，退出
        is_prime[i] = test_primality(x[i]);
    }
}

// 粗粒度版本：1 任务 = GRANULARITY 个元素
const int GRANULARITY = 10;
void worker_coarse() {
    while (true) {
        int i;
        counter_lock.lock();
        i = counter;
        counter += GRANULARITY;   // 一次领取 10 个元素
        counter_lock.unlock();
        if (i >= N) break;
        int end = std::min(i + GRANULARITY, N);
        for (int j = i; j < end; j++)
            is_prime[j] = test_primality(x[j]);
    }
}

int main() {
    for (int i = 0; i < N; i++) x[i] = i * i + 1000;
    std::vector<std::thread> pool;
    for (int t = 0; t < 4; t++)
        pool.emplace_back(worker_fine);   // 换成 worker_coarse 对比
    for (auto& t : pool) t.join();
    std::printf("primes found: %d\n",
                (int)std::count(is_prime, is_prime + N, true));
    return 0;
}
```

【代码做了什么？】

1. 串行版本就是把 `test_primality(x[i])` 循环跑一遍；但由于每个数的素性测试时间不可预测（大质数要试除很多因子），**静态分配**（每人固定 1/4 区间）会导致负载不均。

2. 动态分配：多个 worker 共享一个 `counter`，每次加锁取一个（或一组）下标。先到先得，谁的块大谁自然多做——**完成快的线程自动多干活**，实现良好负载均衡。
3. 细粒度（每任务 1 元素）与粗粒度（GRANULARITY=10）的唯一差别是临界区进入频率：粗粒度版本临界区次数减少 10 倍。

【并行机制解说】

- **线程如何创建**：`std::thread` 创建 4 个 worker，所有 worker 执行同一个 SPMD 函数（`worker_fine`），靠共享 `counter` 区分工作——这正是第 4 讲的 shared address space 模型。
- **工作如何分配**：**动态分配（dynamic assignment）**。锁保护的临界区（`counter++`）是分配机制本身，它引入的串行化是串行程序中不存在的**额外开销（overhead）**，且是串行执行（受 Amdahl 定律约束）——这就是"细粒度同步开销"的量化来源。
- **对应概念**：dynamic assignment、work queue（此处为计数器形式）、task granularity、overhead、critical section。
- **课堂讨论**：细粒度同步开销到底是不是问题？答案取决于任务工作量与临界区开销之比——若每个任务本身很重，临界区开销占比就小，细粒度没问题；若任务很轻，就应增大粒度（讲义给出了粗粒度版本）。

示例 2：共享工作队列（work queue）与任务依赖（Assignment 2 预告）

代码（C++，共享工作队列 + 显式依赖的任务系统）：

```cpp
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <functional>
#include <vector>
#include <cstdio>

// 共享工作队列：worker 取任务、推任务都经过它
class WorkQueue {
    std::queue<std::function<void()>> tasks;
    std::mutex m;
    std::condition_variable cv;
    bool done = false;
public:
    void push(std::function<void()> f) {
        std::lock_guard<std::mutex> lk(m);
        tasks.push(std::move(f));
        cv.notify_one();
    }
    bool pop(std::function<void()>& out) {   // 阻塞取任务
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [&]{ return !tasks.empty() || done; });
        if (tasks.empty()) return false;
        out = std::move(tasks.front());
        tasks.pop();
        return true;
    }
    void finish() {
        std::lock_guard<std::mutex> lk(m);
        done = true;
        cv.notify_all();
    }
};

void worker(WorkQueue& wq) {
    std::function<void()> task;
    while (wq.pop(task)) task();    // 不断取任务执行
}

int main() {
    WorkQueue wq;
    std::vector<std::thread> pool;
    for (int t = 0; t < 4; t++) pool.emplace_back(worker, std::ref(wq));
    for (int i = 0; i < 64; i++)
        wq.push([i]{ std::printf("task %d\n", i); });  // 64 个独立小任务
    wq.finish();
    for (auto& t : pool) t.join();
    return 0;
}
```

【代码做了什么？】

1. `WorkQueue` 用 mutex + condition_variable 实现线程安全队列：`push` 放入任务并唤醒一个等待线程，`pop` 在队列空且未结束时阻塞等待。
2. `main` 放入 64 个独立小任务，4 个 worker 动态领取执行。执行顺序由调度决定（先到先得），不是确定性的。

【并行机制解说】

- **工作如何分配**：动态分配——所有 worker 争抢同一个队列。讲义指出这种**单一共享队列**的缺点：所有 worker 都要在同一个队列上同步（**争用/contention**），临界区成为串行瓶颈。
- **改进方向（本讲后半部分 + Assignment 2）**：**分布式队列**——每个 worker 有自己的队列，本地 push/pop 无争用；只有本地队列空时才**窃取**（steal）别人的工作（此刻线程本来就闲着，同步代价可接受）。这直接引出 work stealing。

- **任务可以不独立（Assignment 2 的核心）**：讲义给出带依赖的任务系统 API：

```
foo_handle = enqueue_task(foo);              // 独立任务
bar_handle = enqueue_task(bar, foo_handle);  // bar 依赖 foo：foo 完成前不能执行
```

任务管理系统（scheduler）负责在依赖满足后才把任务分配给 worker——这就是 Assignment 2 "Scheduling Task Graphs on a Multi-Core CPU"中任务图（task graph）调度库的抽象。 – **对应概念**：work queue、dynamic assignment、task dependency、task graph、contention（在第 6 讲详述）。

示例 3：Cilk 风格的分治并行——并行 Quicksort

代码（Cilk Plus）：

```
// Cilk Plus 代码：可用 Intel ICC 或 GCC 的 -fcilkplus 选项编译
//  （也可用下方 std::async 近似版本在普通 C++ 环境验证同样的思路）
#include <algorithm>

const int PARALLEL_CUTOFF = 1000;   // 问题规模小于此值时串行排序

void quick_sort(int* begin, int* end) {
    if (begin >= end - PARALLEL_CUTOFF) {
        std::sort(begin, end);              // 足够小 → 串行（spawn 开销超过并行收益）
    } else {
        int* middle = partition(begin, end);      // 划分：选 pivot 并分区
        cilk_spawn quick_sort(begin, middle);     // 左半边：可能并行执行
        quick_sort(middle + 1, end);              // 右半边：当前线程继续执行
        // 函数末尾有隐式 cilk_sync：返回前保证左右两边都完成
    }
}
```

【代码做了什么？】

1. 串行 quicksort 递归：`quick_sort(begin, middle)` 与 `quick_sort(middle+1, end)` 是**相互独立的工作**（划分完成后两边互不依赖）。
2. Cilk 版本只在划分（partition）之后 spawn 左半边，右半边由当前线程直接递归执行——**每个线程同时只产生一个可被窃取的"continuation"**。
3. `PARALLEL_CUTOFF`：问题足够小时退回 `std::sort` 串行排序——因为此时 spawn 的开销（创建任务、簿记）超过了并行化带来的收益。

【并行机制解说】

- **工作如何分配（调度过程）**：假设 200 个元素、3 个线程。线程 0 划分出 [0–100] 与 [101–200]，spawn 左半边后把"cont: 101–200"（continuation）放入自己的 dequeue，然后执行 [0–100] 的划分……线程 1/2 空闲时从线程 0 的 dequeue **顶部**偷走一大块（如 "cont: 101–200"），各自继续划分——被偷的 continuation 又会生成新的 continuation 供进一步窃取。
- **同步点**：每个函数末尾的隐式 cilk_sync。被窃取的子任务完成后，通过 block descriptor（见下方示例 4）追踪"还有多少 spawn 未完成"。
- **对应概念**：fork–join、cilk_spawn/cilk_sync、work stealing、continuation stealing、PARALLEL_CUTOFF（任务粒度）、parallel slack。

C++ `std::async` 近似版本（普通 C++ 环境验证思路）：

```
#include <future>
#include <algorithm>

void quick_sort_async(int* begin, int* end) {
    if (begin >= end - PARALLEL_CUTOFF) {
        std::sort(begin, end);
    } else {
        int* middle = partition(begin, end);
        std::future<void> left =
            std::async(std::launch::async, quick_sort_async, begin, middle);
        quick_sort_async(middle + 1, end);   // 当前线程做右半边
        left.get();                          // ≈ cilk_sync：等待左半边完成
    }
}
```

示例 4：child-first 分治 vs 扁平 spawn 循环（工作窃取调度器如何"喂饱"机器）

代码（Cilk Plus）：

```
// 形式 1：扁平 spawn 循环（breadth-first 生成 O(N) 个任务项）
for (int i = 0; i < N; i++) {
    cilk_spawn foo(i);       // 每个 foo(i) 都是独立任务项
}
cilk_sync;

// 形式 2：child-first 递归分治（depth-first，空间占用 ≈ O(T · 单线程栈)）
void recursive_for(int start, int end) {
    while (start <= end - GRANULARITY) {
        int mid = start + (end - start) / 2;   // 对半切
        cilk_spawn recursive_for(start, mid);  // 先执行左半边（child first）
        start = mid;                           // "剩余工作"成为 continuation
    }
    for (int i = start; i < end; i++)
        foo(i);
}
recursive_for(0, N);
```

【代码做了什么？】

1. 形式 1（扁平 spawn 循环）在 Cilk 的 child-first 执行下：线程执行 foo(0)，把"剩余迭代"作为**唯一一个** continuation 入队；若该 continuation 被窃取，窃取者执行 foo(1) 后再把 continuation（i=2）入队——任何时刻队列中基本只有 1 个"剩余工作"项，调用图按**深度优先**遍历，空间占用小（T 线程总存储 ⩽ T × 单线程栈）。
2. 形式 2（递归分治）同样 child-first：线程 0 执行 `recursive_for(0,N)` 时 spawn `recursive_for(0, N/2)` 并立即执行它，把 `recursive_for(N/2, N)` 作为 continuation 入队；下一层递归再把 `(N/4, N/2)` 入队……队列里是一串**大块** continuation（如 (N/2,N)、(N/4,N/2)、...），窃取者拿到任意一块后自己继续细分、又产生新的 continuation——可窃取的工作量随递归深度**指数级增长**。

【并行机制解说】

- **为什么 child-first 调度器"预见到"分治**：① 空间：任何时刻队列中只有"剩余工作"的 continuation，T 线程的工作队列总存储 ⩽ T × 单线程栈存储（可证明）；② **并行度产生速度**：扁平循环每次窃取只"释放"一个 foo（并行度线性爬升，机器填不满）；递归分治形式每次窃取都让窃取者继续细分出更多大块工作（并行度指数增长），**更快地把并行机器填满**——这正是讲义原话 "Code at right generates work in parallel, (code at left does not), so it more quickly fills up parallel machine"（右列代码在并行地产生工作、更快填满机器）；③ 无人窃取时，两种形式的执行顺序都与去掉 spawn 的串行程序一致（利于调试与空间局部性）。
- **窃取顶部的好处**：偷到的是最大的工作块（如 [101–200] 而不是 [1–2]），窃取次数少；每个线程执行的工作局部性最大（深度优先产生的连续区间数据往往在 cache 中相邻）；本地线程与窃取线程操作 dequeue 的两端，可无锁实现。

- **对应概念**：continuation stealing vs child stealing、dequeue、work stealing、parallel slack（"要有比执行能力更多的独立工作，但别多到粒度太细"）。

示例 5：sync 的实现——block descriptor（发生窃取时）

代码/图示（Cilk 运行时行为）：

```
// 假设 3 个线程执行：
for (int i = 0; i < 10; i++) { cilk_spawn foo(i); }
cilk_sync;
bar();

无窃取情形：cilk_sync 是 no-op —— 所有 foo 都由线程 0 顺序完成，没有跨线程依赖。

有窃取情形：
  线程0 执行 foo(0) (id=A)；线程1 偷走 "cont: i=0 (id=A)" 后执行 foo(1)...
  运行时为代码块 A 创建 descriptor：

        ┌────────────────────┐
        │ id = A             │
        │ spawn: 3, done: 1  │   ← 该块已 spawn 3 个、已完成 1 个
        └────────────────────┘

  每当一个 spawn 发生（done 的 continuation 再次 spawn）：spawn+1
  每当一个 foo 完成：done+1
  cilk_sync 返回的条件：spawn == done（该块所有 spawn 的工作全部完成）
  之后：持有 continuation 的线程（可能不是发起 spawn 的线程 0！）继续执行 bar()
```

【代码做了什么？】

1. 无窃取时 sync 无需任何簿记——所有 spawn 的工作都在同一线程顺序完成，sync 是空操作（no-op），零开销。
2. 一旦发生窃取，运行时为每个"包含 spawn 的代码块"创建 **block descriptor**，记录该块 `spawn`（已产生的 spawn 数）与 `done`（已完成数）。每完成一个被窃取/本地执行的子任务更新计数；`spawn == done` 时 sync 满足。
3. `cilk_sync` 之后的代码（如 `bar()`）由**当前持有 continuation 的线程**执行——不一定是发起 spawn 的线程。

【并行机制解说】

- **开销分析（greedy join scheduling 的关键论据）**：descriptor 的创建、计数更新等簿记**只在发生窃取时才发生**；若窃取的是大块工作，窃取应发生得很稀疏。绝大多数时间线程只是本地 dequeue push/pop——这就是 Cilk 调度器"低开销"的来源。
- **对应概念**：cilk_sync 的实现、block descriptor、greedy join scheduling、overhead（簿记开销与窃取频率成正比）。

三、关键要点

1. **高性能编程的第一条铁律（TIP #1）**：先实现最简单的并行方案，**测量**性能，再决定是否值得优化。不要过早引入复杂的分配/调度机制。
2. **三个目标互相矛盾**：负载均衡（把核喂饱）、减少通信（避免停顿）、减少额外开销（调度/同步/分配机制本身的开销）——优化的本质是在三者之间找平衡点。静态 vs 动态分配不是二选一，而是一个连续谱：**尽可能用对工作负载的先验知识**减少负载不均与任务管理开销（极限情况下，若系统全知，就用完全静态分配）。
3. **任务粒度是核心杠杆**：任务数要远多于处理器（利于均衡），又要尽量少（降低管理开销）——"任务太多"与"任务太少"都会拖慢程序；`PARALLEL_CUTOFF`、`GRANULARITY`、parallel slack ≈ 8 都是这个权衡的具体体现。
4. **Cilk 工作窃取调度器的三件套**：① 每线程一个 dequeue，本地从底部 push/pop（无争用、可无锁）；② 空闲线程**随机选 victim** 从顶部窃取（偷最大块、保持局部性、减少窃取次数）；③ **run child first（continuation stealing）**——深度优先、空间有界（≤ T×单线程栈）、无窃取时执行顺序与串行一致。

5. **sync 的开销只在窃取发生时产生**：greedy join 调度下线程永不空等（没事就偷，偷不到才 idle）；簿记开销与窃取频率成正比，而大块窃取保证了低频率。

四、常见陷阱与注意事项

1. **过度细粒度导致同步成为瓶颈**：每个任务都进出临界区，同步开销（串行部分！）可能超过并行收益。先估算"任务执行时间 vs 临界区开销"的比值，再定粒度；粗粒度版本（GRANULARITY）是简单有效的修复。
2. **负载不均被低估**：静态分配下若工作代价不均（P4 做 2 倍工作 → 50% 运行时间串行化，S≈0.2），加速比被 Amdahl 定律锁死。必要时用动态分配/半静态重分配；但动态分配本身有开销——**用先验知识（semi-static）往往比纯动态更优**。
3. **"长任务最后才被调度"**：共享队列按入队顺序（左到右）取任务时，若长任务排在最后，末尾会出现大片空闲（slop）。对策：任务拆小，或**先调度长任务**（需要一定的工作量可预测性）。
4. **单一共享队列的争用**：所有 worker 抢一个队列，临界区串行化。用分布式队列 + 窃取替代（这也是 Assignment 2 的动机之一）。
5. **误用 spawn/sync 语义与线程身份假设**：`cilk_spawn` 只保证"可以并行"，不保证"一定并行"——一个只把 spawn 实现成普通函数调用的 Cilk 实现也是正确的（讲义明确提问过这一点）；`cilk_sync` 才是调度约束。同时不要把"发起 spawn 的线程会在 sync 后继续执行"当成不变量——continuation 可能已被其他线程偷走。跨线程共享状态时，要用 sync 保证依赖，而不是假设线程身份。

五、思考题（带答案）

1. **问**：为什么 Cilk 选择 "run child first"（continuation stealing）而不是 "run continuation first"（child stealing）？给出至少两个理由。 **答**：① 空间：child stealing 在进入任何工作前就生成 O(N) 个任务项（广度优先），而 continuation stealing 任何时刻队列中只有"剩余工作"一个 continuation，可证明 T 线程系统的工作队列存储不超过单线程栈存储的 T 倍；② 并行度产生速度：continuation stealing 沿递归路径立刻产生可窃取的工作，能更快填满并行机器（`recursive_for` 例子）；③ 无窃取时执行顺序与去掉 spawn 的串行程序一致（可预测、利于调试与 cache 局部性）。Cilk 采用 child-first，因此称为 continuation stealing。
2. **问**：讲义说"细粒度任务负载均衡好，但同步开销高"。若你的并行程序因临界区争用而变慢，除了增大任务粒度，还有什么办法？ **答**：① 减少临界区内的操作（只保护必要状态，把重计算移出临界区）；② 用原子指令替代锁（如 `atomic_incr(counter)`，讲义动态分配示例的注释）；③ 用分布式队列 + 工作窃取，让本地 push/pop 无争用（本讲后半部分）；④ 若任务是并行的，考虑把"取任务"频率降到最低（每线程一次取一大块，如 GRANULARITY）。核心思想是**减少对共享资源的访问频率与串行化时间**——这正是第 6 讲"减少争用（contention）"的主题。
3. **问**：实现 cilk_sync 时，为什么"无窃取时它是 no-op"？这如何体现"抽象 vs 实现"？ **答**：无窃取意味着该代码块的所有 spawn 工作都由同一线程顺序完成——sync 的语义（"所有 spawn 的工作已完成"）在顺序执行下自动成立，无需任何簿记。运行时只需在"发生窃取"（跨线程产生了真实的并行/依赖）时才创建 block descriptor 追踪 spawn/done 计数。这体现了抽象（cilk_sync 的语义恒定：等待所有 spawn 完成）与实现（是否产生簿记开销取决于具体的调度结果）的分离——也是 Cilk 调度器低开销的关键设计。

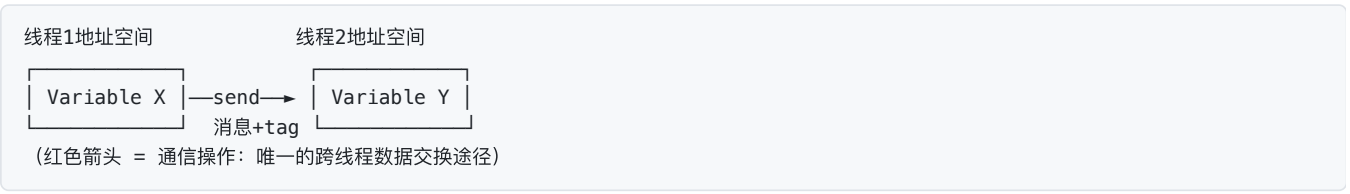
Lecture 6: Program Optimization 2: Locality, Communication, and Contention（程序优化（二）：局部性、通信与争用）（日期：Oct 09）

> **概述**：本讲把"通信"从"机器之间的消息"推广到**扩展内存层次结构中的每一级**（寄存器 ↔ cache ↔ 内存 ↔ 远程内存），系统讲解降低通信代价的四类技术：减少消息数量与开销（bulk transfer/合并消息）、降低通信延迟（利用局部性）、避免争用（contention）、通信与计算重叠（overlap）。核心工具是 **arithmetic intensity（算术强度）** 与 **inherent vs artifactual communication（固有 vs 人为通信）** 的区分；并以 grid solver 为案例演示 1D/2D 分块分配、loop fusion、loop tiling 如何提升算术强度。后半讲介绍性能分析方法论：roofline model（屋顶模型）、high watermark（性能上限水印）实验、硬件 performance counters，以及"固定问题规模测量加速比"的陷阱（超线性加速比之谜）。
>
> **注意**：本讲与 Written Assignment 1（截止 Oct 9）以及后续 Assignment 2/3 的性能分析部分直接相关——"你是 compute-bound、bandwidth-bound 还是 sync-bound？"的判断方法、roofline 与 high watermark 实验思路将贯穿整个课程。

一、核心概念与定义

1. Message Passing（消息传递模型）

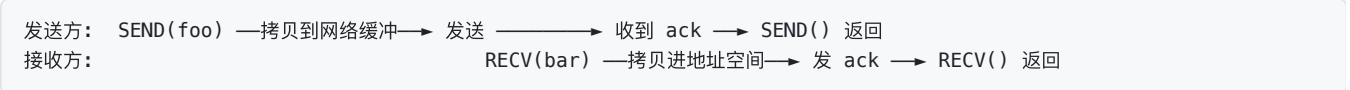
- **定义**：与共享地址空间相对的另一抽象：每个线程运行在**自己的私有地址空间**中，线程之间**只能通过发送/接收消息**交换数据。`send(X, dest, tag)`：把本地变量 X 的内容作为消息发给 dest，并打上标识符 tag；`recv(Y, src, tag)`：接收来自 src 的、带 tag 的消息并存到本地变量 Y。发送消息是线程 1、2 之间交换数据的**唯一**方式。
- **现实类比**：讲义中的经典比喻——**蜗牛邮件（snail mail）**：每个人有自己的邮箱（私有地址空间），要给别人东西只能写信寄过去（send），对方收到信才知道内容（recv）。
- **公式/图示**：



- **实现层面**：硬件**不需要**实现全局共享地址空间，只需要提供节点间消息机制——因此可以把普通商用机器用网络（如 Infiniband）连成大规模并行机，消息传递是**集群与超算**的编程模型。

2. Blocking Send/Receive（阻塞式同步收发）

- **定义**：`send()` 在**收到接收方确认（ack）**、确认消息数据已进入**接收方地址空间**后才返回；`recv()` 在消息数据被**拷贝进接收方地址空间并发出 ack** 后才返回。语义是"收发双方在消息上同步"。
- **现实类比**：挂号信——寄出方要等到收件人签收回执（ack）才算"寄完"；收件人要拿到信并签收，寄件人才算解脱。
- **公式/图示**：



3. Non-Blocking（Asynchronous）Send/Receive（非阻塞异步收发）

- **定义**：`send()` **立即返回**，但调用线程在消息真正发送完成前**不得修改发送缓冲**；`recv()` 只是"登记"未来要接收的意图并立即返回一个句柄（handle），用 `checksend(h)` / `checkrecv(h)` 查询实际完成状态。调用线程可以在等待期间**做其他工作**——即**通信与计算重叠（overlap）**。
- **现实类比**：电子邮件——点"发送"就继续干别的（异步），但邮件正文在发送完成前不能改动；收件人也不用一直守在邮箱前，稍后查收即可。
- **公式/图示**：

```
发送方:  SEND(foo) ──► 返回 handle h1 ──►  (继续做别的计算) ──► CHECKSEND(h1) 通过后
          (此时才能安全修改 foo)                                    方可修改 foo
接收方:  RECV(bar) ──► 返回 handle h2 ──►  (继续做别的计算) ──► CHECKRECV(h2) 通过后
          (此时才能安全读取 bar)                                    方可读取 bar
(红色文字 = 与应用程序线程并发执行的通信过程)
```

4. NUMA（Non-Uniform Memory Access，非均匀内存访问）

- **定义**：多插槽（multi-socket）等现代系统中，不同核心访问同一内存地址的**延迟可能不同**（取决于内存控制器与核心的相对位置），且带宽也可能不同。Intel 的 ring interconnect（四环：request/snoop/ack/data，6 个节点，理论峰值约 435 GB/s）与 SUN Niagara 2 的 crossbar（面积约等于一个核）都是片上互连的实现例子。
- **现实类比**：住在不同宿舍楼的学生去同一个图书馆——离图书馆近的宿舍（本地内存）走得快，远的（远端内存）走得慢，即使去的是同一本书。
- **公式/图示**：

```
      ┌── Core1..4 ── Memory Controller ── Memory ◄── X（核心1-4 访问快）
片上网络┤
      └── Core5..8 ── Memory Controller ── Memory     （核心5-8 访问 X 慢）
```

5. Arithmetic Intensity（算术强度）

- **定义**：`算术强度 = 计算量（如指令数）/ 通信量（如字节数）`。讲义也给出"amount of computation / amount of communication"的公式：若分子是计算的执行时间，比值就是代码的平均带宽需求。`1 / 算术强度` 是 **communication-to-computation ratio（通信计算比）**。由于现代并行处理器"计算能力/可用带宽"的比值很高（回忆第 3 讲逐元素向量乘的例子），**必须**有高算术强度才能高效利用它们。
- **现实类比**：算术强度像"一次进货能支撑多少道菜"——进货（通信）一次很贵，所以要尽量让每次取回来的数据（cache line/消息）被多次计算使用。
- **公式/图示**：

```
算术强度 = amount of computation（指令数）/ amount of communication（字节数）
  · 高算术强度（低通信计算比）：计算密集，好！
  · 低算术强度（高通信计算比）：带宽受限，差！

例（讲义 loop fusion）：add 循环 = 2 load + 1 store per 1 次运算 → 算术强度 1/3
                        fused 循环 = 4 load + 1 store per 3 次运算 → 算术强度 3/5
```

6. Inherent Communication（固有通信）与 Artifactual Communication（人为通信）

- **定义**：**固有通信**是并行算法中**必须发生**的通信，是算法的根本属性（如消息传递 grid solver 中发送 ghost rows）；**人为通信**是除此之外的一切通信，源于系统实现的实际细节（缓存行粒度、缓存容量有限、无效加载等）。减少固有通信要靠好的 assignment（分配）；减少人为通信要靠局部性优化。
- **现实类比**：固有通信像"两家分店必须互相调货"（业务必需）；人为通信像"每次调货必须整车发货，哪怕只需要一件"（运输系统的浪费）。
- **公式/图示**：人为通信的三个来源（讲义）：
- **最小传输粒度**：程序只 load 1 个 4 字节 float，但整条 64 字节 cache line 必须从内存传来——多传了 16 倍；
- **系统操作的冗余**：连续 store 16 个 4 字节 float，整条 cache line 被"load → 整体覆盖 → store 回内存"，load 是多余的（2 倍开销）；
- **有限复制容量**：缓存太小，同一数据在两次访问之间被逐出，导致重复通信（**capacity miss，容量缺失**）。

7. Temporal Locality（时间局部性）与 Loop Tiling/Blocking（循环分块）

- **定义**：时间局部性指"刚访问过的数据很快会被再次访问"。**Blocking（tiling，分块）** 通过**重排计算顺序**减少 capacity miss：把计算组织成小块，让小块内的邻域数据在 cache 中存活到被再次使用。讲义例子：cache line = 4 个网格元素、cache 容量 = 24 个元素时，row-major 逐行遍历在第二行开头需要为每 4 个输出元素加载 3 条 cache line；分块后每 6 个输出元素只需加载 2 条 cache line。
- **现实类比**：去仓库取料——与其每次只取一件（来回跑），不如把接下来要用的料一次搬回工位（分块），减少往返次数。
- **公式/图示**：

```
朴素 row-major 遍历（红线 = 必须重新从内存加载）：
  行0: [■■■■] [■■■■] [■■■■] [■■■■] [■■■■] [■■■■]   （cache 24 元素 = 6 行块）
  行1: 更新时上一行数据已被逐出 → 每 4 个输出元素 load 3 条 cache line

分块遍历：先算左上角小块（含其邻居），邻居在 cache 中 → 每 6 个输出元素 load 2 条 cache line
```

8. Loop Fusion（循环融合）

- **定义**：把多个依次遍历同一数组的循环合并成一个循环，让**同一批数据在一次遍历中被多次使用**，提高算术强度。讲义例子：`E = D + (A+B)*C` 用三个独立循环（add/mul/add，各算术强度 1/3）vs 融合成一个循环（4 load + 1 store per 3 运算，算术强度 3/5）。模块化的代码（如 NumPy 式数组库）更好读，但融合版性能好得多。
- **现实类比**：一次购物跑三家店（三个循环）vs 在一家店买齐（融合）——后者少跑路（少重复加载数据）。
- **公式/图示**：见代码示例 3。

9. Contention（争用）与 Hot Spot（热点）

- **定义**：资源（内存、通信链路、服务器……）有固定吞吐（单位时间事务数）。当**很短时间窗口内大量请求涌向同一资源**时，资源成为热点（hot spot），请求排队，整体操作时间变长。减少争用：复制被争用的资源（本地副本、细粒度锁）、错开访问时间、用树形通信结构（降低争用但无争用时延迟更高）vs 扁平通信（无争用时低延迟但有争用风险）。
- **现实类比**：讲义的办公室答疑例子——Kayvon 3:00–3:20 答疑，多个学生同时从 Bytes Cafe 走来（各 5 分钟路程），到办公室后排成一队：第一个学生 10 分钟搞定，排在后面的学生要 23 分钟；而如果大家**预约错开**（3:00、4:30），每个人都是 10 分钟。问题不在"教授答疑"本身，而在**同时到达导致的排队**。
- **公式/图示**：

```
扁平通信（更新共享变量）：          树形通信（reduce）：
   P1─┐                              P1─┐
   P2─┼─► 共享变量（热点!）            P2─┼─► 部分和 ─┐
   P3─┼─►                            P3─┼─► 部分和 ─┼─► 最终结果
   P4─┘                                P4─┘           │
   高争用风险，无争用时延迟低          争用少，但无争用时延迟更高
```

10. Roofline Model（屋顶模型）与 High Watermark（性能上限水印）

- **定义**：Roofline 是分析程序性能受何限制的模型：横轴是算术强度，纵轴是最大可达指令吞吐——水平区（高算术强度）是 **compute limited（计算受限）**，对角区（低算术强度）是 **memory bandwidth limited（带宽受限）**，屋顶曲线即机器能力上限。High watermark 是性能分析的实验方法：通过修改程序建立"最优情况"的上界——如把所有数组访问改成 `A[0]`（局部性收益上界）、删除所有原子/锁操作（同步开销收益上界）、删掉大部分数学运算但保留同样数据加载（判断内存瓶颈）、或添加数学指令看执行时间是否线性增长（判断是否指令率受限）。
- **现实类比**：屋顶模型像限速牌——车（程序）在平路（计算密集）能跑多快看发动机（计算能力），在上坡（通信密集）能跑多快看坡度（带宽）——永远到不了屋顶之上。
- **公式/图示**：

```
吞吐
 |╱╲ 屋顶 = 机器上限
 | ╲    (水平区: compute limited)
 |  ╲
 |   ╲  (对角区: bandwidth limited)
 └────→ 算术强度
```

11. Ghost Cell（幽灵单元/镜像单元）

- **定义**：消息传递模型中，每个线程的私有数组只保存自己负责的那部分网格；计算边界单元时需要**邻居线程的数据**，于是把远端地址空间的数据**复制一份**到本地数组的边缘（ghost cells）。这些数据"归"其他线程所有（owned by other threads）。
- **现实类比**：两个邻国各自维护一份"边境地图"——边境线对面 1 公里内的地形是复制的（ghost），不归自己管，但规划时需要看。
- **公式/图示**：见代码示例 1。

12. Super-Linear Speedup（超线性加速比）

- **定义**：加速比超过 P（处理器数）的现象。原因通常是**工作集（working set）变小后装进了 cache**：处理器多了，每个处理器分到的数据块小到能放进自己的 cache，消除了大量内存访问；同理，问题太大时单机工作集装不进内存（thrashing 到磁盘），换大机器（内存更多）会显得加速比"惊人"。这提示：**固定问题规模评估机器是有问题的**——问题规模与机器规模之间存在复杂交互（影响负载均衡、开销、算术强度、局部性）。
- **现实类比**：一个人搬 100 箱货需要来回跑很多趟仓库（每次货太多堆不下只能放仓库）；10 个人时每人只需搬 10 箱，全都能放在各自的推车上——每个人"搬得快了"（工作集进了 cache），总加速比超过 10。
- **公式/图示**：

```
258×258 网格在 32 处理器上（每处理器仅 ~310 格）：无收益甚至变慢（通信/计算比太高）
1K×1K 网格在 32 处理器上（每处理器 ~32K 格）：正常加速
大网格 + 足够多处理器：每处理器块变小 → 装进 cache → 超线性加速比
```

二、代码示例与详细解说（本讲重点）

示例 1：消息传递版 grid solver——ghost rows 交换与死锁

背景：回顾第 4 讲的 grid solver（red-black 更新）。消息传递模型下，网格被分成 P 份私有数组，每份含 `rows_per_thread+2` 行（上下各加一行 ghost cells）。每轮迭代，线程需要邻居刚更新的边界行（inherent communication）。

代码 A（MPI 风格，同步/阻塞收发，会死锁！）：

```
// 编译：mpicc -o solver solver.c   运行：mpirun -np 4 ./solver
// 伪代码整理自讲义；localA[i,j] 记法表示扁平数组 localA[i*(N+2)+j]
// MSG_ID_ROW 等为消息标识常量
int N, tid = get_thread_id();
int rows_per_thread = N / get_num_threads();
float* localA = allocate(rows_per_thread + 2, N + 2);  // 含 ghost rows（上下各一行）

void exchange_ghost_rows_deadlock() {
    int bytes = sizeof(float) * (N + 2);   // 一整行的字节数
    // 每个线程先"发送"再"接收" —— 使用同步 send/recv 时必然死锁!
    if (tid != 0)
        send(&localA[1][0],     bytes, tid - 1, MSG_ID_ROW);  // 把第 1 行发给上邻居
    if (tid != get_num_threads() - 1)
        send(&localA[rows_per_thread][0], bytes, tid + 1, MSG_ID_ROW); // 把最后一行发给下邻居
    if (tid != 0)
        recv(&localA[0][0],     bytes, tid - 1, MSG_ID_ROW);  // 从上邻居收 ghost row
    if (tid != get_num_threads() - 1)
        recv(&localA[rows_per_thread+1][0], bytes, tid + 1, MSG_ID_ROW);
}
```

【代码做了什么？】：每个线程想把"自己负责区域的最上一行/最下一行"发给邻居，同时从邻居接收它们的最上行/最下行，填入自己的 ghost cells。但使用**同步 send** 时，send 要等接收方确认；所有线程都先阻塞在自己的 send 上、还没执行到 recv——**循环等待（circular wait），死锁**。

代码 B（死锁修复：奇偶交错收发）：

```
// 偶数线程：先 send 后 recv；奇数线程：先 recv 后 send
// 打破"所有线程同时阻塞在 send"的循环等待
void exchange_ghost_rows_fixed() {
    if (tid % 2 == 0) {          // 偶数线程
        sendDown(); recvDown();  // 先发下行，再收上行
        sendUp();   recvUp();
    } else {                     // 奇数线程
        recvUp();   sendUp();    // 先收上行，再发下行
        recvDown(); sendDown();
    }
}
```

【并行机制解说】

- **数据如何共享/通信**：通信**显式**发生在消息收发中；一次收发一整行（bulk transfer，批量传输一整行而不是逐个元素发消息）；数组下标相对于**本地地址空间**。
- **同步方式**：消息收发本身即同步原语——讲义指出，互斥、barrier、flag 都可以用消息实现（例如把"汇总 diff"实现为所有线程 send 自己的 `my_diff` 给线程 0，线程 0 计算全局 diff 后广播 `done` 标志）。
- **死锁如何避免**：同步收发下，"所有线程先 send 再 recv"会死锁（发送方等接收方，接收方还没到 recv）。修复：奇偶线程**交错顺序**（偶数先发后收、奇数先收后发），保证同一时刻至少有一方的 recv 在等待对方的 send——这是教科书级的死锁避免（打破循环等待）。
- **对应概念**：message passing、blocking send/recv、ghost cell、inherent communication、deadlock。

示例 2：MPI 非阻塞收发（通信与计算重叠）

代码（MPI，`MPI_Isend` / `MPI_Irecv`）：

```
// 编译：mpicc -o solver solver.c   运行：mpirun -np 4 ./solver
#include <mpi.h>

void exchange_ghost_rows_async() {
    int bytes = sizeof(float) * (N + 2);
    MPI_Request reqs[2];
    int nreq = 0;

    // 1. 发布（post）收发请求：立即返回，不等待完成
    if (tid != 0) {
        MPI_Isend(&localA[1][0], bytes, MPI_FLOAT, tid - 1, MSG_ID_ROW,
                  MPI_COMM_WORLD, &reqs[nreq++]);
        MPI_Irecv(&localA[0][0], bytes, MPI_FLOAT, tid - 1, MSG_ID_ROW,
                  MPI_COMM_WORLD, &reqs[nreq++]);
    }
    // ... 对下邻居同理 ...

    // 2. 在通信进行的同时，执行与 ghost rows 无关的计算（overlap！）
    compute_interior_without_borders();

    // 3. 需要用到收发结果时，才等待完成
    MPI_Waitall(nreq, reqs, MPI_STATUSES_IGNORE);
    // 此时才能安全读取 localA[0][*]（接收缓冲）、修改发送缓冲
}
```

【代码做了什么？】

1. `MPI_Isend` / `MPI_Irecv` 只"登记"通信请求并返回 handle，函数立即返回。
2. 在等待消息期间，线程执行不依赖 ghost data 的计算（如内部区域更新）——**通信与计算重叠**，隐藏通信延迟。
3. `MPI_Waitall` 在所有请求完成后返回；此后才可安全访问接收缓冲（`recv` 缓冲未就绪时读取是未定义行为）与修改发送缓冲（消息可能尚未拷贝完）。

【并行机制解说】

- **对比阻塞版本**：阻塞收发把"等待通信完成"变成线程的停顿（stall）；非阻塞收发把这段等待时间**填满计算**。讲义总结的"增加通信/计算重叠"手段：应用侧用异步消息；硬件侧用流水线（pipelining）、多线程、预取（prefetching）、乱序执行（out-of-order execution）——但**需要应用有足够的额外并发**（并发度要大于执行单元数）。
- **对应概念**：non-blocking send/recv、overlap、pipelining（重叠/流水思想的软件形态，硬件流水见讲义总结）、communication latency hiding。

示例 3：提高算术强度——循环融合（loop fusion）

代码（C）：

```
// 目标：E = D + (A + B) * C，数组长度 n

// —— 版本 A：三个独立循环（模块化，例如 NumPy 式数组库的写法）——
void add(int n, float* A, float* B, float* C) {
    for (int i = 0; i < n; i++)
        C[i] = A[i] + B[i];
}
void mul(int n, float* A, float* B, float* C) {
    for (int i = 0; i < n; i++)
        C[i] = A[i] * B[i];
}

float *A, *B, *C, *D, *E, *tmp1, *tmp2;   // 假设已分配
add(n, A, B, tmp1);      // tmp1 = A + B
mul(n, tmp1, C, tmp2);   // tmp2 = (A+B) * C
add(n, tmp2, D, E);      // E = (A+B)*C + D

// —— 版本 B：循环融合（fused）——
void fused(int n, float* A, float* B, float* C, float* D, float* E) {
    for (int i = 0; i < n; i++)
        E[i] = D[i] + (A[i] + B[i]) * C[i];   // 每个元素一趟读完所有输入
}
fused(n, A, B, C, D, E);
```

【代码做了什么？】

1. 版本 A：`tmp1`、`tmp2` 两个临时数组，数据被**反复读写**——`tmp1` 被写一次、读一次；每次循环都要把 A、B、C、D 各自重新从内存加载。
2. 版本 B：单个循环，每个输出元素一次性读入 A[i]、B[i]、C[i]、D[i]，在寄存器里完成加乘加，一次写出。**同一批数据只进 cache/寄存器一次**。

【并行机制解说】

- **算术强度对比（讲义原话）**：版本 A 的 add 与 mul 都是"per math op: 2 loads + 1 store"，算术强度 = 1/3，总体也是 1/3；版本 B 是"per 3 math ops: 4 loads + 1 store"，算术强度 = 3/5。**同样的计算，通信量几乎减半**——在带宽受限的机器上，版本 B 可以快接近 2 倍。
- **代价**：版本 A 更模块化、可组合（数组数学库风格）；版本 B 更难维护。性能 vs 模块化的经典权衡。
- **对应概念**：arithmetic intensity、temporal locality、loop fusion、减少人为/固有通信（数据被重复加载属于可避免的通信）。

示例 4：改善时间局部性——grid solver 的循环分块（loop tiling/blocking）

代码（C）：

```
// Gauss-Seidel 式更新：A[i,j] = 0.2*(A[i,j] + A[i,j-1] + A[i-1,j] + A[i,j+1] + A[i+1,j])
// 假设 row-major 存储：A[i*n + j]
// 讲义参数：cache line = 4 元素，cache 容量 = 24 元素（6 条 line）

// —— 朴素版本：逐行扫描 ——
void gauss_seidel_naive(int n, float* A) {
    for (int i = 1; i < n - 1; i++)
        for (int j = 1; j < n - 1; j++)
            A[i*n + j] = 0.2f * (A[i*n + j] + A[i*n + j-1] +
                                 A[(i-1)*n + j] + A[i*n + j+1] + A[(i+1)*n + j]);
}

// —— 分块版本：把计算组织成 block×block 的小块 ——
void gauss_seidel_blocked(int n, float* A, int block) {
    for (int i0 = 1; i0 < n - 1; i0 += block)
        for (int j0 = 1; j0 < n - 1; j0 += block)
            for (int i = i0; i < i0 + block && i < n - 1; i++)
                for (int j = j0; j < j0 + block && j < n - 1; j++)
                    A[i*n + j] = 0.2f * (A[i*n + j] + A[i*n + j-1] +
                                         A[(i-1)*n + j] + A[i*n + j+1] + A[(i+1)*n + j]);
}
```

【代码做了什么？】

1. 两个版本做完全相同的计算（浮点顺序略有不同，但都收敛到误差阈值内）。
2. 朴素版本按行推进：更新第 i 行的格子时需要第 i−1 行的邻居。当处理到第 2 行第 1 个格子时，之前访问过的 (0,1)、(1,1)、(2,1)、(0,2)、(2,2) 等元素**已被逐出 cache**（cache 只有 24 个元素）——讲义指出该程序**每 4 个输出元素要加载 3 条 cache line**（大量 capacity miss，人为通信）。
3. 分块版本按 block×block 小块推进：小块的邻居行在小块处理期间**留在 cache 中**——讲义数据：**每 6 个输出元素只加载 2 条 cache line**。

【并行机制解说】

- **时间局部性**：分块让"刚访问的数据很快再次被访问"（邻居复用），把 cache miss 变成 hit。
- **为什么是"人为通信"**：这些重复加载不是算法必需的（inherent），而是因为 cache 容量有限、数据在两次访问之间被逐出（capacity miss）——属于 artifactual communication，可通过重排计算顺序消除。
- **与并行化的关系**：这个重排与第 4 讲的 red-black 重排目的一致——**改变访问/计算顺序换取更好的资源利用**（那里换并行度，这里换局部性）。
- **对应概念**：temporal locality、loop tiling/blocking、artifactual communication、capacity miss。

示例 5：生产者–消费者流水线（pipelining：通信与计算重叠的软件形态）

代码（C++，std::thread + 有界队列）：

```cpp
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <cstdio>

// 有界阻塞队列：容量 cap 决定流水线深度（缓冲多少"在途"数据）
template <typename T>
class BoundedQueue {
    std::queue<T> q;
    std::mutex m;
    std::condition_variable not_full, not_empty;
    size_t cap;
public:
    explicit BoundedQueue(size_t c) : cap(c) {}
    void push(T v) {
        std::unique_lock<std::mutex> lk(m);
        not_full.wait(lk, [&]{ return q.size() < cap; });
        q.push(std::move(v));
        not_empty.notify_one();
    }
    T pop() {
        std::unique_lock<std::mutex> lk(m);
        not_empty.wait(lk, [&]{ return !q.empty(); });
        T v = std::move(q.front());
        q.pop();
        not_full.notify_one();
        return v;
    }
};

int main() {
    BoundedQueue<int> q(2);   // 容量 2：最多 2 个"在途"数据项

    std::thread producer([&]{
        for (int i = 0; i < 100; i++) q.push(i);   // 生产阶段
        q.push(-1);                                 // 哨兵：结束信号
    });
    std::thread consumer([&]{
        while (true) {
            int v = q.pop();
            if (v == -1) break;
            std::printf("%d\n", v);                 // 消费阶段
        }
    });
    producer.join();
    consumer.join();
    return 0;
}
```

【代码做了什么？】

1. 生产者线程不断把数据推入有界队列；消费者线程不断取出处理。队列容量 > 0 意味着**生产者不需要等消费者处理完当前项**就能生产下一项——两者在时间上重叠。
2. 把多个这样的队列**串成链**（stage1 → queue → stage2 → queue → stage3）就构成多级流水线：每一级处理完就把结果传给下一级，各级同时工作。

【并行机制解说】

- **与讲义的对应**：讲义在"减少通信代价"的总结中列出：**增加通信/计算重叠**——应用作者用异步通信；硬件实现者用流水线（pipelining）、多线程、预取、乱序执行。本示例是应用侧"重叠"思想的直接体现：**通信（队列传输）与计算（生产/消费）重叠**，隐藏通信延迟，与示例 2 的非阻塞消息收发异曲同工。

- **与工作窃取的联系**：第 5 讲的分布式队列 + 窃取同样是"队列"思想——只是那里的队列存"任务"，这里的队列存"数据流"；两者都靠"本地操作 + 有界同步"降低争用。
- **注意**：讲义未把生产者–消费者流水线作为独立主题，本示例用于直观展示讲义中"increase communication/computation overlap"的方法；真实的软件流水线通常与循环分块（示例 4）结合，把"读数据、算、写数据"拆成流水阶段。
- **对应概念**：pipelining（重叠）、non–blocking 通信思想、contention 的缓解（有界缓冲避免无界争用与拥塞）。

三、关键要点

1. **"通信"无处不在**：把并行系统看作**扩展的内存层次结构**（寄存器 → L1 → L2 → L3 → 本地内存 → 远端内存 1 跳 → N 跳），局部性管理在每一级都重要；访问未在本地满足就会触发与下一级的通信。消息传递只是把"跨处理器通信"显式化。
2. **算术强度是核心指标**：`算术强度 = 计算量/通信量`，越高越好；现代并行处理器"计算能力/带宽"比值很高，**低算术强度的代码必然带宽受限**（回忆第 3 讲：内存 100% 时间在传输，核利用率只由指令吞吐与内存吞吐决定，与延迟/未完成请求数无关）。优化 = 提高算术强度（分块、融合、共享数据）或减少通信量。
3. **区分 inherent 与 artifactual communication**：固有通信靠**好的 assignment** 减少（1D blocked → 2D blocked 的算术强度从 $\propto N/P$ 提升到 $\propto N/\sqrt{P}$，通信随 P 亚线性增长）；人为通信靠**局部性优化**减少（分块消除 capacity miss、融合消除重复加载）。
4. **消息传递的死锁可以且必须主动避免**：同步收发下"全体先 send 后 recv"必然死锁；用奇偶交错顺序或非阻塞收发打破循环等待。非阻塞收发的另一收益是**通信与计算重叠**。
5. **性能分析要"测量 + 建立上界"**：先做最简单的并行方案再测量；用 high watermark 实验判断自己是 compute–bound、bandwidth–bound 还是 sync–bound；用 roofline 模型定位"离屋顶还有多远"；用 performance counters（IPC、L3 hit ratio、bytes read）获取硬数据。**警惕固定问题规模的加速比测量**：问题太小（通信/计算比高）无加速甚至变慢，工作集装进 cache 会产生超线性加速比——评估时要考虑问题规模与机器规模的匹配。

四、常见陷阱与注意事项

1. **死锁：同步 send/recv 的顺序错误**。所有人先 send 后 recv（或先 recv 后 send）会循环等待。修复：奇偶交错（偶数先发后收、奇数先收后发）、或改用非阻塞收发。另一个常见死锁：线程 0 汇总 diff 时，其他线程在等待 `MSG_ID_DONE`，而线程 0 又在等待 `MSG_ID_DIFF`——注意消息顺序与匹配（tag）。
2. **异步收发的缓冲使用时机**：非阻塞 send 返回后**不能马上修改发送缓冲**，非阻塞 recv 返回后**不能马上读取接收缓冲**——必须等 `checksend` / `checkrecv`（MPI 中是 `MPI_Wait` / `MPI_Test`）确认完成。违反会读到旧数据或发送被破坏的数据。
3. **忽略局部性，把带宽当无限**：row–major 扫描导致 capacity miss（每 4 个输出元素 load 3 条 cache line）时，程序是带宽受限的——加核、加频率都救不了；要用分块/融合提高算术强度。模块化的三循环写法（每个算术强度 1/3）在带宽受限机器上比融合版（3/5）慢近一半。
4. **争用被忽视**：所有 worker 抢一个共享队列/共享变量（hot spot）时，同步串行化。对策：复制资源（每线程本地副本 + 最后合并）、错开访问、分布式队列 + 窃取（第 5 讲）。
5. **测量陷阱与低估串行开销**：① 与"并行算法跑在 1 个核上"比加速比（而不是与最优串行程序比）是自我安慰；② 固定问题规模（258×258 网格在 32 处理器上每处理器仅 ~310 格）通信/计算比太高，无收益甚至变慢；③ 超线性加速比不一定代表算法好——可能只是工作集装进了 cache（或从磁盘 thrashing 变为装入内存）；④ "CPU 使用率"（activity monitor）对性能优化几乎没有帮助，要用硬件 performance counters；⑤ 临界区、阻塞收发、barrier 都是串行执行的时间、都受 Amdahl 定律约束，能用非阻塞/重叠解决的就不要用阻塞等待。

五、思考题（带答案）

1. **问**：讲义问"如何从图中看出内存总线已 100% 利用？若内存延迟提高（总线带宽不变、请求可流水化）图示如何变化？若带宽提高呢？" **答**：① 内存总线 100% 利用：图中红色"从内存传输数据"的块**连续不断**、没有空隙（内存永远在忙）；此时核利用率只由

指令吞吐与内存吞吐的比值决定，与延迟、未完成请求数无关。② 延迟提高：红色块之间的"请求在途"时间变长（load 指令发出到数据返回的等待更长），但**总线仍在满负荷传输**（吞吐不变），只是核的空闲时间（红色区域，stall）更多——前提是未完成请求数足够多能持续"喂饱"总线；③ 带宽提高：红色块变短（每条 cache line 传输更快），同样指令序列下核的空闲减少、利用率上升——但若带宽提高到计算成为瓶颈，则进入 compute-limited 区（roofline 的水平区），再加带宽无益。

2. **问**：为什么说"1D blocked 分配下算术强度 $\propto N/P$，而 2D blocked 分配下 $\propto N/\sqrt{P}$"？这对并行机设计意味着什么？ **答**：N×N 网格分给 P 个处理器。1D 分块：每处理器计算 N^2/P 个元素，只需与上下两个邻居交换 2 行 $\approx 2N$ 个元素（通信量不随 P 减少！），算术强度 $\approx (N^2/P)/(2N) = N/(2P)$；P 增大时通信/计算比线性变差。2D 分块：每处理器计算 N^2/P 个元素，与四邻交换的边界长度 $\approx 4N/\sqrt{P}$，算术强度 $\approx (N^2/P)/(N/\sqrt{P}) = N/\sqrt{P}$——通信随 P **亚线性**增长。含义：处理器越多，越要选择**能捕获算法 2D 局部性**的分配；这同时解释了为什么"258×258 小网格在 32 处理器上无收益"（每处理器仅 310 格，算术强度太低）。
3. **问**：假设你的程序性能很差。如何用 high watermark 实验判断它是 compute-bound 还是 bandwidth-bound？ **答**：① 把所有数组访问改成 `A[0]`（消除大部分内存流量）：若执行时间大幅下降，说明程序对内存带宽/延迟敏感（bandwidth/latency-bound），值得投入局部性优化（分块、融合）；若几乎不变，说明瓶颈不在内存。② 删掉大部分数学运算但保留同样的数据加载：若时间下降很少，进一步印证内存瓶颈；若时间大幅下降，则是 compute-bound。③ 反向实验：添加额外数学指令，若时间随运算数线性增长，说明指令率受限（compute-bound）。④ 用 performance counters 直接读数（IPC、L3 miss、bytes read）交叉验证。注意：计算、内存、同步几乎从不完美重叠，单一实验的结论要谨慎，但**性能对上述修改的敏感度能很好地指示主导成本**。

Lecture 7: GPU 架构与 CUDA 编程（GPU Architecture & CUDA Programming）（日期：Oct 14, 2025）

> **概述**：本讲从 GPU 的历史讲起——GPU 原本是为实时 3D 游戏渲染设计的专用处理器，后来人们发现它对"大规模数据上执行相同计算"（数据并行）极其擅长，于是 2007 年 NVIDIA 随 Tesla 架构推出了 CUDA，让 GPU 以通用计算模式运行任意程序。本讲的核心目标是：① 掌握 CUDA 编程抽象（thread / block / grid 两级线程层级、host/device 分离、shared/global memory、`__syncthreads`、原子操作）；② 理解这些抽象在现代 GPU（以 NVIDIA V100 为例）上是如何实现的（SIMT、warp、SM、线程块调度器）。课程反复强调的思考题是：CUDA 究竟是数据并行模型、共享地址空间模型还是消息传递模型？它与 ISPC 的 gang/task、pthreads 有何异同？

注意：本讲对应 **Assignment 3: A Circle Renderer in CUDA**（CUDA 圆形渲染器）——你需要用本讲的全部 CUDA 概念（block/grid 配置、shared memory、`__syncthreads`、scan 等）在 GPU 上实现一个高性能圆形渲染器。

一、核心概念与定义

1. CUDA（Compute Unified Device Architecture）

- **定义**：NVIDIA 于 2007 年随 Tesla 架构推出的"C 风格"编程语言与运行时，用于在 GPU 的 compute mode（通用计算模式）硬件接口上编写程序。它相对底层：CUDA 的抽象与当代 GPU 的能力/性能特征非常贴近，设计目标是保持**低抽象距离**（low abstraction distance）。
- **现实类比**：CUDA 之于 GPU，就像 C 之于 CPU——不给你铺满玫瑰花的抽象（比如自动并行化），而是把硬件的真实结构（线程层级、共享内存、屏障）直接暴露给你，让你自己安排，换来得天独厚的性能控制力。
- **公式/图示**：无；CUDA 是语言+运行时。

2. Kernel（内核，`__global__` 函数）

- **定义**：用 `__global__` 修饰、在 GPU（device）上以 SPMD 方式执行的函数。一次 kernel launch（`kernel<<<grid, block>>>(args)`）会**批量启动**（bulk launch）成千上万个 CUDA 线程，每个线程执行同一份 kernel 代码，但通过内置变量

`threadIdx` / `blockIdx` / `blockDim` 区分自己处理的数据。

- **现实类比**：kernel 就像一张"施工图纸"（函数体），而批量启动就是复印几万份图纸发给几万个工人，每个工人按自己胸牌上的编号（`threadIdx` / `blockIdx`）负责不同工位。注意每个 worker 的程序计数器是独立的——这是与 ISPC gang 的本质区别之一（ISPC 是编译期把实例编译成 SIMD 指令，CUDA 是运行时动态判断）。
- **公式/图示**：`一次 kernel 启动： myKernel<<<numBlocks, threadsPerBlock>>>(args); └─ 启动 numBlocks × threadsPerBlock 个 CUDA 线程`

3. CUDA 线程（CUDA thread）与线程层级：thread / block / grid

- **定义**：CUDA 线程是逻辑控制流（与 pthread 的抽象类似，但实现天差地别——见后文）。线程按两级层级组织：若干线程组成一个 **thread block（线程块）**，若干 block 组成一个 **grid（网格）**。线程 ID 最高可以是 3 维的（`dim3`），方便处理天然 N 维的问题。
- **现实类比**：想象一家工厂：grid 是整个车间（一批订单），block 是班组（一个班组内的工人必须同时在场、能互相递工具——对应共享内存与 `__syncthreads`），thread 是工人。订单（block）之间互相独立，车间调度员想先做哪单就先做哪单。
- **公式/图示**：`Grid（本次 kernel 启动的全部线程，由 numBlocks 指定） ├── Block (0,0) ── 12 个线程：T(0,0) T(1,0) T(2,0) T(3,0) │ T(0,1) T(1,1) ...（2D 情形） ├── Block (1,0) ── 12 个线程 └── Block (2,0) ── 12 个线程 线程全局坐标： i = blockIdx.x * blockDim.x + threadIdx.x j = blockIdx.y * blockDim.y + threadIdx.y` 例：`dim3 threadsPerBlock(4,3); dim3 numBlocks(3,2);` → 6 个 block × 12 线程 = 72 个 CUDA 线程。

4. Host / Device 与分布式地址空间（distributed address space）

- **定义**：CUDA 程序被静态地分成两半：**host 代码**（普通 C/C++，串行跑在 CPU 上）与 **device 代码**（kernel，跑在 GPU 上）。host 与 device 拥有**不同的地址空间**：host memory 与 device global memory 之间靠 `cudaMalloc` / `cudaMemcpy`（如 `cudaMemcpyHostToDevice`）搬运数据。在 host 端直接解引用 device 指针是非法的。
- **现实类比**：就像两座隔海的城市，不能直接隔海递包裹，必须走"港口+轮船"（`cudaMemcpy`）。这正是课程前几讲讲的**分布式内存（消息传递）**风格的地址空间——`cudaMemcpy` 让你联想到什么？它跟 MPI 的 `memcpy` 一样在"两个地址空间之间"移动数据。
- **公式/图示**：`Host（CPU） Device（GPU） ┌──────────────┐ cudaMemcpy ┌──────────────────────┐ │ Host memory │◄────────────►│ Device global memory │ │ 地址空间 │ │ （DRAM，所有线程可读写） │ └──────────────┘ └──────────────────────┘`

5. Device 内存模型：private / shared / global memory

- **定义**：kernel 内部可见三种地址空间：**per-thread private memory**（每线程私有，通常是寄存器）、**per-block shared memory**（块内共享，片上高速存储）、**per-program global memory**（所有线程可见，位于 DRAM）。三种地址空间反映了程序中不同粒度的**局部性**——这是 GPU 高效实现 CUDA 的关键（如果预先知道某些线程访问同一批变量，调度器就能把共享数据放到高速片上存储）。
- **现实类比**：global memory 是"公共仓库"（慢但容量大），shared memory 是"本班组共用的工作台"（快，但只限本班组），private memory 是"个人口袋里的笔记本"（最快）。
- **公式/图示**：见第 4 点的图，补充三层结构：`Device 内部： ┌──┐ │ Global memory（所有 block、所有线程共享） │ │ ┌──────────────────────────────────┐ │ │ │ Shared memory（block 内所有线程共享） │ │ │ │ ┌──────────────────────────┐ │ │ │ │ │ Private memory（线程私有，寄存器） │ │ │ │ │ └──────────────────────────┘ │ │ │ └──────────────────────────────────┘ │ └──┘`

6. Warp（线程束）与 SIMT（Single Instruction, Multiple Thread）

- **定义**：warp 是 32 个连续的 CUDA 线程组成的一组（block 内线程 0–31 为 warp 0，32–63 为 warp 1……）。GPU 硬件把同一个 warp 内 32 个线程的指令流取出后，**动态检查**它们是否执行同一条指令；若是，就用 SIMD ALU 让 32 个线程同时执行——NVIDIA 称之为 **SIMT**。warp **不属于 CUDA 编程模型**，而是现代 NVIDIA GPU 上重要的实现细节。若 warp 内线程指令不一致（如 `if` 分支两侧都执行），则发生**分支发散（divergent execution）**，性能受损。

- **现实类比**：warp 像一列 32 节车厢的火车，共用一个火车头（取指/译码单元）；只要所有车厢目的地一致（同一条指令），火车头就能一次把整列车拖过去。哪节车厢想"变道"（走不同分支），整列车就得分成两趟跑。
- **公式/图示**：`一个 256 线程的 block → 8 个 warp（256 / 32） V100 一个 sub-core：最多可调度/交织 16 个 warp warp 指令执行：16 个 fp32 ALU 跑 32 线程的指令 → 需要 2 个时钟`

7. Shared Memory（共享内存，`__shared__`）

- **定义**：kernel 内用 `__shared__` 声明的、**按 block 分配**的片上高速存储（V100 上 shared + L1 合计 128 KB/SM）。块内所有线程可读写；块与块之间不共享。用途：把 global memory 中会被多次复用的数据一次性搬进片上，避免反复访问 DRAM。
- **现实类比**：做菜时把冰箱（DRAM）里要用的所有食材一次性搬到厨房台面（shared memory）上，后面每个步骤都在台面上取料，而不是每用一次就跑一趟冰箱。
- **公式/图示**：`__shared__ float support[THREADS_PER_BLK + 2];` —— 每个 block 一份，块内线程共享。

8. `__syncthreads()`（块内屏障）与原子操作（atomic）

- **定义**：`__syncthreads()` 是 block 内所有线程必须到达的**屏障（barrier）**：谁先到谁等待，全部到齐才继续。CUDA 还提供**原子操作**（如 `atomicAdd(float* addr, float amount)`），可作用于 global 和 shared 地址；另有 host/device 同步（kernel 返回时隐含对所有线程的屏障）。
- **现实类比**：接力赛的"交接区"——所有队员必须都到达交接区，下一棒才能一起出发。少一个队员（比如某些线程没调用 `__syncthreads()`），全队卡死。
- **公式/图示**：`线程 0: 写 shared ─┐ 线程 1: 写 shared ─┼→ __syncthreads() → 所有线程才允许读 shared 线程 2: 写 shared ─┘`

9. SM（Streaming Multiprocessor）与 sub-core

- **定义**：SM 是 GPU 上的一个"多线程 SIMD 核"，V100 芯片上有 80 个 SM；每个 SM 由 4 个 sub-core 组成，每个 sub-core 有自己的 warp selector、取指/译码单元和一组 SIMD 功能单元（16 个 fp32 mul-add、16 个 int、8 个 fp64、load/store 单元、tensor core 单元）。寄存器堆共 256 KB/SM（每 sub-core 64 KB），分给最多 64 个 warp。
- **现实类比**：SM 是一家"车间"，sub-core 是车间里的 4 条"装配线"，每条装配线同一时刻只伺候一个 warp，但可以在 16 个 warp 之间快速切换（交织执行）来隐藏延迟。
- **公式/图示**：V100 关键数字：`80 SMs × 4 sub-cores × 16 fp32 ALUs = 5,120 fp32 mul-add ALUs = 12.7 TFLOPs （mul-add 计为 2 flops） 最多 80 × 64 = 5,120 个并发 warp = 163,840 个并发 CUDA 线程/芯片 L2 cache 6 MB；HBM 16 GB，带宽 900 GB/sec（4096-bit 接口）`

10. 线程块调度器（Thread Block Scheduler）与工作分配

- **定义**：CUDA 的**核心假设**是：thread block 之间**没有依赖**，可以按**任意顺序**执行。GPU 上的硬件工作调度器（work scheduler）把 block（"工作"）按动态调度策略映射到 SM 上，只要满足资源约束（每 block 需要的线程上下文数、shared memory 字节数）。block 一旦完成，其资源（shared memory、warp 上下文）立即释放给下一个 block。注意 CUDA 程序里**没有 `num_cores` 这个概念**——同一个 kernel 可以不加修改地跑在 6 核或 16 核的 GPU 上（类似数据并行模型里的 forall）。
- **现实类比**：公司接了一批订单（block），调度台按各产线的空闲情况随时派单，先完成先释放产线；派单员不关心订单之间的先后（因为订单之间本来就独立）。
- **公式/图示**：`Grid（1000 个 block）→ GPU Work Scheduler → Core 0 / Core 1（fictitious 双核 GPU） Step 1: host 发送 kernel 启动命令（EXECUTE convolve, NUM_BLOCKS=1000） Step 2: 调度器把 block 0 映射到 core 0（预留 128 线程上下文 + 520B shared） Step 3: 继续映射 block 1、2、3……（交错映射） 核心容量：每 core 只能驻留 2 个 block（3 × 520B > 1.5KB shared） Step 4: block 0 完成 → 资源释放 Step 5: block 4 调度到 core 0 ……（依次类推，共 1000 个）`

11. 内存带宽（Memory Bandwidth）

- **定义**：GPU 从 DRAM 读写数据的速率。CPU 时代内存带宽约 150–300 GB/sec（DDR5），高端 GPU 约 900 GB/sec–1 TB/sec（HBM，4096-bit 接口）。带宽是 GPU 程序最重要的性能天花板之一：shared memory 之所以快，是因为它是**片上存

储，不走 DRAM 带宽。

- **现实类比**：带宽像"港口吞吐量"——每秒能从仓库（DRAM）运多少货到车间。计算再快，货（数据）运不进来也是白搭（这就是后续讲的 bandwidth-bound）。
- **公式/图示**：`带宽 = 总线宽度 × 时钟频率 × 每时钟传输次数`（如 4096-bit × 1.4 GHz × 2 ≈ 900+ GB/s）。

12. GPGPU（General-Purpose computation on GPU）与 CUDA 的诞生

- **定义**：2002–2003 年研究者用"hack"方式把 GPU 当作数据并行机器用：把输出图像大小设成数组大小（如 512×512），画两个恰好盖满屏幕的三角形，让 fragment shader 对每个像素执行一次计算——shader 函数被 map 到 512×512 个元素上。2004 年斯坦福图形学实验室的 **Brook** 语言把 GPU 抽象成流处理器（stream + kernel）。2007 年 NVIDIA Tesla 架构提供首个非图形专用的 **compute mode** 接口：分配 buffer、上传 kernel 二进制、`launch(myKernel, N)` 以 SPMD 方式运行 N 个实例——这比图形接口 `drawPrimitives()` 简单得多，CUDA 由此诞生。
- **现实类比**：GPU 本来是"只会画画的专用打印机"，GPGPU 时代人们发现"只要把计算画成图像"就能让它算任何东西；CUDA 则干脆给打印机装上了"通用打印"按钮。
- **公式/图示**：`hack 方法： 图像 512×512 ↔ 数组 512×512 fragment shader（纯函数，跑在每个像素上） ↔ 对每个数组元素执行 f`

二、代码示例与详细解说

示例 1：`matrixAdd`——2D block/grid 配置与 `<<<>>>` 启动

```
// matrixAdd.cu —— 编译：nvcc -o matrixAdd matrixAdd.cu（需 NVIDIA GPU 与 CUDA Toolkit）
#include <cuda_runtime.h>
#include <cstdio>

const int Nx = 12;
const int Ny = 6;

// ============ device 代码：kernel 定义（运行在 GPU 上） ============
__global__ void matrixAdd(float A[Ny][Nx], float B[Ny][Nx], float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // 列坐标（全局）
    int j = blockIdx.y * blockDim.y + threadIdx.y;   // 行坐标（全局）
    C[j][i] = A[j][i] + B[j][i];
}

// ============ host 代码：串行执行在 CPU 上 ============
int main()
{
    float *A, *B, *C;   // host 指针
    //  （完整程序还需 cudaMalloc 设备内存并用 cudaMemcpy 拷贝数据，见示例 2 说明）
    dim3 threadsPerBlock(4, 3);                      // 每个 block 4×3 = 12 个线程
    dim3 numBlocks(Nx / threadsPerBlock.x,           // 网格 3×2 = 6 个 block
                   Ny / threadsPerBlock.y);
    // 这次启动共创建 72 个 CUDA 线程：6 个 block，每个 12 个线程
    matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
    cudaDeviceSynchronize();                         // host/device 同步
    return 0;
}
```

【代码做了什么？】 – host 端用 `dim3` 声明 `threadsPerBlock(4,3)` 与 `numBlocks(3,2)`（`Nx/threadsPerBlock.x = 12/4 = 3`，`Ny/threadsPerBlock.y = 6/3 = 2`）。 – `matrixAdd<<<numBlocks, threadsPerBlock>>>(A,B,C)` 是一次 **bulk launch**：一次性创建 72 个 CUDA 线程（6 block × 12 线程），每个线程独立执行 kernel 函数体。 – kernel 内每个线程用公式 `i = blockIdx.x * blockDim.x + threadIdx.x`、`j = blockIdx.y * blockDim.y + threadIdx.y` 计算自己在 12×6 矩阵中的全局坐标，然后执行 `C[j][i] = A[j][i] + B[j][i]`。 – 该调用是**同步语义**：host 端调用返回时所有线程都已终止（隐含全线程屏障）。

【并行机制解说】 – 线程如何创建：不是像 `pthread_create` 那样逐个创建（那要分配栈、OS 控制块），而是**一次启动一整网格**的线程，成本极低。 – 工作如何分配：程序里**没有** `num_cores`——block 是"工作单元"，由 GPU 硬件调度器动态映射到任意数量的 SM 上，因此同样的程序能跑在 6 核或 16 核 GPU 上（对应概念：thread block scheduler、数据并行 forall 精神）。 – 数据如何共享：此例无共享，三个数组都在 global memory。 – 同步点：kernel 返回时的隐式屏障；本 kernel 块间无依赖，调度顺序任意。 – 对应概念：**thread/block/grid 层级、kernel 启动语法、host/device 分离**。

示例 2：1D 卷积 v1——每输出元素一个线程（朴素版）

```
// convolve_v1.cu —— 编译：nvcc -o convolve_v1 convolve_v1.cu
#include <cuda_runtime.h>

#define THREADS_PER_BLK 128

// kernel：对长度为 N 的输入做 3 点滑动平均：output[i] = (input[i]+input[i+1]+input[i+2]) / 3
__global__ void convolve(int N, float* input, float* output)
{
    int index = blockIdx.x * blockDim.x + threadIdx.x;  // 线程局部变量：全局线程号
    float result = 0.0f;                                // 线程局部变量：累加器
    for (int i = 0; i < 3; i++)
        result += input[index + i];                     // 3 次 global memory 读
    output[index] = result / 3.f;                       // 1 次 global memory 写
}

// host 代码
int main()
{
    int N = 1024 * 1024;                    // 约 100 万个输出元素
    float *devInput, *devOutput;
    cudaMalloc(&devInput,  sizeof(float) * (N + 2));  // 输入数组（多 2 个边界元素）
    cudaMalloc(&devOutput, sizeof(float) * N);        // 输出数组
    //  （此处应使用 cudaMemcpy(devInput, hostInput, ..., cudaMemcpyHostToDevice)
    //    初始化 devInput 的内容，代码从略）
    // 启动 N/128 = 8192 个 block，共 1M 个线程，每个线程算 1 个输出元素
    convolve<<<N / THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
    cudaDeviceSynchronize();
    return 0;
}
```

【代码做了什么？】 – 每个 CUDA 线程负责**一个输出元素**：先由 `index` 定位自己是第几个输出，然后串行执行 3 次加法累加 `input[index..index+2]`，最后写回 `output[index]`。 – 这是典型的 "one thread per output element"（每个输出元素一个线程）的数据并行分解：1M 个输出 → 1M 个线程 → 8192 个 block。 – 注意 `input` 在 host 端无法直接访问：`devInput` 是设备地址空间指针，host 只能通过 `cudaMemcpy` 与之交互（对应概念：分布式地址空间）。

【并行机制解说】 – 工作分配：每个线程独立处理一个输出，线程之间**零通信、零同步**——这是最容易并行化的模式，GPU 上数万并发线程轻而易举。 – 效率问题（本讲的伏笔）：每个线程需要 3 次 global load（`input[index]`、`[index+1]`、`[index+2]`），且相邻线程的访问窗口**互相重叠**——相邻线程会重复读取同一个输入元素。整个 kernel 共执行 3×128 次 load per block，而输入数据总量只有 130 个元素/block。这是巨大的带宽浪费，引出示例 3。 – 对应概念：**memory bandwidth 意识、数据并行分解（one thread per element）**。

示例 3：1D 卷积 v2——用 shared memory 暂存输入（本讲重点）

```
// convolve_v2.cu —— 编译：nvcc -o convolve_v2 convolve_v2.cu
#include <cuda_runtime.h>

#define THREADS_PER_BLK 128

// kernel：与 v1 相同功能，但先把 block 需要的输入区间搬进 shared memory
__global__ void convolve(int N, float* input, float* output)
{
    __shared__ float support[THREADS_PER_BLK + 2];  // 每 block 分配：128 个 + 2 个 halo
    int index = blockIdx.x * blockDim.x + threadIdx.x;  // 线程局部变量

    // 1) 所有线程协作把本 block 的"支撑区"从 global 载入 shared memory
    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {                              // 前 2 个线程额外加载 2 个 halo 元素
        support[THREADS_PER_BLK + threadIdx.x] =
            input[index + THREADS_PER_BLK];
    }

    __syncthreads();                                    // 2) 块内屏障：全部载入完成后才继续

    float result = 0.0f;                                // 线程局部变量
    for (int i = 0; i < 3; i++)
        result += support[threadIdx.x + i];             // 3) 从 shared memory 读，不再访问 global

    output[index] = result / 3.f;                       // 4) 写结果到 global memory
}

// host 代码与 v1 完全相同：
//   cudaMalloc 分配 devInput(N+2)、devOutput(N)；
//   convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

【代码做了什么？】 – 步骤 1：块内 128 个线程**协作加载**：每个线程把 `input[index]` 写入 `support[threadIdx.x]`；前 2 个线程（`threadIdx.x < 2`）再额外把 `input[index + THREADS_PER_BLK]` 写入 `support[128 + threadIdx.x]`（这是本 block 需要的 2 个跨块边界元素，即 halo）。总共执行 **130 条 load 指令**，而不是 v1 的 3×128 = 384 条。 – 步骤 2：`__syncthreads()` 保证所有 130 个数据都写进 shared memory 之后，任何线程才允许读取。 – 步骤 3：每个线程从 shared memory 读 3 个元素做累加（shared 是片上高速存储，比 DRAM 快得多）。 – 步骤 4：把结果写回 global memory。

【并行机制解说】 – 数据如何共享：`support` 是 **per-block shared memory**——所有线程用它交换数据，这正是"共享地址空间"编程风格（block 内）。 – 同步点：`__syncthreads()` 是**唯一的块内屏障**。它把"写入 shared"与"读取 shared"两个阶段隔开，防止线程 A 读到线程 B 尚未写入的旧数据。注意它只同步**本 block** 的线程，不同 block 之间没有任何同步语义（对应概念：`__syncthreads`、block 独立性）。 – 为什么省带宽：输入数据被**复用**了 3 次（相邻输出窗口重叠），v1 每次复用都走 DRAM，v2 只在开始时走一次 DRAM、之后全部命中片上 shared memory。这是"用 shared memory 显式管理局部性"的经典案例（对应概念：shared memory、memory bandwidth）。 – 运行在 V100 上：128 线程的 block 由 4 个 warp 执行（128/32）；由于块内线程必须"同时在场"（见陷阱 4），4 个 warp 必须同时驻留在同一 SM 上。 – 对应概念：**shared memory**、**`__syncthreads`**、**SIMT/warp（4 warps/block）**。

示例 4：用 shared memory 的树形归约（reduction）内核

```
// reduce.cu —— 编译：nvcc -o reduce reduce.cu
// 目标：把长度为 N 的数组 input 归约为一个总和（每个 block 产出一个部分和）
#include <cuda_runtime.h>

#define THREADS_PER_BLK 256
#define NUM_BLOCKS      64

__global__ void reduce(float* input, float* partial, int N)
{
    __shared__ float sdata[THREADS_PER_BLK];   // 每 block 一块共享累加区
    int tid = threadIdx.x;
    int i   = blockIdx.x * blockDim.x + tid;   // 该线程负责的全局元素下标

    // 1) 每线程把 global 中的一个元素装入 shared memory（越界元素当 0 处理）
    sdata[tid] = (i < N) ? input[i] : 0.f;
    __syncthreads();                           // 确保全部装入

    // 2) 树形归约：每轮参与线程数减半
    for (int s = THREADS_PER_BLK / 2; s > 0; s >>= 1) {
        if (tid < s)
            sdata[tid] += sdata[tid + s];      // 前半线程把后半的累加进来
        __syncthreads();                       // 防止读到自己还没被更新的数据
    }

    // 3) 每个 block 的 0 号线程把部分和写回 global
    if (tid == 0)
        partial[blockIdx.x] = sdata[0];
}

// host 端：把 NUM_BLOCKS 个部分和再相加（可再启动一个小 kernel，或拷回 CPU 求和）
int main()
{
    int N = NUM_BLOCKS * THREADS_PER_BLK;      // 64 × 256 = 16,384 个元素
    float *devInput, *devPartial;
    cudaMalloc(&devInput,    sizeof(float) * N);
    cudaMalloc(&devPartial,  sizeof(float) * NUM_BLOCKS);
    //  （初始化 devInput 后……）
    reduce<<<NUM_BLOCKS, THREADS_PER_BLK>>>(devInput, devPartial, N);
    //  （把 devPartial 拷回 host 求和，或再启动一个 reduce kernel 处理 64 个部分和）
    return 0;
}
```

【代码做了什么？】 – 步骤 1：256 个线程各装一个元素到 `sdata`，`__syncthreads()` 保证装载完成。 – 步骤 2：`for (int s = 128; s > 0; s >>= 1)` 执行 8 轮：第 1 轮线程 0–127 把 `sdata[0..127] += sdata[128..255]`，第 2 轮线程 0–63 把 `sdata[0..63] += sdata[64..127]`……每轮参与线程减半，8 轮后 `sdata[0]` 就是本 block 256 个元素的和。这是 $O(\log_2 256) = 8$ 步的树形归约。 – 步骤 3：每个 block 由 0 号线程把部分和写入 `partial[blockIdx.x]`。 – host 端最后再把 64 个部分和加起来（或再启动一次归约 kernel）。

【并行机制解说】 – 工作分配：每个 block 负责一块连续数据（256 个元素），block 之间完全独立（符合 CUDA 块可任意调度假设）。 – 数据共享与同步：`sdata` 是 shared memory；每轮加法后必须 `__syncthreads()`——否则线程 tid 可能读到相邻线程还没写完的旧值。**归约是"写后读"依赖最密集的 kernel 之一，是理解 `__syncthreads` 的最佳练习**。 – 性能观察：每轮有一半线程闲置（`tid >= s` 的线程直接跳到屏障）——这是朴素树归约的固有浪费，也是后续优化的方向（如 warp shuffle）。它还展示了"block 内 SPMD 协作"：256 个线程不是各自为战，而是作为一个整体协同计算（对应概念：block 内协作、shared memory、`__syncthreads`）。 – 注意：本 kernel 是教学版，实际还有"每线程加载多个元素（grid-stride loop）"等改进；它没有用原子操作，跨 block 的合并靠 host 完成。 – 对应概念：**block 内 SPMD 协作、shared memory 归约、barrier**。

示例 5（进阶/bonus）：persistent threads 编程风格

```
// persistent.cu —— 编译：nvcc -o persistent persistent.cu
// 思路：启动恰好"填满 GPU"的 block 数，让每个线程在 while 循环里反复取工作，
//       完全绕过 GPU 的线程块调度器，由应用自己管理工作分配。
#define THREADS_PER_BLK 128
#define BLOCKS_PER_CHIP 80 * (32 * 64 / 128)   // 针对 V100：80 SM × 2048 线程/SM ÷ 128

__device__ int workCounter = 0;                // global memory 中的工作计数器

__global__ void convolve(int N, float* input, float* output)
{
    __shared__ int startingIndex;              // 本 block 本轮起始下标（shared）
    __shared__ float support[THREADS_PER_BLK + 2];

    while (1) {
        if (threadIdx.x == 0)                  // 0 号线程原子地领取一段工作
            startingIndex = atomicInc(&workCounter, THREADS_PER_BLK);
        __syncthreads();                       // 广播给整个 block
        if (startingIndex >= N) break;         // 没有更多工作了

        int index = startingIndex + threadIdx.x;
        support[threadIdx.x] = input[index];
        if (threadIdx.x < 2)
            support[THREADS_PER_BLK + threadIdx.x] =
                input[index + THREADS_PER_BLK];
        __syncthreads();

        float result = 0.0f;
        for (int i = 0; i < 3; i++)
            result += support[threadIdx.x + i];
        output[index] = result;

        __syncthreads();                       // 防止下一轮覆盖 support 时还有人没读完
    }
}

// host：只启动 BLOCKS_PER_CHIP 个 block（不再 N/128 个）
// convolve<<<BLOCKS_PER_CHIP, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

【代码做了什么？】 – host 只启动恰好填满 GPU 的 block 数（V100 上 80 SM × 2048 线程/SM ÷ 128 = 1280 个 block）。 – 每个 block 的 0 号线程用 `atomicInc` 从全局计数器领取一段起始下标，`__syncthreads()` 广播后，整个 block 处理这 128 个输出；然后回到 while 循环领下一段，直到 `startingIndex >= N`。 – 工作分配从"硬件调度器"转移到"应用自身"——程序员的心理模型变成"所有 CUDA 线程同时跑在 GPU 上"。

【并行机制解说】 – 这是 CUDA 的**反模式/特例**：它要求程序员知道底层 GPU 的核数与每核容量（`BLOCKS_PER_CHIP` 写死了 V100 参数），并**假设 GPU 确实会让所有 block 并发执行**（"Ugg!"——幻灯片原话）。一旦换 GPU 或该假设不成立，程序性能甚至正确性都会出问题。 – 但它清楚地展示了两个概念：① block 内 `__syncthreads()` 充当"小循环内屏障"，把"领取工作→处理→再领取"串成流水；② 原子操作 `atomicInc` 是跨 block 共享 global 变量的唯一同步手段（对应概念：atomic、shared memory、block 调度）。 – 与"线程池"类比：就像 web server 启动时创建固定数量线程等待请求，而不是每来一个请求建一个线程——thread pool 的线程数是核数的函数，不是请求数的函数。

三、关键要点

1. **CUDA 是"批量启动 + 两级层级"的数据并行编程**：一次 kernel launch 创建成千上万个线程；问题分解成"block 集合"（网格），block 之间被假设**无依赖、可按任意顺序调度**（与 ISPC task 极其相似）；block 内部则是**共享地址空间的 SPMD 编程**（与 ISPC gang 相似）。没有 `num_cores`，程序天然可移植到不同规模 GPU。
2. **warp/SIMT 是 CUDA 最重要的实现细节，但不是编程模型的一部分**：32 个线程一个 warp，硬件运行时动态检查 warp 内指令是否一致，一致则用 SIMD ALU 一起执行；不一致则发散执行、性能受损。这与 ISPC 的"编译期生成 SIMD 指令"根本不同——

CUDA 程序不会被编译成 SIMD 指令。

3. **三种 device 地址空间 = 三种局部性**：private（寄存器）/ shared（片上）/ global（DRAM）。用 shared memory 显式管理复用（如 1D 卷积把 384 次 global load 降到 130 次）是把 DRAM 带宽留给真正需要它的数据的关键。
4. **block 内线程必须"同时在场"**：因为 `__syncthreads()` 等机制允许块内线程互相依赖，系统不能先把 128 个线程跑完再跑后 128 个——block 开始时所有线程的寄存器上下文就必须全部分配好（这是对调度的硬约束，也是 block 大小受硬件上下文容量限制的原因）。
5. **同步工具箱只有三样**：`__syncthreads()`（块内屏障）、原子操作（global/shared）、kernel 返回的隐式全线程屏障（host/device 同步）。跨 block 的细粒度同步（如 spin-wait 等待别的 block）在 CUDA 里是**危险甚至非法**的，因为没有任何保证两个 block 会并发运行。

四、常见陷阱与注意事项

1. **block 内线程数超过硬件上限**：CUDA 限制每 block 最多 1024 个线程（V100 及多数现代 GPU）；shared memory 也是有限资源（V100 每 SM 128 KB shared+L1），block 占用过多资源会降低每 SM 可驻留 block 数，甚至无法启动（参见示例 3 的 fictitious core 只能放 2 个 block）。设计 block 大小时要同时考虑线程数、寄存器数、shared 字节数三个维度。
2. **忘记 `__syncthreads()`（或放错位置）**：shared memory 的"写后读"依赖必须用屏障隔开。漏掉屏障是 data race（读旧值）；把屏障放在条件分支里则可能导致**死锁**——因为屏障要求块内**所有**线程都到达，若只有部分线程执行了它，其余线程永远等不到。
3. **shared memory 溢出 / 数组越界**：`support[THREADS_PER_BLK + 2]` 这类"带 halo"的数组，越界写会悄悄破坏相邻 block 的数据（shared 是物理上连续的），且没有运行时错误提示——这是最难调的 bug 之一。
4. **warp 发散（divergent execution）**：`if (threadIdx.x < 2)` 这种分支会让 warp 内部分线程走 A 路径、部分走 B 路径，两条路径串行执行（掩码执行）。少量发散可接受，但大量发散（如按奇偶分叉）会让 SIMT 效率腰斩。设计时尽量让分支与 warp 边界对齐（如 `if (warp_id == 0)`）。
5. **把 block 当 pthread 用**：以为 block 之间可以像线程那样靠共享变量同步。记住 CUDA 只保证 block **可**任意顺序调度——两个 block 用 spin-wait 互相等待（幻灯片中的 `while(atomicAdd(&myFlag,0)==0){}` 例子）在"每 SM 只驻留一个 block"的 GPU 上会**永久死锁**。跨 block 协作应通过 kernel 边界（启动多个 kernel）或原子操作完成。
6. **host 端直接解引用 device 指针**：`devInput[i]` 在 host 代码里是非法的（不同地址空间），必须 `cudaMemcpy`。反之亦然。
7. **默认 launch 是同步的**：host 调用 kernel 会阻塞直到 kernel 完成（CUDA 里其实是"默认串行流"语义），需要 host/device 并发时要用 streams——本讲没展开，但要知道 kernel 返回时刻有一个隐式全线程屏障。

五、思考题（带答案）

Q1：CUDA 是数据并行模型、共享地址空间模型，还是消息传递模型？请分别从"块之间"与"块内部"两个视角回答，并与 ISPC 的 gang/task 类比。

A1：答案在幻灯片总结里非常明确，是三者的"混合体"： – **块之间（grid 层）**：数据并行模型——问题被划分为独立 block，系统把它们调度到任意数量的核上（无 `num_cores`，类似 forall），block 间无依赖；这与 ISPC 的 **task** 几乎一模一样（task 也可按任意顺序调度）。 – **块内部（block 层）**：共享地址空间模型的 SPMD 编程——线程并发运行、通过 shared memory 变量通信、用 `__syncthreads()` 同步；这与 ISPC 的 **gang** 类似，但 warp 不是编译期 SIMD（ISPC gang 编译成 SIMD 指令），而是硬件运行时动态检测的 SIMT。 – **host/device 之间**：分布式地址空间——两个地址空间用 `cudaMemcpy` 搬数据，这又像消息传递模型。 – 所以：CUDA 一次编程同时用到了数据并行（块间）、共享内存（块内）、消息传递（host↔device）三种模型的元素——这正是幻灯片反复提醒"遇到任何并行编程系统先问它的语义是什么"的原因。

Q2：为什么 CUDA 必须为 block 内所有线程预先分配执行上下文（寄存器），而不能像 CPU 那样"先把 0–127 线程跑完再跑 128–255 线程"？

A2：因为 CUDA 允许（也鼓励）块内线程之间产生依赖——最简单的例子就是 `__syncthreads()`：如果线程 0–127 先跑完并越过了屏障，而 128–255 还没开始，那么"屏障"就名存实亡（前面的人不等后面的人）。更一般地，任何 shared memory 通信都假设双

方同时存在。因此 CUDA 的语义是：**block 开始执行时，块内所有线程都已存在且拥有寄存器状态**；如果某线程可运行，它最终一定会被运行（不会死锁）。这给调度器套上了紧箍咒：一个 block 只能整体驻留在一个 SM 上，且占用的上下文数量受 SM 寄存器/线程槽容量限制——这也是为什么 block 不能无限大。

Q3：假设你要在 GPU 上统计一个数组的直方图（值域 0–9）。为什么 `atomicAdd(&counts[A[i]], 1)`（counts 在 global memory）是合法 CUDA 代码？它违反"块间无依赖"的假设吗？

A3：合法，且**不违反**块间无依赖假设。原子操作提供的是**互斥**（mutual exclusion）而不是**顺序依赖**——无论 block 以什么顺序、什么交错方式执行，每个 `atomicAdd` 都只做"读–加–写"的不可分割单元，最终 `counts` 的内容与执行顺序无关（都是把数组 A 中每个值出现次数统计一遍）。调度器仍然可以任意顺序调度 block。真正的麻烦是幻灯片中 block 0/block 1 用 `while(atomicAdd(&myFlag,0)==0){}` 互相 spin-wait 的代码：那是在要求**特定的执行顺序**（必须先跑 block 0），而 CUDA 不提供这种保证——在只放得下一个 block 的 GPU 上直接死锁。记住区分：**原子操作 = 互斥（安全），顺序假设 = 依赖（危险）**。

Q4（进阶）：为什么 warp 的指令流是"标量"指令，但执行效率却接近 SIMD？这和 ISPC 的 gang 有何区别？

A4：GPU 的每个硬件线程（warp 里的一个 lane）执行的是**只有标量指令**的指令流；硬件在取指时发现同一 warp 的 32 个线程正在执行同一条指令，就让 16 个 SIMD ALU 分两个时钟把它执行完（掩码掉不发散的 lane）。所以 SIMD 是**运行时动态合并**出来的，而不是编译期静态生成的。ISPC 则是编译器把 gang 内实例的操作**静态编译**成 SIMD 指令。区别的意义：CUDA 程序里写任意控制流（`if`、循环、函数调用）都不会"编译失败"，代价是发散时性能下降；而 ISPC 对 gang 内控制流的 SIMD 化是编译器的事。这也是为什么 CUDA 线程比 ISPC 实例"更自由"但需要程序员自觉保持 warp 一致性。

Lecture 8: 数据并行思维（Data-Parallel Thinking）（日期：Oct 16, 2025）

> **概述**：前几讲我们习惯从"worker 做什么、如何把工作分配给 worker"的角度思考并行编程；本讲换一个视角：**把算法描述成对数据序列（sequence）的操作**——map、filter、fold/reduce、scan/segmented scan、sort、groupBy、gather/scatter 等。核心思想是：这些操作的**高性能并行实现已经存在**，因此用这些原语写的程序往往能高效跑在并行机器上（前提：不要被内存带宽卡住）。本讲还会深入 scan（前缀和）的并行算法（O(N log N) 朴素算法 vs O(N) 工作高效算法）及其多级实现，并用稀疏矩阵乘法、粒子网格、直方图等实例展示如何把"不规则并行"转化为"规则并行"、把"细粒度同步"转化为"粗粒度同步"。

注意：本讲与 **Assignment 3** 直接相关——幻灯片明确说 Assignment 3 提供了与本讲 `scan_block` 类似的 CUDA scan 代码；此外本讲的数据并行思维（map/reduce/scan）也是后续 Assignment 5（最快速 CUDA kernel）的思维基础。

一、核心概念与定义

1. 数据并行模型（Data-parallel model）与序列（Sequence）

- **定义**：把计算组织成**对元素序列的操作**（例如：对序列的所有元素执行同一个函数）。序列（sequence）是有序的元素集合（C++ 的 `Sequence<T>`、Scala 的 `List[T]`、Pandas Dataframe、PyTorch/JAX Tensor、Haskell 的 `seq T`）。**关键**：与数组不同，程序只能通过特定操作访问序列元素，不能直接按下标访问——这给了实现方重排/并行化的自由。现代著名例子：NumPy 的 `C = A + B`。
- **现实类比**：序列像一条流水线上的"待加工件队列"，工人（worker）只允许通过规定的操作（加工台）触碰工件，不允许伸手到队列中间乱拿——这样流水线才能自由调度。
- **公式/图示**：`C = A + B`（三个等长向量逐元素相加）就是一次 map 级联。

2. Map（映射）

- **定义**：高阶函数（以函数为参数的函数）。把**无副作用**（side-effect free）的一元函数 `f :: a -> b` 应用到输入序列的每个元素上，产生**等长**的输出序列。Haskell：`map :: (a -> b) -> seq a -> seq b`；C++：`std::transform`；JAX：`vmap`。

- **现实类比**：给一摞文件（输入序列）每份盖章（函数 f），得到一摞盖好章的文件（输出序列）——每份文件互不影响，盖章顺序随便。
- **公式/图示**：`a = [3, 8, 4, 6, 3, 9, 2, 8] f(x) = x + 10 b = map f a b = [13, 18, 14, 16, 13, 19, 12, 18] ← 每个元素独立应用 f，顺序可任意`

3. Filter（过滤）

- **定义**：删除序列中不满足谓词（predicate）的元素。输出是输入的子序列（长度 ≤ 输入）。
- **现实类比**：安检——只放行"合格"的行李，不合格的留在外面；通过顺序无关紧要。
- **公式/图示**：`filter f s`，其中 `f :: a -> Bool`。例：过滤掉奇数 → `[3,8,4,6,3,9,2,8]` → `[8,4,6,2,8]`。

4. Fold / Reduce（折叠 / 归约）

- **定义**：把二元操作 `f :: (b,a) -> b` 应用到每个元素和一个累加值上，最终把整个序列"折叠"成一个值。fold left：`fold :: b -> ((b,a) -> b) -> seq a -> b`，初始值（seed）类型为 b。**并行 fold** 还需要一个额外的二元 **combiner** 函数 `comb :: (b,b) -> b`（把子结果合并）；如果 `f :: (b,b) -> b` 本身就是**结合律**二元操作，就不需要 combiner。初始值必须是 f 和 comb 的**单位元（identity）**。
- **现实类比**：合唱团报数——每个人把"前一个人报的数 + 自己的数"传下去（串行 fold）；并行版则是每个小组先各自求和，组长再把各组的和加起来（combiner）。
- **公式/图示**：`串行 fold: fold 10 (+) [3,8,4,6,3,9,2,8] = ((((((10+3)+8)+4)+6)+3)+9)+2)+8 = 53 并行 fold: 把序列分成若干段，每段并行 fold，再用 comb 合并各段结果: sum = comb(comb(seg0, seg1), comb(seg2, seg3))`

5. Scan / 前缀和（Prefix Sum）

- **定义**：给定结合律二元操作 `⊕`，**inclusive scan** 输出 `[a0, a0⊕a1, a0⊕a1⊕a2, ...]`，即每个输出元素是"从开头到当前位置（含）的所有元素的累计结果"；**exclusive scan** 输出 `[I, a0, a0⊕a1, ...]`（不含当前元素，第一个元素是单位元 I）。当 `⊕ = +` 时称为 **prefix sum（前缀和）**。
- **现实类比**：电影院排队——每个人想知道"我前面有多少人"。如果每个人只问前面那个人"你前面有多少人"，队伍排多长就得等多久（串行 scan）；并行 scan 让一小群人各自数完自己的小组后，再用"前面所有小组的总人数"一次性修正每个人的号（工作高效并行 scan）。
- **公式/图示**：`in = [3, 8, 4, 6, 3, 9, 2, 8] scan_inclusive(+) = [3, 11, 15, 21, 24, 33, 35, 43] scan_exclusive(+) = [0, 3, 11, 15, 21, 24, 33, 35]`

6. 并行 scan 的两个算法：Work 与 Span

- **定义**：**Work** = 算法执行的总操作数；**Span** = 最长串行依赖链长度（关键路径）。① 朴素并行 scan（Hillis–Steele 风格，每步跨距翻倍）：**Work = O(N log N)**（比串行算法还低效！），**Span = O(log N)**。② **工作高效（work-efficient）scan**（Blelloch 算法）：**up-sweep（自底向上建树）+ down-sweep（自顶向下修正）** 两阶段，**Work = O(N)**，**Span = O(log N)**。幻灯片特别提醒：要注意常数因子（"but what is the constant?"）。
- **现实类比**：Work 像"总工时"，Span 像"最短工期"。一个算法可以工时很高但工期很短（全员加班并行），也可以工时最优但工期较长。GPU 上有时故意选"低效"算法，因为 SIMD 利用率更高（见下）。
- **公式/图示**（Blelloch up-sweep + down-sweep 骨架，`⊕ = +`）：`Up-sweep（自底向上，d 从 0 到 log2n−1）: forall k 步长为 2^(d+1): a[k + 2^(d+1) − 1] = a[k + 2^d − 1] + a[k + 2^(d+1) − 1] Down-sweep（自顶向下，d 从 log2n−1 到 0）: x[n−1] = 0 forall k 步长为 2^(d+1): tmp = a[k + 2^d − 1] a[k + 2^d − 1] = a[k + 2^(d+1) − 1] a[k + 2^(d+1) − 1] = tmp + a[k + 2^(d+1) − 1]`
- **补充：两处理器（共享内存）实现——"分而治之 + 加基数"**：幻灯片第 18–19 页展示了一个更贴近真实机器的做法：P1 串行 scan 前半段 `[a0..a7]`，P2 串行 scan 后半段 `[a8..a15]`（两段完全独立，可并行）；然后 P2 把前半段的总和 `base = a0−7` 加到后半段的每个元素上。分析：`P1: 串行 scan a0..a7 （8 次加法） P2: 串行 scan a8..a15 （8 次加法，与 P1 并行） P2: 后半段每个元素加 base （8 次加法，只需 base 一个通信值） Work = O(N)，常数只有 1.5（2N 次加法中的一半与另一半并行 + N/2 次修正） 数据访问：空间局部性极好（连续内存）；P2 读 base 的开销在大规模 NUMA 系统上可能更贵，但小型多核系统上几乎可忽略` 这个例子说

明：**scan 的"最优"实现强烈依赖目标机器的规模与内存层级**——两核机器用"串行一半 + 修正一半"；GPU 用"warp 内 SIMD scan + 块内协作"；大规模并行机器才值得用完整的两阶段树形算法。

7. Segmented Scan（分段扫描）

- **定义**：scan 的推广：对输入序列的**连续分区**同时各自做 scan（例如对 `[[1,2],[6],[1,2,3,4]]` 做分段 exclusive scan 得 `[[0,1],[0],[0,1,3,6]]`）。常用 **start-flag 表示法**：一个 flag 序列标记每个分段的起点，与数据序列平行存放。
- **现实类比**：一排人分成几组，每组内部各自报数——"组内前面的人总数"，而不是所有人混在一起报数。
- **公式/图示**：`嵌套序列 A = [[1,2,3],[4,5,6,7,8]] flag: 1 0 0 | 1 0 0 0 0 data: 1 2 3 | 4 5 6 7 8`

8. Gather / Scatter（聚集 / 散开）

- **定义**：**gather(index, input, output)**：`output[i] = input[index[i]]`（按索引序列取数据，可以并行、无冲突）；**scatter(index, input, output)**：`output[index[i]] = input[i]`（把数据按索引序列放到目标位置，可能冲突）。硬件支持：AVX2（2013）支持 SIMD gather 但不支持 scatter；AVX-512 有 scatter 指令；GPU 硬件支持 gather/scatter——但都比连续向量的 load/store 贵得多。
- **现实类比**：gather 像"按购物清单（index）从货架（input）取货"，每件货独立可取；scatter 像"把货按收货地址（index）投递"，两个货可能抢同一个地址（冲突）。
- **公式/图示**：`gather: output = [data[3], data[12], data[4], data[9], ...] scatter: output[index[i]] = input[i] ← 多个 i 可能写同一位置 → 需要原子/排序`

9. GroupByKey（按键分组）与 Sort

- **定义**：**groupByKey**：`Seq (key, T) -> Seq (key, Seq T)`，把相同 key 的元素聚成"序列的序列"。**Sort**：按 key 排序整个序列。两者是构建直方图、稀疏矩阵、粒子网格等结构的基础。
- **现实类比**：把一摞混着各种颜色的纸牌按花色分堆（groupBy），或按点数排成一条龙（sort）。
- **公式/图示**：`(1,3),(2,8),(2,4),(1,6),(3,3),(1,9),(1,2),(2,8) groupByKey ⟶ (1,[3,6,9,2]), (2,[8,4,8]), (3,[3])`

10. Work / Span（工作量 / 跨度）与并行度

- **定义**：见第 6 点。本讲反复用 Work/Span 分析算法：scan 的"理论并行度与元素数成线性关系"，但实践上"只使用填满机器执行资源所需的并行度即可"（避免过度并行带来的额外通信与同步开销）。
- **现实类比**：Work = 总工作量，Span = 关键路径。想快，两个都要小；但 GPU 上有时宁可 Work 大一点也要让 SIMD 通道全部忙起来。
- **公式/图示**：`并行度上界 ≈ Work / Span`（对于理想机器）。

11. 带宽受限（Bandwidth-bound）

- **定义**：如果程序的总执行时间由"需要搬多少数据 / 机器带宽"决定，而不是由计算量决定，则称带宽受限。数据并行方案通常需要**多趟遍历数据**（map → sort → scan → 取末元素……），每一趟都是全量带宽消耗——"if you can avoid being bandwidth bound"是本讲原语的潜台词。
- **现实类比**：数据就是"料"，带宽就是"运料的卡车"。算法再精巧，卡车不够，工地就得干等。
- **公式/图示**：`T ≥ max(总操作数 / 峰值算力, 总字节数 / 带宽)`（roofline 思想，第 9 讲展开）。

12. 数据并行思维的核心方法论

- **定义**：① 把**不规则并行**转化为**规则并行**（如用 sort+scan 替代锁）；② 把**细粒度同步**转化为**粗粒度同步**（如每线程一个原子操作 → 块内归约 + 少量原子）；③ 代价：多次遍历数据、额外带宽与存储。
- **现实类比**：与其让几千人同时抢一支笔签到（细粒度锁），不如先按姓名拼音排序，再数每个首字母有多少人（sort + scan）——大家都只读不抢，最后合并一次。

- **公式/图示**：现代大数据/并行系统的基石：CUDA **Thrust**、**Pandas** Dataframe 操作、**JAX**、**Apache Spark / Hadoop** 都以这些原语为核心。

案例：把 100 万粒子放进 16 个格子的五种解法（幻灯片第 40–46 页）

问题：按 2D 位置把 1M 个粒子放入 16 格均匀网格（构建"二维数组的列表"），供 N-body 近邻查询（只需查周围格子）。五条路线的权衡光谱，就是本讲方法论的全部：

解法	并行粒度	同步	问题
1. 每粒子一线程 + 全局锁	高（按粒子）	全局锁	数千线程抢一把锁，争用爆炸
2. 每粒子一线程 + 每格一把锁	高（按粒子）	每格锁	均匀分布下争用降到 ~1/16，仍不理想
3. 每格一线程，格内扫全部粒子	低（仅 16 任务）	无	GPU 需要数千任务（并行度不足）；每格判定全部粒子，算力浪费 16 倍
4. 部分网格 + 合并	中（N 个 block）	块内同步（shared memory）	争用降 N 倍、同步变便宜；但需合并 N 个网格（额外工作+内存）
5. 数据并行：map + sort + scan	高（始终按粒子）	**完全无锁**	代价：一次全局 sort + 多次遍历（额外带宽）

解法 5 的三步（对应概念 12 的"sort 聚拢 + 差分定位"）：

```
Step 1 (map):    每粒子算所在格子号 grid_cell[i]
Step 2 (sort):   按格子号排序（粒子索引数组随排序置换）
Step 3 (差分):   每线程比较 grid_cell[i] 与 grid_cell[i-1]，
                 不同即写入 cell_starts[this_cell] 与 cell_ends[prev_cell]
                 （首尾元素特判），最终每个格子得到 [start, end) 区间
```

结论：解法 5 保持大并行度、**完全消除细粒度同步**，用一次 sort 和额外遍历（带宽）换来了 GPU 上可行的实现——"This solution maintains a large amount of parallelism and removes the need for fine-grained synchronization... at the cost of a sort and extra passes over the data (extra BW)!"

二、代码示例与详细解说

示例 1：Map——CUDA 内核与 C++ `std::transform` 的对照

```
// map.cu —— 编译：nvcc -o map map.cu
// 功能：b[i] = f(a[i])，其中 f(x) = x + 10（map 原语的一次实现）
#include <cuda_runtime.h>

__global__ void mapAdd10(int N, const int* a, int* b)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N)                      // 边界保护：N 不必是 block 大小的整数倍
        b[i] = a[i] + 10;           // 无副作用：只读 a[i]，只写 b[i]
}

// host 端：mapAdd10<<<ceil(N/256), 256>>>(N, devA, devB);
```

```
// C++ 标准库中的 map（幻灯片原例）：
#include <algorithm>
int f(int x) { return x + 10; }
int a[] = {3, 8, 4, 6, 3, 9, 2, 8};
int b[8];
std::transform(a, a + 8, b, f);      // 输入起止迭代器 + 输出起始迭代器 + 一元函数
```

【代码做了什么？】 – CUDA 版：每个线程负责一个下标 `i`，读 `a[i]`、写 `b[i]`，`f` 是无副作用的纯函数。`if (i < N)` 是"线程数显式、数据集合大小不决定线程数"的典型护栏（对应第 7 讲：launch 的线程数不由数据规模决定）。 – C++ 版：`std::transform(first1, last1, d_first, unary_op)` 就是标准库的 map——输入两个迭代器界定序列、输出起始迭代器、一元操作符。 – 两者都在表达同一件事：`b = map f a`。

【并行机制解说】 – 为什么 map 可任意并行：`f` 无副作用，所以对每个元素的处理**互不干扰、顺序任意**。map 的并行化策略（幻灯片）：`map f s = 把序列 s 分成 P 个子序列 对每个子序列 s_i（并行地）： out_i = map f s_i out = 拼接所有 out_i` – CUDA 的实现：block/grid 就是"划分子序列"，每个线程独立应用 f——**零通信、零同步**，GPU 上天然高效。 – 对应概念：**map、数据并行模型、序列操作**。

示例 2：树形 Reduce——CUDA 树归约（Work O(N)，Span O(log N)）

```
// tree_reduce.cu —— 编译：nvcc -o tree_reduce tree_reduce.cu
#include <cuda_runtime.h>

#define THREADS_PER_BLK 256

// 每个 block 把 256 个元素的子树归约为一个部分和
__global__ void treeReduce(const float* input, float* partial, int N)
{
    __shared__ float sdata[THREADS_PER_BLK];
    int tid = threadIdx.x;
    int i   = blockIdx.x * blockDim.x + tid;

    sdata[tid] = (i < N) ? input[i] : 0.0f;   // 越界补 0（0 是 + 的单位元）
    __syncthreads();

    // 树形归约：跨度每次减半，8 轮后 sdata[0] = 本 block 的和
    for (int s = THREADS_PER_BLK / 2; s > 0; s >>= 1) {
        if (tid < s)
            sdata[tid] += sdata[tid + s];     // 两两相加，跨度减半
        __syncthreads();
    }
    if (tid == 0)
        partial[blockIdx.x] = sdata[0];
}
// host 端再对 partial[0..numBlocks-1] 做第二次归约（可复用同一 kernel 或拷回 CPU）
```

【代码做了什么？】 – 每个线程装载一个元素到 shared memory，然后执行 8 轮"跨度减半"的树形相加：第 1 轮 128 个线程把 256 个元素两两相加 → 128 个部分和；第 2 轮 64 个线程 → 64 个部分和……第 8 轮 1 个线程 → 1 个和。整个过程像一个倒置的二叉树。 – 每轮之后 `__syncthreads()` 保证写方完成、读方再读。

【并行机制解说】 – 树形结构与 Work/Span：`level 0: 256 个元素 level 1: 128 个部分和（128 线程并行） level 2: 64 个部分和（64 线程并行） ... level 8: 1 个总和（1 线程） Work = O(N)（每元素参与一次加法），Span = O(log N)（8 轮串行依赖链）` – 这是 **fold/reduce** 原语的并行实现：每个 block 是一个"子归约"，block 之间靠第二次 kernel 合并——对应"并行 fold = 分段 fold + combiner 合并"。 – 注意每轮闲置一半线程（`tid >= s` 的直接跳过）——朴素实现的固有开销，幻灯片强调"work-efficient scan 的常数因子"时与此同理。 – 对应概念：**reduce/fold、树形归约、Work/Span、shared memory 协作**。

示例 3：`scan_warp`——32 元素 SIMD 并行 scan（Hillis-Steele 风格，幻灯片原码）

```
// 在 32 个 CUDA 线程（一个 warp）上执行 exclusive scan。
// 调用约定：由 32 个线程共同调用，ptr 指向 shared memory 中的 32 个元素；
// 每个线程返回自己下标对应的 exclusive scan 结果
//  （完成后 ptr[] 里保存的是 inclusive scan 结果）。
__device__ int scan_warp(int *ptr, const unsigned int idx)
{
    const unsigned int lane = idx % 32;   // 线程在 warp 内的编号（0..31）
    __syncwarp();                         // warp 内同步（比 __syncthreads 便宜）

    for (int i = 0; i < 5; i++) {         // 5 步，因为 2^5 = 32
        int shift = 1 << i;               // 跨距 1, 2, 4, 8, 16
        if (lane >= shift) {
            int tmp1 = ptr[idx - shift];
            int tmp2 = ptr[idx];
            __syncwarp();                 // 读旧值之前先同步，防止读到被覆盖的新值
            ptr[idx] = tmp1 + tmp2;
            __syncwarp();                 // 写完后同步，防止下一轮读到半更新状态
        }
    }
    return (lane > 0) ? ptr[idx - 1] : 0; // exclusive：返回前一个位置的 inclusive 值
}
```

【代码做了什么？】 – 目标：32 个元素的前缀和（exclusive）。每步跨距翻倍：第 1 步每个线程把 `ptr[idx] += ptr[idx-1]`（跨距 1），第 2 步 `ptr[idx] += ptr[idx-2]`（跨距 2）……第 5 步跨距 16。5 步之后，`ptr[idx]` 里就是从 0 到 idx 的 inclusive scan。 – 每个线程最后返回 `ptr[idx-1]`（lane>0）或 0（lane=0）——即 **exclusive** scan 结果。 – 注意 `__syncwarp()` 的使用：在"读旧值"和"写新值"之间各放一次，保证同一 warp 内 32 个线程步调一致（warp 内同步比块内 `__syncthreads()` 便宜，因为只有一个 warp）。

【并行机制解说】 – 这是**朴素并行 scan**（每轮所有线程可并行，跨距翻倍）的教科书实现。它的复杂度：**Work = N log N = 32×5 = 160 次加法**——比串行 scan 的 31 次加法多得多！ – 为什么幻灯片说"work-efficient 的 scan 在这里反而不划算"：work-efficient（Blelloch）算法需要 up-sweep + down-sweep 两趟，指令数比这个实现多 2 倍以上，而且在 SIMD 上利用率低（up-sweep 时每轮线程数减半，down-sweep 时同样有半空通道）。**在一个 warp 内（32 个 lane 全忙）跑 N log N 的 Hillis-Steele 反而 SIMD 利用率最高**——"并行度只要够填满机器就行"的活例子。 – 对应概念：**scan、inclusive/exclusive、Work vs Span、SIMD 利用率**。

示例 4：`scan_block`——多 warp 协作的块级 scan（幻灯片原码，Assignment 3 同款）

```
// 在 CUDA thread block 内执行 scan。假设 ptr 指向 shared memory,
// 数组长度 == block 内线程数（Assignment 3 中提供了类似代码）。
__device__ void scan_block(int* ptr, const unsigned int idx)
{
    const unsigned int lane    = idx % 32;   // 线程在 warp 内的编号
    const unsigned int warp_id = idx >> 5;   // 线程所在 warp 在 block 内的编号

    // Step 1: 每个 warp 先对各自的 32 个元素做 scan_warp（部分 scan）
    int val = scan_warp(ptr, idx);
    //  （所有线程都参与：同 warp 线程通过 shared 缓冲 ptr 通信）

    // Step 2: 每个 warp 的 31 号线程把本 warp 的扫描结果（最后一个元素）
    //         拷进 block 级 shared 的紧凑区域 ptr[0..numWarps-1]
    if (lane == 31) ptr[warp_id] = ptr[idx];
    __syncthreads();

    // Step 3: 只有 warp 0 对这 numWarps 个"基数"做一次 scan_warp,
    //         得到每个 warp 的偏移基数（inclusive）
    if (warp_id == 0) scan_warp(ptr, idx);
    __syncthreads();

    // Step 4: 所有线程把本 warp 的基数加到自己的部分 scan 结果上
    if (warp_id > 0)
        val = val + ptr[warp_id - 1];        // ptr[warp_id-1] 是前序 warp 的累计基数
    __syncthreads();

    ptr[idx] = val;                          // 写回最终 exclusive scan 结果
}
```

【代码做了什么？】 – Step 1：每个 warp 内先做 32 元素的 scan_warp（示例 3）——得到"warp 内局部 scan"。 – Step 2：每个 warp 的 31 号线程把该 warp 的**局部总和**（= 局部 inclusive scan 的最后一个值）写入 `ptr[warp_id]`（紧凑区域，只占 numWarps 个槽）。 – Step 3：warp 0 对 numWarps 个基数再做一次 scan_warp，得到"每个 warp 前面所有 warp 的总和"（warp 间的累计基数，inclusive 存放在 ptr[]）。 – Step 4：每个非 0 号 warp 的线程把自己的局部 scan 结果加上"前面 warp 的累计基数"，得到**全局** exclusive scan。最后 `__syncthreads()` 保证写完才让下一个使用 ptr 的阶段开始。

【并行机制解说】 – 这是**多级（heterogeneous）scan 策略**的典范：warp 内用一种算法（SIMD 友好的 Hillis-Steele），warp 之间用另一种策略（先收缩成 numWarps 个基数、再扫描基数、再广播）。幻灯片称之为"算法在不同层级采用不同策略"——这是 scan 实现的关键洞见。 – 同步点：两次 `__syncthreads()`（Step 2→3、3→4）是**块级**屏障；`scan_warp` 内部的 `__syncwarp()` 是**warp 级**屏障。块内通信完全走 shared memory。 – 对应概念：**scan、分段扫描思想、多级实现、块内协作**。

从块级 scan 到百万级 scan：三 kernel 流水（幻灯片第 24 页）

超过一个 block 能容纳的元素（如 100 万元素、每 block 1024 元素）时，"扫描基数"这一步本身也超过一个 block 的能力，需要把"计算 → 汇总 → 修正"拆成**三个 kernel launch**（每次 launch 之间有一次隐式全局屏障）：

```
Kernel Launch 1（逐块局部 scan）：
  Block 0  Scan ──► 局部 scan 结果 + 基数 base[0]
  Block 1  Scan ──► 局部 scan 结果 + 基数 base[1]
  ...      ...
  Block N-1 Scan ──► 局部 scan 结果 + 基数 base[N-1]
                （每个 block 只依赖自己的数据——并行）

Kernel Launch 2（扫描基数，规模小，一个 block 足够）：
  base[0..N-1] 做一次 scan ──► 每个 block 的累计偏移

Kernel Launch 3（逐块修正）：
  Block 0  Add base[0]  ──► 全局 scan 结果
  Block 1  Add base[1]  ──► 全局 scan 结果
  ...      ...
  Block N-1 Add base[N-1] ──► 全局 scan 结果
                （每个 block 又只依赖自己的数据——并行）
```

要点：每次 kernel launch 返回时隐式同步了所有线程，所以"基数必须先于修正"的依赖由 launch 边界天然满足——这就是 CUDA 中"用粗粒度同步（kernel 边界）替代细粒度同步（跨 block 通信）"的标准手法；其代价是数据被多遍历一遍（带宽）。

示例 5：用 gather + map + segmented scan 实现稀疏矩阵乘法（幻灯片实例）

```
目标：y = M·x，其中 M 是稀疏矩阵（大部分元素为 0），x 是稠密向量。
CSR（compressed sparse row）表示：
  values     = [3, 1, 2, 4, 2, 6, 8]        // 所有非零元素（按行展开）
  cols       = [0, 2, 1, 2, 1, 2, 3]        // 每个非零元素的列号
  row_starts = [0, 2, 3, 4]                 // 每行在 values 中的起始下标

5 步数据并行算法（每步都是一种原语）：
Step 1 (gather):  gathered[i] = x[cols[i]]
      gathered = [x0, x2, x1, x2, x1, x2, x3]
Step 2 (map):    products[i] = values[i] * gathered[i]
      products = [3x0, x2, 2x1, 4x2, 2x1, 6x2, 8x3]
Step 3 (构造 flag): 由 row_starts 生成 start-flag：每行起点为 1
      flags = [1, 0, 1, 1, 1, 0, 0]
Step 4 (segmented scan): 对 (products, flags) 做 inclusive segmented scan (+)
      [3x0, 3x0+x2, 2x1, 4x2, 2x1, 2x1+6x2, 2x1+6x2+8x3]
Step 5 (取每段末元素): 取每个 flag=1 段落的最后一个元素
      y = [3x0+x2, 2x1, 4x2, 2x1+6x2+8x3]
```

【代码做了什么？】 – 把"每行一个点积"（各行的非零数不同 → 不规则并行）改写为**四条整齐的序列操作**：先 gather 出每列对应的 x 值，再 map 乘上 values，再按行分段做 segmented scan 求和，最后取每段末元素作为该行结果。 – 复杂度：所有步骤都是 O(非零元素数) 的规则数据并行操作——没有锁、没有每行独立的线程（那会造成行间负载不均，破坏 SIMD）。

【并行机制解说】 – 为什么这是"数据并行思维"的胜利：直接按行并行，不同行非零数不同 → SIMD 利用率灾难（warp 内各 lane 工作量不同）；改写为"按非零元素并行 + segmented scan"后，**每个 lane 的工作量完全相同**——把不规则并行变成了规则并行（对应概念：segmented scan、gather、map）。 – 代价：多次遍历数据（gather → map → scan → 取末），带宽开销上升——这正是第 50 页总结里"带宽 hungry"的注脚。 – 对应概念：**segmented scan、gather、map、数据并行思维**。

示例 6：数据并行直方图——map + sort + scan 的组合拳（幻灯片第 47–49 页）

```
// histogram.cu（示意）—— 只用 map、sort、scan 类原语构造大规模并行直方图
// 目标：统计 input[0..N-1] 落入 NUM_BINS 个 bin 的数量（f 把值映射为 bin id）

// 阶段 1（map）：每个线程算自己元素的 bin id
__global__ void compute_bin(float* input, int* bin_ids)
{
    int thread_index = blockIdx.x * blockDim.x + threadIdx.x;
    bin_ids[thread_index] = f(input[thread_index]);
}

// 阶段 3（map + 差分检测）：排序后，扫描相邻元素，定位每个 bin 的起点
__global__ void find_starts(int* bin_ids, int* bin_starts)
{
    int thread_index = blockIdx.x * blockDim.x + threadIdx.x;
    if (thread_index == 0 || bin_ids[thread_index] != bin_ids[thread_index - 1])
        bin_starts[bin_ids[thread_index]] = thread_index;   // 该 bin 首次出现的下标
}

// 阶段 4（每 bin 一线程）：由 bin 起点算出 bin 大小（要跳过空 bin）
__global__ void bin_sizes(int* bin_starts, int* histogram_bins, int num_items, int num_bins)
{
    int thread_index = blockIdx.x * blockDim.x + threadIdx.x;   // 每线程一个 bin
    if (bin_starts[thread_index] == -1) {
        histogram_bins[thread_index] = 0;                       // 空 bin
    } else {
        // 找下一个非空 bin 的起点，两者之差即本 bin 大小
        int next_idx = thread_index + 1;
        while (next_idx < num_bins && bin_starts[next_idx] == -1)
            next_idx++;
        histogram_bins[thread_index] = (next_idx < num_bins)
            ? bin_starts[next_idx] - bin_starts[thread_index]
            : num_items - bin_starts[thread_index];
    }
}

// host 端流程：
//   bin_ids[N]  ← 由 compute_bin 填充；bin_starts[NUM_BINS] 初始化为 -1
//   sort(N, bin_ids, sorted_bin_ids);          // 阶段 2：按 bin id 排序（元素聚拢）
//   launch<<<N>>>     find_starts(sorted_bin_ids, bin_starts);   // 阶段 3
//   launch<<<NUM_BINS>>> bin_sizes(bin_starts, histogram_bins, N, NUM_BINS); // 阶段 4
```

【代码做了什么？】 – 阶段 1（map）：`compute_bin` 每线程算一个元素的 bin id——纯并行。 – 阶段 2（sort）：按 bin id 排序，让同一 bin 的元素聚拢成连续段（排序时元素数组本身也要随之置换，但直方图只关心计数）。 – 阶段 3（差分检测）：排序后"相邻两个元素的 bin id 不同"就说明这里是某 bin 的起点——`find_starts` 每线程检查自己与前一个元素，把起点下标写进 `bin_starts[bin_id]`。`thread_index == 0` 与最后一个元素是两个特例。 – 阶段 4（每 bin 一线程）：`bin_sizes` 由"本 bin 起点 − 下一个非空 bin 起点"得大小；用 while 跳过中间的空 bin（`bin_starts == -1`）。

【并行机制解说】 – 为什么不用"每个元素原子地给 bin 加 1"：原子方案不是错误（第 7 讲论证过原子互斥合法），但在 GPU 上**争用是灾难**——数据分布集中时几千个线程抢同一 bin 的原子单元，串行化到吞吐趋零。数据并行方案**零细粒度同步**：全程只有 map/sort/scan 式的并行遍历与只读比较，全部线程从不互斥写同一位置。 – 三种原语各司其职：map 负责"逐元素计算"，sort 负责"把相同 key 聚拢"（这是 groupByKey 的核心机制），scan/差分负责"把连续段边界变成可并行的计数"。 – 代价：sort 本身是超线性工作 + 多次遍历数组的带宽开销（"at the cost of a sort and extra passes over the data (extra BW)!"）。 – 对应概念：**map、sort、groupByKey（排序聚拢）、数据并行直方图、带宽代价**。

三、关键要点

1. **用"对序列的操作"思考算法**：map、filter、reduce、scan、segmented scan、sort、groupBy、gather/scatter 的高性能并行实现已经存在（Thrust、Pandas、JAX、Spark/Hadoop 全是它们撑起来的），把程序改写成这些原语的组合，就自动获得了并行能

力——前提是**别被带宽卡住**。

2. **理解依赖是关键**：`x = a + b; y = b*7; z = (x-y)*(x+y)`——没有依赖的操作可以并行（`a+b` 与 `b*7` 并行），有依赖的必须等待（z 依赖 x、y）。scan 的难点全在"每个输出都依赖前面所有输入"这条长依赖链上。
3. **并行 scan 是 Work/Span 权衡的教科书**：朴素算法 Work O(N log N)、Span O(log N)；工作高效算法（up-sweep+down-sweep）Work O(N)、Span O(log N)。但 GPU 上常选"低效"算法：warp 内 32 个 lane 全忙的 N log N 版，比"高效但指令多 2 倍以上、通道利用率低"的 Blelloch 版更快。
4. **多级实现（hierarchy）**：scan 在不同层级用不同策略——warp 内 SIMD scan、块内 warp 间协作（收缩基数 + 再扫描 + 广播）、跨块用多 kernel launch。目标永远是：减少 work、减少通信/同步、匹配内存层级。
5. **数据并行思维的三大功效与代价**：把不规则并行→规则并行；把细粒度同步→粗粒度同步（甚至零同步）；代价是**多趟遍历 → 带宽压力大**。粒子网格的五种解法完美演示了这条光谱：全局锁（细粒度、高争用）→ 每格锁 → 按格并行（并行度不足）→ 部分结果+合并 → **sort+scan 的纯数据并行方案（无锁、大并行度，代价是一次 sort 和额外带宽）**。

四、常见陷阱与注意事项

1. **以为 scan 的输出可以"独立并行计算"**：map 可以任意乱序，但 scan 的每个输出都依赖前缀，不能直接并行——必须用跨距翻倍或 up/down-sweep 等专门算法；把 inclusive/exclusive 搞混（exclusive 的第一个元素是单位元 I，不是 in[0]）也是高频错误。
2. **忽略结合律要求**：并行 scan/reduce 要求 `⊕` 是**结合律**操作，且（并行 fold 时）初始值必须是单位元。浮点加法"近似结合"但不严格结合——并行归约的顺序不同，结果可能与串行版本有微小数值差异（这是语义上可接受的，但要知道）。
3. **`__syncwarp()` / `__syncthreads()` 放错位置**：scan_warp 中"读旧值→同步→写新值→同步"的顺序一旦错乱（比如漏掉写后同步），warp 内线程会读到半更新的数据。块级 scan 中 Step 2→3、3→4 的 `__syncthreads()` 一个都不能少。
4. **带宽盲区**：数据并行方案（如稀疏矩阵乘法、粒子网格的 sort 方案）需要多趟全量遍历，每趟都是 O(N) 带宽。若算术强度低，最终瓶颈是带宽而不是并行度——"efficient implementations only leverage as much parallelism as required"。
5. **把 block 当线程池 / 线程数当元素数**：GPU 需要**海量**并行度（V100 可并发 163,840 个 CUDA 线程）；只有 16 个格子的"按格并行"方案并行度严重不足，而"每粒子一原子操作"又争用爆炸——要在并行度、同步开销、带宽之间取平衡（粒子网格 5 种解法就是这份权衡的完整案例）。
6. **对不规则数据强行 SIMD**：行长度不同的稀疏矩阵如果"一行一线程"，warp 内 lane 工作量差异巨大（SIMD 利用率灾难）；正确姿势是先压平成规则序列 + segmented scan（示例 5）。

五、思考题（带答案）

Q1：为什么幻灯片说朴素并行 scan（Hillis-Steele，Work = N log N）"比串行算法还低效"，却在 CUDA 的 `scan_warp` 中仍然使用它？在什么条件下你会选择 work-efficient（Blelloch）算法？

A1：从 Work（总操作数）看，N log N > N，朴素算法确实更"费算力"；但 GPU 的性能模型不是只看 Work，而是看**执行时间 ≈ max(Work/并行度, Span) × 常数**。在单个 warp（32 lane）里，Hillis-Steele 每轮 32 个 lane 全忙，5 轮搞定（Span = 5），时间 ≈ 160/32×每指令时间；而 Blelloch 需要 up-sweep（每轮线程减半）+ down-sweep（同样有半空轮），虽然总加法数约 2N，但指令数多 2 倍以上且 SIMD 通道利用率低，实际更慢。**选择原则**：当可用并行度（lane/核数）≥ 数据规模时（如 warp 内 32 元素），用高并行度低 span 的朴素算法；当数据规模远大于可用并行度、且每元素工作较重时，用 work-efficient 算法把总工作量压下来。这也是"efficient implementations only leverage as much parallelism as required"的注脚。

Q2：在"粒子网格"问题（把 100 万个粒子按 2D 位置放进 16 个格子）的五种解法中，为什么"数据并行（sort+scan）"方案最终胜出？它牺牲了什么？

A2：前四种方案要么争用严重（全局锁：数千线程抢一把锁）、要么并行度不足（16 个格子只有 16 个任务，远小于 GPU 需要的数万）、要么浪费算力（每格扫描全部粒子 = 16 倍粒子-格子判定）。sort+scan 方案：Step 1 map 算每个粒子的格子号（并行于粒子）；Step 2 按格子号 sort（粒子索引同步置换）；Step 3 并行扫描排序后的数组，用"相邻元素格子号不同"定位每格 start/end。它

的优点：并行度始终是 O(粒子数)，且**完全不需要锁**（排序天然把同类聚在一起，start/end 由位置关系确定）。牺牲：一次全局 sort 和多次额外遍历带来的**带宽开销**（"at the cost of a sort and extra passes over the data (extra BW)!"）。这正体现了本讲总结：用规则并行替代不规则并行、用粗粒度（甚至零）同步替代细粒度同步，代价是带宽。

Q3：如何只用 map、sort、segmented scan 构造一个大规模并行直方图？为什么不能用"每个元素原子地给对应 bin 加 1"替代？

A3：数据并行直方图三阶段：① **map**：`compute_bin`——每个线程算 `bin_ids[i] = f(input[i])`；② **sort**：按 bin_id 排序得到 `sorted_bin_ids`（同类聚拢）；③ **find_starts + bin_sizes**：并行扫描排序数组，`bin_starts[bin_id]` 记录每个 bin 首次出现的下标；再按"下一个非空 bin 的 start 减当前 start"算出每个 bin 的大小（`bin_sizes` kernel 用 while 跳过空 bin）。至于"原子加 1"方案：它并非不正确（第 7 讲已论证原子互斥不破坏块间调度），但在 GPU 上**争用**是灾难——如果数据分布集中，成千上万个线程同时对少数几个 bin 做 `atomicAdd`，原子单元会串行化到接近零吞吐；而 sort+scan 方案完全没有细粒度同步，全部线程只读。代价依然是 sort 和额外遍历的带宽。

Lecture 9: 在 GPU 上高效评估 DNN（Efficiently Evaluating DNNs on GPUs: Transformers and ConvNets）（日期：Oct 21, 2025）

> **概述**：本讲把前几讲的所有工具（数据并行、共享内存、SIMD、带宽分析）汇聚到现代 AI 推理上：如何高效调度 DNN 的每一层。核心主线有三条：① **算术强度（arithmetic intensity）与 roofline 模型**——程序要么是 compute bound 要么是 bandwidth bound，而"更快更宽的硬件"与"提高算术强度的程序改造"会改变这个平衡；② **把卷积映射为矩阵乘法**——explicit GEMM（im2col 物化矩阵）与 implicit GEMM（不物化、在 shared memory 里按块构造），以及分块（blocked/tiled）GEMM 如何通过缓存/共享内存复用提升算术强度；③ **层融合（layer fusion）**——conv+scale/bias+maxpool、softmax、transformer 注意力（FlashAttention 式分块 softmax）如何通过融合减少中间数据的 DRAM 往返。最后展望：为什么 GPU 是 DNN 的好平台（张量核、高算术强度、cuDNN），以及为什么它可能不是最优平台（专用硬件 TPU/NPU/Neural Engine）。

注意：本讲对应 **Assignment 4: Fused Conv+MaxPool on the Trainium2 Accelerator**（Trainium2 卷积+池化融合）——你需要在 AWS Trainium2 上把卷积层与 max pooling 融合实现，正是本讲"层融合"主题的实战。

一、核心概念与定义

1. 算术强度（Arithmetic Intensity）与 Roofline 模型

- **定义**：算术强度 = 每搬运 1 字节数据执行多少次算术操作（Ops/BW）。**Roofline** 把"机器峰值吞吐（ops/sec）"对"算术强度"画成曲线：算术强度低时吞吐被带宽压住（**bandwidth bound 区**，斜率为带宽的直线）；算术强度高时达到峰值算力（**compute bound 区**，水平线）。两个推论：① 同样内存系统下把峰值算力提高 → 程序更容易落入带宽受限区；② 提高程序算术强度（程序改造）→ 更容易达到 compute bound。
- **现实类比**：算术强度像"每趟卡车运的料能做出多少件成品"。料运得少（低强度）时，工厂速度取决于卡车（带宽）；料运得足（高强度）时，工厂速度取决于机器（算力）。
- **公式/图示**：`Throughput (Ops/sec) | ┌── compute bound (水平线 = 峰值算力) | / | / | / bandwidth bound (斜线 = 带宽) | / └──────────/──────────→ Arithmetic Intensity (Ops/BW) (1/4 1/2 1 2 4 8 16) 程序时间 ≈ max(总操作数/算力, 总字节数/带宽)`

2. 流水线重叠与双缓冲（Pipelining / Double Buffering）

- **定义**：把"数据加载（load）→ 计算（arithmetic）→ 写回（store）"三个阶段**重叠**执行：上一块数据在计算时，下一块数据同时在加载。代价是**片上存储成本**——必须同时持有"正在处理的数据"和"正在传输的数据"两份缓冲，即 **double buffering（双缓**

冲)。

- **现实类比**：餐厅后厨"流水线备餐"——A 桌在炒菜时，B 桌的食材已经在切配（切配与炒菜并行），而不是炒完 A 再切 B。代价是需要两张案板（两份缓冲）。
- **公式/图示**：`无重叠：[load][compute][store] [load][compute][store] ... 有重叠：[load0][compute0][store0] [load1][compute1][store1] [load2][compute2][store2] ... 片上存储 = 正在算的块 + 正在传的块（双缓冲）`

3. Loop Fusion（循环融合）

- **定义**：把多个遍历同一数据的循环合并成一个循环，**提高算术强度**——中间结果不再落 DRAM，而是留在寄存器/片上。例：计算 `E = D + (A+B)*C`，朴素写法是三个独立循环（add、mul、add），每个循环"2 次 load + 1 次 store 对应 1 次运算"（算术强度 1/3）；融合成一个循环后是"4 次 load + 1 次 store 对应 3 次运算"（算术强度 3/5）。整体算术强度从 1/3 提升到 3/5。
- **现实类比**：洗菜、切菜、炒菜三件事如果每件都先把菜搬回仓库再取出来做（中间结果落 DRAM），不如在一个厨房里一气呵成——省掉两次"仓库往返"。
- **公式/图示**：`程序1（3 个循环）：add(A,B)→tmp1; mul(tmp1,C)→tmp2; add(tmp2,D)→E 每循环：2 load + 1 store / 1 op → 算术强度 1/3（总） 程序2（融合）：E[i] = D[i] + (A[i]+B[i])*C[i] 4 load + 1 store / 3 ops → 算术强度 3/5`

4. 卷积层（Convolutional Layer）

- **定义**：全连接层是"每个输出连所有输入"；卷积层是**局部连接**（每个输出只看输入的一个小窗口，如 3×3）且**同一层所有单元共享同一组参数（weights + bias）**。卷积核（filter）可看作一个"模式检测器"：输出像素的幅度 = 滤波器对输入局部区域的"响应"（如 Sobel 梯度检测核：水平梯度核 `[[-1,0,1],[-2,0,2],[-1,0,1]]` 响应水平梯度）。现代 CNN（如 Inception、MobileNet）由大量 Conv + Pool + ReLU 层堆叠。
- **现实类比**：卷积核像"印章"——同一个印章盖到图像的每个位置（共享参数），盖出来的"印痕"（响应图）标出哪些位置出现了印章上的图案。
- **公式/图示**：`output[j][i] = Σ_jj Σ_ii input[j+jj][i+ii] * weights[jj][ii] （单通道 3×3 卷积） 多滤波器：输出 W × H × num_filters（每滤波器一张响应图） 卷积 + ReLU + Pool：W×H×C → W×H×K → W/2×H/2×K（pooling 减半空间分辨率）`

5. GEMM（General Matrix Multiply，稠密矩阵乘）与 im2col（Explicit GEMM）

- **定义**：GEMM 是 `C += A × B` 的稠密矩阵乘法，是现代 AI 的"内核中的内核"——全连接层、卷积层、transformer 的注意力块都归结为 GEMM。**im2col（explicit GEMM）**：把卷积"展开"成矩阵乘法——为每个输出位置构造一行输入窗口（拉平成向量），得到 (W×H) 行 × (R×S×C) 列的"卷积矩阵"，再与滤波器矩阵相乘。代价：矩阵带 0-padding、存储开销 O(N)（滤波器有 N 个元素时），且**物化该矩阵使 DRAM 流量增加 R×S 倍**（3×3 卷积就是 9 倍）——读激活张量来"拼矩阵"本身就要多读 9 遍数据。
- **现实类比**：im2col 像"把滑动窗口的每一帧都截图存档"——每个窗口的照片（矩阵行）都要实际写出来才能交给 GEMM 库；照片数量巨大（每个输出位置一张），占地方、费流量。
- **公式/图示**：`3×3 卷积 = 矩阵乘：[w0 w1 ... w8] [窗口0拉平: 0 0 0 0 x00 x01 0 x10 x11] [ ... ] × [窗口1拉平: 0 0 0 x00 x01 x02 x10 x11 x12] [ ... ] [窗口2拉平: ... ] num_filters×9 (W×H) 行 × 9 列（每列一个输入通道×空间位置） = (W×H) × num_filters 的响应图`

6. 分块 GEMM（Blocked / Tiled GEMM）与层次化分块

- **定义**：朴素三重循环 GEMM 算术强度极低（不利用 A、B 的时间局部性）；**分块**（blocking/tiling）把计算组织成 C 的小块：算 C 的一个 BLOCKSIZE_J×BLOCKSIZE_I 子块时，让所需 A、B 子块**驻留在缓存里反复复用**（假设 BLOCKSIZE 选得足够小）。进一步**层次化分块**：L2 级块 → L1 级块 → 寄存器级块，逐级匹配内存层级。自检问题：BLOCKSIZE 是不是越大越好？（不是——块要能装进缓存/寄存器文件，太大就装不下、反而驱逐自己。）
- **现实类比**：盖一堵大墙（C），与其每次搬一块砖（一个标量）来回跑仓库，不如把一大片砖（A、B 子块）一次搬到脚边（缓存/shared memory），在脚边砌完一片再搬下一片。
- **公式/图示**：`朴素：for j for i for k: C[j][i] += A[j][k]*B[k][i] ← 每步都从 DRAM 取 A、B 分块：for jblock for iblock for kblock: for j for i for k（块内）: C[jb+j][ib+i] += A[jb+j][kb+k]*B[kb+k][ib+i] ← A、B 子块在缓存/片上驻留期间被复用 BLOCKSIZE 次 层次：jblock2/iblock2/kblock2（L2 级）→ jblock1/...（L1 级）→ 寄存器级（未画出）`

7. Implicit GEMM（隐式 GEMM，不物化矩阵）

- **定义**：explicit im2col 要物化整个卷积矩阵（DRAM 流量 × R×S、额外存储）；**implicit GEMM** 的改进实现是**只把卷积矩阵的一个子块物化在 GPU 片上 shared memory 里**，用调好的 shared-memory GEMM 例程（如 CUTLASS）做子块乘法——**不需要额外片外存储，也不增加 DRAM 流量**。索引仍直接指向原始权重张量和激活张量。
- **现实类比**：不再"每帧截图存档"，而是"投影仪边放边截取当前一帧"——需要用哪一帧，现场从原始视频（激活张量）里截哪一帧，看完即弃，不占相册（DRAM）。
- **公式/图示**：`explicit GEMM: 构造完整卷积矩阵（DRAM 流量 ×9，3×3 卷积）→ 调 GEMM 库 implicit GEMM: 按块从激活张量现场构造子矩阵 → shared memory 中的子块 GEMM → 写回 （无额外片外存储、无额外 DRAM 流量）`

8. Attention（注意力）与 Softmax

- **定义**：transformer 的注意力块：设 Q、K、V 都是 N×d 矩阵（N = 序列长度，d = 嵌入维度），计算 $S = Q \cdot K^T$（N×N 分数矩阵），对 S 的**每一行**做 softmax 得 P，再 $O = P \cdot V$（N×d 输出）。注意 N 可达数千 → N^2 矩阵太大，**朴素实现需要 N^2 空间**（"Trouble!!!"）。softmax 行向量 x：$m(x)=\max_i x_i$，$f(x)=[e^{x_1-m},\ldots,e^{x_B-m}]$，$l(x)=\Sigma f(x)_i$，$\mathrm{softmax}(x)=f(x)/l(x)$（减 max 保证数值稳定）。
- **现实类比**：注意力像"信息检索打分"——Q 是查询、K 是键、V 是值；S 是查询与键的匹配分数，softmax 把分数变成"权重"，O 是值的加权平均。N^2 矩阵就像"每对词都写一张评分卡"——长句子时卡片堆满仓库。
- **公式/图示**：$S = Q\ K^T$ （N×d × d×N = N×N） $P = \mathrm{softmax}(S)$ （逐行 softmax） $O = P\ V$ （N×N × N×d = N×d）

9. 分块 softmax 与 Fused Attention（FlashAttention 思路）

- **定义**：softmax 可以**分块计算**：把行向量 x 分成块 $x^{(1)}$、$x^{(2)}$，则 $m(x) = \max(m(x^{(1)}), m(x^{(2)}))$；$f(x) = [e^{m(x^{(1)})-m(x)} \cdot f(x^{(1)}),\ e^{m(x^{(2)})-m(x)} \cdot f(x^{(2)})]$；$l(x) = e^{m(x^{(1)})-m(x)} \cdot l(x^{(1)}) + e^{m(x^{(2)})-m(x)} \cdot l(x^{(2)})$。**Fused attention（FlashAttention）**：for each j（Q 块）：for each i（K/V 块）：加载 Q_i、$K_j{}^T$、V_j、O_i 块 → 算 $S_{ij} = Q_i K_j{}^T$ → 行方向算 M_{ij}、P_{ij}、l_{ij} → 把 $P_{ij} V_j$ 按缩放累加进 O_i。效果：**从不物化 N^2 矩阵**（省内存），算术强度高（读 3 个块做 2 次矩阵乘 + 若干行求和，O 块常驻缓存）；代价：每步 i 循环要**重新缩放之前累加的 O**（额外计算）。
- **现实类比**：不再把全部评分卡堆满仓库再统一折算，而是"算一批、折算一批、累加进结果"，仓库里永远只有当前一批卡片——空间 O(N)，时间上多花一点"重新折算"的功夫。
- **公式/图示**：`for each j（外层，Q 块）：for each i（内层，K^T/V 块）：加载 Q_i, K_j^T, V_j, O_i（4 个块） S_ij = Q_i K_j^T M_ij, P_ij, l_ij = 行方向 max/exp/sum（分块 softmax） O_i = 缩放后的 O_i + P_ij V_j（按 m/l 重新缩放）`

10. 层融合（Layer Fusion）

- **定义**：把相邻层的计算融合进同一个循环/kernel，避免中间结果落 DRAM。例：Conv → Scale/Bias → MaxPool 序列：如果分开跑，conv 输出（可高达 1 GB）要先写 DRAM、scale/bias 再读一遍、pool 再读一遍——**带宽灾难**。融合方案：scale/bias 是逐元素操作，**可以在 conv 每算出一个元素后立即执行**；max pool 可以在每算完一个 2×2 输出区域后立即取最大值。融合后 DRAM 流量从"N×H×W×K 写 + 2 次读"变成只写 pool 输出 N×H/2×W/2×K。
- **现实类比**：工厂流水线不再把每道工序的半成品全部运回仓库，而是"边加工边传"——每件工件做完前一道立即进下一道，仓库只存最终成品。
- **公式/图示**：`未融合：Conv →(写 N×H×W×K)→ Scale/Bias →(写/读 N×H×W×K)→ MaxPool →(写 N×H/2×W/2×K) 融合后：Conv + Scale/Bias + MaxPool 一个 kernel： 每个输出元素算出后立即 scale+bias；每算完 2×2 区域立即取 max 只写 N×H/2×W/2×K（最终结果）`

11. 张量核（Tensor Cores）

- **定义**：NVIDIA SM 内的专用矩阵乘单元（第 7 讲结尾预告"数百 TFLOPs 的张量核"，本讲展开其用途）：为低精度（如 FP16）矩阵乘提供远超普通 fp32 ALU 的吞吐。注意：本讲主要把它们当作"为什么 GPU 是 DNN 好平台"的证据（V100 有大量 tensor core，需要大量并行工作才能喂饱——N=1、P=Q=64 时输出 524K 元素=2 MB；N=32、P=Q=256 时输出 256M 元素=1 GB）。

- **现实类比**：张量核是"印钞机"（专印矩阵乘这张钞票），普通 ALU 是"点钞机"——印得快，但一次要印一大批才划算（需要大 batch 大矩阵才能喂饱）。
- **公式/图示**：`D = A×B + C`，单条指令完成一个小矩阵块（如 4×4×4 或 8×8×4）的乘加。

12. 低精度（Low Precision）与 DNN 优化三路径

- **定义**：DNN 权重与中间激活常用 16-bit、8-bit 值，正在向 4-bit 推进（极端是 1-bit）。幻灯片总结的优化技术分三类：① **更好的算法**（手工设计模型：深度、宽度、滤波器数、stride；以及自动搜索高效拓扑 NAS）；② **软件优化**（对性能关键操作做好的调度：loop blocking/tiling、fusion——通常人工调优，研究界在努力自动化，如 torch.compile、cuDNN backend、Triton）；③ **近似手段**（模型压缩：低比特精度）。
- **现实类比**：三条路分别是"换更省的配方（模型）"、"改进做菜工序（调度）"、"用更便宜的食材（低精度）"——可以同时用。
- **公式/图示**：精度 32-bit → 16-bit → 8-bit → 4-bit → 1-bit（带宽与存储需求随位数线性下降）。

二、代码示例与详细解说

示例 1：Loop Fusion 提升算术强度（幻灯片原例）

```
// program1.c —— 三个独立循环，中间结果落内存
void add(int n, float* A, float* B, float* C) {
    for (int i = 0; i < n; i++)
        C[i] = A[i] + B[i];
}
void mul(int n, float* A, float* B, float* C) {
    for (int i = 0; i < n; i++)
        C[i] = A[i] * B[i];
}
// 计算 E = D + ((A + B) * C)
add(n, A, B, tmp1);      // 每个循环：2 load + 1 store 对应 1 op（算术强度 1/3）
mul(n, tmp1, C, tmp2);   // 每个循环：2 load + 1 store 对应 1 op（算术强度 1/3）
add(n, tmp2, D, E);      // 整体算术强度 = 1/3
```

```
// program2.c —— 融合成一个循环
void fused(int n, float* A, float* B, float* C, float* D, float* E) {
    for (int i = 0; i < n; i++)
        E[i] = D[i] + (A[i] + B[i]) * C[i];   // 4 load + 1 store 对应 3 op（算术强度 3/5）
}
// 程序 1 → 程序 2 的变换就叫 loop fusion
```

【代码做了什么？】 – 程序 1：三个独立循环各遍历一次数组，`tmp1`、`tmp2` 两个中间数组**完整落内存**，每个循环都是"2 次 load + 1 次 store 换 1 次算术"。 – 程序 2：一个循环内完成 `E[i] = D[i] + (A[i]+B[i])*C[i]`，中间值 `A[i]+B[i]` 留在寄存器里，不再写/读内存。

【并行机制解说】 – 算术强度：1/3（程序 1）→ 3/5（程序 2），提升 80%。在 roofline 图上，程序 1 很可能落在带宽受限区，程序 2 可能够到 compute bound 区。 – 融合的两个前提：① 循环有**相同遍历结构**（同一 i 域）；② 中间结果**无跨元素依赖**（`tmp1[i]` 只被 `mul` 的 `tmp1[i]` 使用——逐元素独立）。这正是第 8 讲"理解依赖"的直接应用。 – 融合的代价：无额外片上存储需求（中间值就在寄存器里），比双缓冲还便宜；但对"循环体变复杂"的 kernel 要小心寄存器压力。 – 对应概念：**loop fusion、算术强度、roofline、bandwidth bound vs compute bound**。

示例 2：im2col——把卷积展开成 GEMM（伪代码）

```
// im2col_pseudo.cpp —— 伪代码：把 3×3 卷积映射为矩阵乘（explicit GEMM）
// 输入：input  W×H 单通道图像；weights  num_filters×9（每个滤波器 9 个权重）
// 输出：output W×H×num_filters
// 设 im2col_matrix 为 (W*H) 行 × 9 列的矩阵，每行是"以某输出像素为中心的 3×3 窗口拉平"

// Step 1: 构造 im2col 矩阵（关键：这是"物化"步骤，需要 0-padding 越界窗口）
for (int outIdx = 0; outIdx < W * H; outIdx++) {
    int j = outIdx / W, i = outIdx % W;          // 输出像素 (i, j)
    for (int jj = 0; jj < 3; jj++)
        for (int ii = 0; ii < 3; ii++) {
            int sy = j + jj - 1, sx = i + ii - 1; // 输入坐标（-1 表示 padding 边界）
            im2col_matrix[outIdx][jj * 3 + ii] =
                (sy >= 0 && sy < H && sx >= 0 && sx < W) ? input[sy * W + sx] : 0.f;
        }
}
// Step 2: 一次 GEMM 完成所有输出：output_matrix = im2col_matrix × weightsᵀ
//          ((W*H)×9  ×  9×num_filters  =  (W*H)×num_filters)
//         可以直接调用任何调优 GEMM 库（BLAS/cuBLAS）

// 多输入通道版：im2col 每行长度变为 9×C（3×3×C），weights 变为 num_filters×(9×C)
// 批量版：batch 内每张图都构造一份 im2col 矩阵
```

【代码做了什么？】 – Step 1 把"滑动窗口"物化为行向量：每个输出像素对应一行 9 个（或 9×C 个）元素，越界位置填 0（padding）。这是 im2col 的全部"魔法"——卷积的窗口结构被编码进矩阵布局里。 – Step 2 把"9 次乘加"变成"9 维行向量 × 权重矩阵列"的点积——整层卷积变成一次标准 GEMM，直接复用调优矩阵乘库。

【并行机制解说】 – 并行性：GEMM 有海量并行度（每个输出元素独立），这正是 GPU 需要的（对应第 8 讲"暴露大量并行度"）。 – 代价（本讲强调）：**物化矩阵的 DRAM 流量是输入的 R×S 倍**（3×3 → 9 倍）——构造 im2col 矩阵时要反复读激活张量的重叠窗口；还占用大量额外存储。这就是示例 3 引入 implicit GEMM 的动机。 – 对应概念：**im2col / explicit GEMM、卷积→矩阵乘映射、带宽开销**。

示例 3：分块（tiled）GEMM——CUDA shared memory 实现

```
// tiled_gemm.cu —— 编译：nvcc -o tiled_gemm tiled_gemm.cu
// 计算 C = A*B (M×N = M×K · K×N)，用 shared memory 分块复用
#include <cuda_runtime.h>

#define TILE 16   // 每个 block 计算 C 的一个 TILE×TILE 子块

__global__ void tiledGemm(const float* A, const float* B, float* C,
                          int M, int N, int K)
{
    __shared__ float As[TILE][TILE];   // A 子块（片上）
    __shared__ float Bs[TILE][TILE];   // B 子块（片上）

    int row = blockIdx.y * TILE + threadIdx.y;   // C 子块内的行
    int col = blockIdx.x * TILE + threadIdx.x;   // C 子块内的列

    float acc = 0.0f;                            // 寄存器累加器（私有）

    for (int kk = 0; kk < K; kk += TILE) {
        // 1) 协作加载 A、B 子块到 shared memory
        As[threadIdx.y][threadIdx.x] = A[(blockIdx.y * TILE + threadIdx.y) * K + kk + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(kk + threadIdx.y) * N + blockIdx.x * TILE + threadIdx.x];
        __syncthreads();                         // 2) 屏障：确保子块加载完成

        // 3) 块内小 GEMM：累加 As×Bs
        for (int k = 0; k < TILE; k++)
            acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();                         // 4) 屏障：防止覆盖 As/Bs 前有人还在读
    }

    C[row * N + col] = acc;                      // 写回全局
}
// host 端：dim3 block(TILE,TILE); dim3 grid(N/TILE, M/TILE);
//          tiledGemm<<<grid, block>>>(dA, dB, dC, M, N, K);
```

【代码做了什么？】 – 每个 block 负责 C 的一个 TILE×TILE 子块；外层 `kk` 循环沿 K 维滑动，每轮把 A、B 的相应子块协作装入 shared memory，块内线程各自累加一个输出元素（`acc` 在寄存器里）。 – 两次 `__syncthreads()`：装完子块后（防止读到旧数据）与子块用完后（防止下一轮覆盖时还有人没读完）。

【并行机制解说】 – 算术强度：朴素三重循环每个 `C[j][i]` 都从 DRAM 取 A[j][k]、B[k][i]（每 1 次乘加 ~2 次 DRAM 访问）；分块后，每块 As、Bs 在 shared memory 驻留期间被复用 TILE 次——**DRAM 访问量 ÷ TILE**。这就是"compute partial result for block of C while required blocks of A and B remain in cache"（幻灯片语）在 GPU 上的直译：shared memory 就是 GPU 的"cache"。 – 自检题：BLOCKSIZE（TILE）越大越好吗？不是——块必须**装得进** shared memory（还有寄存器压力），太大放不下，且会降低每 SM 可驻留的 block 数。 – 完整工程里还有层次化分块（L2 → L1 → 寄存器）与向量化变体（splat + muladd 向量化 i 循环；预转置 B 以便向量化最内层 k 循环；以及寄存器块 `C_accum[SIMD_WIDTH]` 同时向量化 j、i 两维——幻灯片 37–39 页三种方案）。当 i 维很小时方案 1 不好用，需要预转置（方案 2）；方案 3 假设 A、C 也预转置，在 SIMD_WIDTH×SIMD_WIDTH 的寄存器块上做乘加。 – 对应概念：**分块/tiling、算术强度、shared memory、多级内存层级**。

示例 4：注意力分数计算——朴素版与融合（分块 softmax）版

```
# attention_naive.py —— 朴素注意力：物化 N×N 矩阵（N 大时内存爆炸）
import numpy as np
N, d = 1024, 64
Q = np.random.randn(N, d); K = np.random.randn(N, d); V = np.random.randn(N, d)

S = Q @ K.T                    # N×N 分数矩阵（N=1024 时 4 MB；N=10万时 40 GB！）
m = S.max(axis=1, keepdims=True)
P = np.exp(S - m)              # f(x)：减行最大值（数值稳定）
l = P.sum(axis=1, keepdims=True)
P = P / l                      # softmax 归一化
O = P @ V                      # N×d 输出
```

```
# attention_fused.py —— 融合（FlashAttention 思路）：按块扫描，从不物化 N×N
import numpy as np
N, d, BLOCK = 1024, 64, 128
Q = np.random.randn(N, d); K = np.random.randn(N, d); V = np.random.randn(N, d)

O = np.zeros((N, d))
m_prev = np.full(N, -np.inf)   # 每行的 running max
l_prev = np.zeros(N)           # 每行的 running sum of exp

for j in range(0, N, BLOCK):           # Q 块
    Qb = Q[j:j+BLOCK]                  # BLOCK×d
    for i in range(0, N, BLOCK):       # K/V 块
        Kb, Vb = K[i:i+BLOCK], V[i:i+BLOCK]
        Sij = Qb @ Kb.T                # BLOCK×BLOCK 分数块
        m_ij = Sij.max(axis=1)         # 块的 row-max
        Pij = np.exp(Sij - m_ij[:, None])          # 块内 f(x)
        l_ij = Pij.sum(axis=1)                     # 块内 l(x)
        # 用新的 m 重新缩放已累加的 O 与 l，再并入本块
        m_new = np.maximum(m_prev[j:j+BLOCK], m_ij)
        alpha = np.exp(m_prev[j:j+BLOCK] - m_new)  # 旧部分的缩放
        beta  = np.exp(m_ij - m_new)               # 新块的缩放
        O[j:j+BLOCK] = O[j:j+BLOCK] * alpha[:, None] + (Pij * beta[:, None]) @ Vb
        l_prev[j:j+BLOCK] = l_prev[j:j+BLOCK] * alpha + l_ij * beta
        m_prev[j:j+BLOCK] = m_new
O = O / l_prev[:, None]                # 最终归一化
```

【代码做了什么？】 – 朴素版：一次性算出 N×N 的 S，逐行 softmax，再乘 V。三步各读写一次 N×N 矩阵——**内存 $O(N^2)$ 且算术强度低**（每步整个矩阵从 DRAM 进出）。 – 融合版：外层 j 循环遍历 Q 块，内层 i 循环遍历 K/V 块；每步只物化 BLOCK×BLOCK 的 S_{ij} 块，立刻算 M/P/l 并**按新 max 重新缩放**已累加的 O 与 l，然后并入 $P_{ij}V_j$。循环结束用 running l 归一化。

【并行机制解说】 – 分块 softmax 的数学依据（幻灯片 63 页）：`m(x)=max(m(x⁽¹⁾),m(x⁽²⁾))`、`f(x)=[e^{m(x⁽¹⁾)-m(x)}f(x⁽¹⁾), e^{m(x⁽²⁾)-m(x)}f(x⁽²⁾)]`、`l(x)=e^{m(x⁽¹⁾)-m(x)}l(x⁽¹⁾)+e^{m(x⁽²⁾)-m(x)}l(x⁽²⁾)`——所以 softmax 可以在块间"增量"计算。 – 收益：① **内存**：从不物化 N^2 矩阵（省到 O(N)）；② **带宽/算术强度**：每个 i 步读 3 个块（Q、K、V）、做 2 次矩阵乘 + 若干行求和，O 块常驻缓存——高算术强度、低 DRAM 往返。 – 代价：每步 i 循环必须**重新缩放**之前累加的 O（比朴素版多的计算量）；以及需要保存每行的 running m/l。 – GPU 上这正是 FlashAttention 的核心结构（幻灯片以 Thunderkittens 实现为例）。 – 对应概念：**attention、分块 softmax、fused attention / FlashAttention、算术强度**。

示例 5：融合 Conv + Scale/Bias（+ MaxPool 思路）

```
// fused_conv_scalebias.c —— 把 scale/bias 融合进卷积循环（幻灯片 57 页）
// 未融合时：Conv 输出整层后，Scale/Bias 再遍历一遍（中间结果落内存）
float input[IMAGE_BATCH_SIZE][INPUT_HEIGHT][INPUT_WIDTH][INPUT_DEPTH];
float output[IMAGE_BATCH_SIZE][INPUT_HEIGHT][INPUT_WIDTH][LAYER_NUM_FILTERS];
float layer_weights[LAYER_NUM_FILTERS][LAYER_CONVY][LAYER_CONVX][INPUT_DEPTH];
float scale[LAYER_NUM_FILTERS], bias[LAYER_NUM_FILTERS];

// 假设卷积 stride = 1
for (int img = 0; img < IMAGE_BATCH_SIZE; img++)        // 所有 batch 图像
    for (int j = 0; j < INPUT_HEIGHT; j++)
        for (int i = 0; i < INPUT_WIDTH; i++)           // 所有输出像素
            for (int f = 0; f < LAYER_NUM_FILTERS; f++) {  // 所有输出通道
                float tmp = 0.0f;
                for (int kk = 0; kk < INPUT_DEPTH; kk++)    // 累加所有输入通道响应
                    for (int jj = 0; jj < LAYER_FILTER_Y; jj++)
                        for (int ii = 0; ii < LAYER_FILTER_X; ii++)
                            tmp += layer_weights[f][jj][ii][kk]
                                 * input[img][j+jj][i+ii][kk];
                output[img][j][i][f] = tmp * scale[f] + bias[f];  // ← 融合：算完即 scale+bias
            }

// 课堂练习（幻灯片）：如何再融合紧随其后的 max pool（取输出矩阵 2×2 块的最大值）？
// 提示：把黄色循环（i、j）按 2×2 分块——每个 2×2 输出块算完后立即取 max 写 pool 结果，
//       而不是先把整层输出写内存再读回来 pool。
```

【代码做了什么？】 – 七重循环的"直接实现"（batched conv）：每个输出元素 = 所有输入通道上 3×3 窗口的加权和（共享滤波器权重，局部连接）。 – 融合点：`output[...][f] = tmp * scale[f] + bias[f]`——scale/bias 是逐元素操作，紧跟在每个 tmp 算完之后，`tmp` 还在寄存器里就完成，**根本不写中间层**。

【并行机制解说】 – 为什么必须融合：幻灯片算过账——conv 输出可达 **1 GB**（N=32、P=Q=256 情形），"dumping 1 GB to memory and reading it back just to scale, then rereading to pool"是灾难。融合后 DRAM 只写最终 pool 输出（N×H/2×W/2×K），读写量减少约一个数量级。 – maxpool 融合方法（课堂练习答案）：把 i、j 循环按 2×2 分块——每算完一个 2×2 的 conv 输出区域，立即在寄存器里取 max 写入 pool 输出；这样中间 conv 输出永远不出片上（这正是 Assignment 4 的任务）。 – 与示例 1 的 loop fusion 一脉相承，只是融合对象从"数组级"变成"张量层级"。现代框架里：TensorFlow 曾把少数融合算子写死；cuDNN backend 由编译器现场生成融合实现（无运行时开销、中间结果不经内存）；torch.compile 等编译器在自动做类似调度。 – 对应概念：**layer fusion**、**带宽**、**convolutional layer**、**算术强度**。

三、关键要点

1. **"上一张幻灯片你就知道了软件侧性能优化的几乎一切"**：程序是 compute bound 还是 bandwidth bound，取决于机器算力、带宽与程序算术强度的对比；重叠通信与计算需要双缓冲（片上存储成本）；数据移动耗能、片上存储资源与计算资源此消彼长——**现代 AI 优化的软件侧核心就是提高算术强度、减少数据移动**。
2. **卷积的两种 GEMM 化路线**：explicit GEMM（im2col）物化矩阵、直接调库，但 DRAM 流量 ×R×S、存储爆炸；implicit GEMM 只把子块物化在 shared memory，用 CUTLASS 等调优子块 GEMM——**不增加 DRAM 流量**。DNN 层尺寸千差万别（MobileNet 的 1×1/3×3 dw/逐点卷积、Inception 多分支），没有银弹库，所以 CUTLASS/Triton/Thunderkittens/NKI 这类"可编程原语"层至关重要。
3. **分块（blocking）是提升算术强度的通用手术刀**：朴素 GEMM 每步都访问 DRAM；分块让 A、B 子块在缓存/shared memory 里复用 BLOCKSIZE 次；层次化分块逐级匹配 L2 → L1 → 寄存器。自检：BLOCKSIZE 不是越大越好——块必须能驻留（cache 容量、shared memory 容量、寄存器压力）。
4. **融合 = 消除中间数据的 DRAM 往返**：conv+scale/bias+maxpool、softmax 逐行、FlashAttention 分块 softmax——共同点都是"中间结果留在片上，算完即用"；FlashAttention 还展示了分块数学（增量 max/exp/sum 重缩放）如何让"不物化 N^2 矩阵"成为可能，代价是额外的重缩放计算。

5. **GPU 是 DNN 的好平台但未必最优**：高算术强度的矩阵乘正对 GPU 的"flop 富矿"（5120 个 fp32 ALU + 张量核），且有 cuDNN 等高度优化的 kernel 库；但"通用处理器真的需要吗？"——TPU、NPU、Neural Engine、IPU 等专用硬件正是下节课（专门化加速）的主题。低精度（16/8/4/1-bit）是另一条通用优化路径。

四、常见陷阱与注意事项

1. **只优化算力不优化带宽（或反之）**：在带宽受限区盲目加计算资源毫无收益——roofline 告诉你要先看程序在哪个区。幻灯片反问："这是 compute bound 还是 BW bound？"每次优化前先回答这个问题。
2. **BLOCKSIZE 贪大**：分块 GEMM 的块必须能放进 cache/shared memory/寄存器文件；太大放不下、太小复用不足。层次化分块要逐级匹配内存层级，且寄存器级的分块（最内层）往往被忽略。
3. **im2col 的隐性成本**：物化卷积矩阵使 DRAM 流量 ×R×S（3×3 就是 9 倍）并需要大额存储——如果直接拿 explicit GEMM 实现卷积而不考虑这一点，性能会被带宽拖垮；implicit GEMM 才是生产级选择。
4. **softmax/attention 的数值与空间陷阱**：① 不减行 max 直接 exp 会溢出（数值稳定性）；② 朴素实现物化 N×N 矩阵，长序列时空间 $O(N^2)$ 直接爆内存；③ 融合实现里"重缩放旧 O"这步最容易写错（必须用新 m 重缩放，而不是直接用旧 m）。
5. **忘记融合的带宽账**：分开实现 Conv → Scale/Bias → MaxPool 时，1 GB 级中间张量被写了又读、读了又写——看似每层都"简单"，实际带宽开销是数量级的。作业里最容易犯的错是"先把 conv 输出存下来再 pool"，而正确的做法是每算完 2×2 区域就地取 max。
6. **忽视 DNN 层的多样性**：不同层的矩阵维度天差地别（MobileNet 的 1×1 逐点卷积是纯 GEMM、3×3 depthwise 卷积几乎没有通道复用、FC 层 1024×1000……），"一种调度打天下"不成立——这也是 cuDNN 提供多种算法（direct/implicit gemm/winograd 等）的原因。
7. **数值精度想当然**：低精度（FP16/INT8）会改变结果；张量核对低精度有巨大吞吐优势，但精度-性能权衡需要验证（且不同硬件支持不同）。

五、思考题（带答案）

Q1：把卷积映射为矩阵乘法时，为什么 implicit GEMM 比 explicit GEMM（im2col）好？"不物化矩阵"到底省了什么？

A1：explicit GEMM 为了把卷积喂给通用 GEMM 库，必须先把"卷积矩阵"（(W×H) 行 × (R×S×C) 列）整体物化到内存——每个输入元素会被复制到 R×S 个不同的窗口行里，因此**读激活张量的 DRAM 流量放大 R×S 倍**（3×3 卷积 = 9 倍），还要占用巨额额外存储（尤其大 batch 时输出可达 1 GB 级）。implicit GEMM 不构造完整矩阵，而是**每次只在 GPU 片上 shared memory 里物化一个子块**（索引仍直接指向原始权重/激活张量），用调优的 shared-memory GEMM（CUTLASS）做子块乘法——子块来自 DRAM 的流量与直接卷积相同，**不增加任何片外流量，也不需要额外片外存储**。省下的是"9 倍读放大 + 大矩阵存储 + 构造矩阵的额外遍历"。

Q2：FlashAttention 风格的融合注意力，为什么"多做了计算"却仍然更快？它多做了哪些计算？

A2：分块 softmax 要求每并入一个新块 S_{ij} 时，用新出现的行最大值 m_new 重新缩放**已经累加进 O 的所有旧块**（乘以 e^{m_prev−m_new}），同时 running l 也要同步重缩放——这就是"额外的计算"，朴素版没有这步（因为它一次看到整行）。但换来的是：① **空间从 $O(N^2)$ 降到 $O(N)$**（从不物化 N×N 分数矩阵，长序列不再爆内存）；② **算术强度大幅提升**——每个 i 步只读 Q、K、V 三个块、做两次矩阵乘（$Q_iK_j^\top$ 与 $P_{ij}V_j$）+ 少量行归约，O 块常驻缓存，DRAM 往返从"每步读写整个 N×N 矩阵"降到"每步读写 3 个 N×d 块"。在 GPU 上（带宽是稀缺资源、矩阵乘有张量核）"多算一点、少搬很多"几乎总是净赢——这正是 FlashAttention 的原理。

Q3：为什么要对 DNN 做"层融合"（如 Conv+Scale/Bias+MaxPool）？请用 roofline 的视角解释，并说明 Assignment 4 里你会如何实现 maxpool 的融合。

A3：单独执行每一层时，每层都把自己的输出完整写 DRAM、下一层再完整读回来——中间张量（N×H×W×K，可到 1 GB）被反复搬运，而这些层的算术强度极低（scale/bias 每元素 1 次运算、pool 每 4 元素 1 次取 max），在 roofline 上必然落在带宽受限区，吞吐被带宽摁死。融合后中间结果不落 DRAM，只有最终输出（N×H/2×W/2×K）写内存——DRAM 流量减一个数量级，算术强度大幅提升，程序向 compute bound 区移动。maxpool 融合的实现：把输出像素循环按 2×2 分块，每个线程（或每组线程）算完 2×2

四个 conv 输出后，在寄存器里取 max 直接写 pool 输出——conv 的中间 2×2 块从不离开片上。这与示例 1 的 loop fusion 和示例 3 的分块是同一思想在不同层级的应用：**让数据在最近的存储层级被复用**。

Lecture 10: Hardware Specialization（硬件专用化）（日期：Oct 23）

> **概述**：本讲回答一个贯穿全课程的问题——我们前面几讲都在讲如何高效利用多核 CPU 和 GPU，那么为什么这些"通用处理器"仍然不够高效？本讲从**能耗（energy）**的视角重新审视计算：专用硬件（ASIC、FPGA、DSP、领域专用加速器）通过减少指令流开销、缩短数据搬运距离、用数据流（dataflow）方式组织计算，可以在每瓦性能（perf/watt）上比通用 CPU 提升 10～1000 倍。本讲以 Google TPU、NVIDIA Tensor Core、可重构数据流架构（如 Plasticine）为案例，给出"理想 AI 加速器"的特征清单，并讨论"硬件彩票"（hardware lottery）现象：硬件与算法的演化会相互塑造。
>
> **注意**：本讲对应 Assignment 4（在 AWS Trainium2 加速器上编写并优化 fused Conv+MaxPool kernel），作业将让你亲身体验"软件管理片上存储 + 专用计算引擎"的编程模型，与本讲的 tiled programming、dataflow、片上数据搬运等概念直接相关。

一、核心概念与定义

1. Energy efficiency（能效）

- **定义**：单位能量能完成的运算量。幻灯片给出关键关系：

$$\text{Power} = \frac{\text{Ops}}{\text{second}} \times \frac{\text{Joules}}{\text{Op}}$$

即功耗 = 吞吐率 × 每操作能耗。因此，**提升能效（energy efficiency）要么提高每焦耳运算数，要么降低每运算能耗**。幻灯片强调：更好的能效 ⇒ 专用化（specialization，fixed-function），并追问"专用化带来的改进幅度有多大？（What is the magnitude of improvement from specialization?）"——本讲后面的数据（10×、100–1000×）就是对这个问题的回答。 – **现实类比**：一辆卡车（通用处理器）什么都能运但空驶率高、每吨公里耗油大；一条专用输油管道（专用硬件）只能运油，但单位运量的能耗低几个数量级。代价是管道建造成本高、只能运油。 – **公式/图示**：`能效 (perf/watt) = Performance / Power 提高能效的两条路： ① 同样功率做更多运算（专用指令、数据流、消除指令开销） ② 同样运算花更少能量（减少数据搬运、缩短距离、低位宽数据格式）` – **背景动机（energy-constrained computing）**：本讲开篇指出两类能耗受限场景——(1) 移动设备：电池寿命有限、无风扇散热受限（heat dissipation without fan）；(2) 超算与数据中心：机器规模巨大（数十万颗 CPU/GPU），供电与散热（cooling）都是硬约束。而 **AI 的需求正以指数速度增长（AI demands are growing exponentially），数据中心严重受能耗约束**——所以"能效"不是锦上添花，而是 AI 能否继续扩张的第一性约束。

2. Hardware specialization / fixed-function（硬件专用化 / 固定功能）

- **定义**：为特定计算（如视频编码、FFT、DNN 推理）定制电路，把"程序控制"变成"固定数据通路"。相比通用处理器，专用硬件可以省掉指令取指、译码、调度等"控制开销"。
- **现实类比**：瑞士军刀 vs 专用螺丝刀——瑞士军刀什么都能干但每样都不够趁手；专用螺丝刀拧螺丝又快又省力。
- **规则（来自幻灯片）**：与高质量 C 代码在 CPU 上相比——
- 吞吐优化的处理器（GPU core）：约 **10×** perf/watt（假设代码能很好映射到宽数据并行、且为 compute-bound）；
- 固定功能 ASIC：可接近 **100–1000×** 甚至更高 perf/watt（假设 compute-bound 且不是浮点数学密集型）。
- **能效 vs 可编程性光谱**（第 19 页，Pat Hanrahan 设计）：`能效 低 ←————————————————→ 高 Energy-optimized CPU（最好编程） 可编程 DSP GPU（吞吐优化） ~10× 领域专用加速器（TPU） ~20×（DSL 编程，如 DNN） FPGA/可重构逻辑 ~50×??? ASIC 100–1000×（不可编程，设计费数千万美元）` 注意光谱两端：DSP 最易编程；FPGA "difficult to program（making it easier is

active area of research）"；ASIC "Not programmable + costs 10–100's millions of dollars to design/verify/create"。典型固定功能场景：视频编解码、音频播放、Camera RAW 处理、神经网络（future?）。

3. Programmability overhead（可编程性开销）

- **定义**：现代处理器执行一条指令要经过取指、译码、依赖/冒险检查、选择执行资源、读寄存器堆、数据搬移、写回、地址翻译、uop 缓存等一系列步骤。幻灯片（图源 Eric Chung）用 H.264 视频编码能耗分解说明：即使使用 SIMD，**功能单元（functional units）消耗的能耗占比依然很小**，大部分能耗花在寄存器读取（RF）、流水线控制（Ctrl）、流水线寄存器（Pip）、指令取指与指令缓存（IF）、数据缓存（D-$）上。
- **现实类比**：一个 100 人的研究所，真正动手做实验的只有几个人，其余都在开会、审批、填表（控制/管理开销）。专用硬件相当于把"开会审批"全部砍掉，只留做实验的人。
- **量化**（幻灯片，第 32 页 "Amortize overhead of instruction stream control using more complex instructions"）：
- 半精度 FMA（fused multiply-add）的"可编程性开销"约占 **2000%**；
- 半精度 DP4（vec4 dot product）约占 **500%**；
- 半精度 4×4 MMA（matrix-matrix multiply + accumulate）仅占 **27%**。
- 核心原则：**用一条复杂指令摊销（amortize）大量运算的指令流处理成本**（Key principle: amortize cost of instruction stream processing across many operations of a single complex instruction）。
- **复习问题（幻灯片原题）**：SIMD 执行如何降低某些类型计算的开销？这些计算需要具备什么性质？——答案：SIMD 把 N 个运算合并进一条指令，指令流开销被 N 摊销；前提是运算本身可宽数据并行（相同操作、无分支）、数据布局连续。

4. DSP / VLIW（数字信号处理器 / 超长指令字）

- **定义**：DSP 是可编程处理器，但指令流控制通路更简单；通过复杂指令（SIMD/VLIW）让每条指令完成多个运算，摊销控制成本。VLIW（very-long instruction word）：**一条指令同时指定多个不同的运算**（与 SIMD 的"一条指令对多份数据做同一运算"形成对比）。
- **现实类比**：普通工人一次只能搬一件货（RISC）；VLIW 工人拿着清单一次搬五件不同的货（每个货位干什么写死在指令里），省去了逐个分配的中间管理。
- **实例**：Qualcomm Hexagon DSP（用于 Google Pixel 手机），其 FFT 最内层循环每周期执行 **29 个 RISC 级操作**。

5. ASIC（Application-Specific Integrated Circuit，专用集成电路）

- **定义**：为单一用途定制的集成电路，不可编程。代价是设计/验证/流片成本高达数千万～数亿美元。
- **实例 1——FFT**（Chung et al. MICRO 2010）：ASIC 用约 CPU 单核 **1/1000 的芯片面积**、约 **1/100 的功耗** 达到同样性能；GPU core 比 CPU core 面积效率高约 5–7×。
- **实例 2——Anton 超算**（DE Shaw Research）：Anton 1（2008）专为分子动力学蛋白质模拟设计，512 个 ASIC 计算粒子-粒子相互作用，配吞吐优化的 FFT 子系统与为 N-body 通信模式定制的低延迟网络；Anton 3（2025）比同时代 GPU 快约 **20 倍**。
- **现实类比**：专线大巴（ASIC）与出租调度中心（通用 CPU）——大巴线路固定但每公里成本低得多，前提是客流（工作负载）足够稳定。

6. FPGA（Field Programmable Gate Array，现场可编程门阵列）

- **定义**：ASIC 与处理器之间的"中间地带"：芯片上排列逻辑块（logic blocks），通过可编程互连连接。程序员定义的逻辑直接由 FPGA 实现，无需流片。
- **LUT 机制**：可编程查找表。Xilinx Virtex-7 的 LUT6 是 6 输入 1 输出，可视为 **64 元素真值表**；例如 6 输入 AND 用 1 个 LUT6；40 输入 AND 通过级联 8 个 LUT6 实现（延迟 = 3）。
- **现代 FPGA**：大量面积用于"硬门"（hard gates）——SRAM 存储块、DSP 块（乘法器）、CPU（ARM、RISC-V）；用硬件描述语言（Verilog 等）编程。AWS EC2 F1/F2 提供云端 FPGA。
- **能效定位**：约 **50×?**（幻灯片标注"jury still out"，尚无定论），编程困难（如何让它更好编程是活跃研究领域）。

- **现实类比**：乐高积木（FPGA）——你可以随时拆了重搭成任何结构（ASIC 的能力），但每个积木块本身比定制浇筑的钢筋混凝土（ASIC）低效。

7. Domain-specific accelerator（领域专用加速器）

- **定义**：介于"完全可编程"与"完全固定"之间：在有限领域内可编程，通常通过领域专用语言（DSL，如 DNN 场景）编程。约 **20×** 能效（幻灯片以 Google TPU 为例）。
- **现实类比**：食堂的"套餐窗口"——菜品组合固定几种（领域受限），但比"点菜窗口"（通用）出餐快得多，又比"中央厨房专线"（ASIC）灵活。
- **关键词**：TPU 的关键指令（v1 幻灯片）：read host memory、write host memory、read weights、matrix_multiply / convolve、activate——指令集小到只有几个操作。
- **TPU v1 的芯片面积分配**（第 51 页，Jouppi et al. 2017）：**算术单元约占芯片的 30%**，控制逻辑占面积很小（"Note low area footprint of control"）——对比通用 CPU 中大量面积被控制/缓存占据，这正是"专用化省面积"的直接证据。
- **GPU 为什么对 AI 不是最优平台？**（第 22 页的提示问题）——AI 模型计算特征接近稠密矩阵乘（high arithmetic intensity），GPU 有丰富 FLOPs 与 cuDNN 等优化库，但 GPU 本质仍是**通用可编程处理器**：它为任意数据并行程序保留了指令流、调度、缓存等机制，而这些对 AI 工作负载是"多余的"（hint: is a general purpose processor needed?）。专用硬件可以把这些开销全部去掉，把面积和功耗让给算术单元。

8. Systolic array（脉动阵列）

- **定义**：由大量同构处理单元（PE）排成阵列，数据像"脉搏"一样在相邻 PE 之间逐周期流动；权重驻留在各 PE（或从 FIFO 流入），输入/部分和沿阵列传播。这是**数据驱动（data-driven，wavefront）**执行，而非指令驱动。
- **现实类比**：工厂流水线上的工人传递半成品——每个工人（PE）只做自己那一小步（乘加），把结果传给下一个工位，原料从一端流入，成品从另一端流出；没有人需要"下令"每个工位做什么（固定功能），只需让物料流动。
- **SIMD vs Systolic array 对比**（幻灯片表格）：

特征	SIMD	Systolic Array
Dataflow	Control-driven（指令驱动）	Data-driven（wavefront 数据驱动）
Locality（数据重用）	Limited	Temporal + Spatial（时间与空间）
Communication	Global（寄存器/内存）	Local（相邻 PE）
Control	Centralized（集中控制）	Distributed（分布式）
Efficiency（perf/mm^2, perf/Watt）	Medium	Very high

- **如何构建更大的矩阵乘**（第 60–63 页）：实际 GEMM 远大于 4×4（例：A=8×8，B=8×4096，C=8×4096）。做法：让脉动阵列的累加器（assume 4096 accumulators）分时处理不同的输出列块——把 B 的 4096 列切分成列块，权重（A 的行/列）按块复用，阵列反复执行小规模 MMA，把结果累加到 4096 个累加器中。即：**片上阵列做"原子 MMA"，大 GEMM 由阵列+累加器+分块调度组合完成**——这也是后续 tensor core（H100 989 TFLOPS）与 tiled programming（16×16/32×32 tile）的雏形。

9. Dataflow / Streaming dataflow（数据流 / 流式数据流）

- **定义**：把计算组织成数据流图：AI 模型本身就是一个 dataflow graph（Weights → GEMM1 → Pool → GEMM2 → SoftMax → Sum …）。专用硬件直接把中间数据在计算单元之间流动，**用"在芯片上按空间布局调度计算"代替逐条取指执行**。理想情况下没有指令 ⇒ 没有取指/译码开销；极端异步：没有顺序指令执行。
- **现实类比**：水流过一连串水轮（每个水轮做固定工作）发电，而不是"一个机器人反复读说明书操作同一个水轮"。水轮之间用管道（数据通路）直连，中间产物不落地。
- **关键特性**（理想加速器清单）：tiled tensors（16×16、32×32）实现 GEMM 最大 TFLOPS 与低指令开销；异步计算、异步访存、异步 chip-to-chip 通信（重叠 compute/memory/communication）；compute-unit 到 compute-unit 的通信；fusion 与 pipelining；streaming dataflow；避免 off-chip 数据访问。

10. Tensor core + TMA + Tiled programming（张量核 / 张量内存加速器 / 分块编程）

- **定义**：GPU 上的专用矩阵乘单元（tensor core）执行一条 MMA 指令完成整个矩阵分块乘加；TMA（Tensor Memory Accelerator，H100）用专用指令异步搬运张量分块；tiled programming（CUTLASS、Triton、Thunderkittens）以 16×16、32×32 的 tile 为编程原语，目标是 GEMM 最大 TFLOPS、低指令开销。
- **关键事实**（幻灯片）：
- A100 的每个 SM：64 个 fp32 mul-add ALU、32 个 int32 ALU、4 个 tensor core（执行 8×4 × 4×8 矩阵乘加，A、B 为 fp16，D 为 fp32 累加）；GA100 共 108 个 SM → 19.5 TFLOPs fp32 + 312 TFLOPs fp16/32 mixed（tensor cores）。
- H100（2022，TSMC 4nm，800 亿晶体管）：第四代 tensor core、TMA、CUDA cluster、HBM3 最高 80GB；144 个 SM；tensor core（systolic array MMA）**989 TFLOPS fp16**，SIMD 134 TFLOPS fp16 / 67 TFLOPS fp32。
- "All the TFLOPS are in the Tensor Cores"：幻灯片显示各代 GPU 的浮点算力中 tensor core 占比 89%、50%（或 0%？）、94%、96%、98%——绝大多数算力来自张量核。
- **H100 的分层结构**（第 38–39 页）：CUDA 层级 Grid → Cluster → Thread Block → Threads/SIMD Lanes，对应计算层级 GPU → CPC → SM → SIMD Lanes，对应存储层级 80 GB HBM/50 MB L2 → 256 KB shared memory/SM → 256 KB shared memory → 1 KB RF/thread、64 KB/SM partition。**Thread block cluster** 是最多 16 个 thread block 的集合，保证每个 thread block 在不同 SM 上**同时执行**（支持跨 SM 协作）。每个 SM 内：4 个 warp scheduler（每周期各取指/译码 1 个 warp），64 KB 寄存器/sub-core、256 KB 寄存器/SM，按最多 64 个 warp 划分；含 SIMD fp32/int/fp64 单元、LSU、4 个 16×16×16 [fp16 fp16 fp32] tensor core 单元。
- **TMA 细节**（第 40 页）：专用指令高效搬运数据；异步地把张量的一个区域从 global 加载/存储到 shared memory；**拷贝描述符（copy descriptor）描述区域；单个线程**发起 TMA 操作（cuda::memcpy_async）；拷贝完成时**信号屏障**；硬件完成地址生成与数据搬运——程序员不需要逐元素寻址。
- **B100 的变化**（"Not your father's CUDA"）：寄存器带宽限制 tensor core；张量数据放 SMEM 与 TMEM；**单个线程执行 MMA ⇒ 不再有 warp 的概念**；编程步骤：tcgen05.alloc 分配 TMEM 与描述符 → cp.async.bulk.tensor 用 TMA 预取/流式搬运 tile（配合 mbarrier）→ tcgen05.mma batch 启动异步 MMA（配合 tcgen05.commit）→ tcgen05.fence 排序与回收。
- **现实类比**：tile 编程就像"集装箱运输"——不再逐件搬运（标量）或逐托盘搬运（SIMD），而是整个集装箱（16×16 tile）用专用吊机（tensor core + TMA）一次吊装。

11. Numerical formats（数值格式：BF16 / BF8）

- **定义**：为 AI 计算设计的低精度浮点格式，降低每运算能耗与面积。
- BF16：1 符号位 + 8 指数位 + 7 尾数位。**与 FP32 相同范围（range），但精度更低**。
- BF8 E4M3：1+4+3，最大值 448。
- BF8 E5M2：1+5+2，最大值 57344。
- **现实类比**：记账用"元"（FP32）需要更多位数；估预算用"万元"（BF16）范围一样大、但精度粗——对神经网络这种"对噪声不敏感"的计算足够，且省电省面积。
- **公式**：FP32 表示 $\approx -1^{S} \times (1 + M \times 2^{-23}) \times 2^{E-127}$；BF16 把尾数位砍到 7 位，保留 8 位指数（范围同 FP32）。

12. Hardware lottery（硬件彩票）

- **定义**（Sara Hooker 定义）：当一个研究想法获胜，是因为它恰好适合当时可用的软件与硬件，而不是因为它普遍优于其他研究方向时，就发生了"硬件彩票"现象。
- **幻灯片图示**：TPU 擅长 dense matrix multiply（运算强度 OI ∝ n）→ 人们设计 Transformer 模型 → Transformer 模型主导 → 硬件进一步为 MM 专用化。硬件与算法形成正反馈循环。
- **现实类比**：中彩票者把成功归因于自己的"策略"，但其实只是抽到了适合当时机器的号码；当机器变了，同一策略可能一文不值。
- **延伸解读**：这个循环也解释了第 48 页的观察——"AI is the driving force behind new architectures, compilers, and system design"（AI 正在重新定义计算）：TPU3、Apple Neural Engine、AWS Trainium 2、Cerebras Wafer Scale Engine、SambaNova 等纷纷为 AI 造芯。算法研究者与硬件设计师需要意识到：**当前架构偏好可能只是历史偶然，而不是最优解**——这也正是课程鼓励学生思考"通用 vs 专用"长期权衡的原因。

13. 补充：理想 AI 模型加速器的特征清单（贯穿全讲的 checklist）

幻灯片第 23–29 页反复出现一张"特征–原因"表，作为评估任何加速器（GPU、TPU、数据流芯片）的标尺，在此汇总：

特征（Feature）	原因（Why?）
Tiled tensors（16×16、32×32）	GEMM 最大 TFLOPS、低指令开销
异步计算（asynchronous compute）	重叠计算与访存
异步内存访问（asynchronous memory access）	重叠计算与访存
异步 chip-to-chip 通信	重叠计算、访存与通信
计算单元间通信（compute-unit-to-compute-unit comm.）	数据在片上直接流动
Fusion 与 Pipelining	减少中间结果落盘
Streaming dataflow	数据流式执行、无指令开销

第 47 页用这张表"打分"了 NVIDIA GPU：Tiled tensors ✅（CUTLASS/Triton/TK 生态）、异步计算 ✅（mma_async）、异步访存 ✅（TMA+TMEM）、异步芯片间通信 ❓（TB Cluster，仍在探索）——说明**即使是最先进的 GPU，离"理想加速器"也还有一步之遥**，而这正是数据流架构（第 68–70 页，Plasticine/SambaNova）试图补齐的部分。

二、代码示例与详细解说（本讲重点）

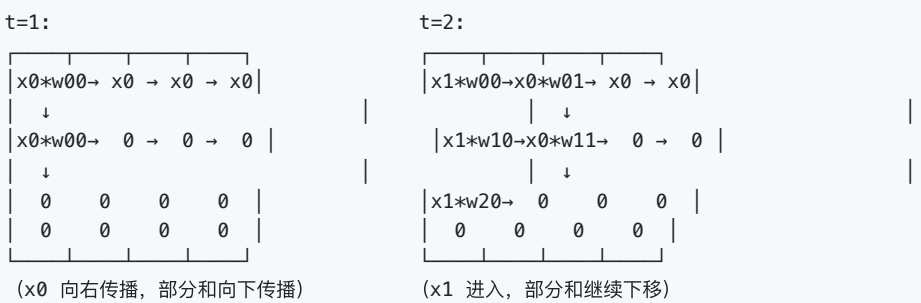
示例 1：脉动阵列的 PE 级伪代码（矩阵–向量乘 y = W·x）

```
// 伪代码：4x4 脉动阵列执行 y = W·x（W 为 4x4 权重矩阵）
// 硬件结构：
//   - 16 个 PE 排成 4x4 网格
//   - 权重 w_ij 预先装入 PE(i,j)（或由 Weights FIFO 逐周期供给）
//   - 输入向量 x 从左侧逐元素流入，向右传播（每个周期前移一格）
//   - 部分和从上往下传播，最后一行 PE 输出到 32-bit accumulator
//   - 每个 PE 内部：1 个乘法器 + 1 个加法器 + 1 个局部寄存器

// 每个 PE(i, j) 每周期执行的逻辑：
loop forever:
    x_in   = receive_from_left(PE(i, j-1))   // 从左边邻居拿输入
    acc_in = receive_from_top(PE(i-1, j))    // 从上方邻居拿部分和
    acc_out = acc_in + w[i][j] * x_in        // 乘加：本 PE 的唯一运算
    send_to_right(PE(i, j+1), x_in)          // 输入继续向右流动
    send_to_bottom(PE(i+1, j), acc_out)      // 部分和向下流动

// 时序（wavefront）：第 t 周期，x0 到达 PE(0,0)；
// t=0: x0*w00 在 PE(0,0) 累加
// t=1: x0 传到 PE(0,1)，x1 进入 PE(0,0)；PE(0,0) 把部分和 x0*w00 传给 PE(1,0)
// t=2: PE(0,2) 得 x0*w02，PE(1,1) 得 x0*w10+x1*w11，PE(2,0) 得 x0*w20 …
// 最终：PE(3,j) 依次输出 y_j = Σ_i w_ij * x_i（对应列累加）
```

数据流动示意（wavefront 的三次快照）：



【代码做了什么？】 这段伪代码描述了一个 4×4 脉动阵列如何计算矩阵–向量乘 $y = Wx$。所有 16 个 PE **每一拍（cycle）同时工作**：每个 PE 把从左邻收到的输入 x_i 与本地权重 w_{ij} 相乘，加到从上邻收到的部分和上，然后把输入继续向右传、把部分和继续向下传。权重事先固定（或者通过 Weights FIFO 逐拍注入），整个计算没有"取指令、译码、判断分支"等任何控制流操作——PE 内部只有一条固定的乘加数据通路。幻灯片用多张图逐拍展示了 x_0w_{00}、$x_0w_{00} + x_1w_{01}$、$x_0w_{00} + x_1w_{01} + x_2w_{02}$ 等部分和在阵列中"涌动"的过程，这就是 wavefront（波前）的含义。

【并行机制解说】 这里的并行性是**空间并行（spatial parallelism）+ 时间流水（temporal pipelining）**的结合：16 个 PE 同时做不同的乘加（空间），同时数据像流水线一样逐拍推进（时间）。与 SIMD 的根本区别在于**通信模式**：SIMD 的每条指令都要访问全局寄存器/内存（global communication，受寄存器带宽限制），而脉动阵列只做**相邻 PE 之间的本地通信**（local，neighbor PEs）——数据在芯片上"走最短的路"，不需要经过集中式寄存器堆。这对应本讲概念 8（systolic array）与概念 9（dataflow）：控制是分布式的（每个 PE 没有自己的指令流），能效极高（perf/mm²、perf/Watt 都是 "very high"）。矩阵–矩阵乘（Y=WX）时，需要多组 4×32-bit accumulator 来容纳输出列（幻灯片第 58 页的"Notice: need multiple 4x32bit accumulators to hold output columns"）。

示例 2：能耗对比——为什么专用硬件省电（减少指令取指开销）

```
// 能耗数量级对比（幻灯片 [Dally / Olson] 的 ballpark 数据）
// 单位：皮焦耳 pJ（10^-12 焦耳）
//
//  整数运算 (integer op)                    ≈ 1 pJ
//  浮点运算 (floating point op)             ≈ 20 pJ
//  读 64 bit 本地小 SRAM（片内 1mm 远）      ≈ 26 pJ
//  读 64 bit 低功耗移动 DRAM (LPDDR)         ≈ 1200 pJ
//
// 注：以上仅为"做逻辑运算本身"的能耗，不含指令译码、寄存器装载等开销
```

能耗数量级速查表（设计决策时随身携带）：

操作	能耗	相对整数运算
整数运算	~1 pJ	1×
浮点运算	~20 pJ	20×
读 64 bit 片内 SRAM（1mm）	~26 pJ	26×
读 64 bit LPDDR	~1200 pJ	1200×

```
// 反例推演：在通用 CPU 上算 y = W·x (4x4)
// 每条标量指令都要：取指(IF) + 译码 + 冒险检查 + 读寄存器堆(RF) + 搬数据 + 运算 + 写回
// H.264 编码实测能耗分解 [Hameed ISCA 2010]：功能单元(FU)占比很小，
// 大头是 RF、Ctrl、Pip、IF、D-$
// 专用加速器做法：把"指令开销"全部砍掉
//   FMA（半精度标量乘加）   → 可编程性开销 ≈ 2000%    （指令开销是运算的 20 倍）
//   DP4（半精度 vec4 点积）→ 可编程性开销 ≈ 500%
//   4x4 MMA（矩阵乘加）     → 可编程性开销 ≈ 27%
```

【代码做了什么？】 这个"示例"其实是一组设计准则与数量级数据，用来回答"为什么通用处理器效率低、专用硬件为什么省电"。幻灯片给出两条主线：(1) **执行一条指令本身很贵**——现代处理器执行一条指令要读指令、译码、检查依赖/冒险、找执行资源、控制寄

存器堆 SRAM、搬数据、运算、写回、转 uop、访问 uop 缓存、地址翻译、访问 icache 等（第 8 页）；H.264 编码的能耗分解显示功能单元消耗的占比很小，绝大部分能耗在"伺候指令"上。(2) **数据搬运比运算贵得多**——读一次 LPDDR（1200 pJ）相当于做 1200 次整数运算（1 pJ）或 60 次浮点运算（20 pJ）。所以设计系统时第一原则永远是"减少数据搬运"。

【并行机制解说】 这个示例解释了两个能效来源：(a) **用复杂指令摊销控制成本**——一条 MMA 指令完成 64 次乘加，指令流处理成本被摊销到 64 个运算上，可编程性开销从 2000%（标量 FMA）降到 27%；这正是 SIMD 的延伸思路（回顾课上问题：SIMD 如何降低开销？要求运算可宽数据并行、无分支）。(b) **固定功能消除控制**——ASIC/脉动阵列根本没有指令流，能效可达 CPU 的 100–1000×（对应概念 2、3、8）。而"减少数据搬运"则引出 tiled programming 与数据流架构（概念 9、10）：数据尽量留在片上 SRAM、在计算单元间直接流动，而不是反复进出 DRAM。这也是本讲后面"理想 AI 加速器"特征清单的由来。

示例 3：Tiled GEMM 的调度伪代码（数据流加速器 / GPU tensor core）

```
// 伪代码：以 16x16 / 32x32 tile 为单位调度 GEMM: C[M,N] = A[M,K] * B[K,N]
// 编程模型对应：CUTLASS / Triton / Thunderkittens 的 tiled 抽象
// 硬件目标：最大化 tensor core 利用率，最小化数据搬运

GEMM_TILED(A, B, C, M, N, K):
    # 外层循环：遍历输出 tile
    for m_tile in range(0, M, 16):          # 输出行方向分块
        for n_tile in range(0, N, 16):      # 输出列方向分块
            acc[16][16] = 0                 # 片上累加器（寄存器/SMEM）
            # 内层循环：沿 K 方向累加
            for k_tile in range(0, K, 32):
                # 异步搬运：把 A、B 的 tile 从 global memory 搬到 SMEM
                a_tile = async_load(A[m_tile:m_tile+16, k_tile:k_tile+32])   # TMA
                b_tile = async_load(B[k_tile:k_tile+32, n_tile:n_tile+16])   # TMA
                wait_tiles_ready(a_tile, b_tile)            # mbarrier 同步
                # 异步计算：tensor core 执行 MMA，与下一次搬运重叠
                mma_async(acc, a_tile, b_tile)              # 16x16x32 MMA
            store_tile(C[m_tile:m_tile+16, n_tile:n_tile+16], acc)   # 异步写回
```

【代码做了什么？】 这是"理想 AI 加速器"编程模型的骨架：把大 GEMM 切成 16×16（或 32×32）的 tile，两层循环遍历所有输出 tile，内层沿 K 方向累加。关键动作有三个：**异步加载**（把 A、B 的 tile 用 TMA 之类的专用单元搬进片上 SRAM，同时干别的）、**屏障同步**（mbarrier 等 tile 就绪）、**异步 MMA**（tensor core 执行矩阵乘加，同时下一批 tile 已经在搬运）。最后把累加结果写回 HBM。幻灯片用三张"理想加速器"表格强调：tiled tensors 的目的就是 GEMM 最大 TFLOPS + 低指令开销；异步 compute / 异步 memory / 异步 chip-to-chip communication 的目的都是**重叠计算与访存**。

【并行机制解说】 这里的并行来自三个层面：(1) **数据并行**：不同输出 tile 之间完全独立，可由不同 SM/线程块并行计算；(2) **流水线并行**：同一 tile 的"搬运下一块、计算当前块、写回上一块"三个阶段通过 asynchrony 重叠——这正是"异步（非阻塞）执行"（第 24 页：Start later operations before earlier operations are complete），避免访存时计算单元空转；(3) **SIMD 级并行**：tensor core 本身是 systolic-array 形式的 MMA 单元（H100 的 989 TFLOPS fp16 全部来自这些单元）。软件侧（CUTLASS/Triton/Thunderkittens）负责把 tile 布局、流水阶段、屏障精确映射到硬件；硬件侧（TMA + mbarrier + async MMA）提供低开销的异步原语。这对应本讲概念 9、10——"GEMM 计算便宜，但数据搬运贵（面积、瓦、纳秒）"，所以整个调度的核心目标是让数据尽量在片上流动、让 tensor core 永不空闲。

示例 4：AI 模型 = 数据流图 → 可重构数据流处理器映射（Plasticine）

```
// 伪代码：把 AI 推理模型（数据流图）映射到可重构数据流架构
// 硬件：Plasticine [Prabhakar, Zhang et al. ISCA 2017]
//   PCU (Pattern Compute Unit)：可配置的计算单元
//   PMU (Pattern Memory Unit)：可配置的存储单元
//   S  (Switch)：片上互连开关

// 模型（以 CNN 为例）：
//   Sample → GEMM1(卷积/全连接) → Pool → GEMM2 → SoftMax → Sum → 输出

// 映射方式：把图中每个算子布局到芯片上的不同 PCU/PMU，
// 数据沿物理连线“流”过整条链，而不是每步取指令执行
place(GEMM1,  pcu[0..3]);    # 4 个 PCU 做 GEMM1 的分块计算
place(Pool,   pcu[4]);       # 1 个 PCU 做池化
place(GEMM2,  pcu[5..8]);    # 4 个 PCU 做 GEMM2
place(SoftMax,pcu[9]);       # 1 个 PCU 做 softmax
route(S, GEMM1_out -> Pool_in);    # 开关把中间结果直接送进下一级
route(S, Pool_out  -> GEMM2_in);
route(S, GEMM2_out -> SoftMax_in);
# 运行时：无指令流、无全局时钟同步，
# 数据到达即计算（token 控制，无需 lock 同步）

# 更细粒度：FlashAttention 类算子可按 tile 流水化
# Tile0..Tile15：QK^T → Mask → Softmax → Dropout → ×V，
# 每个 tile 在独立的 PCU/PMU 链上流式执行（kernel fusion 的效果）
```

【代码做了什么？】 这段伪代码描述"AI 模型 ⇒ 数据流处理器"的映射思想（第 68 页）：因为 AI 模型本身是数据流图（GEMM + 并行模式 map/filter/reduce 等），所以可以用**可重构数据流架构**（Plasticine 是研究原型，SambaNova 是商用版）把图的每个节点"铺"在芯片的不同计算/存储单元上，用开关网络把中间数据在单元间直接流动。运行时没有指令取指/译码，没有顺序指令执行，**数据到达即计算**（第 69 页："No instructions ⇒ No instruction fetch/decode overhead；Extreme asynchrony: no sequential instruction execution"）。下半部分展示 kernel fusion 的 tile 级流水：FlashAttention 的 QK^T、Mask、Softmax、Dropout、×V 按 tile 流式执行，中间结果永远不落回 off-chip 内存。

【并行机制解说】 这是"空间调度计算（schedule computation by laying it out spatially on the chip）"的极致体现（第 71 页总结）：(1) 不同算子在不同 PCU 上**空间并行**；(2) 同一算子的不同 tile 在不同 PCU/PCU 链上**流水并行**（metapipeline/粗粒度流水，第 70 页）；(3) 数据在单元间**直接流动**，避免经过 DRAM，从而规避"读 LPDDR 1200 pJ vs 片内 SRAM 26 pJ"的巨大能耗差（概念 9、11 结合）。与 GPU 相比，数据流架构没有 kernel launch 开销、没有 lock 同步，只有 token 级数据流控制。这对应概念 9（dataflow）、概念 2（specialization）、概念 3（无指令开销）——也是"理想加速器"清单最后两项（compute-unit-to-compute-unit comm.、fusion & pipelining、streaming dataflow）的实现方式。

FlashAttention 的 tile 级数据流（第 70 页）：把 attention 的计算（QK^T → Mask → Softmax → Dropout → ×V）按 tile 切分（Tile 0..15），每个 tile 在独立的 PMU/PCU 链上**流式执行**：Q 与 K^T 的 GEMM 在 PCU 上算，Mask/Softmax/Dropout 各占一个 PMU/PCU 级，结果直接流入 ×V 的下一级。16 个 tile 的处理形成一条 **MetaPipeline（元流水线）**——这是本讲末尾埋下的伏笔，第 11 讲将详细展开 metapipelining 的编程模型。

三、关键要点

1. **能效是新的性能指标**：Power = Ops/s × Joules/Op。移动设备、超算、数据中心、AI 全部受能耗约束；AI 需求指数增长而数据中心能耗受限，"每焦耳运算数"比"峰值 FLOPS"更能决定系统价值。
2. **通用处理器的低效主要来自控制与数据搬运，而不是运算本身**：H.264 能耗分解中功能单元占比很小；读一次 LPDDR（1200 pJ）≈ 1200 次整数运算（1 pJ）。结论：**减少指令流开销 + 减少数据搬运 = 能效提升的两大杠杆**。
3. **能效梯度**（相对高质量 C 代码）：GPU ~10×；领域专用加速器（TPU）~20×；FPGA ~50×（未定论）；ASIC 100—1000×。**可编程性与能效此消彼长**：programmability adds overhead ⇒ reduces efficiency。
4. **脉动阵列 / 数据流 = 专用硬件的核心组织方式**：数据驱动（wavefront）、相邻 PE 本地通信、分布式控制、时间+空间双重数据重用，perf/mm² 与 perf/Watt 都远高于 SIMD。

5. **GPU 正在走向专用化**：tensor core 占了绝大多数 TFLOPS（H100 989/1123 ≈ 88%+），并引入 TMA、TMEM、tcgen05、FP8/FP4、Transformer Engine 等专用机制；"All the TFLOPS are in the Tensor Cores"，程序员必须学会用 tile 级、异步的编程模型（CUTLASS/Triton/Thunderkittens）才能榨出性能。
6. **硬件彩票提醒我们硬件与算法互相塑造**：TPU 擅长 dense MM（OI ∝ n）塑造了 Transformer 的统治地位，Transformer 又反过来推动硬件进一步为 MM 专用化。
7. **从"为什么专用"到"怎么编程专用"**：本讲结尾把问题抛给下一讲——H100/B100 的异步机制（TMA、TMEM、tcgen05）让编程变得极其复杂（"Not your father's CUDA"），需要 ThunderKittens 等 DSL；数据流架构（Plasticine、SambaNova）则用"无指令 + metapipelining"提供更简单的编程模型。

四、常见陷阱与注意事项

1. **只算峰值性能、忽略能耗**：幻灯片反复强调 FFT 案例——ASIC 用 1/1000 面积、1/100 功率达到同等性能。真实系统中（数据中心、手机）能耗预算才是硬约束；"更快但更耗电"的方案往往不可行。注意 ASIC 100–1000× 的前提是 compute-bound 且**非浮点数学密集型**，不要套用到所有负载。
2. **忽略指令流开销**：在通用处理器上写标量循环时，90% 以上的能耗可能花在取指/译码/寄存器堆上而不是运算上。优化时不要只盯着 ALU 利用率，还要看指令吞吐与开销摊销（用 FMA/DP4/MMA 这类复合指令摊销控制成本）。
3. **忽视数据搬运（memory wall）**：读 LPDDR 比做整数运算贵三个数量级。即使算法运算量最优，如果数据反复进出 DRAM，能效和性能都会崩塌。设计加速器/ kernel 的第一原则是"减少 off-chip 数据访问"（tiling、fusion、流式数据流）。
4. **加速器利用率不足**：专用硬件只对匹配的负载高效。把不规则、低运算强度（arithmetic intensity）的代码硬塞给 tensor core / 脉动阵列，利用率可能远低于峰值——"assuming code maps well to wide data-parallel execution and is compute bound"这个前提不能丢。
5. **低估专用化的成本与风险（hardware lottery）**：ASIC 设计/验证/流片要数千万～数亿美元，且押注的算法可能被新研究取代（Transformer 时代硬件跟着 MM 走）。FPGA/可重构/DSA 是"可编程性与能效的折中"，但编程难度是活跃研究问题，不要以为"专用 = 免费"。
6. **把数值格式当 FP32 用**：BF16 与 FP32 范围相同但精度低，BF8 精度更低；低精度格式省能耗与面积，但需要理解精度–能耗权衡，训练/推理中要做精度管理（如混合精度累加 fp32）。
7. **忽视"面积效率"这个维度**：FFT 案例中 ASIC 用 1/1000 芯片面积达到同等性能——面积小意味着同样大小的晶圆能产出更多芯片、成本更低。评估专用硬件时 perf/mm² 与 perf/watt 同样重要。

五、思考题（带答案）

Q1：幻灯片给出 H.264 编码中功能单元能耗占比很小的数据。请用 Power = Ops/s × Joules/Op 解释：为什么"用 SIMD 提升每指令运算数"能提升能效？这种提升有什么前提条件？

答案：SIMD 把 N 个标量运算合并到一条指令里，指令流（取指、译码、控制）的能耗只付一次，摊到 N 个运算上，即"每操作能耗 Joules/Op"下降，而 Ops/s 不变（或上升），所以 Power 上升但每运算能耗下降，perf/watt 提升。前提是：运算本身可宽数据并行（相同操作、无分支）、且是 compute-bound（运算时间占主导，指令开销占比大才有得省）。这正是课堂上"SIMD 如何降低开销"的复习问题。

Q2：为什么脉动阵列（systolic array）比 SIMD 能效更高？请从数据流、通信、控制三个维度对比，并说明这对"数据搬运能耗"（26 pJ SRAM vs 1200 pJ LPDDR）意味着什么。

答案：三个维度：(1) 数据流——SIMD 控制驱动（每条指令从寄存器堆取数据），脉动阵列数据驱动（wavefront，数据自己流动）；(2) 通信——SIMD 走全局寄存器/内存（集中式，受寄存器带宽限制），脉动阵列只走相邻 PE（本地、分布式）；(3) 控制——SIMD 集中控制（指令流开销大），脉动阵列分布式/无控制（固定功能）。因为数据只在相邻 PE 间移动、且可时间+空间重用，数据搬运距离短、次数少，绕开了"读 DRAM 1200 pJ"的天价能耗，主要花在便宜的片上 SRAM（26 pJ）甚至寄存器级流动上。

Q3：假设你要为一家公司设计一个"理想 AI 模型加速器"。请根据幻灯片给出的特征清单（tiled tensors、异步 compute/memory/communication、fusion & pipelining、streaming dataflow），说明为什么这五条能同时提升"compute-bound 模型的性能上界"与"BW-bound 模型的性能上界"。

答案：tiled tensors（16×16/32×32）保证 GEMM 达到最大 TFLOPS 且指令开销低——直接决定 compute-bound 模型（如大矩阵乘）能贴住算力 roofline；高内存带宽 + 异步 memory access 让 BW-bound 模型（如逐元素/attention 类）能贴住带宽 roofline；异步 compute 与异步 chip-to-chip communication 把"访存、通信、计算"三段重叠，使任何一段都不成为短板；fusion & pipelining + streaming dataflow 把中间结果留在片上流动，既降低 off-chip 数据访问（省能耗、提带宽利用率），又让多个算子的执行流水重叠。五条合起来就是：**让计算永远不空闲、让数据尽量不离开芯片、让搬运与计算重叠**——这正是一个加速器同时逼近计算与带宽两个性能上界的充分条件。

Lecture 11: Programming Systems for Specialized Hardware（面向专用硬件的编程系统）（日期：Oct 28）

> **概述**：第 10 讲讲了"为什么需要专用硬件"，本讲回答"**怎么给专用硬件编程**"。专用硬件（TPU 脉动阵列、H100/B100 tensor core、SambaNova SN40L 数据流架构）的能效来自消除指令流开销与异步化，但这也让编程变得极其复杂。本讲以三条主线展开：(1) GPU 侧——NVIDIA 引入 TMA、TMEM、tcgen05 等异步机制后，裸 CUDA 已难以驾驭，需要用 ThunderKittens 这类嵌入式 DSL 把"16×16 tile + asynchrony + producer-consumer 流水"封装起来；(2) 数据流侧——SambaNova 用"数据并行模式（map/zip/reduce）+ metapipelining"这种**以数据为中心**的模型，让程序员以简单方式获得极致异步；(3) 对比——GPU 上 Llama 3.1 8B 每 token 约 800 次 kernel 调用，而 RDU 上一个 kernel 融合整个 decoder，每 token 仅 3 次调用，凸显 kernel fusion 与同步开销的重要性。
>
> **注意**：本讲对应 Assignment 4（在 AWS Trainium2 加速器上编写 fused Conv+MaxPool kernel）。Trainium2 与 H100/B100 类似，拥有专用计算引擎（Tensor Engine 做 128×128 矩阵运算、Vector Engine 做向量运算）与软件管理的片上存储（SBUF/PSUM），需要你用"tile 分块 + 显式数据搬运 + 计算/搬运重叠"的方式编程——正是本讲的核心思想。

一、核心概念与定义

1. Programmability vs Efficiency（可编程性与能效的权衡）

- **定义**：能效与可编程性成反比——"Programmability adds overhead ⇒ reduces efficiency"（可编程性带来开销，从而降低能效）。第 10 讲的能效梯度再次出现：Energy-optimized CPU（最好编程）→ GPU ~10× → 领域专用加速器 ~20×（TPU）→ FPGA ~50×? → ASIC 100–1000×（几乎不可编程）。
- **现实类比**：请一个"什么都会的全能秘书"（通用 CPU）效率低但省心；请一群只会干一件事的专家（专用硬件）效率高，但你必须精确告诉每个专家干什么、怎么衔接（编程难）。
- **公式/图示**：能效 vs 可编程性是光谱的两端；本讲所有编程系统的目标都是：**在不损失太多能效的前提下，让专用硬件变得可编程**。

2. Asynchronous (nonblocking) execution（异步/非阻塞执行）

- **定义**：在较早的操作完成之前就启动较晚的操作。第 4 页对比了同步执行（LD0→ST0→AO0 串行等待，每步等上一步完成）与异步执行（LD_a0 发出后不等完成就发 ST_a0、AO_a0，多个 LD/ST/AO 同时进行）。实现方式：软件+硬件配合的**异步指令与同步原语**，以及硬件乱序执行（out-of-order execution）。
- **现实类比**：同步点菜 = 等第一道菜吃完才点第二道；异步 = 一次性把整桌菜都下单，厨房并行做，先上的先吃。吞吐量大幅提升，但你要学会"不等结果就先干别的，之后再来收结果"。

- **公式/图示**：`同步：LD0 →(等)→ ST0 →(等)→ AO0 →(等)→ LD1 →(等)→ ST1 →(等)→ AO1 … 异步：LD0, ST0, AO0, LD1, ST1, AO1, LD2, ST2, AO2 全部发出，互不等待`

3. Tiled programming model（分块编程模型）

- **定义**：以 16×16、32×32 等形状的"张量块（tile）"为基本编程原语，而不是标量或向量。为什么？因为 GEMM 要打满 tensor core 峰值、指令开销要低。第 10 讲列出 tiled 编程模型：CUTLASS、Triton、Thunderkittens。
- **现实类比**：集装箱运输 vs 散货运输——散货（标量）一件件装卸慢且贵；集装箱（tile）整箱吊装，效率高一个量级，但你需要标准化的装箱方案（tile 布局与搬运计划）。
- **关键点**：tile 的形状要**匹配硬件计算单元的形状**（如 H100 tensor core 是 16×16 输入块、TMEM 16×16×16）；"Use 16 x 16 tiles of fp16 data ⇒ matches Tensor core compute"。

4. Tensor core programming（B100：tcgen05 与 TMEM）

- **定义**：B100 上 tensor core 的编程模型发生了革命性变化（"Not your father's CUDA"）：寄存器带宽限制 tensor core，张量数据放 SMEM 与 **TMEM**（tensor memory），**单个线程即可执行 MMA ⇒ 不再有 warp 的概念**。编程步骤：tcgen05.alloc（分配 TMEM 与描述符）→ cp.async.bulk.tensor（用 TMA 异步预取/流式搬 tile，配 mbarrier 协调）→ tcgen05.mma batch + tcgen05.commit（启动异步 MMA 批量）→ tcgen05.fence（排序与回收）。
- **现实类比**：以前是"几十个工人（warp 线程）一起抬一件货（矩阵）"；现在是"一个工人按下一个按钮，专用起重机（单线程 + 硬件单元）把整箱货吊到位"。控制更简单，但你要学会操作起重机（TMEM 分配、描述符、异步提交）。
- **公式/图示**：B100 tensor core 数据流：Global(HBM) --TMA--> SMEM --cp.async.bulk.tensor--> TMEM --tcgen05.mma--> 结果。

5. TMA（Tensor Memory Accelerator，张量内存加速器）

- **定义**：NVIDIA 的专用数据搬运单元，用**专用指令**高效搬运数据：异步地从 global memory 加载/存储一个张量区域到 shared memory。由**拷贝描述符（copy descriptor）描述区域，单个线程**发起 TMA 操作（cuda::memcpy_async），拷贝完成时**信号屏障**（mbarrier），硬件完成地址生成与数据搬运。A100 有 LDGSTS（bypass L1），H100 的 TMA 更进一步。
- **为什么省电**（第 29 页）：消除数千条指令与内存寻址开销；消除不必要地经过 L1 与寄存器的数据搬运。
- **现实类比**：以前搬货要一箱箱人工登记地址（每箱一条指令、每个地址一次寻址）；TMA 是"整批快递单一次性生成、仓库自动分拣直送"，省掉中间的人工中转站（L1/寄存器）。
- **相关术语**：Warpgroup = 128 个连续线程；PTX（Parallel Thread Execution）= NVIDIA 的虚拟指令集架构（virtual ISA）。

6. Embedded DSL（嵌入式领域专用语言：ThunderKittens）

- **定义**：ThunderKittens（TK）是"Embedded CUDA DSL template library"——一个嵌入 CUDA 的模板库，而不是独立语言。它提供模板化的数据类型（register tiles：寄存器上的 2D 张量，含 height/width/layout；register vectors：1D；shared memory tiles/vectors）与操作（initializer、unary op 如 exp、binary op 如 mul、row/column op 如 row_sum）。
- **设计三原则**（第 36 页）：① **16×16 tile 为基本数据类型**（TK 管理 layout、提供基本操作）；② **处处异步**（暴露原语让用户管理，追求顶级性能）；③ **高层 GPU 协调模式**（如 producer–consumer 处理）。
- **现实类比**：TK 是给 GPU 的"标准集装箱物流系统"——你只需说明"我要运哪些 16×16 的箱子、从哪到哪"，箱子规格、吊装顺序、交接点（barrier）由系统帮你标准化。

7. Producer–consumer pipeline（生产者–消费者流水线）

- **定义**：把 kernel 执行组织成流水：Producer 从 global memory 异步加载 tile 到 shared memory → 中间寄存器暂存 → Consumer（tensor core）计算 → 结果经 shared memory 写回 global memory。幻灯片第 37 页给出 tile 处理流水：Global Memory(HBM/L2) → Shared Memory → Registers → Tensor cores → Shared Memory → Global Memory，并标注 Producer / Consumer / Finish 阶段。

- **现实类比**：餐厅流水线——备菜员（producer）不断把食材（tile）摆到备餐台（shared memory），厨师（consumer）只管炒（compute），收盘员（finish）把成品端走。备菜员不能等厨师炒完才备下一份（异步、多级缓冲）。
- **关键点**：让 tensor core 永不空闲（>90% TFLOPS）的核心手段：**重叠访存与计算 ⇒ 使用 asynchrony**。

8. Dataflow architecture / RDA（数据流架构 / 可重构数据流架构）

- **定义**：把计算"铺"在芯片上：数据在计算单元（PCU）与存储单元（PMU）之间通过开关（switch）直接流动，无指令 ⇒ 无取指/译码开销，极端异步（无顺序指令执行）。SambaNova SN40L RDU 是商用实现：**1040 个 PCU+PMU、638 TFLOPS (bf16)、520 MB 片上 SRAM、64 GB HBM、1.5 TB DDR**；PCU 做 systolic + SIMD 计算（16×8 bf16）；PMU 高地址生成灵活性与带宽（0.5 MB）；AGCU（Address Generator and Coalescing Unit）是访问片外内存与 IO 的"门户"。
- **现实类比**：河流（数据）流经一串水车（PCU）——每个水车做固定的一步工作，水车间用渠道（switch/PMU）直连，没有"调度员逐条下令"。
- **公式/图示**：AI 模型（数据流图：GEMM + map/filter/reduce 等并行模式）⇒ 数据流架构 = 把图直接映射到 PCU/PMU 网格。

9. Data parallel patterns（数据并行模式）

- **定义**：可组合的计算原语：MM（矩阵乘）、Map（逐元素映射）、Zip（按位置合并）、Reduce（归约）、Gather、Scatter…… 程序 = 模式组合，编译器负责 Tiling、Parallelization、Metapipelining、Place & Route、Codegen。关键特性："Flexible scheduling in space and time ⇒ spatial execution"（在空间和时间上灵活调度 ⇒ 空间执行）。
- **现实类比**：乐高积木——标准件（模式）种类有限，但组合方式无穷；拼装说明书（程序）由编译器自动翻译成"每个积木放哪"（布局布线）。
- **示例**：简化 Softmax = Map(exp) → Reduce(+) → Zip(÷)，三个模式组合即可表达；GPU 上要写三个 kernel，RDU 上可在片上流水式执行。

10. Metapipelining（元流水线）

- **定义**：**层级化的粗粒度流水线：一个"流水线的流水线"**。它利用**嵌套循环并行性**：把并行模式（循环）转换成流式流水线——在循环体内插入流水级（pipe stages），各流水级并行执行，**重叠多个循环迭代的执行**；级间中间数据用**双缓冲（double buffers）**存储；能处理执行时间不均衡的流水级；与 tiling 配合良好，缓冲可改变访问模式（如转置数据）；**metapipelining 在 fusion 失效时仍然有效**。
- **现实类比**：工厂车间里的流水线，每条流水线上又有多个工位——"流水线的流水线"。外层流水线处理"批次"，内层流水线处理"批次内的零件"，两层同时并行。
- **公式/图示**：`METAPIPE(M/MM) { LOAD_TILE(A); METAPIPE(N/NN) { LOAD_TILE(B); MAT_MUL; BUFFER; STORE_TILE } }`——外层沿 M 分块、内层沿 N 分块，形成两级流水。

11. Kernel fusion（kernel 融合）

- **定义**：把多个计算步骤（多个 kernel）融合成一个 kernel，中间结果不落回 off-chip 内存，从而：提高数据局部性（high data locality）、消除 kernel 启动与同步开销（zero extra launch overheads）。Llama 3.1 8B 案例：GPU 上 Tensor-RT LLM 每 token 约 10 个 kernel、融合有限（K1..K10，low kernel fusion、low data locality、high launch & synchronization overheads）；RDU 上**一个 kernel 融合整个 decoder**（K0），每 decoder 一次 kernel 调用，**3 次调用/token vs GPU ~800 次调用/token，kernel 调用减少约 100×**。
- **现实类比**：去食堂点"一荤一素一汤"分开三次排队（三个 kernel、三次启动）；还是"套餐窗口"一次取齐（融合 kernel）——省掉三次排队时间，菜也不用端回桌上再端出来（中间结果留在片上）。
- **关键数据**：SN40L 520 MB 片上 SRAM vs H100 100 MB（5× 优势）；数据流融合消除了 GB 级 off-chip 中间结果流量。

12. Compute-communication overlap（计算-通信重叠）

- **定义**：让通信（如 AllReduce）与计算（如 GEMM、权重加载）同时进行，互不阻塞。RDU 上"Fully overlap allreduce with weight load and compute；Allreduce does not consume HBM capacity or bandwidth"——把 AllReduce 与 Down GEMM 等流水化（第 59 页、第 39 页的 Pipelined AllReduce with Compute, no HBM traffic!）。

- **现实类比**：一边开车一边听导航播报（通信与驾驶并行），而不是"停车听完导航再开"。
- **公式/图示**：RDU 0 与 RDU 1 之间：Down GEMM → Add → AllReduce，三者重叠，通信不再占用 HBM 带宽。

二、代码示例与详细解说（本讲重点）

示例 1：ThunderKittens 编写 GEMM kernel（Step 1：定义 layouts）

```
// ThunderKittens (TK) — Embedded CUDA DSL template library
// 目标：在 H100 上打满 tensor core 的 GEMM
// Step 1: Define layouts (定义数据布局)
#include "kittens.cuh"
#include "prototype.cuh"
using namespace kittens;
using namespace kittens::prototype;
using namespace kittens::prototype::lcf;

struct matmul_layout {
   using  a_global_layout = gl<bf16, 1, 1, -1, -1, st_bf<64, 64>>;  // A 的 TMA 描述符：64x64 tile
   using  b_global_layout = gl<bf16, 1, 1, -1, -1, st_bf<64, 256>>; // B 的 TMA 描述符：64x256 tile
   using  c_global_layout = gl<bf16, 1, 1, -1, -1>;                  // C 不需要 TMA 描述符
   struct globals        { a_global_layout A; b_global_layout B; c_global_layout C; };
   struct input_block    { st_bf<64, 64> a[2]; st_bf<64, 256> b; } // 输入 tile：共享内存
   struct finish_block   { st_bf<64, 256> c[2]; };                 // 结果 tile：共享内存
   struct consumer_state { rt_fl<16, 256> accum; };                // 累加器：寄存器 tile
};
```

【代码做了什么？】 这是 TK GEMM 的第一步：**声明整个 kernel 的数据布局**。`gl<bf16, ...>` 是 global memory 布局（同时生成 TMA 描述符，TMA 需要知道从 HBM 的哪里、以什么步幅搬运多大的块）；`st_bf<64,64>` 是 shared memory 中的 tile（bf16、64 行 64 列）；`rt_fl<16,256>` 是寄存器文件中的累加器 tile（fp32 累加）。注意 A 是 64×64 tile、B 是 64×256 tile、C 是 64×256——这些形状与后续 consumer warpgroup 的分工（8 个 consumer warpgroup 各负责一部分）严格对应。**程序员写的是"数据长什么样"，而不是"每个线程怎么搬数据"**——这正是 TK 作为 DSL 的价值：布局与描述符由模板类型自动管理。

【并行机制解说】 对应本讲概念 3（tiled programming）、6（embedded DSL）、5（TMA）。并行性体现在：(1) 以 64×64/64×256 的 tile 为搬运与计算单位，匹配 tensor core 的 16×16 MMA 形状（由 16×16 基本 tile 组合而来）；(2) `gl<...>` 直接生成 TMA 描述符，意味着访存由 TMA 硬件异步完成，无需每元素一条加载指令——消除了数千条指令与寻址开销（第 29 页 "Eliminates 1000's of instructions and memory addressing overhead"）；(3) 布局类型化后，编译器/模板能在编译期推导 tile 在 shared memory 与寄存器间的排布，程序员不再手工计算 bank conflict 与地址偏移。这就是"低指令开销、高 tile 复用"的软件化表达。

示例 2：ThunderKittens 的 producer–consumer 流水（Step 2 & Step 3）

```
// Step 2: Define pipeline and producers（定义流水线与生产者）
struct matmul_template {
   using layout = matmul_layout;
   static constexpr int NUM_CONSUMER_WARPS=8, INPUT_PIPE_STAGES=4;
   // 8 个活跃 consumer warp，4 级输入流水
   static constexpr int PRODUCER_BARRIER_ARRIVALS=1, CONSUMER_BARRIER_ARRIVALS=2;

   struct producer {
      __device__ static void setup(producer_setup_args<layout> args) {
         warpgroup::decrease_registers<40>(); // 生产者少占寄存器，留给消费者
      }
      __device__ static void load(producer_load_args<layout> args) {
         if(warpgroup::warpid() == 0) {   // 只需一个 warp（其实一个线程）发起 TMA
            tma::expect(args.inputs_arrived, args.input); // 告诉 mbarrier 期待多少字节
            for(int i = 0; i < 2; i++) {  // 加载两个 A tile（每个 consumer warpgroup 一个）
               tma::load_async(args.input.a[i], args.globals.A,
                               {blockIdx.x*2+i, args.iter}, args.inputs_arrived);
            }
            // 加载 B tile：一个 64x256 tile，所有 consumer warpgroup 共享
            tma::load_async(args.input.b, args.globals.B,
                            {args.iter, blockIdx.y}, args.inputs_arrived);
         }
      }
   };

   // Step 3: Compute!（消费者）
   struct consumer {
      __device__ static void setup(consumer_setup_args<layout> args) {
         warpgroup::increase_registers<232>(); // 消费者多占寄存器（累加器）
         zero(args.state.accum);               // 累加器清零
      }
      __device__ static void compute(consumer_compute_args<layout> args) {
         // 模板会等输入 tile 就绪后再调用本函数
         warpgroup::mma_AB(args.state.accum, args.input.a[warpgroup::groupid()],
                           args.input.b);           // tensor core 执行 MMA
         warpgroup::mma_async_wait();               // 等待异步 MMA 完成
         if(warpgroup::laneid() == 0) arrive(args.inputs_finished); // 标记内存可复用
      }
      __device__ static void finish(consumer_finish_args<layout> args) {
         int wg = warpgroup::groupid();
         warpgroup::store(args.finish.c[wg], args.state.accum);  // 先存到共享内存
         warpgroup::sync();  // 在 SMEM 中重组，便于对 HBM 合并（coalescing）写
         warpgroup::store(args.globals.C, args.finish.c[wg],
                          args.state.accum, {blockIdx.x*2+wg, blockIdx.y});
      }
   };
};
```

【代码做了什么？】 这是 TK GEMM 的"流水线骨架"：**生产者（producer）**用 TMA 把 A、B 的 tile 异步搬入 shared memory（只用一个 warp 里的一个线程发起，`tma::load_async` 配 `tma::expect` 与 mbarrier `inputs_arrived`）；**消费者（consumer）**等输入就绪后调用 `warpgroup::mma_AB` 让 tensor core 做矩阵乘加，`mma_async_wait` 等异步 MMA 完成，然后 `arrive` 通知"这块 shared memory 可以复用了"；**finish** 阶段把累加结果先写回 shared memory 重组（提高对 HBM 的合并写效率），再写回 global memory。常量 `INPUT_PIPE_STAGES=4` 表示 4 级输入流水（多缓冲），`NUM_CONSUMER_WARPS=8` 表示 8 个 consumer warp 并行。生产者主动 `decrease_registers<40>()`、消费者 `increase_registers<232>()`——寄存器在生产者与消费者之间动态分配。

【并行机制解说】 对应概念 2（asynchrony）、7（producer–consumer pipeline）、4（tensor core 编程）。并行机制是**流水线并行（pipelining）+ 数据并行**：(1) 4 级输入流水让"搬运 tile k+1"与"计算 tile k"重叠，tensor core 永远不空等访存（第 35 页：Make sure compute is never idle; Overlap memory access and compute ⇒ use asynchrony）；(2) 8 个 consumer warp 各自负责一部分输出列（`a[warpgroup::groupid()]`），是数据并行；(3) TMA + mbarrier 让同步原语极轻量（arrive/wait 而非 lock）；(4) MMA 由 tensor core 硬件执行（16×16 输入、fp32 累加），单条指令完成整个矩阵块运算，摊销指令开销。TK 的贡献是：把这些原本散落

在 CUDA/PTX 深处的机制（mbarrier、TMA descriptor、async MMA、寄存器分配）封装成类型化、可组合的模板原语，让程序员用"声明布局 + 写 producer/consumer 回调"的方式表达，而不是手写 PTX——"A Simple Embedded DSL for AI kernels"。

示例 3：SambaNova 的 Matmul Metapipeline（数据流 DSL）

```
// SambaNova RDU 上的矩阵乘 metapipeline（数据流编程模型）
// 以数据为中心：程序描述"数据如何分块、如何在片上流动"
auto format = DataFormat::kBF16;
int64_t M = args::M.getValue();   // 输出行数
int64_t N = args::N.getValue();   // 输出列数
int64_t K = args::K.getValue();   // 归约维

auto A = INPUT_REGION("A", (M, K), format);    // 声明输入张量区域
auto B = INPUT_REGION("B", (K, N), format);
auto C = OUTPUT_REGION("C", (M, N), format);

auto MM = 256;  // M 方向 tile 大小（假设能整除 M）
auto NN = 64;   // N 方向 tile 大小（假设能整除 N）

METAPIPE(M / MM, [&]() {                      // 外层流水：沿 M 分块
    auto a_tile = LOAD_TILE(A, a_tile_shape); // 从 AGCU/HBM 加载 A 的 tile
    METAPIPE(N / NN, [&]() {                  // 内层流水：沿 N 分块
        auto b_tile = LOAD_TILE(B, b_tile_shape, row_par = 4); // 4 路行并行加载
        auto c = MAT_MUL(a_tile, b_tile);     // PCU 执行矩阵乘
        auto c_tile = BUFFER(c);              // 结果进片上缓冲（可转置/改变访问模式）
        STORE_TILE(C, c_tile);                // 写回 HBM
    });
});
```

【代码做了什么？】 这是 SambaNova 数据流编程模型下的 GEMM：程序员声明输入/输出张量区域（INPUT_REGION/OUTPUT_REGION），然后写一个**两层嵌套的 METAPIPE**——外层沿 M 分块加载 A 的 tile，内层沿 N 分块加载 B 的 tile、执行 MAT_MUL、把结果 BUFFER 后 STORE_TILE 写回。与普通 C++/CUDA 循环不同，这里的"循环"不是指令级的，而是**声明式的流水结构**：编译器会把外层/内层循环转换成片上的**流式流水线**（第 51–53 页）：LOAD_TILE 映射到 AGCU（地址生成与合并单元）→ a_tile/b_tile 进 PMU（片上存储）→ MAT_MUL 在 PCU 上执行 → c_tile 经 BUFFER 缓冲（双缓冲、可改变数据布局）→ STORE_TILE 经 AGCU 写回。

【并行机制解说】 对应概念 8（dataflow）、9（data parallel patterns）、10（metapipelining）。并行机制是**层级流水（pipeline of pipelines）**：(1) 外层 METAPIPE(M/MM) 与内层 METAPIPE(N/NN) 各自成为流水级，**多个迭代重叠执行**（第 49 页：Overlap execution of multiple loop iterations）；(2) 中间数据（a_tile、b_tile、c_tile）用双缓冲存储，且 `row_par=4` 指定 B 的加载按 4 路行并行；(3) BUFFER 不只是存储——缓冲可以**改变访问模式（如转置）**，这是融合做不到的（第 49 页：Buffers can be used to change access pattern; Metapipelining can work when fusion does not）；(4) 数据在 AGCU/PMU/PCU 之间直接流动，无指令流、无 kernel 启动开销。程序员写的是"以数据为中心的流水描述"，硬件把计算按空间布局（spatial execution）铺开——这就是"Can we have asynchrony with a simpler programming model?（数据中心的视角）"的答案。

示例 4：简化 Softmax 的数据并行模式表达（Map/Reduce/Zip）

```
# SambaNova 数据流编程：用可组合的并行模式表达计算
# 简化 Softmax：softmax(x)[i] = exp(x[i]) / sum_j exp(x[j])
# 模式：Map(exp) → Reduce(+) → Zip(/)

def simplified_softmax(x):
    # 1) Map：逐元素求 exp（每个元素独立）
    e = map(exp, x)                # e[i] = exp(x[i])
    # 2) Reduce：归约求和（得到分母）
    s = reduce(add, e)             # s = Σ e[i]
    # 3) Zip：按位置两两相除（每对元素独立）
    y = zip(div, e, broadcast(s))  # y[i] = e[i] / s
    return y

# 编译流水：Tiling → Parallelization → Metapipelining → Place&Route → Codegen
# 片上执行：三个模式被布局到不同 PCU，数据流式流过：
#   PMU(x) → PCU(Map:exp) → PCU(Reduce:+) → PMU(s) → PCU(Zip:/) → 输出
```

【代码做了什么？】 这是一个"以数据为中心"的编程示例：把 softmax 拆成三个**可组合的并行模式**——Map（逐元素 exp）、Reduce（求和）、Zip（逐元素相除）。程序员只描述"数据变换"，不描述"哪个线程/哪个周期做什么"。编译器负责后续的 Tiling（分块）、Parallelization（并行化）、Metapipelining（流水化）、Place & Route（布局布线）与 Codegen（代码生成），最终把模式图映射到 RDU 的 PCU/PMU 网格上。第 48 页的"SIMPLIFIED SOFTMAX"图正是这个例子的硬件映射示意。

【并行机制解说】 对应概念 9（data parallel patterns）、8（dataflow）。并行机制：(1) **每个模式内部是数据并行**——Map/Reduce/Zip 天然适合大量 PCU 并行处理不同数据元素；(2) **模式之间是流式流水并行**——exp 的输出直接流入 reduce 的输入、再流入 zip，中间结果不落 HBM（kernel fusion 效果）；(3) **空间执行（spatial execution）**——计算按"在芯片上布局"来调度（第 48 页：Flexible scheduling in space and time ⇒ spatial execution），而 GPU 需要多次 kernel launch（每次启动 + 同步开销）。这解释了第 57 页的对比：同样一个模型，GPU 上"Low kernel fusion, Low data locality, High Launch and Synchronization Overheads"，RDU 上"High kernel fusion: One kernel call for per decoder ⇒ High data locality, Zero Kernel extra launch overheads"。

三、关键要点

1. **能效来自专用化，但专用化让编程变难，DSL/编程系统是桥梁**：H100 的异步机制（TMA、TMEM、tcgen05、mbarrier）让裸 CUDA 编程变得"不是父辈的 CUDA"（Not your father's CUDA）；ThunderKittens 这类嵌入式 DSL 把复杂性封装为"16×16 tile + asynchrony + producer–consumer"原语。
2. **异步（重叠）是打满专用硬件的必要条件**：tensor core 要 >90% TFLOPS，就必须"计算永远不空闲、访存与计算重叠"——同步执行（每步等待）会让访存延迟直接变成空闲时间。
3. **数据流（dataflow）编程模型以数据为中心，天然获得极致异步**：SambaNova 用"并行模式（MM/Map/Zip/Reduce/Gather/Scatter）+ metapipelining"编程，无指令流、无 kernel 启动、无 lock 同步（token 控制）；Metapipeline 是"流水线的流水线"，能处理融合（fusion）处理不了的不规则/需要改布局的场景。
4. **Kernel fusion 的价值量化**：Llama 3.1 8B 推理，GPU 上每 token ~800 次 kernel 调用，RDU 上融合整个 decoder 后仅 3 次——**100× 更少的 kernel 调用**，消除了 GB 级 off-chip 中间结果流量与大量启动/同步开销。
5. **GPU kernel 的重要性**：2025 年 NVIDIA 单季营收 >$47B，AI kernel 在价值数亿美元的 GPU 集群上运行数月；FlashAttention-2 在 A100 上约 70% 效率、到 H100 掉到 ~35%，两年后 FlashAttention-3 才回到 ~65%——**低质量 kernel 会浪费数十亿美元的计算资源**。
6. **通信与计算必须重叠**：多 socket 扩展时通信时间占比上升，不做重叠时通信成为瓶颈（GPU 需要巨大互连带宽）；RDU 把 AllReduce 与权重加载/计算完全重叠，通信不消耗 HBM 容量与带宽。

四、常见陷阱与注意事项

1. **把异步当同步用**：写 TK/CUDA 时若在每个异步操作后立刻等待（sync/wait），流水线退化为同步执行，访存延迟全部暴露，tensor core 利用率暴跌。正确做法是让 producer 提前多级预取（如 INPUT_PIPE_STAGES=4），消费者尽量"晚等"。
2. **tile 形状与硬件不匹配**：tensor core 吃 16×16（fp16）的块，如果你用与硬件形状不匹配的 tile（如 8×8 或非对齐尺寸），MMA 无法全速执行，指令开销摊销失败。TK 中 16×16 tile 是基本类型，CUTLASS 中 tile 形状要与 SM 资源严格核算。
3. **忽略同步/启动开销的放大效应**：GPU 上每个 kernel launch、每次 kernel 间同步都有固定开销；模型级联几十个 kernel 时，这些开销与中间结果落 HBM 的流量会被放大（Llama 8B 案例：~800 calls/token）。融合 kernel 或使用持久化 kernel（如 RDU 的"一个 kernel 跑所有 decoder"）是正解。
4. **数据搬运仍是最贵的**：即使有 TMA，数据一旦必须落 HBM 就付出 1200 pJ/64bit 量级的代价。别把中间结果随便写回 global memory；利用 fusion、片上缓冲（BUFFER）、双缓冲把数据留在 SRAM 里流动（SN40L 520 MB vs H100 100 MB 的 5× SRAM 优势就是这么用的）。
5. **寄存器/shared memory 预算失衡**：TK 示例中生产者主动降寄存器（40）、消费者占用（232）——如果所有 warp 都贪心占寄存器，SM 上能驻留的 warp 变少，流水深度下降。资源分配是性能的一部分。
6. **误以为 metapipelining 万能**：metapipelining 能处理 fusion 失效的场景（缓冲改布局、不均衡流水级），但片上有存储/单元是有限资源（PCU/PMU 数量、SRAM 容量），无限流水会导致布局布线失败或利用率下降。

五、思考题（带答案）

Q1：为什么 H100 需要 ThunderKittens 这类 DSL，而早期 CUDA（如 V100 时代）相对"好写"？请从硬件机制变化的角度回答，并说明 DSL 具体封装了哪些复杂性。

答案：因为硬件越来越专用化（第 33 页：Nvidia chips becoming more specialized）。V100 只有 tensor core；到 A100 加了 sparsity、FP8、Transformer Engine、异步执行、distributed SHMEM；到 H100 加了第四代 tensor core、TMA、异步拷贝；到 B100 甚至引入 TMEM、tcgen05、FP4、解压引擎，且"单个线程执行 MMA ⇒ 不再有 warp"。程序员要直接驾驭这些机制（TMA 描述符、mbarrier、异步 MMA 提交/回收、寄存器分配、TMEM 分配）极其繁琐且易错。TK 封装了：(1) 布局管理（gl/st_bf/rt_fl 类型自动生成 TMA 描述符与内存排布）；(2) 异步原语（load_async/mma_async/mbarrier arrive-wait）；(3) 高层协调模式（producer-consumer 流水、4 级多缓冲）；(4) 资源分配策略（生产者/消费者寄存器配比）。程序员写"布局声明 + producer/consumer 回调"，复杂机制由模板与编译器处理。

Q2：SambaNova 的 metapipelining 与 GPU 上的 kernel fusion 有什么本质区别？为什么说"metapipelining 在 fusion 失效时仍然有效"？

答案：kernel fusion 是把多个算子合并成一个 kernel、中间结果留在片上，本质仍是"指令流 + 集中同步"的执行（GPU 上仍有 kernel launch、grid/sync、L1/L2 往返）。metapipelining 是把嵌套循环转换成**层级化的数据流流水线（pipeline of pipelines）**：各级是物理上独立的 PCU/PMU，级间通过双缓冲与 token 控制流动，无锁同步、无指令取指、无启动开销。fusion 失效的场景（第 49 页）：(1) 算子间访问模式需要改变（如转置）——fusion 无法在中间插入布局转换，而 metapipelining 的 BUFFER 可以改变访问模式；(2) 流水级执行时间不均衡——双缓冲与流水结构天然吸收不均衡；(3) 需要重叠多个循环迭代——融合只是"减少中间落盘"，metapipelining 还重叠了迭代间执行。一句话：fusion 优化"一次执行内的数据局部性"，metapipelining 把"整个计算变成一条永不停止的流水线"。

Q3：RDU 上"一个 kernel 融合整个 decoder（Llama 3.1 8B）、3 calls/token"，而 GPU 上约 800 calls/token。请分析：这是否意味着 RDU 一定比 GPU 快？GPU 该如何缩小这一差距？

答案：不一定。kernel 调用次数少只消除了"启动/同步开销"与"中间结果 HBM 往返"，但推理性能还受峰值算力、HBM 带宽、片上 SRAM 容量、能效等因素约束（第 58 页也承认 "HBM BW limits inference performance"，关键是 overlap 让 HBM 始终忙碌）。RDU 的 520 MB SRAM（5× H100）使其能容纳整个 decoder 的中间状态，这是"一个 kernel 融合"可行的前提；GPU 缩小差距的路径：(1) 用 CUTLASS/TK/Triton 写融合 kernel（如 FlashAttention 系列把 attention 融合，FlashAttention-3 把效率从 35% 拉回 ~65%）；(2) 使用 CUDA Graphs / persistent kernels 减少 launch 开销；(3) 用 producer-consumer 流水 + TMA 多缓冲让权重加

载与计算重叠（GPU 也可以做到"HBM 始终忙碌"）；(4) 增大 L2 驻留（L2 Cache Residency 机制）。所以核心结论是：**kernel 次数是表象，本质是"中间数据是否留在片上 + 计算/访存/通信是否重叠"**——这正是第 57 页两种实现的真正差异。

Lecture 12: Mapping AI Applications to the Datacenter Computer（将 AI 应用映射到数据中心计算机）（日期：Oct 30）

> **概述**：前两讲讨论了"为 AI 设计专用硬件"与"如何编程专用硬件"。本讲把视角从单芯片提升到**数据中心规模**：AI 模型越来越大（从 1.7B 到 1T 参数），单颗加速器装不下、算不动，必须把模型与计算**映射（map）到成千上万台加速器组成的集群**。本讲内容分两大块：(1) **存储系统基础**——从 CPU vs GPU 内存、3D 堆叠的 HBM，到 DRAM 的工作原理（row buffer、bank、burst mode、DIMM、memory controller 调度），解释"内存墙"为何是数据中心计算的根本瓶颈；(2) **数据中心级并行**——DGX SuperPOD 集群拓扑、通信原语（AllReduce/ReduceScatter/AllGather/All-to-All）、AI 模型中的并行维度（DP/TP/PP/EP/SP/CP）、分布式矩阵乘的 K 维切分 + reduce-scatter 合并、计算-通信重叠，以及训练中的细粒度流水并行（micro-batch）。最后回到能耗主题：减少数据搬运与专用处理是降低能耗的两大思想。
>
> **注意**：本讲与 Assignment 4（Trainium2 加速器编程）的片上存储管理/数据搬运思想一脉相承，也为 Assignment 5（GPU kernel 优化）与期末的"数据中心规模"讨论打基础。*补充说明：本讲幻灯片聚焦"数据中心存储系统 + 并行策略"，未展开 LLM serving（连续批处理、KV cache 管理、prefill/decode 等）细节，这些内容以"补充阅读"形式附在本讲末尾，供延伸学习。*

一、核心概念与定义

1. CPU vs GPU memory（CPU 与 GPU 的内存系统差异）

- **定义**：CPU 通过 64-bit memory bus 连接 DRAM；GPU 通过 **1024-bit**（甚至 6144-bit）bus 连接 HBM。GPU 用极宽的接口换取极高带宽，以支撑成千上万个核心同时发出的访存请求。
- **现实类比**：CPU 像"窄而深的通道"（延迟优先，靠缓存层级缩短距离）；GPU 像"宽而浅的河面"（带宽优先，靠宽度吞吐海量数据）。
- **公式/图示**：带宽 ≈ 总线宽度 × 频率 × 每周期传输次数（DDR4 2400：64-bit × 1.2GHz × 2 = 19.2 GB/s/通道，双通道 38.4 GB/s，CAS 约 13 ns）。

2. HBM（High-Bandwidth Memory，高带宽内存）

- **定义**：通过 **3D 堆叠** DRAM 芯片实现的超高带宽内存：DRAM 芯片层层堆叠，用**硅通孔（TSV, through-silicon-vias）**穿过芯片连接；堆叠底层是"逻辑层"（memory controller），负责管理处理器请求；硅中介层（silicon interposer）作为 DRAM 堆与处理器之间的高带宽互连。HBM 的接口宽度为 **1024-bit/stack**。
- **三大优势**（第 6 页）：**More Bandwidth**（更多带宽）、**High Power Efficiency**（高能效）、**Small Form Factor**（小尺寸）。
- **现实类比**：把一摞煎饼（DRAM 层）用吸管（TSV）竖着串起来，再放在一个底座（逻辑层/中介层）上——比平铺一堆煎饼（传统 DRAM 排列）省地方、连接更密、传输更快。
- **关键数据**：H100 用 6 个 HBM3 堆 × 1024-bit = **6144-bit 接口，3.2 TB/s 峰值带宽，80 GB 容量**；P100（2016）4×HBM2 = 4096-bit、720 GB/s、16 GB；AMD Fury（2015）4096-bit、512 GB/s。

3. Scale up vs Scale out（纵向扩展 vs 横向扩展）

- **定义**：Scale up = 把单个节点做强（更大内存、更快互连、更多算力）；Scale out = 用标准节点堆出大规模集群（通过网络互连）。数据中心 AI 计算两者都需要：单机内 scale up（NVLink 全连接），机群间 scale out（InfiniBand 胖树）。
- **现实类比**：scale up 是把一个工位升级成"全能超人"；scale out 是招 1000 个普通工人并配好对讲机网络。

- **图示**：DGX SuperPOD 1K GPU 集群：140 个 DGX A100 节点（1120 块 GPU）组成一个 GPU POD；DGX A100 节点 = 2× AMD EPYC 7742 CPU + 8× A100 GPU + NVLink 3.0 全连接交换机；用 Mellanox HDR 200 Gb/s InfiniBand **全胖树（full fat-tree）**互连；计算网络与存储网络分离，支持自适应路由与 SharpV2 卸载。

4. Communication primitives（通信原语）

- **定义**：多节点协同的基本消息传递操作（rank = 一个加速器节点）：
- **AllReduce**：所有节点得到所有数据的归约结果，**AllReduce = ReduceScatter + AllGather**；
- **ReduceScatter**：先归约，再把结果按 rank 分片（每个节点各拿一部分）；
- **AllGather**：每个节点把自己的分片广播汇总，最终所有节点都拥有完整数据；
- **All-to-All**：每个节点把自己的数据分片发送给所有其他节点（如 rank 0 的 A0,A1,A2,A3 分别发给 rank 0..3）。
- **现实类比**：全班合写一份报告——AllReduce 是"每人把自己的部分发给大家并汇总成完整版"；ReduceScatter 是"先汇总统计，每人领回自己负责的那一段"；All-to-All 是"每人把自己写的章节分别发给所有同学"（像交换名片）。
- **公式/图示**：`AllReduce = ReduceScatter + AllGather 分布 GEMM 例子：inputA[MxK] × inputB[KxN] = out[MxN]，沿 K 维切分到 S 个 RDU 每个 socket 算 [MxK/S] × [K/SxN] = [MxN] 的部分结果 → S 份部分结果 → S-way reduce-scatter 合并`

5. Parallelism in AI models（AI 模型中的并行维度）

- **定义**：现代 AI 模型（Transformer）的张量有三个维度：**Batch_dim（批维度）**、**Sequence_dim（序列维度）**、**Hidden_dim（隐藏维度）**，加上模型自身的"层"维度。并行策略按切分哪个维度划分（第 36 页）：
- **Data Parallel (DP)**：按 batch 切分数据，各副本持有完整模型；
- **Tensor Parallel (TP)**：按 hidden_dim 切分权重/激活（如把权重矩阵劈成多块，各卡算一部分）；
- **Pipeline Parallel (PP)**：按层切分，不同层在不同设备上，数据流水式流过；
- **Expert Parallel (EP)**：Mixture-of-Experts 中不同 expert 放在不同设备；
- **Sequence Parallel (SP) / Context Parallel (CP)**：按序列/上下文长度切分。
- **通信模式对应**（第 37 页）：TP → ReduceScatter + AllGather 或 AllReduce；PP → Send-Receive；EP → All-to-All；DP → ReduceScatter + AllGather 或 AllReduce。
- **现实类比**：一个大型报告团队——按章节分工（PP：各写各的章节）、按主题分工（TP：每人负责一个主题列）、按读者分组复制（DP：每人服务一批读者）。
- **公式/图示**：`Activation Tensor 的维度： [Batch, Sequence, Hidden] × Layers Weight Tensor 的维度： [Hidden, Hidden] × Layers (+ vocab) 并行维度：DP(批) / SP·CP(序列) / TP(隐藏) / PP(层) / EP(专家)`

6. Distributed matrix multiply（分布式矩阵乘）

- **定义**：把单个超大 GEMM（如 M=24576, K=131072, N=8192）分发到多个加速器：**沿 K 维切分**，每个 socket 计算 [MxK/S] × [K/SxN] = [MxN] 的部分结果，得到 S 份 [MxN] 部分结果后做 **S-way reduce-scatter** 合并成最终 [MxN]。
- **现实类比**：多个小组各自算一部分"公共因子的乘积"（K 维是共享的求和维），最后把大家的中间结果汇总归约。
- **关键点**：选择切分维度决定了通信量——切 K 维只需一次 reduce-scatter；切 M/N 维则需要 AllGather 完整输入。

7. Compute-communication overlap（计算-通信重叠）

- **定义**：让通信（如 AllReduce）与计算（如 GEMM）并行进行。RDU 上把 AllReduce 与 Down GEMM 等流水化（"Pipelined AllReduce with Compute, no HBM traffic!"），使通信**不消耗 HBM 容量与带宽**；GPU 上若不做重叠，随着 socket 数增加通信时间占比上升，通信成为瓶颈（"GPUs need large interconnect bandwidth to get high utilization"）。
- **量化数据**（第 41 页，BS=16, M=24576, K=131074, N=8192，总 844.44 TFLOPs）：8/16/32 个 RDU 时，compute roofline 时间为 66.3/33.1/16.5 ms，reduce-scatter 时间为 8.6/9.7/15 ms；**不重叠的理论峰值利用率仅 88.5%/77%/52%**，而**重叠后实测利用率 72%/75%/79%**——32 socket 仍保持 70%+ 利用率。

- **现实类比**：厨师炒菜（计算）的同时，传菜员（通信）把上一桌的菜端出去——传菜不占用灶台，出菜率（利用率）不随桌数增加而崩。

8. Pipeline parallelism & micro-batch（流水并行与微批）

- **定义**：朴素流水并行（模型按层切分到多设备）存在**计算资源利用不足（under-utilization）与整体吞吐低（low overall throughput）**的问题——因为每层设备只在自己的数据到达时才干活，存在大量空闲（pipeline bubble）。**细粒度流水并行（fine-grained pipeline parallelism）**：把 mini-batch（每次迭代处理的样本数）**拆成多个 micro-batch**，让 forward 与 backward 计算在 micro-batch 之间流水化——设备 1 算完 micro-batch 1 的 forward 立刻传给设备 2，同时开始 micro-batch 2 的 forward，层层推进。
- **现实类比**：流水线上第一个工位不必等整个订单做完才开工——把大订单拆成小件（micro-batch），前一件还没做完，后一件已经开工，各工位始终忙碌。
- **公式/图示**：`朴素： F1 → F2 → F3 → B3 → B2 → B1（大部分时间只有一两个设备在干活） 微批： micro-batch1: F1→F2→F3→B3→B2→B1 micro-batch2: F1→F2→F3→B3→B2→B1 （错位流水，设备始终忙碌）`

9. DRAM basics（DRAM 工作原理）

- **定义**：DRAM 阵列由"1 晶体管 + 1 电容"存储单元组成（电容存电荷，读取是破坏性的）；每行约 2 Kbit 数据读入 **row buffer**；数据通过少量 **data pins（8 bit）** 传输给 memory controller。读一个字节的步骤：**Precharge（预充电，就绪位线，~10 ns）→ Row activation（行激活，把行读入 row buffer，~10 ns）→ Column selection（列选择）→ Transfer（数据上总线，~10 ns）**；若访问的是**已激活行（row hit）**，可跳过前两步，延迟大大降低——**DRAM 访问延迟不是固定的**！
- **现实类比**：图书管理员找书——预充电=把书架归位，行激活=把整个书柜搬到台面（row buffer），列选择=抽出要找的那本；如果连续找同一书柜的书（row hit），就不用反复搬书柜，快得多。
- **关键机制**：**burst mode**（一个命令批量传输多个连续列，摊销延迟）、**多 bank**（各 bank 共享引脚但可流水：一个 bank 在传输数据时另一个 bank 在做 precharge/activate，实现高引脚利用率）、**DIMM**（8 个 DRAM 芯片组成 64-bit 接口的模块，最小传输粒度 64 bit）。

10. Memory controller（内存控制器 = 访存请求调度器）

- **定义**：内存控制器接收 LLC 的 load/store 请求，负责：(1) 把物理地址映射到 DRAM 的 bank/row/column 几何结构；(2) 在相互冲突的目标（**最大化吞吐、最小化延迟、最小化能耗**）之间调度几十到几百个未完成的请求。常见调度策略：**FR-FCFS（first-ready, first-come-first-serve）**——优先服务当前已打开行的请求（最大化 row locality），其余按 FIFO；还会把多个小请求**合并**成大连续请求（利用 burst mode）。
- **现实类比**：机场塔台调度——优先让"跑道已清空"的航班（开放行）先起飞，其余按先来后到排队；把去同一目的地的小包裹合并成一次运输（合并请求）。
- **公式/图示**：`CPU → LLC(L3) → Memory Controller（按 bank 分队列）→ 64-bit bus → DRAM 调度目标：吞吐↑、延迟↓、能耗↓（互相冲突，需权衡） FR-FCFS：先服务开放行（row hit），再按 FIFO 服务其他行`

11. Data movement energy cost（数据搬运的能耗成本）

- **定义**：移动数据比计算贵得多（第 46–47 页）：
- 整数运算 ≈ 1 pJ；浮点运算 ≈ 20 pJ；
- 读 64 bit 片内小 SRAM（1mm 远）≈ 26 pJ；读 64 bit LPDDR ≈ **1200 pJ**；
- （另一组数据：fp32 数学运算 ~0.9 pJ；片内 SRAM 访问 ~5 pJ；LPDDR 读 32 bit ~640 pJ）
- 推论：以 10 GB/s 从内存读数据 ≈ **1.6 瓦**，而整个移动 GPU 的功率预算约 1 瓦；iPhone 16 电池约 14 瓦时。
- **结论**：**利用局部性极其重要（Exploiting locality matters!!!）**；在面向能效优化代码时，**重新计算（recompute）比存储再重载（store + reload）更划算**。
- **现实类比**：为了喝一杯水（一次运算）专程开车去水库（DRAM）取水，油费（能耗）是水本身价值的几百倍；不如把水存在手边（SRAM）或干脆用自来水（重算/近存计算）。

12. Locality & near-memory processing（局部性与近存处理）

- **定义**：解决内存瓶颈的多层次方案（第 72 页总结）：应用程序员——调度计算以最大化局部性（最小化数据搬运）；新硬件——智能 DRAM 请求调度、把数据搬到离处理器更近处（深层缓存层级、3D 堆叠）、加宽内存系统、研究在内存"内部/附近"执行有限计算（近存计算）、硬件加速压缩。**通用原则**：把数据存储放在处理器附近；把计算搬到数据存储处；数据压缩（用额外计算换取更少数据传输）。
- **现实类比**：与其每次都从市中心仓库（DRAM）取货，不如在工厂旁边建分仓（缓存/HBM）、甚至让小型加工设备直接开进仓库（近存计算）。

二、代码示例与详细解说（本讲重点）

示例 1：分布式矩阵乘（沿 K 维切分 + ReduceScatter 合并）

```
# 伪代码：S 个加速器（RDU/GPU）协同计算 C = A[M,K] @ B[K,N]
# 参数：BS=16, M=24576, K=131072, N=8192, S=4（示例为 4 个 socket）
# 策略：沿 K 维切分 → 各算部分结果 → S-way reduce-scatter 合并

import numpy as np
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
S    = comm.Get_size()          # 4 个 socket

# 1) 数据分片：A 每行完整 [M, K]，但只给本 rank K/S 列；B 每列完整 [K, N]，
#    但只给本 rank K/S 行（K 维切分）
A_local = load_tile("A", rows=range(M),    cols=range(rank*K//S, (rank+1)*K//S))  # [M, K/S]
B_local = load_tile("B", rows=range(rank*K//S, (rank+1)*K//S), cols=range(N))      # [K/S, N]

# 2) 本地 GEMM：每个 socket 计算一个 [M, N] 的部分结果
partial = A_local @ B_local        # [M, K/S] × [K/S, N] = [M, N]（K 维上只累加了 1/S）

# 3) S-way reduce-scatter：把 4 份 [M, N] 部分结果按列归约合并
#    ReduceScatter：先归约（逐元素相加），再把结果分片
out_local = np.zeros((M, N // S))
for k in range(S):
    # 每个 rank 把自己的 partial 中属于目标 rank 的列分片发出去
    col_chunk = partial[:, k*(N//S):(k+1)*(N//S)]
    comm.reduce_scatter_block(col_chunk, out_local, op=MPI.SUM)

# 最终：out_local 是 C 的 1/S 列分片，所有 rank 各持一份 → 完成分布式 GEMM
```

【代码做了什么？】 代码实现"分布式矩阵乘"：把 GEMM 的 **K 维（归约维）切分到 S 个加速器**。每个 socket 只持有 A 的 [M, K/S] 列分片与 B 的 [K/S, N] 行分片，本地算出一个 [M, N] 的"部分结果"——注意这个部分结果在 K 维上只累加了 1/S 项，所以**需要把 S 份部分结果逐元素相加**才是最终 C。第 3 步用 `reduce_scatter_block`（即幻灯片第 33 页的 ReduceScatter 原语）：先做归约（SUM），再把结果按列分片分发给各 rank——每个 rank 最终持有 C 的 1/S 列。这正是幻灯片第 38 页"Scale-up mapping（Example shown for 4 RDUs）"的软件表达：`GEMM(inA[MxK/4], inB_0[K/4xN]) → out0[MxN]` ×4 → Reduce-scatter → `out[MxN]`。

【并行机制解说】 对应本讲概念 4（communication primitives：ReduceScatter/AllReduce）、5（tensor parallelism：切 hidden/K 维）、6（distributed matrix multiply）。并行机制：S 个 socket **数据并行**地计算不同的 K 分片（每份工作量为 1/S），然后通过 **reduce-scatter** 通信合并。选 K 维切分的好处：每个 socket 的本地 GEMM 仍是完整的 [M,N] 形状（不损失计算形状），通信只发生在最后一步（一次 S-way reduce-scatter）；这等价于 AllReduce = ReduceScatter + AllGather 的前半段。注意本示例未做重叠——这正是示例 3 要解决的"通信成为瓶颈"问题。真实系统（第 39–41 页）会把 reduce-scatter 与 GEMM **流水化重叠**，才能在多 socket 下保持 70%+ 利用率。

示例 2：细粒度流水并行（micro-batch 流水训练）

```
# 伪代码：4 个设备（layer 分片）上的细粒度流水并行训练
# 思想：mini-batch（如 512 样本）拆成多个 micro-batch（如 4 × 128 样本），
#       forward 与 backward 在 micro-batch 间流水推进，消除 pipeline bubble

NUM_DEVICES   = 4          # 模型按层切成 4 段：设备 d 持有第 d 段层
MINI_BATCH    = 512
MICRO_BATCHES = 4
micro_size    = MINI_BATCH // MICRO_BATCHES   # 128 样本/micro-batch

# 每个设备 d 的执行逻辑（1F1B 风格的错位流水，简化为示意）：
for device in range(NUM_DEVICES):
    # forward 阶段：micro-batch i 的前向按层顺序穿过各设备
    for i in range(MICRO_BATCHES):
        x = recv_from(device-1, micro_batch=i)     # 接收上一层设备的激活（send-receive）
        h = my_layers.forward(x)                   # 本设备段的 forward
        send_to(device+1, h, micro_batch=i)
    # backward 阶段：梯度反向穿过各设备（错位：设备 0 先做 backward）
    for i in reversed(range(MICRO_BATCHES)):
        g = recv_from(device+1, micro_batch=i)
        grad = my_layers.backward(g)               # 本设备段的 backward
        send_to(device-1, grad, micro_batch=i)

# 关键：设备 0 算完 micro-batch 0 的 forward 后，立即开始 micro-batch 1 的 forward，
# 而不是等整批 forward 全部结束——各设备始终有活干（pipeline 填充）
```

【代码做了什么？】 代码示意"细粒度流水并行"：模型按层切到 4 个设备（pipeline parallelism），每次迭代的 mini-batch（512 样本）拆成 4 个 micro-batch（128 样本）。每个设备对每个 micro-batch 依次做 forward（从上游收激活、算自己那一段、传给下游），再反向做 backward。与朴素流水的区别在于**错位推进**：设备 0 完成 micro-batch 0 的 forward 后立刻处理 micro-batch 1，设备 1 收到 micro-batch 0 后开始干活的同时设备 0 已进入 micro-batch 1——流水被"填满"。设备间用 **Send-Receive** 通信（对应第 37 页表格：PP → Send-Receive）。

【并行机制解说】 对应本讲概念 8（pipeline parallelism & micro-batch）、5（PP 维度）。并行机制：**层间流水并行（spatial：不同层在不同设备）+ 时间流水（temporal：多个 micro-batch 在流水线中重叠）**。朴素的"一次只跑一个 batch"会让流水线大部分时间处于"只有一两个设备在算"的空闲状态（第 42 页：Under-utilization of compute resources, Low overall throughput）；拆 micro-batch 后，每个设备在稳态下几乎始终忙碌。第 44 页的表格展示了实际大模型的并行配置（如 145B 模型：TP=8, PP=8, Model parallel=64, DP=24, 1536 GPUs, batch=2304, 44% peak flops；1T 模型：TP=8, PP=64, DP=6, 3072 GPUs, batch 3072, 49% peak flops）——**pipelining schedule、microbatch size、degree of pipeline/tensor/data parallelism** 共同决定通信量、pipeline bubble 大小与内存占用。

示例 3：计算–通信重叠（Pipelined AllReduce with Compute）

```
# 伪代码：把 AllReduce 与 GEMM 重叠，让通信不阻塞计算、不占 HBM 带宽
# 场景：Llama 3.1 8B 的 Down GEMM 之后需要跨设备 AllReduce 累加梯度/激活
# 硬件：RDU（数据流），通信与计算在物理上并行执行

# 朴素（无重叠）：通信期间计算单元空闲，HBM 被通信流量占用
def no_overlap(down_out):
    partial = down_gemm(down_out, Wdown)     # 计算阶段
    allreduce(partial)                       # 通信阶段（等！计算空闲）
    return add(partial, residual)            # 后续计算

# 重叠（流水化）：把 AllReduce 拆成块，边算边通信
def with_overlap(down_out, num_chunks=4):
    result = []
    # 每个 chunk：先发自己的部分（reduce 的开始），继续算下一个 chunk
    for i in range(num_chunks):
        chunk = down_gemm_chunk(down_out, Wdown, i)   # 算第 i 块（与通信并行）
        partial = reduce_start(chunk)                 # 发起该块的归约（异步）
        result.append(partial)
    allreduce_finish(result)                          # 收尾：所有块的归约完成
    return add(result, residual)
# RDU 上：AllReduce 走专用的芯片间通路（不经过 HBM），与 Down GEMM/权重加载完全重叠

# 关键数据（第 40–41 页）：
#   不重叠时：8/16/32 sockets 的理论峰值利用率 88.5%/77%/52%（通信占比上升）
#   重叠后：  实测利用率 72%/75%/79% —— 32 socket 仍保持 70%+
```

【代码做了什么？】 代码对比"朴素 AllReduce"与"重叠 AllReduce"：朴素版在 GEMM 与 AllReduce 之间严格串行（通信时计算空闲）；重叠版把结果/梯度切成块，算一块、异步发起一块的归约、同时继续算下一块——通信与计算在时间上重叠。在 RDU 数据流架构上，AllReduce 走**芯片间专用通路**（第 59 页："Fully overlap allreduce with weight load and compute; Allreduce does not consume HBM capacity or bandwidth"），因此通信既不占用 HBM 带宽也不占用计算单元。第 39 页展示了 Llama 3.1 8B 中 "Down GEMM → Add → AllReduce" 在 RDU 0 与 RDU 1 之间的流水化示意（Pipelined AllReduce with Compute, no HBM traffic!）。

【并行机制解说】 对应本讲概念 7（compute–communication overlap）、4（AllReduce）。这是数据中心规模最关键的性能思想：**多 socket 扩展时，通信时间随 socket 数增加而增长**（8.6→9.7→15 ms），若不重叠，通信在 32 socket 时占掉近一半时间（利用率从 88.5% 掉到 52%）；重叠后即使 32 socket 也能维持 70%+ 利用率。要做到重叠需要硬件支撑：独立的通信通路（不占用 HBM）、异步通信原语、数据流式的 token 控制（无需 lock）。GPU 上"需要巨大的互连带宽才能获得高利用率"（第 40 页），因为 GPU 的通信与计算难以像 RDU 那样完全解耦。

示例 4：DRAM 行缓冲访问（row hit vs row miss 的延迟差异）

```
# 时序示意：读两个字节，展示 DRAM 行缓冲机制对延迟的影响
# 硬件：DRAM 阵列（2 Kbit/row）、row buffer、8-bit data pins、memory controller
# 假设 DDR3-1600 时序，各步骤 ~10 ns（约几个 memory clock）

# 第一次读字节 X（行 R，列 0）：row miss（最坏情况）
t=0    Precharge (PRE)          准备位线，~10 ns         # 位线未就绪
t=10   Row activate (RAS)       行 R 数据读入 row buffer，~10 ns
t=20   Column selection (CAS)   选列 0，~10 ns
t=30   Transfer                 数据上 8-bit bus → memory controller

# 第二次读同行的字节 X+1（row hit！最佳情况）
t=40   Column selection (CAS)   行已在 row buffer，跳过 PRE 和 RAS
t=50   Transfer                 直接传输，~10 ns
# 结论：row hit 延迟 ≈ row miss 延迟的 1/3（跳过 precharge + activate）

# 进一步优化：
# 1）Burst mode：一条 CAS 命令批量传输连续多列，摊销 PRE/RAS 开销
# 2）多 bank 流水：bank 0 在传输数据时，bank 1 同时做 precharge/activate
# 3）控制器调度（FR-FCFS）：优先服务开放行（最大化 row locality）
# 4）物理地址按字节粒度交错（interleaved）分布在多个芯片上：
#    64B 缓存行 = 8 个芯片并行各传 8 bit → 首次 64 bit 并行到达，而非串行
```

【代码做了什么？】 这是"DRAM 工作原理"的时序示意（第 48–62 页）：读一个字节要经过 Precharge → Row activate → Column select → Transfer 四步（各约 10 ns）；如果下一次访问落在**同一行**（row hit），可以跳过前两步。幻灯片用"读一个 64-byte cache line 的错误方式 vs 正确方式"对比说明：若 64B 缓存行全部由同一颗芯片串行服务，要 8 次列选+传输；若物理地址**按字节粒度交错**分布在 8 颗芯片上（DIMM 内），8 颗芯片**并行**各传 8 bit，首次 64 bit 同时到达——这解释了为什么"物理地址到 DRAM bank/row/column 的映射"是内存控制器的重要职责。

【并行机制解说】 对应本讲概念 9（DRAM basics）、10（memory controller）。这个示例展示了**存储系统中的三层并行/流水机制**：(1) **burst mode**——一次命令批量传多个连续列，摊销固定开销（类似"批量传输"摊销启动成本）；(2) **多 bank 流水**——不同 bank 共享 data pins，但 precharge/activate/传输可以在不同 bank 间流水进行（bank 0 传输时 bank 1 激活），实现高引脚利用率（第 56 页图：RAS/CAS/PRE 交错在不同 bank 上）；(3) **多芯片并行（DIMM/channel）**——8 芯片并行提供 64-bit 接口，或双通道（dual channel）两个 controller 独立发命令进一步加宽。这些机制共同决定"数据搬运成本"（第 11 概念），也是理解为什么"减少 off-chip 数据访问"能同时省能耗（1200 pJ vs 26 pJ）与提性能（row hit vs row miss）的基础。

三、关键要点

1. **内存是数据中心计算的根本瓶颈**：从 64-bit（CPU）到 6144-bit（H100 HBM3）的总线加宽、3D 堆叠（TSV/中介层）、多 bank 流水、burst mode、智能调度（FR-FCFS）——所有技术都在对抗同一件事：**让数据更快、更省地到达计算单元**。数据搬运能耗（LPDDR 1200 pJ vs 片内 SRAM 26 pJ）比运算本身贵两个数量级，"Exploiting locality matters!!!"。
2. **AI 模型有多个可切分的维度，每个维度对应一种并行策略**：DP（batch）、SP/CP（sequence）、TP（hidden）、PP（layers）、EP（experts）；每种策略对应特定通信原语（TP→RS+AG/AR、PP→Send-Receive、EP→All-to-All、DP→RS+AG/AR）。真实训练把多种并行组合使用（如 1T 模型：TP8 × PP64 × DP6 = 3072 GPUs）。
3. **分布式 GEMM 的本质是"切分 + 归约"**：沿 K 维切分 → 各算 [M,N] 部分结果 → reduce-scatter 合并；**通信与计算必须重叠**，否则 socket 数一多通信占比飙升（32 socket 不重叠利用率仅 52%），重叠后可持续 70%+。
4. **细粒度流水并行用 micro-batch 消除 pipeline bubble**：mini-batch 拆成 micro-batch，forward/backward 错位流水，各设备稳态忙碌；pipelining schedule、microbatch size、并行度配置共同决定通信量、bubble 与内存占用。
5. **降低能耗的两大思想：用对处理器（专用化）+ 少搬数据（局部性/近存/压缩）**：能效优化时，**重新计算比存储再重载更划算**（recompute beats store-and-reload）。
6. **从单芯片到数据中心的"映射"是全栈问题**：应用层（局部性调度）、架构层（智能调度、3D 堆叠、加宽内存、近存计算）、系统层（集群拓扑、通信原语、并行策略）协同解决内存墙。

四、常见陷阱与注意事项

1. **把"峰值带宽"当"实际带宽"**：H100 的 3.2 TB/s 是峰值；DRAM 访问延迟不固定（row hit vs row miss 差 2–3 倍），且引脚利用率受 latency 限制（第 54 页：Data pins in use only a small fraction of time）。只有行局部性好、请求被合并、bank 流水充分时才能接近峰值。
2. **忽视物理地址映射与交错（interleaving）**：64B 缓存行若全由同一芯片串行服务，延迟放大 8 倍；正确的字节粒度交错让 8 芯片并行。程序员看不到映射细节，但**访问模式（顺序 vs 随机、跨行 vs 同行）**直接决定 row hit 率——顺序访问、块状访问（tiling）是最安全的。
3. **多节点扩展时不考虑通信比例**：socket 数翻倍，计算时间减半但通信时间可能上升（8.6→15 ms）；不做 compute–communication overlap，利用率随规模崩塌（88.5%→52%）。**规模越大，越要重叠**。
4. **分布式 GEMM 选错切分维**：切 K 维只需一次 reduce–scatter；切 M/N 维需要 AllGather 完整输入，通信量更大。切分维度的选择要与后续算子（如 attention、norm）的通信需求一起考虑。
5. **流水并行直接套朴素 pipeline**：朴素按层切分、一次跑一个 batch 会有大量空闲（bubble），吞吐很低；必须拆 micro–batch 并设计流水 schedule（1F1B 等），否则"并行"反而更慢。
6. **只优化计算不优化数据搬运**：在能效与性能上，数据搬运都是大头（1200 pJ vs 20 pJ 浮点运算；10 GB/s 读取 = 1.6 W ≈ 整个移动 GPU 的功率预算）。**优先减少 off–chip 流量**（融合、tiling、重算、压缩），而不是盲目堆算力。

五、思考题（带答案）

Q1：为什么 H100 用 6144–bit 接口（6×HBM3×1024–bit）而不是继续加宽 64–bit 的 DDR 总线？请从"带宽 vs 引脚/功耗"与"3D 堆叠"两个角度解释，并说明为什么这对数据中心 AI 计算重要。

答案：加宽平面总线受限于引脚数量、走线面积与功耗（每根引脚都要驱动长距离信号）；HBM 用 **3D 堆叠**把 DRAM 层叠起来、用 **TSV（硅通孔）** 提供超高密度的垂直连接、通过硅中介层与处理器近距离互连——距离短 → 更宽的接口（1024–bit/stack）可行且更省电（第 5 页：Increase bandwidth, reduce power by chip stacking）。数据中心 AI 模型（如 1T 参数）的权重加载与激活交换需要 TB/s 级带宽，64–bit 总线根本喂不饱上千个 SM/PCU 的访存需求；HBM 的"更多带宽、高能效、小尺寸"三大优势让它成为 GPU/加速器的标配（H100 80 GB @ 3.2 TB/s）。这也是"内存墙"问题在硬件侧的主要解法之一。

Q2：给定一个 145B 参数的 Transformer（序列长 2048、词表 51200），第 44 页表格建议 TP=8、PP=8、DP=24、1536 块 GPU。请解释：为什么 TP 要切 hidden 维、PP 要切层、DP 要复制模型，三者各自解决什么问题？它们各自的主要代价是什么？

答案：(1) **TP（切 hidden）**解决"单卡装不下单层权重/激活"：把每层权重按 hidden 维劈开，多卡协作算一层；代价是每层后都需要 AllReduce/RS+AG 通信（通信量随 hidden 增大而增大）。(2) **PP（切层）**解决"整模型放不下"：不同层放不同设备，数据流水流过；代价是引入 pipeline bubble 与设备间 Send–Receive 通信（可通过 micro–batch 流水缓解）。(3) **DP（复制模型）**解决"吞吐不够"：多份完整模型副本并行处理不同 batch，是扩展吞吐的主要手段；代价是每步梯度 AllReduce（通信量随模型大小线性增长）。三者组合让 145B 模型在 1536 卡上达到约 44% 峰值 flops——多维度并行是"放得下 + 算得快 + 通得了"的权衡结果。

Q3：第 41 页数据显示：32 socket 时 reduce–scatter 理论时间 15 ms、compute roofline 16.5 ms，若不重叠理论利用率仅 52%，重叠后实测 79%。请解释为什么"不重叠时利用率会低于 50%（不是 16.5/(16.5+15)≈52%）"，以及重叠为什么能超过这个比例。

答案：不重叠的利用率上限 ≈ compute/(compute+comm) = 16.5/(16.5+15) ≈ 52%，这是"通信完全串行"的理论上界；52% 正是这个式子（第 41 页"Theoretical Peak utilization without overlap"）。实际可能更低，因为还有 kernel 启动、同步、负载不均等额外开销。重叠后通信与计算并行，理论上利用率可接近 100%（只要通信通路与计算通路不争抢资源），实测 79% 已接近"通信完全隐藏"的极限（剩余 21% 来自同步开销、流水线填充/排空、负载不均等）。这也解释了第 40 页的结论：GPU 上通信时间随 socket 数增加而增加、不重叠时通信成为瓶颈，因此 **GPU 需要巨大的互连带宽**（把通信时间压短），而 **RDU 靠重叠**（把通信藏到计算背后）在同样的互连上获得高利用率——两种不同的"躲开通信瓶颈"策略。

六、补充阅读：LLM Serving 相关概念（非本讲幻灯片内容）

> **说明**：本讲 Fall 2025 幻灯片聚焦数据中心存储系统与并行策略，未展开 LLM 推理服务（serving）的实现细节。以下为与该主题直接相关、常被问到的概念简介（基于业界通用知识，供延伸学习，不作为本讲考试范围依据）。

- **LLM serving（推理服务）**：把训练好的大模型部署为可响应在线请求的服务。与训练（追求吞吐、可批处理）不同，serving 还要满足**延迟（latency）约束**——每个用户的请求都要尽快得到回复。
- **Prefill（预填充）阶段**：处理用户 prompt 的阶段：一次性把整个 prompt 的所有 token 并行计算（GEMM 密集、算力受限、compute-bound），产出首个输出 token 并写入 KV cache。
- **Decode（解码）阶段**：逐 token 自回归生成阶段：每步只算一个新 token，但需要读取该序列全部的 KV cache——**访存密集、带宽受限（memory-bound）**，计算强度远低于 prefill。
- **KV cache（键值缓存）**：Transformer 自回归解码时，把已生成 token 的 Key/Value 张量缓存起来，避免每步重复计算历史 token 的注意力；其大小随序列长度与 batch 规模线性增长，常成为 serving 的内存瓶颈（HBM 容量受限时可用 KV cache compression——本讲 HBM4 幻灯片中提到的"KV cache compression"正是指这类技术）。
- **Batching / continuous batching（批处理 / 连续批处理）**：把多个请求合并成一批同时处理以提高 GPU 利用率（特别是把 compute-bound 的 prefill 与 memory-bound 的 decode 混批，或不断把新到的请求插入正在执行的批次，避免等待整批完成）；连续批处理显著提升 serving 吞吐，代价是需要动态管理每个序列的 KV cache 与调度策略。

这些概念与第 12 讲的主题紧密相连：prefill/decode 的访存特征差异决定了"为什么带宽与局部性如此重要"，KV cache 的管理体现了"数据搬运成本主导"的设计原则，而 continuous batching 则是"在数据中心规模最大化加速器利用率"的 serving 侧答案。

Lecture 13: Domain-Specific Programming Systems and AI-Driven Performance Optimization（日期：2025-11-06, Thursday）

> **概述**：本讲聚焦"如何提高性能优化工作的生产力"。核心想法有三条：(1) 提高抽象层次（Domain-Specific Languages, DSL），把"算法"与"调度"分离，代表作是图像处理 DSL —— Halide；(2) 智能搜索（automatic search / autotuning），让编译器在巨大的调度空间中自动寻找高性能实现；(3) 新兴的第三条路 —— 利用现代 LLM 的代码生成与问题求解能力，构建"AI 智能体"来自动写 kernel、profile、反思、迭代优化。课程最终指出：性能优化既需要专家知识又枯燥费时，是自动化（DSL 自动调度 + LLM 智能体）最有价值的应用场景之一。
>
> **注意**：本讲内容与 Assignment 4（Trainium2 上的 Fused Conv+MaxPool kernel，11 月 13 日截止）以及 Assignment 5（"写出世界上最快的 CUDA kernel"，12 月 4 日截止）的优化思路直接相关——你在作业中学到的 profile→定位瓶颈→改写 kernel 的循环，正是本讲"AI 智能体优化循环"中人类要做的同一件事。

一、核心概念与定义

1. Performance / Productivity / Generality 三角（理想并行语言）

- **定义**：任何并行编程语言都在三个维度上权衡：**Performance**（能否榨干硬件性能）、**Productivity**（写起来是否省力）、**Generality**（能否表达各种不同问题）。不存在三者兼得的"理想并行编程语言"（slide 中借用 Pat Hanrahan 的经典设计）。
- **现实类比**：就像"好吃、便宜、快"不可能同时满足的餐厅三难（不可能三角）。C++/ISPC/CUDA 性能强、通用性强，但生产力极低；Python 生产力高但性能差。
- **图示**：

```
          Performance（性能）
              /\
             /  \
            /    \
           / 理想语言 \
          /   （不存在）\
         /______________\
  Productivity        Generality
   （生产力）           （通用性）
```

2. DSL（Domain-Specific Language，领域特定语言）

- **定义**：针对某一特定领域、表达能力受限（restricted expressiveness）的编程语言；通常是高层、声明式（declarative）且确定性的（deterministic）。
- **现实类比**：餐馆菜单而不是通用菜谱大全。菜单只列本店能做的菜（受限表达力），点菜就是"声明我要什么"（declarative），厨房怎么做（调度）由后厨决定。
- **要点**：限制表达力正是换取性能与生产力的手段——系统因为知道"你只会做这几种菜"，才能为每一种菜准备最优做法。

3. Domain-Specific Programming System（领域特定编程系统）

- **定义**：围绕 DSL 构建的完整系统。核心思想是**提高表达程序的抽象层次**，目标是：(a) 快速为某台目标机器写出高性能程序；(b) 一份程序在不同机器上都能高效运行。做法是引入**针对应用领域的高层编程原语**（primitives）。
- **现实类比**：快递公司分拣系统。你只需写"把包裹送到 X 城市"（高层原语），分拣系统（领域知识）自动决定用哪条运输线路、哪种车辆（算法选择）、如何合并拼车（并行化策略），甚至物流网络本身（硬件）也为这种抽象优化。
- **关键句（slide 8）**：优化不止是"把软件高效映射到硬件"——**硬件平台本身也可以针对这些抽象来优化**。代价是：**通用性/完备性的损失**（loss of generality/completeness）。

4. Halide（图像处理 DSL）

- **定义**：一个嵌入 C++ 的、用于描述**图像处理操作序列**的简单 DSL（Jonathan Ragan-Kelley、Andrew Adams 等人，SIGGRAPH 2012 / PLDI 2013）。已在 Google 手机相机管线（HDR+、人像模式等）、Instagram、Adobe 等生产环境中使用。
- **现实类比**：写图像处理就像写"烹饪配方"：你只管描述"最终菜品长什么样、每步用什么原料"，至于用什么锅、多大火、先切后炒还是先炒后切（循环顺序、向量化、多核并行）是另一层的事情。
- **核心抽象**：Halide **Func**（函数）把整数坐标映射到值（如像素颜色）；**Halide expression** 是无副作用（side-effect free）的表达式，描述如何用其他函数的值计算某点上的值。程序本质是一张 **DAG**（数据流图）。

5. Algorithm / Schedule 分离（算法与调度分离）

- **定义**：Halide 把程序拆成两层——**algorithm**（"做什么"：声明式描述每个输出像素如何由输入计算而来）与 **schedule**（"怎么做"：循环顺序、分块 tiling、向量化 vectorize、多核并行 parallel、中间量在哪里计算 compute_at）。程序员用一组**调度原语**给出高层"草图"，由 Halide 编译器机械地生成底层平台特定代码（pthreads、AVX intrinsics 等）。
- **现实类比**：电影导演 vs 摄影组。导演（程序员）决定"这场戏拍什么内容、机位大致怎么摆"（algorithm + 高层 schedule），摄影、灯光、场务（编译器）负责把细节落地。
- **要点**：Halide 的哲学是——**程序员负责算法，并对如何高效调度有直觉；系统（编译器）不"聪明"，只负责机械地把调度草图翻译成目标机器上的具体机制**。

6. 调度原语（Scheduling Primitives）

- **定义**：Halide 提供的描述 N 维域迭代方式的指令，可同时指定**迭代顺序**与**并行化方式**（多线程、SIMD 向量化）。常用组合：
- `tile(x, y, xi, yi, W, H)`：把外层 (x,y) 循环按 W×H 分块，产生内部 (xi,yi) 循环；
- `vectorize(xi, 8)`：把 xi 循环用 8 宽 SIMD 指令实现；
- `parallel(y)`：把 y 循环用线程并行；

- `compute_at(out, x)` / `compute_root()`：决定中间 Func 在哪个循环层级计算（影响局部性与中间存储大小）。
- **现实类比**：装修时给工人下"施工指令"——"每个房间（tile）内，先铺地板再刷墙（循环顺序），地板用 8 块并行铺（vectorize），不同房间分给不同工人（parallel）"。

7. 算术强度与局部性（Arithmetic Intensity & Locality）

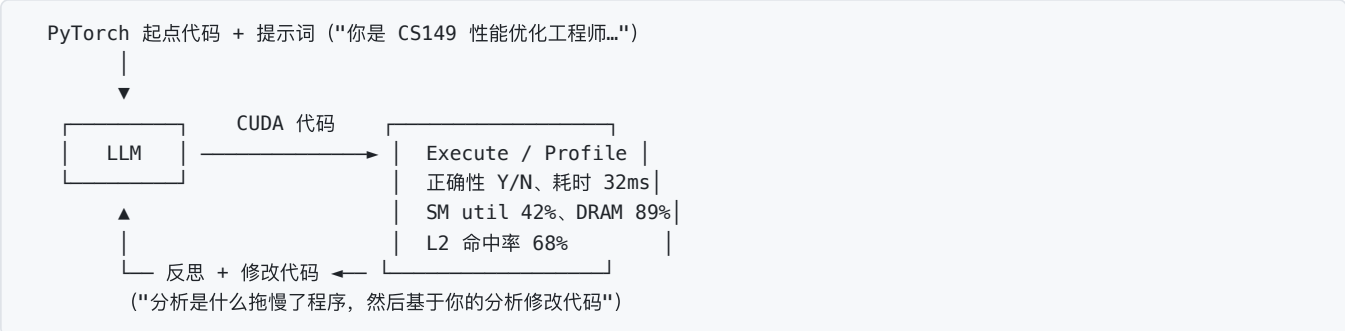
- **定义**：3x3 box blur 直接实现的总工作量是 9×WIDTH×HEIGHT（N×N 滤波器为 N^2×W×H）；利用可分离性（separable filter）改成两遍 1D 滤波后为 6×W×H（N×N 为 2N×W×H），**算术强度降低 2 倍**（每像素的乘加次数变少）。局部性分析关注：每个数据被复用几次、是否重复加载、缓存行是否被充分利用。
- **现实类比**：买食材做 9 道菜——"直接法"每道菜都单独去菜市场买齐 9 种配料（重复加载）；"两遍法"先集中采购一批、再做中间处理，食材（数据）被反复利用。
- **公式**：
- 直接 2D 滤波：`work = N² × WIDTH × HEIGHT`
- 两遍可分离滤波：`work = 2N × WIDTH × HEIGHT`
- 分块版（CHUNK_SIZE=16）：`work = (34/16) × 3 × WIDTH × HEIGHT ≈ 6.4 × W × H`，随 CHUNK_SIZE 增大趋近理想值 6×W×H。

8. 自动调度 / 自动调优（Autoscheduler / Autotuning）

- **定义**：把"找好调度"建模为**在调度空间中进行序列化决策的搜索问题**：从 DAG 末端开始，对每个节点 N 依次决定 (1) N 在现有循环嵌套中的位置（即 `compute_at` 到哪一层）；(2) N 的 tile 大小。然后用**搜索算法**（greedy search、beam search）在数十万乃至上百万个候选调度中寻找代价最小的一个。
- **现实类比**：装修公司报价系统——对"地板、墙、吊顶"每道工序，系统枚举"在哪个房间做、用什么尺寸"的所有组合，用一个快速估价模型挑出最便宜的方案。
- **关键数据（slide 41）**：代价估计用一个简单 **MLP（多层感知机）**，每个调度只需几十微秒（1.4M 个调度在 166 秒内测完）；该 MLP 在大量随机生成的 Halide 程序上训练（把程序编译执行得到真实代价作为标签），实际输出 27 个系数，代入一个手工设计的代价模型。

9. LLM 智能体优化循环（LLM Agent Optimization Loop）

- **定义**：把"性能优化工程师"的工作流程自动化：给定起点代码（如 PyTorch）+ 提示词（"你是 CS149 性能优化工程师，请把它改写成高性能 CUDA"）→ LLM 生成 kernel → **执行/Profile**（正确性 Y/N、耗时、SM 利用率、DRAM 利用率、L2 命中率等统计）→ 把 profile 统计反馈给 LLM 让它**反思**瓶颈原因并**修改代码** → 循环直到达标。这就是"通过反思进行试错"（trial and error via reflection）。
- **现实类比**：带教实习生——导师（profile 工具）每轮给出"这版哪里慢"的诊断，实习生（LLM）据此改下一版，而不是一次性写出完美代码。
- **图示**：



10. KernelBench 与 DNN DSL（Triton / CUTLASS-CuTe / TileLang）

- **定义**：**KernelBench** 是一个包含数百个 PyTorch kernel 的基准测试集，LLM 智能体的目标是自动产出**又快又正确**的 CUDA kernel。另外，为 DNN 编写的 DSL（Triton、CUTLASS/CuTe、TileLang）也能帮助自动化：LLM 拼装的是**高层高性能原语**而不是手写底层 CUDA，因此更不容易出错/幻觉；挑战在于"用得少的语言训练数据少，LLM 容易写错，但会随着时间改善"。
- **现实类比**：给装修工人提供标准化预制件（DSL 原语）而不是让他现场砌砖（底层 CUDA）——出错率低、速度快，前提是工人熟悉这些预制件的规格。

11. LLM 智能体的四种自我改进思路（Idea 1–4）

- **定义**（slides 50–54）：
- **Idea 1：基于经验微调（fine-tune）**——用大量同类任务的经验微调一个专用 LLM，需要大量任务样本与微调大模型的能力。
- **Idea 2：维护"优秀例题"数据库**——智能体不断积累"优质 kernel 解决方案库"（如用 Thunderkittens 或 CuTe 写的高质量方案），新问题来时**检索最相关的例题**辅助生成；库里不仅存方案，还存**一系列优化决策的序列**。
- **Idea 3：从经验中优化提示词（prompt optimization）**——不直接给相关例子，而是由一个 prompt optimizer 观察优化循环的历史轨迹（trajectories），总结出重要事实与原则，更新给 LLM 的提示词。
- **Idea 4：穷举搜索 + LLM 智能体结合**——把 Halide 式自动调优的穷举搜索技术与上述 agentic 思路结合；优化成本极高，但能取得一些最好的结果。
- **现实类比**：实习生成长三阶段——(1) 送去集训班（fine-tune）；(2) 建立自己的"错题本/优秀作业本"并考前翻看（retrieval DB）；(3) 把老师批改意见总结成自己的"工作守则"（prompt optimization）。

12. "正确的表示"（The Right Representation）原则

- **定义**（slide 28）：任何系统的设计核心是**为任务选择正确的表示**。好的表示应满足：(a) **生产力**——体现人思考问题的自然方式；(b) **让系统能提供服务**——如正确性/资源界/类型检查等保证，以及性能服务（并行化、向量化、专用硬件利用）。Halide 的贡献在于：任务从"表达图像处理计算"变成了"为特定 Halide 程序生成高效实现"——正是因为算法/调度两种表示被分开，系统才能插手优化。
- **现实类比**：用坐标纸画工程图（结构化表示）才能让 CAD 自动算面积、做有限元分析；随手素描（自由表示）画得再像，软件也帮不上忙。

13. 调度搜索空间与代价模型（Cost Model）

- **定义**（slides 40–41）：调度空间由"每个节点的 compute_at 层级 × tile 尺寸"的所有组合构成，可达数十万以上候选。搜索需要**代价估计**：本讲采用 AI 方法——一个简单 **MLP** 对"程序 + 调度"打分，单次只需**几十微秒**（1.4M 个调度 166 秒测完）；MLP 在**随机生成的 Halide 程序库**上训练（把这些程序真实编译执行得到代价作为标签），实际输出 **27 个系数**，代入**手工设计的代价公式**得到最终估计值。
- **现实类比**：买房时用"估价模型"快速给几十万套房打分，而不是每套都实地看房（实地看房 = 真实编译执行，太贵）。

14. 可移植性与硬件协同设计（Portability & HW Co-Design）

- **定义**（slide 8）：领域特定编程系统的两大目标之一是"**写一份程序，在不同机器上高效运行**"（write one program, run it efficiently on different machines）。更进一步：**优化不止是软件到硬件的映射——硬件平台本身也可以针对这些抽象进行优化**（the hardware platform itself can be optimized to the abstractions）。代价是通用性/完备性的损失。
- **现实类比**：标准集装箱（抽象原语）既让货主只写一次"装箱单"，也让码头、轮船、卡车（硬件）全部按集装箱规格设计——物流全链条都因统一抽象而高效，但散货（通用性）就没法用这套系统。

二、代码示例与详细解说（本讲重点）

示例 1：朴素的 3×3 box blur（直接 2D 滤波）——"性能不好的起点"

代码（C）：

```
int WIDTH = 1024;
int HEIGHT = 1024;
float input[(WIDTH+2) * (HEIGHT+2)];   // 输入图像，四周各留 1 像素边界
float output[WIDTH * HEIGHT];
float weights[] = {1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9,
                   1.f/9, 1.f/9, 1.f/9};

for (int j=0; j<HEIGHT; j++) {
  for (int i=0; i<WIDTH; i++) {
    float tmp = 0.f;
    for (int jj=0; jj<3; jj++)
      for (int ii=0; ii<3; ii++)
        tmp += input[(j+jj)*(WIDTH+2) + (i+ii)] * weights[jj*3 + ii];
    output[j*WIDTH + i] = tmp;
  }
}
```

【代码做了什么？】 – 这是 CS149 幻灯片上的原始"朴素版"：对每个输出像素 `(i,j)`，累加其 3×3 邻域 `(jj,ii)` 与 3×3 权重模板 `weights` 的逐元素乘积，得到模糊后的像素值。 – 计算总量是 `9 × WIDTH × HEIGHT` 次乘加：对 N×N 滤波器推广为 `N² × WIDTH × HEIGHT`。 – 输入数组刻意做成 `(WIDTH+2) × (HEIGHT+2)`，即四周有一圈 padding，这样边界像素的 3×3 邻域也能无分支地访问。

【并行机制解说】 – 这段代码**尚未做任何并行优化**：没有多线程、没有 SIMD、也没有调整循环顺序。它是本讲性能讨论的"基准起点"。 – 从并行角度观察：输出像素彼此独立（每个 `output` 元素只依赖 `input` 的 3×3 邻域），因此天然可并行——这正是 Halide 用**声明式表达式**（`out(x,y) = f(in 邻域)`）能够表达、并由调度器自动生成并行循环的原因。 – 对应概念：**算术强度 / 局部性 / 提升抽象层次**——人类直接手写并行版本（如 slide 22 的优化 C++）虽然能快 10 倍，但代码变得"看不懂、只适用于 SSE、只适用 CPU"；Halide 想解决的就是这个问题。

示例 2：可分离两遍滤波 + 分块（chunked）——"手工优化之旅"

代码（C）（对应 slide 20 的 chunked version 2，CHUNK_SIZE=16）：

```
int WIDTH = 1024;
int HEIGHT = 1024;
int CHUNK_SIZE = 16;
float input[(WIDTH+2) * (HEIGHT+2)];
float tmp_buf[WIDTH * (CHUNK_SIZE+2)];   // 中间缓冲：只放 CHUNK_SIZE+2 行
float output[WIDTH * HEIGHT];
float weights[] = {1.f/3, 1.f/3, 1.f/3};

for (int j=0; j<HEIGHT; j += CHUNK_SIZE) {          // 外层：按 CHUNK 行处理
  // 第一步：水平模糊，产出本 chunk 需要的 CHUNK_SIZE+2 行 tmp_buf
  for (int j2=0; j2<CHUNK_SIZE+2; j2++)
    for (int i=0; i<WIDTH; i++) {
      float tmp = 0.f;
      for (int ii=0; ii<3; ii++)
        tmp += input[(j+j2)*(WIDTH+2) + i+ii] * weights[ii];
      tmp_buf[j2*WIDTH + i] = tmp;
    }
  // 第二步：垂直模糊，由 tmp_buf 的 3 行生成 CHUNK_SIZE 行输出
  for (int j2=0; j2<CHUNK_SIZE; j2++)
    for (int i=0; i<WIDTH; i++) {
      float tmp = 0.f;
      for (int jj=0; jj<3; jj++)
        tmp += tmp_buf[(j2+jj)*WIDTH + i] * weights[jj];
      output[(j+j2)*WIDTH + i] = tmp;
    }
}
```

【代码做了什么？】 – 利用 3×3 box 滤波的**可分离性**（separable）：2D 模板 = 水平 1D 模板 ⊗ 垂直 1D 模板，于是先做水平 1D 模糊（读 input 写 tmp_buf），再做垂直 1D 模糊（读 tmp_buf 写 output）。 – 直接两遍法总工作量是 `6 × W×H`（每像素 6 次乘加），比直接 2D 法的 `9 × W×H` 少 1/3；代价是需要 `W × (H+2)` 的中间存储，且多了对 tmp_buf 的读写流量。 – 分块版把中间缓冲压缩到 `W × (CHUNK_SIZE+2)`（只保留产出当前 chunk 输出所需的最少行数），让**整块 tmp_buf 能装进缓存**，捕获"生产者–消费者局部性"。 – 工作量核算：每个 chunk 第一步 `18×3×WIDTH`（水平模糊 CHUNK_SIZE+2=18 行）、第二步 `16×3×WIDTH`；摊到整幅图是 `(34/16)×3×W×H = 6.4×W×H`，**随着 CHUNK_SIZE 增大趋近理想值 6×W×H**（因为重叠的边界行占比变小）。

【并行机制解说】 – 这一步的"并行"其实发生在**缓存层级**（cache-level parallelism of the memory system）：tmp_buf 的所有读写都命中缓存，避免把中间结果写回内存再读回——这是**利用局部性减少片外通信**，与多核/向量化正交。 – 但 slide 21 明确指出"还没做完"：循环尚未为多核并行化、未用 SIMD、未做循环展开等基本优化。手工把这些全做上（slide 22 的优化 C++）需要：SSE 向量 intrinsics、256×32 分块迭代顺序最大化命中率、按行切分图像多核执行、把两遍融合使 tmp 数据直接从缓存读取——最终比两遍朴素版快约 10 倍，但代码可读性极差且不可移植。 – 对应概念：**局部性 / 算术强度 / "手工优化 = 低生产力"**——这正是 Halide 想用"调度"这一层抽象自动化掉的工作。

示例 3：Halide——算法与调度分离

代码（C++，嵌入 Halide DSL）（对应 slide 23/29 的完整版本，含 bright 与查表）：

```
#include "Halide.h"
using namespace Halide;

int main() {
    Var x, y, xi, yi;
    Func blurx, blury, bright, out;
    Buffer<uint8_t> in    = load_image("myimage.jpg");   // 输入照片
    Buffer<uint8_t> lookup = load_image("s_curve.jpg");  // 255 像素 1D 查找表

    // ---- 算法描述（algorithm：声明"做什么"，无任何循环）----
    blurx(x,y) = 1/3.f * (in(x-1,y) + in(x,y) + in(x+1,y));      // 水平 1D 模糊
    blury(x,y) = 1/3.f * (blurx(x,y-1) + blurx(x,y) + blurx(x,y+1)); // 垂直 1D 模糊
    bright(x,y) = min(blury(x,y) * 1.25f, 255);                  // 提亮 25% 并截断
    out(x,y) = lookup(bright(x,y));                              // 查表对比度增强

    // ---- 调度描述（schedule：声明"怎么做"）----
    out.tile(x, y, xi, yi, 256, 32)   // 2D 分块 256×32
       .vectorize(xi, 8)              // 内部 xi 循环 8 宽 SIMD
       .parallel(y);                  // 外部 y 循环多线程
    blurx.compute_at(out, x)          // blurx 在每个 tile 内按需计算
         .vectorize(x, 8);            // 内部 x 循环 8 宽 SIMD

    // 在 1024×1024 域上执行整条流水线
    Buffer<uint8_t> result = out.realize(1024, 1024);
    return 0;
}
```

【代码做了什么？】 – 算法部分：`blurx(x,y)` 是"坐标 (x,y) 处的水平模糊值 = 输入图像同一行三个相邻像素的加权平均"；`blury` 在 blurx 上再做垂直 1D 模糊；`bright` 提亮并 clamp 到 255；`out` 用 bright 的值查 s_curve 查找表。四个 Func 形成一张 DAG：`in → blurx → blury → bright → out`（外加 `lookup`）。 – 每个 Halide 表达式都是**无副作用的纯函数定义**：它只说明"要算某点的值需要哪些其他点的值"，**不规定迭代顺序、不规定哪些中间值要存下来**——迭代整个域是隐式的（`realize` 触发）。 – 调度部分：`out.tile(x, y, xi, yi, 256, 32)` 生成外层 (x,y) 与内层 (xi,yi) 的两级循环；`.vectorize(xi,8)` 把 256 宽的内层 xi 循环改成 8 宽 SIMD（编译器自动处理 256+2 不能被 8 整除的边界条件）；`.parallel(y)` 用线程并行化外层 y；`blurx.compute_at(out, x)` 表示 blurx 在**每个输出 tile 内部**按需计算（只需分配 258×34 左右的小缓冲），`vectorize(x,8)` 让 blurx 的计算也向量化。

【并行机制解说】 – 并行如何实现？**全部由编译器完成**：程序员只给了 4 行高层"调度草图"，Halide 编译器机械地把它展开成等价的并行循环嵌套（pthreads + AVX intrinsics + 边界条件处理）——这正是 slide 35 所说"系统（编译器）不聪明，它提供的是把调度细节机械落地到目标机器机制（pthreads、AVX 等）的服务"。 – 数据共享/同步点：多线程并行的是外层 tile 循环，每个线程处理不同的 y-tile，输出像素互不重叠，因此**线程间无数据竞争**；共享的是只读的 input/lookup。blurx 是每个线程私有的小块缓冲（生产者–消费者局部性在单线程内由缓存捕获）。 – 对应概念：**Algorithm/Schedule 分离、调度原语、DSL 提升抽象层次**。Halide 的代价是通用性受限：只支持规则 N-D 域上的前馈流水线（加上 reduction 与固定深度递归的特别支持），且要求编译器能推断所有依赖（slide 36）——"受限表达力"换来的是自动化的可能性。

示例 4：自动调度 = 序列化决策 + 搜索（autoscheduler 伪代码）

代码（伪代码）（对应 slide 39–41 的调度搜索建模）：

```
// 输入：Halide 程序 DAG（节点 = Func），如 in → blurx → blury → out
// 输出：一个完整的 schedule（每个节点的 compute_at 位置 + tile 大小）

function AUTOSCHEDULE(dag):
    schedule = EMPTY_SCHEDULE
    // 从 DAG 末端（输出节点）开始，逆向依次调度每个节点
    for node in REVERSE_TOPOLOGICAL_ORDER(dag):
        candidates = []
        for loop_level in ALL_LOOP_LEVELS(schedule):        // 决策 1：compute_at 放哪层
            for (tw, th) in ALL_TILE_SIZES:                 // 决策 2：tile 尺寸
                s = schedule + PLACE(node, loop_level, tw, th)
                cost = COST_MODEL(dag, s)                   // MLP 估计：几十微秒/次
                candidates.append((cost, s))
        schedule = BEAM_SEARCH_PICK(candidates)             // 保留 beam 宽度的最优分支
    return schedule

function COST_MODEL(dag, s):
    // 用训练好的 MLP 估算：(1) 该 schedule 的循环开销、缓存局部性、
    //                      (2) 向量化/并行的收益……
    // MLP 不直接输出代价，而是输出 27 个系数，代入手工设计的代价公式
    coeffs[27] = MLP_ENCODE(dag, s)
    return HANDCRAFTED_COST(coeffs)      // 估计吞吐量/像素每秒
```

【代码做了什么？】 – 外层循环按**逆拓扑序**（从输出往前）逐个 Func 决定两件事：它应该 `compute_at` 到当前循环嵌套的哪一层、以及它的 tile 尺寸。每做一个局部决策，就用代价模型给"当前部分完成的调度"打分（slide 40 图中每个节点旁的数字就是该部分调度的估计代价）。 – 搜索策略用 **greedy search / beam search**：每次保留代价最低的若干候选分支继续扩展，而不是枚举全部（全部可能多达几十万个以上）。 – 代价模型是 **AI 驱动的**：一个简单 MLP 用几十微秒即可给一个调度打分——slide 41 报告 1.4M 个调度 166 秒测完；MLP 在**大量随机生成并真实编译执行的 Halide 程序**上训练，输出 27 个系数交给手工设计的代价公式。

【并行机制解说】 – 这个"并行"是**元层面的并行搜索**：不是程序运行时的并行，而是"调度空间"中的并行搜索与加速。MLP 之所以能快到几十微秒/次，是因为它把"编译+执行+测量"这一昂贵流程替换为一次前向推理。 – 结果（slide 42–43）：autoscheduler 生成的调度**与已知最好的手工调度相当**（在 CPU 上做图像处理，想手工写出更好的调度相当困难）；且能大幅节省专家时间——图中人类专家（Dillon、Andrew）花几十分钟手工调参达到的吞吐量，自动调度器很快就能达到甚至超越。 – 对应概念：**自动调优 / 智能搜索**。课程强调：Halide 调度抽象之所以能自动化，正是因为**高层的调度原语让"所有可能调度"这个空间可以被干净地枚举**（slide 44）——反观 C++，"搜索所有可能的 C++ 程序排列"根本无从谈起。

示例 5：LLM 智能体优化循环（KernelBench 场景）

代码（伪代码）（对应 slide 46 的 "trial and error via reflection"）：

```
PROMPT_INIT = "You are a performance optimization engineer in CS149.
               Please rewrite the following PyTorch code as high performance
               code in CUDA. Keep in mind the code optimization principles
               we discussed in class…"

PROMPT_REFLECT = "You are an optimization engineer in CS149. Given the input
                  code and the profiling statistics produced by running the
                  code on an H100 GPU, reflect on what might be slowing the
                  program down. Then, given the code and your reflection,
                  make an edit to the code to address the reason for the
                  slowdown that you identified."

kernel = LLM_GENERATE(pytorch_code, PROMPT_INIT)
for round in 1..MAX_ROUNDS:
    result = EXECUTE_AND_PROFILE(kernel)        // 在 H100 上编译运行
    stats = { correct: result.passes,           // 正确性 Y/N
              time_ms: result.time,             // 耗时：32 ms
              sm_util: result.sm_util,          // SM 利用率：42%
              dram_util: result.dram_util,      // DRAM 利用率：89%
              l2_hit: result.l2_hit_rate }      // L2 命中率：68%
    if stats.correct and stats.time_ms < TARGET: break
    kernel = LLM_EDIT(kernel, stats, PROMPT_REFLECT)   // 反思 + 修改
return kernel   // 最终：又快又正确的 CUDA kernel
```

【代码做了什么？】 – 第一阶段：给 LLM 起点代码（PyTorch）与角色提示词，生成第一版 CUDA kernel。提示词刻意让 LLM "扮演 CS149 性能优化工程师"，并提醒它运用课堂上学过的优化原则（局部性、算术强度、向量化、占用率等）。 – 第二阶段（循环）：把 kernel 放到真实硬件（如 H100）上执行并 profile，得到**正确性、耗时、SM 利用率、DRAM 利用率、L2 缓存命中率**等统计；把这些统计原样喂给 LLM，让它**反思**"什么在拖慢程序"，然后基于反思**修改代码**；循环直到正确且达标。 – 评价基准：**KernelBench**——一个含数百个 PyTorch kernel 的基准集，目标就是让 LLM 智能体自动产出快且正确的 CUDA kernel。

【并行机制解说】 – 这里的"并行/加速"体现为**闭环自动化**：人类专家的"写代码→跑→profile→分析→改"循环被 LLM + 工具执行。它依赖两个前提：可执行、可测（profile 工具给出可解释的硬件计数器），以及 LLM 能把"统计数字 ↔ 代码结构"联系起来（如 DRAM 利用率高但 SM 利用率低 → 访存受限 → 需要改善数据复用/合并访问）。 – 与之互补的是 **DNN 领域 DSL**（slide 48）：让 LLM 拼装 Triton / CUTLASS-CuTe / TileLang 的高层原语，而不是手写裸 CUDA——正确性失误/幻觉更少；但冷门语言训练数据少，LLM 容易写错（预计随时间改善）。 – 对应概念：**LLM 智能体优化循环、KernelBench、AI 驱动优化**。课程还列出四条进化路线（slide 50–54）：fine-tune 专用 LLM、建立例题数据库做检索增强、用经验优化提示词、以及把穷举自动调优与 LLM agentic 结合（成本极高但效果最好）。

示例 6：遗传算法式自动调优（OpenTuner 风格）——搜参数 vs 搜调度

代码（Python 伪代码，OpenTuner/遗传算法风格）（补充"智能搜索"一类的经典形态）：

```
import random

# 可调参数空间（以 CUDA kernel 为例）：每个参数 = kernel 的一个变体维度
PARAM_NAMES = ["block_size_x", "block_size_y", "unroll_factor", "use_shared_mem", "vector_width"]
PARAM_SPACE = {
    "block_size_x":   [32, 64, 128, 256],
    "block_size_y":   [1, 2, 4, 8],
    "unroll_factor":  [1, 2, 4, 8],
    "use_shared_mem": [True, False],
    "vector_width":   [1, 2, 4],
}

def random_config():
    return {p: random.choice(v) for p, v in PARAM_SPACE.items()}

def evaluate(config):
    """把参数代进 kernel 模板、编译、在目标机器上运行并计时（真实测量）"""
    kernel_src = generate_kernel_from_template(config)   # 模板实例化
    compile_and_load(kernel_src)
    return measure_time_ms()                             # 越小越好

def mutate(cfg):
    cfg = dict(cfg)
    p = random.choice(PARAM_NAMES)
    cfg[p] = random.choice(PARAM_SPACE[p])               # 随机改一个参数
    return cfg

def crossover(a, b):
    return {p: (a[p] if random.random() < 0.5 else b[p]) for p in PARAM_NAMES}

def genetic_autotune(pop_size=16, generations=50):
    population = [random_config() for _ in range(pop_size)]
    for gen in range(generations):
        # 评估整代（可并行：每个个体独立编译+运行）
        scored = sorted([(evaluate(c), c) for c in population])
        best_time, best_cfg = scored[0]
        print(f"gen {gen}: best = {best_time} ms  {best_cfg}")
        # 精英保留 + 交叉 + 变异，生成下一代
        elites   = [c for _, c in scored[:pop_size // 4]]
        children = [crossover(random.choice(elites), random.choice(elites))
                    for _ in range(pop_size // 2)]
        children += [mutate(random.choice(elites)) for _ in range(pop_size - len(elites) - len(children))]
        population = elites + children
    return best_time, best_cfg

best_time, best_cfg = genetic_autotune()
print("best:", best_time, best_cfg)
```

【代码做了什么？】 – 把 kernel 优化建模为**参数搜索**：每个"配置"是 block 尺寸、循环展开、共享内存、向量宽度等参数的组合；`evaluate` 负责把配置实例化成真实代码、编译并在目标机器上**实测计时**（真实硬件反馈，不是模型估计）。 – 遗传算法流程：随机初始化种群 → 每代按实测性能排序 → 精英保留（top 1/4）→ 交叉（两个精英各取一半参数）→ 变异（随机改一个参数）→ 生成下一代；跑若干代收敛到最优参数。 – 这是 OpenTuner 等经典 autotuner 的思想骨架，也是 slide 54 "Idea 4" 中"exhaustive search based techniques (like the Halide autotune)"的另一形态。

【并行机制解说】 – **并行的两个层面**：(1) **评估并行**——种群中每个个体的"编译+运行+计时"互相独立，可多核/多机并行执行（与 Halide autoscheduler 用 MLP 加速代价估计形成对比：这里用真实测量，代价高但最准确）；(2) **程序运行时的并行**——最终找到的配置本身决定了 kernel 如何并行（block 尺寸、SIMD 宽度等）。 – **与 Halide autoscheduler 的区别与联系**：Halide 在**结构化的调度空间**（compute_at 层级 × tile 尺寸）上搜索，用 MLP 代价模型快速打分（几十微秒/个，可测 140 万个）；遗传调优在**参数空间**上搜索，用真实硬件测量（每次要编译+运行，慢但准）。二者都体现"**智能搜索**"这一本讲主题，也预示 slide 54 的结论：把穷举/搜索类方法与 LLM agentic 方法结合，代价极高但效果最好。 – 对应概念：**autotuning / 智能搜索 / 代价模型 vs 实测**。

三、关键要点

1. **性能优化是稀缺技能且枯燥**：能写高性能 C++/ISPC/CUDA 的程序员很难找（"Proof by assignments 1–4"），而且换一台机器、换一个略有不同的问题就要重来一遍——这是自动化（DSL + 自动调度 + LLM 智能体）的最佳战场。
2. **提高抽象层次是第一步**：DSL 用"受限表达力"换取"系统可以利用领域知识做高性能实现 + 硬件可针对抽象优化"，代价是通用性/完备性损失。Halide 是教科书级案例：算法/调度分离后，程序员只需写"算法 + 4 行调度草图"。
3. **调度可以且应该被搜索**：Halide 的调度原语不仅提高专家生产力，更重要的是把"所有可能调度"变成了**可枚举的空间**，从而让贪婪/束搜索 + MLP 代价模型实现自动调度，效果可媲美人类专家（Adams 2019）。
4. **LLM 智能体正在改变优化范式**：通过"生成 → 执行/profile → 反思 → 修改"的闭环，LLM 能自动把 PyTorch 改写为高性能 CUDA（KernelBench）；用高层 DNN DSL 拼装原语可大幅降低幻觉风险。
5. **开放问题**：LLM 智能体能否成为"优秀的 CS149 学生"？代价是多少 token？而真正产生价值的到底是 **DSL 设计**还是 **LLM 智能体**，是值得争论的问题（slide 55）。

四、常见陷阱与注意事项

1. **过早写底层优化代码**：一上来就写 SSE/AVX intrinsics 的代码（slide 22 那种"快 10 倍但没人看得懂"），可读性、可移植性极差。应先确认瓶颈（profile），再考虑用 DSL/高层抽象表达，最后才落到平台特定优化。
2. **忽视算术强度与局部性就谈并行**：直接 2D 滤波（9×W×H）比两遍可分离（6×W×H）多 50% 工作量；中间缓冲放不进缓存时，两遍法会产生大量**非固有的内存流量**（tmp_buf 读写是实现伪影，不是计算本身需要的）。分块（chunking）是捕获 producer–consumer 局部性的关键。
3. **把"算法"和"调度"混为一谈**：在 Halide 中改 schedule 不会改变算法结果（都是声明式的、确定性的），但会剧烈改变性能；反之，用命令式循环手写时，算法与调度纠缠在一起，任何一处重排都可能引入 bug。
4. **误以为 DSL 是万能的**：Halide 只支持规则 N–D 域、前馈流水线（+ reduction、固定深度递归），且要求依赖全部可推断；超出领域，抽象反而成为束缚（通用性损失）。
5. **对 LLM 生成代码盲目信任**：LLM 输出必须经过"执行 + profile + 正确性检查"的闭环验证；在冷门语言（如新 DSL）上幻觉率更高；检索到的"例题解决方案"也要注意适配当前问题，而不是照抄。

五、思考题（带答案）

Q1：为什么"两遍可分离滤波 + 分块"能把工作量从 9×W×H 降到接近 6×W×H，而且分块大小越大越接近理想值？ **A1**：可分离性把 2D 模板（9 次乘加/像素）拆成两个 1D 模板（3+3=6 次乘加/像素），所以计算量降到 6×W×H。分块只影响**中间缓冲的重叠行**：CHUNK_SIZE=16 时每 chunk 需算 18 行水平模糊（多出的 2 行是与相邻 chunk 重叠的边界行），摊到每 chunk 是 18+16=34 行的工作量对应 16 行输出，即 (34/16)×3×W×H ≈ 6.4×W×H；CHUNK_SIZE 越大，重叠行占比 2/CHUNK_SIZE 越小，工作量趋近 6×W×H。同时分块让 tmp_buf 装进缓存，消除了两遍法中读写中间结果的片外流量。

Q2：Halide 的 schedule 为什么能被"自动搜索"？换成一个任意 C++ 程序，同样的思路为什么行不通？ **A2**：Halide 的调度原语（tile/vectorize/parallel/compute_at）构成了一套**小而完备的决策空间**——每个 Func 的 compute_at 层级与 tile 大小都是有限枚举，整个调度空间是"干净"的，可以建模成序列化决策并用贪婪/束搜索遍历（每个候选用 MLP 代价模型打分）。任意 C++ 程序的"等价变换"空间（循环重排、分块、融合、向量化、展开的组合）没有这样的规范表达，无法枚举，也就无法系统搜索——这正是 slide 44 的论点："Consider searching over all possible permutations of a C++ program"。

Q3：LLM 智能体优化循环中，profile 统计（SM 利用率、DRAM 利用率、L2 命中率）为什么是关键？如果只有"正确/错误"二元反馈，会发生什么？ **A3**：profile 统计提供了**可操作的中间信号**，让"反思"有据可依——例如 SM 利用率 42% 但 DRAM 利用率 89%，说明 kernel 是访存受限（memory-bound），应该减少冗余访存、改善数据复用或合并访问；L2 命中率低则提示改善局部性。若只有二元反馈，LLM 只能盲目试错（运气成分大、收敛慢），因为"错了"并不告诉它错在哪个优化维度。这正是"trial and error via reflection"中 reflection 的价值所在。

Q4（开放题）：课程最后提出的争论——"真正带来成功的价值，到底在 DSL 设计，还是在 LLM 智能体？"你如何从三者的关系论证？ A4（要点）：(1) DSL（如 Halide、Triton、CuTe）提供了**干净、可枚举、语义清晰**的中间表示——没有算法/调度分离，就没有可搜索的调度空间，LLM 也无从"拼装高层原语"（slide 48 明说 LLM 拼装 DSL 原语比裸写 CUDA 幻觉更少）；(2) 但 DSL 本身不解决"选择哪个调度"——那是智能搜索（autoscheduler）与 LLM 反思要做的；(3) 因此更合理的观点是**二者互补**：DSL 定义搜索/生成的空间，智能体（搜索算法或 LLM）在其中导航；LLM 的独特价值在于能利用自然语言知识（优化原则、历史经验、例题库）跨问题迁移，而搜索的价值在于穷举与可验证。这也是 slide 54 "Idea 4"（穷举搜索 + LLM agentic 结合）效果最好的原因。

本讲笔记基于 Stanford CS149 Fall 2025 Lecture 13 幻灯片（raw/aiperfoptimization.txt）撰写；Halide 示例参考 Ragan-Kelley/Adams 的 SIGGRAPH 2012 / PLDI 2013 工作，自动调度参考 Adams et al., SIGGRAPH 2019 "Learning to Optimize Halide with Tree Search and Random Programs"。

Lecture 14: Cache Coherence（缓存一致性）（日期：2025-11-11, Tuesday）

> **概述**：本讲讨论共享内存多处理器上的**缓存一致性问题（cache coherence problem）**：现代处理器为了性能在各自的私有缓存中复制内存内容，导致不同处理器可能对同一内存位置观察到不同值。本讲先给出"coherence"的严格定义与两条不变量（SWMR、Data-Value），然后重点讲解基于**失效（invalidation）**的写回一致性协议 **MSI** 及其改进版 **MESI**（状态转换图、总线事务 BusRd/BusRdX/BusWB、总线嗅探 snooping），再简要介绍可扩展的**目录式一致性（directory-based coherence）**，最后从程序员视角讨论**伪共享（false sharing）**等由一致性协议引起的"伪通信"开销。
>
> **注意**：本讲与 Assignment 2（多核 CPU 上的任务图调度）直接相关——你在作业中使用的共享变量、互斥锁与屏障背后的正确性保证，正是本讲的一致性协议提供的；理解 coherence 也能帮你解释作业中"看似无关的线程互相拖慢"这类性能现象（通常是伪共享）。

一、核心概念与定义

1. Cache Coherence Problem（缓存一致性问题）

- **定义**：现代处理器把内存内容复制到本地缓存中以获得性能。问题在于：**不同处理器的缓存可能持有同一内存位置的不同值**，使得"读 X 应返回最近写入 X 的值"这一直观预期被破坏。
- **现实类比**：同一份合同原件（内存）被复印成多份放在不同办公室（各核缓存）。A 办公室在自己的复印件上改了数字（store），B 办公室看自己的旧复印件（load），两边看到的"合同"不一致。
- **图示**（写回缓存下的乱象，来自 slide 11–12）：

```
P1 $   P2 $   P3 $   P4 $   mem[X]   Action
 0      0      0      0       0      初始 foo=0
 1      0      0      0       0      P1 store X
 1      0      0      0       2      P1 load Y（迫使 X 从 P1 缓存被换出）
 0      1      0      0       0      P3 load X → miss，从内存拿到旧值 0!
 0      1      0      0       2      P3 store X
 …                    （各处理器看到不同值 = 不一致）
```

2. Coherence（一致性）的严格定义

- **定义**（slide 16）：一个内存系统是 **coherent** 的，当且仅当：对每个内存位置，所有处理器对该位置的全部操作存在一个**假设的串行顺序（hypothetical serial order）**，且该顺序与执行结果一致，并且满足：

- 任一处理器发出的操作，在该顺序中保持其**程序内发出顺序**（program order）；
- 每次读返回的值 = 该串行顺序中**最后一次写**到该位置的值。
- **现实类比**：多位老师批改同一份作业（同一地址）时，必须约定一个"批改先后顺序"，且每位老师看到的批改结果符合这个顺序；顺序由"谁先交"（程序序）决定，而不是"谁手快"（时间）决定。
- **关键点**：coherence 只约束**同一个地址**上的操作顺序。

3. SWMR Invariant（单写多读不变量）

- **定义**（slide 17）：对任意地址 x，在任意时间段（epoch）内：
- **Read-Write epoch**：只有一个处理器可以写 x（同时也可以读）；
- **Read-Only epoch**：可以有任意多个处理器只读 x。
- **现实类比**：接力棒（单写权）只有一个人握着；其余人只能围观（读），等接力棒传到自己手上才能写。
- **图示**：

```
地址 x 的时间线：
  Read-Write    Read-Only      Read-Write    Read-Only
   (只有 P0)    (P0,P1,P2)     (只有 P1)    (P0,P1)
|──────────────|──────────────|──────────────|──────────────|→ time
```

4. Data-Value Invariant（数据值不变量 / 写串行化）

- **定义**（slide 17）：一个 epoch 开始时地址 x 的值，等于其**上一个 Read-Write epoch 结束时**的值。换句话说，写必须被"串行化"——每次写的结果成为后续所有读看到的值。
- **现实类比**：黑板上的值每被擦掉重写一次（写 epoch），所有学生下一节课看到的都是最新值；不允许"某个学生还看到上一版"。

5. Write-Through vs Write-Back Cache（写直达 vs 写回缓存）

- **定义**：
- **Write-through**：每次写都穿透到内存，内存始终是最新值；实现一致性简单，但**每个写操作都占用内存带宽**，带宽需求极高（slide 23）。
- **Write-back**：写只改缓存行并置 **dirty bit**，行被换出时才写回内存；吸收了大量写流量，但**dirty 状态现在意味着"独占所有权"**，需要更复杂的一致性协议。
- **现实类比**：写-through 像"每写一个字立刻传真给总部存档"（安全但费钱费线）；写-back 像"先在本地草稿本上改，定稿后才把整页寄回总部"（省事，但总部可能不知道本地改了）。
- **缓存行结构**（slide 6/27）：`[ Tag | Line state | Dirty bit | Data (现代 Intel 为 64 字节) ]`。

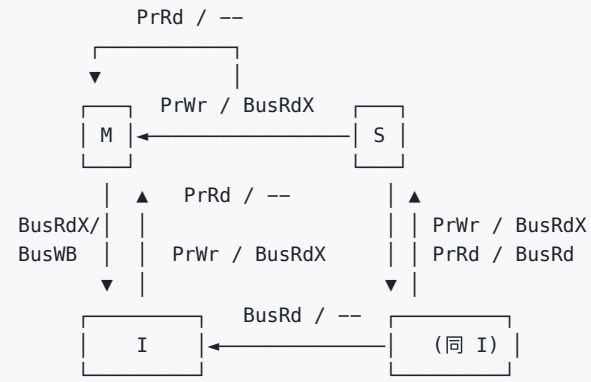
6. Snooping（总线嗅探）

- **定义**（slide 21）：基于**广播**的一致性方案。所有与一致性相关的活动都广播到系统中所有处理器（更准确地说，广播到各处理器的**缓存控制器**）；每个缓存控制器"嗅探"（snoop）总线上的内存操作，并按一致性协议作出响应。
- **现实类比**：公司群里喊话——任何改动都在大群里@所有人，每个人看到消息后检查自己手上有没有相关文件并处理。
- **要点**：缓存控制器现在要响应**两个方向**的事件：(1) 本地 CPU 的 LD/ST 请求；(2) 芯片互连上广播的一致性活动。

7. MSI 协议（Invalidation-Based Write-Back Protocol）

- **定义**（slide 28–30）：基于失效的写回一致性协议。每个缓存行有三种状态：
- **I（Invalid）**：本缓存无有效副本；
- **S（Shared）**：行在一个或多个缓存中有效，**内存副本是最新的**；
- **M（Modified）**：行只在**恰好一个**缓存中有效（即 dirty / exclusive），该缓存必须负责在别人读时提供数据。

- 两种处理器操作：**PrRd**（处理器读）、**PrWr**（处理器写）；三种总线事务：**BusRd**（读一份副本，无意修改）、**BusRdX**（取独占副本，打算修改）、**BusWB**（把 dirty 行写回内存）。
- **现实类比**：图书馆借书规则——M 状态 = 你借了唯一一本并在上面批注（别人要看你必须把批注版给他）；S 状态 = 多人都借了同一本书且内容与馆藏一致；I 状态 = 你没借这本书。
- **状态转换图**（slide 29）：



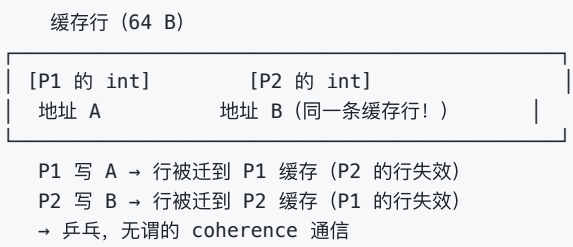
（图例：A / B 表示"观察到动作 A 时，采取动作 B"；实线 = 处理器发起，虚线 = 总线发起）

8. MESI 协议（Exclusive Clean 状态）

- **定义**（slide 34–35）：MSI 的改进——即使程序**完全没有共享**，MSI 也要为"读后写"付两次总线事务（I→S 的 BusRd，S→M 的 BusRdX）。MESI 增加 **E（Exclusive Clean）** 状态：行未修改、但**只有本缓存有副本**（内存副本仍有效）。于是 **E→M 升级不需要任何总线事务**（本地直接改）。"MESI，不是 Messi！"
- **现实类比**：MSI 像"买书必须先在登记簿上登记再借（每次都要跑柜台）"；MESI 像"发现这本书只有我借了，我直接在书上批注，不用再跑柜台报备"。
- **四种状态**：I / S / M / E；读请求若发现**没有其他缓存断言 shared**，则进入 E 而非 S。

9. False Sharing（伪共享）

- **定义**（slide 43）：两个处理器**写入不同的地址**，但这些地址**映射到同一条缓存行**。缓存行在两个写处理器的缓存之间"乒乓"（ping-pong）传递，产生大量由一致性协议驱动的通信——这些通信是**完全人为的（artifactual）**，因为程序本身在这些地址之间没有任何数据依赖（没有真实共享）。
- **现实类比**：两个学生坐在同一张长桌两端各自写自己的作业（不同地址），但桌子（缓存行）只有一个，谁一低头写字，对方就得把桌面的东西收走再放回来（缓存行被整行迁移）。
- **图示**：



10. Directory-Based Coherence（目录式一致性）

- **定义**（slide 36–37）：嗅探方案需要**广播**才能得知其他缓存中行的状态，扩展性差。目录式方案把每行的状态信息集中存放在一个**目录（directory）**中：目录条目记录该行在所有缓存中的状态，缓存按需查目录，通过**点对点（point-to-point）"按需告知"消息**维护一致性，而不是广播。仍须维持 SWMR 与写串行化两条不变量。
- **现实类比**：图书馆总台账（目录）记录"这本书在谁手上"；别人借书时只需问总台账，而不必向全楼喊话。

- **实例**（Intel Core i7）：**L3 充当集中目录**（L3 是 inclusive cache，L2 里的行必在 L3 中，因此 L3 知道每行在哪些 L2 中）；一致性消息只发给**包含该行的 L2**，而不是广播给所有 L2（i7 互连是 ring，不是 bus）。目录规模：P=4（核数）× M（L3 行数）。

11. 3Cs Cache Miss Model（三类缺失）与 AMAT

- **定义**：缺失分为 **Cold（冷缺失）**、**Capacity（容量缺失）**、**Conflict（冲突缺失）**。多处理器下还要额外考虑 **True Sharing（真共享）/ False Sharing（伪共享）** 与 **Upgrade（升级）** 造成的缺失（slide 44 按缓存行大小拆解各类缺失率）。平均访存时间 **AMAT = Σ frequency × latency**，且 **AMAT_Multiprocessor > AMAT_Uniprocessor**（slide 39）。
- **现实类比**：AMAT 像"通勤平均时间"——既取决于坐哪趟车（访问频率），也取决于每趟车多慢（各级延迟）；多核下多了"找别人要数据"的绕路，平均通勤变长。
- **延迟表**（Core i7 Xeon 5500，约值）：L1 命中 ~4 cycles；L2 命中 ~10 cycles；L3 命中（行未共享）~40 cycles；L3 命中（行在别的核共享）~65 cycles；L3 命中（行在别的核且 modified）~75 cycles；本地 DRAM ~30 ns（~120 cycles）；远端 DRAM ~100 ns（~400 cycles）。

二、代码示例与详细解说（本讲重点）

示例 1：伪共享（False Sharing）演示——相邻 int 计数器互相拖慢

代码（C++11，std::thread 版）（对应 slide 42 的 pthread 演示，改写为现代 C++）：

```cpp
#include <atomic>
#include <chrono>
#include <cstring>
#include <iostream>
#include <thread>
#include <vector>

constexpr int NUM_THREADS = 8;
constexpr int MANY_ITERATIONS = 100'000'000;   // 每个线程累加次数
constexpr int CACHE_LINE_SIZE = 64;            // 现代 x86 缓存行大小

// 版本 1：朴素版——每个线程一个相邻的 int（伪共享！）
void worker_plain(int* counter) {
    for (int i = 0; i < MANY_ITERATIONS; i++)
        (*counter)++;                          // 写自己的计数器
}

// 版本 2：填充版——每个计数器独占一条缓存行
struct PaddedCounter {
    int counter;
    char padding[CACHE_LINE_SIZE - sizeof(int)];
};

void worker_padded(PaddedCounter* pc) {
    for (int i = 0; i < MANY_ITERATIONS; i++)
        pc->counter++;
}

template <typename Worker, typename Arg>
double run(Worker worker, Arg* array) {
    std::vector<std::thread> threads;
    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < NUM_THREADS; i++)
        threads.emplace_back(worker, &array[i]);
    for (auto& t : threads) t.join();
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double>(t1 - t0).count();
}

int main() {
    // 版本 1：int counter[NUM_THREADS]，相邻元素共享缓存行
    int* plain = new int[NUM_THREADS]();
    double t1 = run(worker_plain, plain);

    // 版本 2：PaddedCounter counter[NUM_THREADS]，每个元素独占缓存行
    PaddedCounter* padded = new PaddedCounter[NUM_THREADS]();
    double t2 = run(worker_padded, padded);

    std::cout << "plain  (false sharing): " << t1 << " s\n";
    std::cout << "padded (no  sharing)  : " << t2 << " s\n";
    delete[] plain;
    delete[] padded;
    return 0;
}
```

【代码做了什么？】 – 8 个线程各自重复累加自己的计数器 `counter[i]`——线程之间**没有任何真实的数据共享**：没有锁、没有跨线程读写同一变量，程序逻辑上完全正确。 – 版本 1 把 8 个 `int` 放在连续数组里：相邻元素（如 `counter[0]` 与 `counter[1]`）落在**同一条 64 字节缓存行**内。 – 版本 2 用 `struct PaddedCounter` 让每个计数器独占一条缓存行（int 后填充到 64 字节对齐）。 – 幻灯片上的实测（8 线程、4 核系统）：**版本 1 耗时 14.2 秒，版本 2 仅 4.7 秒**——快了约 3 倍，而两者的"算法"完全相同。

【并行机制解说】 – 为什么逻辑上独立却慢 3 倍？因为缓存一致性的粒度是**缓存行**而不是单个 int。线程 1 写 `counter[0]` 时，缓存控制器必须先获得该行的**独占权**（写回协议下即 M 状态），于是广播 BusRdX，使持有同行的其他缓存（线程 2 的）将该行**失效**；线程 2 再写 `counter[1]` 时又要重新把行抢回来。两个线程交替写同一条行 → 缓存行在两个核之间**乒乓**，每一次乒乓都是一轮总线事

务 + 失效传播（几十到上百 cycle 的延迟），且随线程数增加而恶化。 – 这对应本讲的 **false sharing** 概念：通信完全由一致性协议驱动、与程序语义无关（"No inherent communication, this is entirely artifactual communication (cache lines > 4B)"）。修复方式就是**填充 padding / 对齐**，让不同线程写的数据落在不同缓存行。 – 关键点：即使没有 `std::atomic`、没有任何同步，单靠 `volatile int` 级别的"看似独立"的写，也会因为缓存行粒度而产生性能耦合——这是 cache-coherent 架构编程中必须警惕的隐藏通信。

示例 2：MSI 协议的缓存控制器状态机（模拟器）

代码（C++，模拟 MSI 协议的缓存控制器）（对应 slide 28–30 的协议逻辑）：

```
#include <cstdio>

enum State { I, S, M };
enum Event { PrRd, PrWr, BusRd, BusRdX, BusWB };

// 模拟"一个缓存控制器"对事件 A 的响应：更新自身状态，并决定是否发出总线事务
// 返回值 = 该控制器需要在总线上发出的事务（0 表示无）
struct Controller {
    State st = I;

    int onEvent(Event e) {
        int tx = 0;                         // 0 = 无总线事务
        switch (e) {
        case PrRd:                          // 本地处理器读
            if (st == I)      { st = S; tx = /*BusRd*/ 1; }  // 从 I 读：发 BusRd 拿共享副本
            else              { /* S 或 M 命中，直接读 */ }
            break;
        case PrWr:                          // 本地处理器写
            if (st == M)      { /* 已独占，直接写（不通知别人） */ }
            else if (st == S) { st = M; tx = /*BusRdX*/ 2; } // 升级：必须 BusRdX 让别人失效
            else              { st = M; tx = /*BusRdX*/ 2; } // 从 I 写：BusRdX 取独占
            break;
        case BusRd:                         // 嗅探到别人要读
            if (st == M)      { st = S; tx = /*BusWB*/ 3; }  // 我是唯一持有者：写回内存供其读取
            break;                          // I/S 状态无需动作
        case BusRdX:                        // 嗅探到别人要独占写
            if (st == M)      { st = I; tx = /*BusWB*/ 3; }  // 必须失效；若 dirty 先写回
            else if (st == S) { st = I; }                    // 必须失效（否则不"唯一"了）
            break;
        case BusWB:                         // 嗅探到写回，无本地动作
            break;
        }
        return tx;
    }
};

int main() {
    Controller c;
    // 场景（对应 slide 31 的例子）：P1 读 x → P3 读 x → P3 写 x → P1 读 x
    printf("PrRd -> state=%d tx=%d\n", c.st, c.onEvent(PrRd));    // S
    printf("PrRd -> state=%d tx=%d\n", c.st, c.onEvent(PrRd));    // S（S 中读仍命中）
    printf("PrWr -> state=%d tx=%d\n", c.st, c.onEvent(PrWr));    // M, tx=BusRdX
    printf("BusRdX(from P2) -> state=%d tx=%d\n", c.st, c.onEvent(BusRdX)); // I
    return 0;
}
```

【代码做了什么？】 – 实现一个缓存控制器的**状态机逻辑**：根据本地事件（PrRd/PrWr）与总线嗅探事件（BusRd/BusRdX/BusWB）更新行状态，并决定是否发出总线事务。 – 关键分支：本地**写**只有在 M 状态才能"静默"进行；S 或 I 状态写都必须发 **BusRdX**（即使行在本地缓存中有效，只要它是 S 状态，也必须广播 BusRdX 让别人失效——因为"多个缓存同时持有同一行"时，本地无法独占）。 – 嗅探到别人发 BusRdX 时，本控制器必须把行**失效**（M 状态还需先写回，即发 BusWB），否则别人

拿不到独占权。 – 模拟了 slide 31 的场景：P1 读 x（S）→ P3 读 x（S）→ P3 写 x（P1 被 BusRdX 失效为 I，P3 进入 M）→ P1 再读 x 会 miss，数据来自持有 M 的 P3（而非内存）。

【并行机制解说】 – **并行如何实现**：所有缓存控制器**独立地**运行同一套协议逻辑（slide 30："all caches are carrying out this logic independently to maintain coherence"），通过总线上的广播消息"协作"维持两条不变量： 1. **SWMR**：只有 M 状态的行可以被写，而 M 意味着"其他所有缓存都收到了失效消息"——于是任意时刻只有一个写者； 2. **Data-Value（写串行化）**：当某缓存需要数据而另一个缓存处于 M 时，数据由 M 缓存通过 BusWB 提供（而不是内存里的旧值）；**总线本身串行化了所有事务**，从而给所有操作一个一致的全局顺序。 – 同步点在哪：每一次 **BusRdX** 都是一次"隐性同步"——它宣告"我要独占写入"，所有相关缓存必须在此之前处理完自己手上的副本（失效或写回）。 – 对应概念：**MSI 协议、invalidation、snooping、SWMR/Data-Value 不变量**。注意本例展示的是"单个地址"的协议行为——这正是 coherence（同地址排序）要保证的；不同地址之间的排序问题留给 Lecture 15 的 consistency。

示例 3：MSI vs MESI——"读后写"需要几次总线事务?

代码（C++，事务计数对比）（对应 slide 34 的论点：MESI 消除"无共享也要付两次事务"的低效）：

```
#include <cstdio>

// 统计"读取地址 X，然后写入地址 X"这一最常见模式所需的总线事务数
int main() {
    // ---- MSI：读 → 进入 S；写 → 必须 BusRdX 升级 ----
    // 事务 1：BusRd（I → S）
    // 事务 2：BusRdX（S → M，向所有其他缓存广播失效）
    int msi_transactions = 2;
    printf("MSI : read-then-write needs %d bus transactions\n", msi_transactions);

    // ---- MESI：读时若没有其他缓存断言 shared，进入 E（独占、干净）----
    // 事务 1：BusRd（I → E，其他缓存无人持有 → 进入 E 而非 S）
    // 事务 2：无！E → M 是本地升级，不需要任何总线事务
    int mesi_transactions = 1;
    printf("MESI: read-then-write needs %d bus transaction\n", mesi_transactions);

    // 关键前提：MESI 的 E 状态必须"确认没有别人也持有该行"。
    // 总线协议用"共享信号"（shared line）实现：读事务在总线上发出时，
    // 若有其他缓存持有该行，它会"断言 shared"，本缓存就进入 S 而非 E。
    bool another_cache_asserts_shared = false;  // 典型单线程/无共享场景
    if (another_cache_asserts_shared) {
        // 有其他缓存持有 → 进入 S，写时仍要 BusRdX 升级
        printf("MESI: line is shared -> write needs BusRdX upgrade (like MSI)\n");
    } else {
        // 无其他缓存持有 → 进入 E，写是"静默升级"
        printf("MESI: line is exclusive-clean -> silent E->M upgrade, 0 extra tx\n");
    }
    return 0;
}
```

【代码做了什么？】 – 对比同一场景（先读后写同一地址）在两种协议下的总线事务数。MSI 必须付两次：BusRd（I→S）+ BusRdX（S→M，广播失效）。 – MESI 引入 E 状态：读请求若**没有其他缓存断言 shared**，则进入 E；此时本地写只是 **E→M 的静默升级**，无需任何总线事务——**即使程序完全没有共享，也省掉一次广播**（slide 34："This ineffciency exists even if application has no sharing at all"）。 – 代码同时说明 E 与 S 的判定机制：总线上的"共享信号"（shared line）——读事务广播时，其他持有该行的缓存会断言 shared，本缓存据此选择进入 S 还是 E。

【并行机制解说】 – 这是**协议级优化**的典型例子：一致性协议设计需要权衡"每次操作付多少通信成本"。MESI 的洞察是**把"独占"（exclusivity）与"所有权/脏"（dirty/ownership）解耦**：E 状态的行"只有我有，但内存副本仍有效"——所以它既不违反 SWMR（只有我持有，我可以独占写），又不需要像 M 那样承担"必须把最新数据写回/提供给他人"的责任。 – 对程序员的意义：即便你的程序从不共享数据（每个核只碰自己的数据），缓存一致性机制依然在后台产生通信；MESI 把"私有数据的常见访问模式"（读一次然后反复写）的通信降到最低。这提醒我们：**性能不仅取决于程序逻辑，还取决于硬件协议为你的访问模式付出的通信代价**。 –

对应概念：**MESI、BusRd/BusRdX、E 状态、协议开销**。注意 MESI 仍保留 MSI 的失效语义——只要有人真正共享（进入 S），写就仍需 BusRdX 广播失效，保证 SWMR。

示例 4：目录式一致性（Directory-Based Coherence）草图

代码（伪代码/C++ 风格）（对应 slide 36–37 的目录思想）：

```
// 目录条目：记录一行在哪些缓存中、以及当前所有权状态
struct DirEntry {
    enum State { UNCACHED, SHARED, MODIFIED } state;
    int owner;                    // MODIFIED 时的唯一持有者（缓存编号）
    std::vector<int> sharers;     // SHARED 时的读者列表
};

// 集中式目录（例如 Intel Core i7 的 L3 扮演的角色）
class Directory {
    std::unordered_map<uint64_t, DirEntry> entries;
public:
    // 请求者 c 想读地址 addr：只通知"需要知道"的缓存，而不是广播
    void handleReadRequest(int c, uint64_t addr) {
        auto& e = entries[addr];
        if (e.state == MODIFIED) {
            // 数据在 owner 手里：让 owner 把最新数据转发给 c（并写回内存）
            sendMessage(e.owner, "forward data", addr);
            e.owner = -1;                    // 转为共享
            e.sharers = {c};
            e.state = SHARED;
        } else {
            // 内存有最新副本：直接回数据，把 c 加入 sharers
            e.sharers.push_back(c);
        }
    }

    // 请求者 c 想写地址 addr：逐一点对点地让所有 sharers 失效（无广播）
    void handleWriteRequest(int c, uint64_t addr) {
        auto& e = entries[addr];
        if (e.state == SHARED) {
            for (int s : e.sharers)
                if (s != c) sendMessage(s, "invalidate", addr);  // 只通知持有者！
            e.sharers.clear();
        } else if (e.state == MODIFIED && e.owner != c) {
            sendMessage(e.owner, "invalidate + writeback", addr);
        }
        e.owner = c;                         // 目录记住新 owner
        e.state = MODIFIED;
    }
};
```

【代码做了什么？】 – 目录为每条缓存行维护：状态（UNCACHED / SHARED / MODIFIED）、MODIFIED 时的 owner、SHARED 时的 sharers 列表。 – 读请求：若行在别的缓存处于 MODIFIED，目录让 owner **转发数据**（并写回内存），再记录读者；否则直接从内存回数据。 – 写请求：目录**只向 sharers 列表里的缓存**发送 invalidation 消息（point-to-point），而不是广播给所有缓存；随后把 owner 记为请求者。消息只在"需要知道"的缓存间传递（"need to know" basis）。 – 对应 slide 37 的 Intel Core i7 实现：L3 是 inclusive cache（L2 中的行必在 L3），因此 L3 能当集中目录；目录维护"哪些 L2 含有该行"的列表，一致性消息只发给这些 L2（i7 互连是 ring 而非 bus）。

【并行机制解说】 – **并行如何实现**：一致性串行化的"裁判"从**总线**换成**目录**——目录成为串行化点（serialization point），所有状态转换决策由目录做出，因此依然满足 SWMR 与写串行化两条不变量，但通信方式从"广播给所有人"变成"点对点通知相关者"，**可扩展性**大大提高（广播成本随处理器数量增长，目录只随共享关系增长）。 – 同步点：目录集中了"谁是当前 owner / 谁在共享"的信息，写请求的失效确认（invalidation acknowledgement）通过目录汇聚，形成全局一致的写顺序。 – 代价：目录本身是存储开销（条目

数 = 缓存行数 × 每条目信息），且所有请求都要经过目录（多一跳延迟）——这是"可扩展性"与"单点延迟"的权衡。 – 对应概念：**directory-based coherence、点对点消息、序列化点**。这是对 snooping 广播方案（"scalability limited by ability to broadcast"）的直接回应。

示例 5：用 VTune / 系统工具观察一致性开销（命令示例）

代码（命令行）（对应 slide 40 "Use VTune to learn about memory system performance"）：

```
# 1. 用 VTune 的 memory-access 分析收集缓存与内存指标
vtune -collect memory-access -knob sampling-interval=1 -result-dir=vtune_out ./my_program

# 2. 查看关键指标：缓存缺失、带宽、以及 NUMA/远程访问
vtune -report summary -result-dir=vtune_out
# 关注指标：
#   - L1/L2/L3 命中率与缺失率（对比单线程基线，评估 coherence 带来的额外缺失）
#   - DRAM 带宽利用率（写回、伪共享都会抬高带宽）
#   - NUMA 远程内存访问占比（NUMA 系统中一致性/访存延迟更高）

# 3. Linux perf 快速查看缓存行为
perf stat -e cache-misses,cache-references,LLC-load-misses,LLC-store-misses ./my_program
```

【代码做了什么？】 – 给出用硬件性能计数器观察缓存/一致性开销的标准流程：VTune 的 memory-access 分析 + `perf stat` 的 cache 事件。 – 关键判断依据：**对比单线程基线的缓存缺失率增量**——多处理器下增加的缺失往往来自一致性协议（true sharing、false sharing、upgrade），而 NUMA 系统还可能出现"本地内存也 miss"的远程访问延迟（slide 39 的 AMAT 分析）。

【并行机制解说】 – 这是"从程序员视角理解 coherence 开销"的实操：coherence 使**通信时间成为并行开销的一部分**——它表现为更高的缓存缺失率、更高的内存访问延迟（如 L3 命中但行在其他核：~65 cycles；行在其他核且 modified：~75 cycles；远端 DRAM：~100 ns）。AMAT = Σ frequency × latency，多核下两者都可能变差。 – 只有百分之零点几的额外缺失（"Only a fraction of a % of these can be significant!"）就可能显著影响性能，所以需要用工具量化，而不是凭感觉。 – 对应概念：**communication overhead、AMAT、false sharing 的量化**。如果发现缺失率异常升高，下一步通常就是检查是否伪共享（示例 1 的 padding 修复）。

三、关键要点

1. **缓存一致性问题是"复制"带来的**：共享地址空间这个抽象并不是由单一存储单元实现的——数据既在内存中又被复制到各处理器私有缓存中，因此"读 X 应返回最近写 X 的值"需要协议来保证；**这不是互斥问题，加锁无法修复**（slide 12："Is this a mutual exclusion problem? Can you fix the problem by adding locks? NO!"）。
2. **coherence 有精确的定义**：对每个地址存在一个与所有观察一致的串行顺序；每处理器按程序序执行；读返回串行顺序中最后一次写。实现层面由两条不变量落地：**SWMR（单写多读）** 与 **Data-Value（写串行化）**。
3. **基于失效的写回协议（MSI/MESI）是核心机制**：只有 M（或 E）状态的缓存能本地静默写；想写必须先通过 **BusRdX** 获得独占权（广播失效他人副本）；嗅探到别人要独占时自己必须失效（dirty 先写回）。**MESI 的 E 状态让"无共享"场景免掉一次总线事务**。
4. **一致性通信是程序员要付的隐性开销**：伪共享（不同地址同缓存行）会产生完全人为的乒乓通信——同一逻辑程序的性能可差 3 倍（14.2s vs 4.7s）；padding/对齐是标准修复。
5. **广播不扩展，目录可扩展**：snooping 的可扩展性受"能否广播给所有缓存"限制；目录式一致性（如 i7 的 L3 目录）用点对点消息把通信限制在"需要知道"的缓存之间。

四、常见陷阱与注意事项

1. **以为一致性问题是锁能解决的**：把共享变量"用锁保护"并不能修复缓存不一致——协议层面的一致性错误（读旧值）发生在内存系统，而不是临界区；锁只解决"多写者互斥"的语义问题（而 coherence 本就不允许并发写者在协议层面同时写）。

2. **忽视伪共享**：为每个线程分配"独立的"相邻变量（如 `int counter[NUM_THREADS]`）看似无共享，实则同缓存行乒乓。写高性能代码时，线程私有数据要 **padding 到缓存行边界**（或用 `alignas(64)`），并意识到 `volatile`/普通写都可能触发。
3. **误解 dirty bit 的含义**：写回缓存中 dirty 不只是"内存过期"，它意味着**独占所有权**（M 状态）——持有 M 行的缓存必须负责在别人读时提供数据（写回），否则别人会从内存拿到旧值。
4. **忽略通信延迟的层级差异**：同样一次"L3 命中"，行未共享 ~40 cycles、行在别的核共享 ~65 cycles、行在别的核且 modified ~75 cycles、远端 DRAM 数百 cycles——设计共享数据结构时应尽量让高频访问的数据**独享缓存行**并减少跨核写共享。
5. **对缓存行大小/协议做硬编码假设**：缓存行是 64B（现代 Intel）但不同架构不同（slide 44 显示缺失率随行大小显著变化，且分解为 cold/capacity/true sharing/false sharing/upgrade）；协议可能是 MSI/MESI/目录式的混合（如 i7 的 L3 目录 + L1/L2 嗅探式行为）。可移植代码应使用 `std::hardware_destructive_interference_size`（C++17）或对齐宏，而不是写死 64。

五、思考题（带答案）

Q1：为什么在 MSI 中，即使行已经在本地缓存且处于 S 状态，写它仍然必须发出 BusRdX 事务？ A1：因为 S 状态意味着**其他缓存可能也持有该行的副本**。若本地直接从 S 改为 M 而不广播失效，别的缓存仍保留旧副本，之后它们读到的将是旧值——违反 SWMR 不变量（同一时刻只能有一个写者，且写后其他缓存不得再持有有效副本）。BusRdX 的作用就是"宣告我要独占写入"，迫使所有持有者失效（若它们处于 M，还须先写回）。这正是 slide 33 强调的："Read-exclusive transaction is required even if line is valid (but not exclusive... it's in the S state)"。

Q2：伪共享和真共享（true sharing）有什么区别？为什么伪共享是"人为的"（artifactual）通信？ A2：真共享指多个处理器**访问同一地址**（比如同一把锁、同一个累加器），通信是程序语义必需的；伪共享指多个处理器**访问不同地址**（如 `counter[0]` 和 `counter[1]`），仅因这些地址落在**同一条缓存行**而被迫以整行为粒度通信——程序本身在这些地址间没有任何数据依赖，因此这种通信完全是缓存行粒度（64B）大于数据粒度（4B int）的产物。修复方法：padding/对齐使每个线程的数据独占缓存行，或重新布局数据结构。

Q3：为什么说"总线"对 MSI 协议的正确性如此重要？如果把总线换成非广播的互连（如 ring），MSI 还会正确工作吗？ A3：MSI 依赖两点总线特性：一是**广播**（BusRd/BusRdX 能到达所有缓存，保证"所有持有者都被通知失效"）；二是**顺序性/原子性**（总线在同一时刻只处理一个事务，天然为所有一致性操作提供一个全局串行顺序，从而满足 Data-Value 不变量中的写串行化）。若换成 ring 之类的非广播互连，MSI 的失效消息无法到达所有缓存、也没有天然的总序——所以 Intel Core i7 改用**目录式**一致性：L3 作集中目录、按需点对点通知，由目录充当新的串行化点（slide 37）。这解释了"广播不可扩展 → 目录方案"的演进逻辑。

本讲笔记基于 Stanford CS149 Fall 2025 Lecture 14 幻灯片（raw/cachecoherence.txt）撰写；性能数据（14.2s vs 4.7s、各层级延迟）均引自幻灯片原文。

Lecture 15: Implementing Synchronization + Memory Consistency（实现同步 + 内存一致性）（日期：2025-11-13, Thursday）

> **概述**：本讲先回顾 Lecture 14 的缓存一致性与 MSI/MESI 协议（slides 1–20），然后切入新主题：**内存一致性（memory consistency）**。Coherence 只约束"同一地址"的操作顺序，而 consistency 定义"不同地址"上的读写以何种顺序对其他线程可见。幻灯片讲清四个问题：(1) 顺序一致性（sequential consistency，Lamport 1976）是什么、为什么"00/10"输出不该出现；(2) 为什么需要**放松一致性**（relaxed consistency）——写缓冲与乱序执行为了隐藏内存延迟，会重排内存操作（TSO/PC/PSO/WO/RC）；(3) 同步如何"拯救"这一切——**fence**、**acquire/release**、read-modify-write（如 CAS）等原语恢复排序保证；(4) 语言层内存模型：C11/C++11 与 Java 5 承诺 **"SC for DRF"**（无数据竞争的程序获得顺序一致性）。本讲代码示例覆盖自旋锁、票锁、CAS 与 acquire/release 等同步原语的实现，展示"同步原语如何在放松的硬件上强制出 SC 行为"。
>
> **注意**：本讲与 Assignment 2（任务图调度）中你使用的 mutex / condition variable / barrier 直接对应——这些库原语内部正是用自旋、CAS、acquire/release 与 fence 实现的；理解本讲能帮你判断"我的同步写对了吗"。另外第 15 讲（Nov 13）当天是 Assignment 4 截止日（Trainium2 Fused Conv+MaxPool），下周二是期中考试（覆盖 Lectures 1–14）。

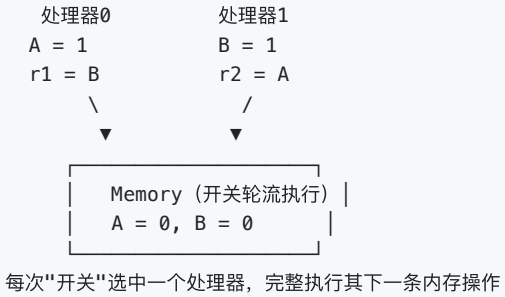
一、核心概念与定义

1. Memory Coherence vs Memory Consistency（一致性 vs 一致性模型）

- **定义**（slides 24–25）：**coherence** 定义**同一内存位置**上读写的观察行为——所有处理器必须对"对 X 的读写顺序"达成一致（能把涉及 X 的所有操作放到一条时间线上）。**consistency** 定义**不同位置**（如 X 与 Y）上读写的可见顺序——coherence 只保证"对 X 的写最终会传播"，consistency 决定"对 X 的写何时传播（相对于对其他地址的读写）"。
- **现实类比**：coherence 像"会议室白板上内容的唯一性"（同一块白板只有一个内容）；consistency 像"你先后说出的两句话，听众听到的先后顺序"（不同句子 = 不同地址）。
- **金句（slide 25）**：coherence 的目标是让并行机的内存系统表现得**好像缓存不存在**；而 consistency 定义的是**有没有缓存都要遵守**的、不同地址读写行为的规范。

2. Sequential Consistency（顺序一致性，SC）

- **定义**（slide 30，Lamport 1976，图灵奖 2013）：所有内存操作按照**某个全局串行顺序**执行，就像操作单一共享内存；且**每个线程的操作保持程序序**（program order）。SC 系统维持全部四种内存操作排序：**W→R、R→R、R→W、W→W**（写 X 必须先于后续读 Y 提交，依此类推）。
- **现实类比**（slide 31 的"开关隐喻"）：所有处理器按程序序发出 load/store，**内存端有一个开关**：随机选中某个处理器，把它的一条内存操作完整执行完，再选下一个……同一时刻只有一条操作在内存上执行。
- **图示**：



3. Program Order 与四种内存操作排序（Memory Operation Orderings）

- **定义**（slide 27）：程序定义了一串 load/store（即这些操作的 program order）。四种排序约束：
- **W_X→R_Y**：对 X 的写必须先"提交"（结果可见）于后续对 Y 的读；
- **R_X→R_Y**：对 X 的读必须先提交于后续对 Y 的读；
- **R_X→W_Y**：对 X 的读必须先提交于后续对 Y 的写；

- **W_X→W_Y**：对 X 的写必须先提交于后续对 Y 的写。
- **现实类比**：做饭时"放盐（写 A）必须在尝味（读 B）之前完成"这类工序约束；"先烧水再下面"是 W→W 约束。
- 注意：这里"提交"（commit）指**结果对其他处理器可见**，而不仅仅是执行单元执行完。

4. 经典例子：A/B 程序（"00"为什么不该出现）

- **定义**（slide 28–29）：初始 A = B = 0；处理器 0 执行 `A = 1; print B;`，处理器 1 执行 `B = 1; print A;`。可能输出 "01"、"10"、"11"，但**不应输出 "00" 或 "10"**。
- **happens-before 图**：把"必须发生的事件顺序"画成有向图；若某输出导致图中出现**环**（一个事件必须发生在它自己之前），则该输出不可能。
- **现实类比**：两位同学各写一道题并互改——如果"甲必须先改到乙的答案、乙又必须先改到甲的答案"，这就是环，物理上不可能。
- **图示**（"10" 为什么不可能：r1=0 要求 (2) 先于 (1)，r2=1 要求 (3) 先于 (4)，而程序序要求 (1) 先于 (2)、(3) 先于 (4)……构成环）：

```
要打印 "10"：r1 = B 得 0 → (2) 必须先于 (1) 完成
            r2 = A 得 1 → (3) 必须先于 (4) 完成
程序序：(1) → (2)，(3) → (4)
综合：(2)→(1)→…→(3)→(4)→… 需要 (2) 先于 (4) 且 (4) 先于 (2) → 环 → 不可能
```

5. Relaxed Consistency（放松一致性）

- **定义**（slide 37）：放松模型**允许违反某些**内存操作排序约束，以换取性能——具体放松哪几条，决定了模型名称：**TSO**（放松 W→R）、**PC**（Processor Consistency，放松 W→R 且允许他人提前读新值）、**PSO**（Partial Store Order，再放松 W→W）、**WO/RC**（Weak Ordering / Release Consistency，几乎全部可重排）。
- **现实类比**：快递可以"先送近的再送远的"（重排）以省时间——只要不违反"必须亲自签收的件不能让别人代签"这类关键约束；放松得越多，省的时间越多，但你需要自己加"加急件标记"（fence）来保证关键件顺序。
- **动机（slide 38–39）**：内存访问在一致性系统中可能要做很多事（找数据、发失效等），写操作要几百个 cycle；若两条操作**互不冲突**（如 `A=1` 与 `r1=B`），没必要等第一条完成——重排/重叠它们能隐藏延迟。

6. Write Buffer（写缓冲）与 TSO

- **定义**（slide 40–43）：处理器把写放入**写缓冲**（write buffer）后即可继续执行后续指令，不必等写真正到达缓存/内存；读时**先查自己的写缓冲**。代价：**处理器自己的读可以"越过"自己的写**（W→R 被放松）→ 出现 SC 下不可能的行为：`r1 = r2 = 0`。
- **现实类比**：先记账后付款——你（处理器）在账本上记"已付"（写缓冲）就继续忙别的，供应商（其他处理器）要等钱真的汇出才看到"已付"。
- **关键事实（slide 43）**：**每个现代处理器都有写缓冲**（Intel x86、ARM、RISC-V 皆是）；x86 用的是**一种未完全定义的 TSO（Total Store Order）**。TSO 只放松 W→R，**W→W 仍保持**（同一线程的写不重排）。
- **图示**：

```
处理器0                      处理器1
A = 1 ──► [写缓冲]           B = 1 ──► [写缓冲]
r1 = B ──► (读自己的缓冲？)   r2 = A ──► (读自己的缓冲？)
      │                            │
      ▼                            ▼
      └──────────► Memory ◄────────┘
两个处理器的写都还"堵"在缓冲里，对方读不到 → r1 = r2 = 0 可能!
```

7. Fence（内存屏障 / 内存围栏）

- **定义**（slide 50）：**fence（memory barrier）** 指令**阻止重排**：fence 之前的**所有**内存操作必须全部完成，fence 之后的任何内存操作才能开始。它是恢复排序保证的"万能工具"，但**很昂贵**（付出的是放松模型想省下的那部分性能）。
- **现实类比**：工地上的"验收关卡"——所有已完工的工序必须验收完毕（之前的操作全部可见），才能开始下一批工序。

- **实例（slide 51）**：x86 提供 `_mm_lfence`（等所有 load 完成）、`_mm_sfence`（等所有 store 完成）、`_mm_mfence`（等所有内存操作完成）；ARM 的一致性模型非常放松，需要更多显式屏障。
- **图示**：

```
可重排的读和写……
═══════ MEMORY FENCE ═══════      ← 之前的操作全部完成、全部可见
可重排的读和写……
═══════ MEMORY FENCE ═══════
```

8. Acquire / Release 语义（同步原语的内存序）

- **定义**：**release**（释放）：释放操作**之前**的所有内存操作，在释放之后对其他线程可见（"发布"出去）；**acquire**（获取）：获取操作**之后**的所有内存操作，必须能看到获取之前已被 release 发布的所有内容（"领取"回来）。二者成对出现，构成**单向屏障**：release 是"下行屏障"（前面的不许越过我），acquire 是"上行屏障"（后面的不许越过我）。比全量 fence 便宜，因为它只约束一个方向。
- **现实类比**：发布朋友圈（release）：发之前发生的事，朋友都能看到；刷到朋友圈（acquire）：你接下来看到的（之后的操作）以这条朋友圈为基准。
- **要点**：这是 C++11 `std::atomic` 中 `memory_order_release` / `memory_order_acquire` 的语义，也是锁（lock/unlock）、屏障（barrier）等库原语的内部基石。

9. Data Race（数据竞争）与 DRF

- **定义**（slide 52–53）：两个处理器对**同一内存位置**的访问构成**冲突**，如果至少一个是写；若冲突访问**没有被同步操作排序**（如 fence、release/acquire、barrier），程序就是 **unsynchronized** 的，包含 **data race**——输出取决于处理器相对速度（**非确定**）。
- **现实类比**：两个人同时改同一份 Word 文档（冲突写），且没有任何"审阅锁定"（同步），最后谁存盘谁赢——结果不可预测。
- **关键定理（slide 54）**：**同步化程序（data-race-free，DRF）在非 SC 系统上也得到 SC 结果**——"If there are no data races, reordering behavior doesn't matter"：访问被同步排序，同步强制出顺序一致性。实践中绝大多数程序通过锁、屏障等**同步库**写成 DRF 程序，而不是靠临时读写共享变量。

10. 语言级内存模型：SC for DRF

- **定义**（slide 59）：现代语言 **C11/C++11** 与 **Java 5** 保证：**无数据竞争的程序（DRF）获得顺序一致性（SC）**——编译器负责针对目标硬件插入必要的同步（fence 等）来兑现这一承诺。**如果你的程序有数据竞争，语言不提供任何保证**（绝大多数程序员会认为有 race 的程序就是 buggy 的）。
- **现实类比**：航空公司承诺"准时到达"（SC for DRF），但前提是你**按规则托运**（不用同步就乱写共享变量 = 不按规则）；违规者后果自负。
- **实践建议**：**用同步库**（std::mutex、std::atomic 等），不要手工裸写内存序。

11. 同步原语：Spinlock / Test-and-Set / CAS / Ticket Lock（本讲代码部分的概念）

- **定义**：这些都是"把放松的硬件拉回 SC 行为"的实现手段：
- **test-and-set**（TAS）：原子指令，读取并置位某内存单元，返回旧值（如 `std::atomic_flag::test_and_set`）；`xchg` 在 x86 上加 `lock` 前缀即原子。
- **spinlock（自旋锁）**：等待者用 TAS 循环"自旋"抢锁，直到成功；忙等（busy-wait）浪费 CPU，且高竞争下产生大量一致性通信。
- **test-and-test-and-set（TTAS）**：先普通读（test）看锁是否空闲，空闲才 TAS——把"每轮测试都写"降为"仅在锁释放时发一次失效"，一致性通信从 O(P)/次释放降到 O(P)/释放。
- **CAS（compare-and-swap）**：原子地"若当前值等于期望值则写入新值，返回旧值/是否成功"（`std::atomic::compare_exchange_strong`）；是构建无锁数据结构与各种同步的万能积木。
- **ticket lock（票锁）**：取号（fetch_add 自己的 ticket）→ 等待叫号（读 now_serving 直到等于自己的号）；**获取锁只需读**，每释放一次锁只产生一条失效（O(P) 通信总量），且**先来先得（公平）**。

- **现实类比**：TAS 锁像"进门就抢把手"（乱抢、拥堵）；TTAS 像"先隔着玻璃看门开没开，开了才去抢"；票锁像银行取号——先取号（原子加一），然后坐着等叫号（只读显示屏）。

二、代码示例与详细解说（本讲重点）

示例 1：自旋锁（Spinlock）——`std::atomic_flag::test_and_set`

代码（C++11）：

```
#include <atomic>

class SpinLock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;   // 初始为 clear（未锁定）
public:
    void lock() {
        // 反复"测试并置位"：若返回 false，说明之前是 clear（我们抢到了锁）
        while (flag.test_and_set(std::memory_order_acquire)) {
            // 抢锁失败：忙等（自旋）——可加 pause/yield 降低功耗与总线压力
        }
    }
    void unlock() {
        flag.clear(std::memory_order_release);  // 释放：置回 clear
    }
};

// 使用示例：用自旋锁保护一个计数器
#include <thread>
#include <vector>
#include <cstdio>

int main() {
    SpinLock lock;
    int counter = 0;
    std::vector<std::thread> threads;
    for (int t = 0; t < 4; t++) {
        threads.emplace_back([&] {
            for (int i = 0; i < 100000; i++) {
                lock.lock();      // acquire：保证拿到锁后能看到锁保护的数据
                counter++;        // 临界区：互斥访问
                lock.unlock();    // release：保证临界区的写对其他线程可见
            }
        });
    }
    for (auto& th : threads) th.join();
    std::printf("counter = %d (期望 400000)\n", counter);
    return 0;
}
```

【代码做了什么？】 – `lock()`：`flag.test_and_set()` 是原子指令——读出旧值并把 flag 置 1。若旧值为 0（锁空闲），我们抢锁成功；若旧值为 1（已被持有），返回 true，进入 `while` 循环**忙等**（spinning），不断重试直到抢到。 – `unlock()`：`flag.clear()` 原子地把 flag 置 0，释放锁。 – 主程序用 4 个线程各加 10 万次计数器，靠自旋锁保证互斥，最终 counter 应恰为 400000（无数据竞争）。

【并行机制解说】 – **硬件如何支持原子性**：`test_and_set` 对应硬件的 read-modify-write（RMW）原子指令——在 x86 上是 `lock xchg` 之类（原子前缀保证"读+写"不可分割）；**原子性**是同步的根基：若"读旧值"和"写新值"之间插进别的线程，抢锁逻辑就崩了。 – **为什么这里要 acquire/release**：在放松的硬件（如 ARM）上，`unlock()` 用 release 保证"临界区内的所有写（如 counter++）在锁释放时对下一个持锁线程可见"；`lock()` 用 acquire 保证"拿到锁之后的操作能看到前一个持锁线程 release 的所有内容"。正是这对 acquire/release 把放松模型"拉回" SC 行为——这对应 slide 54 的论点：同步化的（DRF）程序在非 SC 系统上

得到 SC 结果。 – **代价**：高竞争时自旋锁产生大量一致性通信（每次 test_and_set 都是一次写，会失效其他缓存中的 flag——即"一锁释放，所有等待者同时抢"，一致性流量大）；且忙等浪费 CPU。这是"同步原语让排序更严格（slide 50）"的代价一面。 – 对应概念：**test-and-set、spinlock、acquire/release、fence 的替代品**。

示例 2：票锁（Ticket Lock）——公平且通信高效

代码（C++11，std::atomic）：

```
#include <atomic>
#include <thread>
#include <vector>
#include <cstdio>

class TicketLock {
    std::atomic<int> next_ticket{0};   // 发号器：下一位客人的号码
    std::atomic<int> now_serving{0};   // 叫号屏：当前服务到几号
public:
    void lock() {
        int my_ticket = next_ticket.fetch_add(1, std::memory_order_relaxed);
        // 取号：原子地把 next_ticket 加一并取回自己的号
        while (now_serving.load(std::memory_order_acquire) != my_ticket) {
            // 等待叫号：只读（没有写！）—直到 now_serving 变成自己的号
        }
    }
    void unlock() {
        // 叫下一个号：把 now_serving 加一
        now_serving.fetch_add(1, std::memory_order_release);
    }
};

int main() {
    TicketLock lock;
    int counter = 0;
    std::vector<std::thread> threads;
    for (int t = 0; t < 8; t++) {
        threads.emplace_back([&] {
            for (int i = 0; i < 50000; i++) {
                lock.lock();
                counter++;
                lock.unlock();
            }
        });
    }
    for (auto& th : threads) th.join();
    std::printf("counter = %d (期望 400000)\n", counter);
    return 0;
}
```

【代码做了什么？】 – `lock()`：第一步**取号**——`fetch_add` 原子地把 `next_ticket` 加 1，并返回自己的号码 `my_ticket`；第二步**等待**——循环**只读** `now_serving`，直到它等于自己的号码。 – `unlock()`：把 `now_serving` 加 1（"叫下一个号"）。 – 与自旋锁的关键区别：**等待期间不写任何共享变量**（只有 `fetch_add` 取号时写一次）。

【并行机制解说】 – **为什么通信少**：自旋锁里每个等待者反复 `test_and_set`（每次都是一次**写**，触发缓存行失效，抢锁风暴）；票锁等待者只做**读** `now_serving`——读不会失效别人的副本，所有线程可以同时读自己的缓存副本，**只有 `unlock` 的那一次 `fetch_add` 写会发出一条失效**。于是每次释放锁只产生 O(P) 总量的一致性通信（每个等待者至多被失效一次），而自旋锁是每轮测试 O(P)。 – **公平性**：取号顺序 = 获得锁的顺序（FIFO），先来先得——解决了 TAS 锁"释放时所有等待者一拥而上、谁快谁得"的不公平。 – **acquire/release 的角色**：等待者用 acquire 读 `now_serving`，确保"看到自己的号被叫到"之后，能看到前一持锁者 release 出的临界区数据；unlock 用 release 把临界区写"发布"出去。同样地，这一对 acquire/release 保证整个程序（DRF）呈现 SC 行为。 – 对应概念：**ticket lock、fetch_add、acquire/release、一致性通信开销**。

示例 3：CAS（compare-and-swap）——原子更新与自旋 CAS 锁

代码（C++11，`std::atomic::compare_exchange_strong`）：

```
#include <atomic>
#include <thread>
#include <vector>
#include <cstdio>

// 用 CAS 实现"原子加一"（无锁的 fetch_add 替代品，教学演示）
void atomic_increment(std::atomic<int>& x) {
    int old = x.load(std::memory_order_relaxed);
    do {
        int expected = old;
        // 若 x 仍等于 expected，则把 x 写成 old+1，返回 true；否则返回 false 且 expected 被更新为当前值
        if (x.compare_exchange_strong(expected, old + 1, std::memory_order_relaxed))
            return;                     // 成功
        old = expected;                 // 失败：有人抢先改过，用最新值重试
    } while (true);                     // CAS 循环（compare-exchange loop）
}

// 用 CAS 实现一个"自旋 CAS 锁"（用 0=空闲、1=占用 表示锁）
class CASLock {
    std::atomic<int> state{0};
public:
    void lock() {
        int expected = 0;               // 期望锁是空闲的
        // 只有"state 从 0 变成 1"成功才代表抢到锁；失败则重试
        while (!state.compare_exchange_strong(expected, 1, std::memory_order_acquire)) {
            expected = 0;               // 恢复期望值，继续自旋
        }
    }
    void unlock() {
        state.store(0, std::memory_order_release);
    }
};

int main() {
    // 场景 A：8 个线程各做 5 万次 CAS 原子加
    std::atomic<int> x{0};
    std::vector<std::thread> threads;
    for (int t = 0; t < 8; t++)
        threads.emplace_back([&] { for (int i = 0; i < 50000; i++) atomic_increment(x); });
    for (auto& th : threads) th.join();
    std::printf("x = %d (期望 400000)\n", x.load());

    // 场景 B：CAS 自旋锁保护计数器
    CASLock lock;
    int counter = 0;
    std::vector<std::thread> threads2;
    for (int t = 0; t < 8; t++)
        threads2.emplace_back([&] { for (int i = 0; i < 50000; i++) { lock.lock(); counter++; lock.unlock(); } });
    for (auto& th : threads2) th.join();
    std::printf("counter = %d (期望 400000)\n", counter);
    return 0;
}
```

【代码做了什么？】 – `atomic_increment`：标准的 **CAS 循环**——先读旧值，`compare_exchange_strong(expected, old+1)` 原子地检查"当前值是否还是 expected"，是则写新值并返回成功；否则 `expected` 被更新为最新值，循环重试。这等价于硬件提供的 RMW 原子加，但在语义上完全由 CAS 构建。 – `CASLock`：锁状态 0/1；`lock()` 用 CAS 把 0 换成 1——只有"从 0 变 1"成功者获得锁；失败者恢复 expected 后重试（自旋）。 – 两个场景都验证最终结果恰为 400000。

【并行机制解说】 – **CAS 的原子性从哪来**：`compare_exchange_strong` 在硬件上是一条原子 RMW（x86 的 `lock cmpxchg`）——"比较 + 交换"作为**不可分割**的一步完成。它是实现所有更高级同步（锁、屏障、无锁队列）的"万能积木"，也正是 slide 50 提到的"per-address 同步原语：read-modify-write / compare-and-swap"。 – **CAS 锁与 TAS 锁的差别**：TAS 每次失败都会**写**（置位），产生一致性失效；CAS 锁失败时只是**比较**（读），写只在成功时发生——与 TTAS 类似，减少了竞争时的缓存乒乓。但 CAS 锁仍不公平（释放时所有等待者一起抢）。 – **为什么需要 acquire/release**：与示例 1 同理——`unlock` 的 release 保证临界区写可见，`lock` 的 acquire 保证进入临界区后能看到前者的写；在放松硬件上，没有这对语义，`counter++` 可能被重排到锁外，DRF 前提被破坏。 – 对应概念：**CAS、RMW 原子指令、自旋锁变体、acquire/release**。

示例 4：acquire/release 内存序——生产者-消费者交接

代码（C++11，`std::atomic` + `memory_order`）（演示"为什么必须用 acquire/release，relaxed 会出错"）：

```
#include <atomic>
#include <thread>
#include <cstdio>
#include <cassert>

std::atomic<bool> ready{false};
int payload = 0;                    // 普通（非原子）共享数据

void producer() {
    payload = 42;                           // (1) 先写数据
    ready.store(true, std::memory_order_release);   // (2) release：把 (1) 发布出去
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) { /* 等待 */ }  // (3) acquire
    // 关键问题：这里读 payload 一定得到 42 吗？
    // 答：一定！release-acquire 形成同步关系：acquire 之后的操作
    //     能看到 release 之前的所有写（包括非原子的 payload）。
    assert(payload == 42);
    std::printf("payload = %d\n", payload);
}

int main() {
    std::thread t1(producer), t2(consumer);
    t1.join(); t2.join();
    return 0;
}

// 对比：如果把 (2)(3) 都改成 memory_order_relaxed，程序就是有数据竞争的——
// 编译器/硬件可以重排 (1) 与 (2)，消费者可能看到 ready==true 但 payload 还是 0。
```

【代码做了什么？】 – 生产者先写普通变量 `payload = 42`，再用 `memory_order_release` 写 `ready`；消费者用 `memory_order_acquire` 自旋读 `ready`，等到 true 后读 `payload` 并断言它等于 42。 – 关键保证：**release-acquire 配对形成 happens-before 关系**——`ready.store(release)` 之前的所有写（含非原子 `payload`），对执行 `ready.load(acquire)` 的线程**可见**。因此断言必然成立。 – 注释中对比：若都用 `relaxed`，`payload=42` 与 `ready=true` 可以被重排（编译器或乱序硬件），消费者可能看到 `ready==true` 却读到 `payload==0`——这就是 slide 47 中 PSO 的例子（`A=1; flag=1; while(flag==0); print A;` 可能打印旧值）的现代 C++ 版本。

【并行机制解说】 – **为什么这对原语足够**：这是"同步原语如何把放松硬件拉回 SC"的最小例子——release/acquire 是**单向屏障**：release 保证"前面的操作不越过我"（下行），acquire 保证"后面的操作不越过我"（上行）。与全量 fence 相比，它只约束一个方向，因此更便宜；这正是锁、屏障等库原语的内部机制（slide 54：同步库把复杂性封装起来，程序员只需用 lock/unlock、barrier）。 – **fence 在其中的位置**：`std::atomic_thread_fence(std::memory_order_seq_cst)` 或 x86 的 `_mm_mfence` 是全量屏障（slide 50：所有内存操作完成前，后面的操作不能开始）；acquire/release 是它的"减配版"。在真正的 x86（TSO）上，普通 store/load 已经隐含部分顺序，但 ARM（非常放松）上必须显式使用这些语义。 – **数据竞争是禁区**：`payload` 是非原子变量，但它被 release/acquire 排序，所以程序是 **DRF** 的，语言保证 SC 结果；若改用 relaxed（或干脆不用原子），`payload` 的读写就成了 **data**

race（slide 52–53），程序输出不确定——语言不再提供任何保证（slide 59 的"SC for DRF"）。 – 对应概念：**acquire**、**release**、**fence**、**happens-before**、**data race**、**DRF**、**SC for DRF**。

示例 5："00/10/11/01"问题——用 happens-before 判断合法输出

代码（C++11 伪代码 + 分析）（对应 slide 28–29 的经典问题）：

```
#include <atomic>
#include <thread>
#include <cstdio>

// 初始 A = B = 0（在 SC 系统上）
std::atomic<int> A{0}, B{0};

void p0() {
    A.store(1);            // (1)
    printf("%d", B.load()); // (2) 输出 r1
}

void p1() {
    B.store(1);            // (3)
    printf("%d", A.load()); // (4) 输出 r2
}

// 在顺序一致性（SC）系统上，可能的输出：
//   "01"：顺序 (1)(2)(3)(4) 或 (3)(4)(1)(2) 的混合
//   "11"：两线程都先写完再读：如 (1)(3)(2)(4)
//   "10"：？  r1=0 要求 (2) 先于 (1)；r2=1 要求 (3) 先于 (4)
//         程序序又要求 (1) 先于 (2)、(3) 先于 (4)
//         → 需要 (2)→(1)→…→(3)→(4)→…→(2)，成环 → SC 下不可能！
//   "00"：？  两个读都先于两个写 → (2) 先于 (1) 且 (4) 先于 (3)，
//         与程序序 (1)→(2)、(3)→(4) 矛盾 → 成环 → SC 下不可能！
// 结论：SC 下只可能 "01" 或 "11"（取决于开关先执行哪个线程的哪条指令）。
// 但在 TSO（写缓冲）或更放松的模型下，"00" 是可能的：
// 两线程的写都还堵在自己的写缓冲里，对方读不到 → r1 = r2 = 0。
```

【代码做了什么？】 – 四个语句按程序序排列：P0 写 A、读 B；P1 写 B、读 A。在 SC（"开关"每次完整执行一条内存操作）下枚举所有交错，只有 "01" 和 "11" 合法。 – 用 happens-before 图论证：某个输出合法，当且仅当所需的先后关系**不构成环**；"00" 和 "10" 都会导致环（一个事件必须发生在自己之前），因此不可能。 – 注释指出：一旦引入写缓冲（TSO），"00" 变成可能——这正是"放松一致性"改变程序可见行为的具体演示（slide 41：`Can r1 = r2 = 0? SC: No. Write buffers: Yes!`）。

【并行机制解说】 – **这是 consistency 与 coherence 的对照实验**：coherence（Lecture 14）保证每个地址（A 或 B）各自的写串行化，但**不保证跨地址的可见顺序**；consistency 才决定"P0 写 A 与 P1 读 A 之间的相对时间"。所以 slide 24 说：coherence 是关于同一地址的，consistency 是关于不同地址之间的。 – **为什么现代硬件允许"00"**：性能。写 A 需要几百个 cycle（一致性系统里要定位数据、发失效等），P0 没必要干等；把写放进写缓冲、继续执行读 B（与写 A 无冲突）能隐藏延迟（slide 38–39、42 的性能对比）。代价就是 W→R 排序被放松 → TSO。 – **程序员怎么办**：要么接受"我的程序是 DRF 的，用同步库，SC for DRF 保证正确结果"；要么（只有系统程序员/同步库作者才需要）显式用 fence 或 acquire/release 恢复特定排序（slide 50–51、55）。 – 对应概念：**sequential consistency**、**happens-before**、**TSO**、**write buffer**、**data race**。

三、关键要点

1. **Coherence ≠ Consistency**：coherence 管"同一地址"的读写顺序（让系统表现得像没有缓存）；consistency 管"不同地址"之间的可见顺序（有没有缓存都得遵守的规范）。一致性问题是**复制（缓存）**引起的，放松一致性问题则是**重排（内存操作）**引起的——与缓存是否存在无关（slide 46）。

2. **SC 是黄金标准，但代价是性能**：SC（Lamport 1976）要求所有操作存在一个全局串行顺序且每线程保持程序序（四种排序 W→R / R→R / R→W / W→W 全保留）。但写操作要几百 cycle，为了隐藏延迟，**每个现代处理器（x86/ARM/RISC-V）都有写缓冲**，实际执行比 SC 更放松（x86 ≈ TSO）。
3. **放松模型 = 选择性放弃排序**：TSO 只放松 W→R（PC 还允许别人提前读新值）；PSO 再放松 W→W（`A=1; flag=1` 可能被看到反序）；WO/RC 几乎全放。放松越多性能越好，程序员需要付出的"补排序"工作越多。
4. **同步是解药，但要 DRF 才免费**：fence（全量屏障）、acquire/release（单向屏障）、RMW/CAS 等原语可以恢复排序；但**有数据竞争的程序（冲突访问未被同步排序）输出不确定**。好消息：**同步化的（DRF）程序在非 SC 系统上也得到 SC 结果**——所以绝大多数程序员用同步库写正确程序，而不用关心硬件模型。
5. **语言也承诺 SC for DRF**：C11/C++11、Java 5 保证 DRF 程序获得顺序一致性，编译器负责插入必要同步；有 race 则无任何保证。**实践原则：用同步库，别手工裸写内存序**。

四、常见陷阱与注意事项

1. **用普通变量 + 原子标志做"消息传递"却忘掉 acquire/release**：`flag.store(true, relaxed)` + `payload = 42` 可被重排，消费者可能看到"flag 已置位但 payload 还是旧值"——这是典型的 data race，在 ARM 等放松架构上几乎必然出错（x86 上碰巧常对，形成"在我的机器上能跑"的错觉）。正确做法：release/acquire 配对（示例 4）。
2. **自旋锁的忙等（busy-wait）浪费与缓存乒乓**：test-and-set 自旋锁在竞争激烈时，每次测试都是一次写、触发整条缓存行失效，性能骤降；应使用 TTAS（先读后 TAS）、ticket lock（只读等待）或加 `pause` / `yield`；在单核系统上自旋锁还可能死锁（持锁线程被抢占，等待者永远自旋）。
3. **把"锁了"等同于"内存序正确"**：锁（或任何同步）只有在**正确使用**时才提供排序保证——临界区内外的共享访问都必须被同一把锁保护；漏保护一次访问就产生数据竞争，整个程序的保证归零（race 是"全有或全无"的）。
4. **用错 memory_order 或过度放松**：`relaxed` 只保证原子性、不保证顺序——只适合计数器等"不需要顺序"的场景；`seq_cst` 最安全但最贵。常见错误是把 `compare_exchange` 循环里的 `expected` 忘了更新（无限循环），或把 `memory_order_acquire` / `release` 用反（acquire 配 store、release 配 load 是错的）。
5. **以为"现代 x86 上跑得对"就万事大吉**：x86 是（未完全定义的）TSO，很多重排不会发生；但编译器在 O2 下也会重排（语言层内存模型管的是"编译器 + 硬件"总和），换到 ARM/RISC-V（非常放松）或换编译器优化级别，bug 立刻暴露。**可移植的正确性必须依赖语言内存模型（SC for DRF），而不是某个硬件的巧合行为**。

五、思考题（带答案）

Q1：为什么说"缓存一致性"与"内存一致性（consistency）"是两件不同的事？请用 A/B 程序（A=1; print B / B=1; print A）说明。
A1：coherence 只要求**每个地址单独**存在一个与所有观察一致的串行顺序（SWMR + 写串行化）：A 上的写和读、B 上的写和读各自有序，但**不规定 A 的顺序与 B 的顺序之间的相对关系**。consistency 才规定跨地址的相对可见时间：SC 下 P0 的"写 A"必须在其"读 B"之前提交（W→R 约束），所以 r1=0 与 r2=1 不能同时成立（"10"不可能），"00"也不可能。一旦硬件用写缓冲放松 W→R（TSO），"00"就成为合法输出——地址 A、B 各自仍然 coherent，但程序整体不再 SC。这正说明：coherence 是关于"复制的缓存"，consistency 是关于"重排的操作"。

Q2：为什么票锁（ticket lock）在高竞争下的性能优于 test-and-set 自旋锁？它与缓存一致性协议有什么关系？ A2：test-and-set 锁的每个等待者在每一轮自旋中都执行一次**写**（置位指令），每次写都会通过一致性协议（如 BusRdX）使其他缓存中该锁的副本**失效**——于是"一锁释放，所有等待者同时抢、互相失效"，产生 O(P) 次失效/轮。票锁的等待者只**读** `now_serving`（读不产生失效，所有线程可同时命中自己的缓存副本），唯一的写是取号时的一次 `fetch_add` 和释放时的一次 `fetch_add`——每次释放只产生**一条失效**（O(P) 总量）。此外票锁按取号顺序放行（FIFO 公平），避免 TAS 锁的"抢锁风暴"。这正是 Lecture 14 的 coherence 通信开销在同步原语设计中的直接体现。

Q3：什么是 "SC for DRF"？它为什么能让绝大多数程序员"忘记"内存一致性模型的存在？ A3："SC for DRF"是 C11/C++11 与 Java 5 语言内存模型的承诺：**只要程序无数据竞争（所有冲突访问都被同步操作排序），程序的行为就与顺序一致性系统上的执行一致**——编译器负责针对具体硬件（x86 的 TSO、ARM 的弱序）插入必要的 fence/屏障来兑现该承诺。因为绝大多数程序通过同步库（std::mutex、barrier 等，其内部实现如示例 1–3 所示）写成 DRF 程序，所以应用层程序员只需保证"该同步的地方同步了"，不需

要知道底层是 TSO 还是弱序；只有同步库实现者、内核/驱动开发者与无锁数据结构作者才需要直面 memory model（slide 23、55、59）。

本讲笔记基于 Stanford CS149 Fall 2025 Lecture 15 幻灯片（raw/sync_consistency.txt）撰写；同步原语示例（自旋锁、票锁、CAS、acquire/release）为支撑幻灯片第 50/54 页论点的标准实现，其正确性依赖 C++11 原子库的内存序语义。

Lecture 16: Fine-Grained Locking and Lock-Free Programming（日期：Nov 20）

> **概述**：本讲前半部分从"实现锁"出发：先厘清死锁（deadlock）、活锁（livelock）、饥饿（starvation）三个易混淆术语，然后对比 test-and-set、test-and-test-and-set、ticket lock 等锁实现的性能特征（尤其是 cache coherence 流量）。后半部分转向"使用锁"：用细粒度锁（hand-over-hand locking）在有序链表中获得并行度，并介绍无锁（lock-free）数据结构的基础——单读单写队列、基于 CAS 的无锁栈，以及著名的 ABA 问题。本讲为下一讲"事务内存"（transactional memory）做铺垫：CAS 的本质作用就是检测"操作期间数据结构是否被其他线程修改过"。

一、核心概念与定义

1. Deadlock（死锁）

- **定义**：系统中有若干操作尚未完成，但由于每个操作都持有着别的操作需要的资源，导致**没有任何操作能继续推进**的状态。死锁的必要条件有四条：**mutual exclusion（互斥）**、**hold and wait（持有并等待）**、**no preemption（不可抢占）**、**circular wait（循环等待）**。幻灯片强调：死锁和活锁关乎**程序正确性**；饥饿则主要是**公平性**问题。
- **现实类比**：旧金山十字路口四辆车同时抢行、互不相让，谁都动不了（幻灯片原话：在 SF 死锁"happens all the time"）。更生动的例子是 National Geographic 那张蚂蚁围成圈的图：每只蚂蚁都在等前面的蚂蚁让路。
- **公式/图示**：循环等待即资源依赖图中存在环：

```
线程 A ──持有──> 资源 R1 ──被 B 等待──> 线程 B ──持有──> 资源 R2 ──被 A 等待──> 线程 A
          （环！谁都无法继续）
```

计算机系统里的经典例子（幻灯片 Example 2）：两个线程互相往对方的有限 work queue 里塞消息，队满时发送方阻塞等待，于是 A 等 B 腾出空间、B 等 A 腾出空间，双双卡死。

2. Livelock（活锁）

- **定义**：系统**在不停地执行大量操作**，但没有任何线程取得**有意义的进展**。典型计算机系统场景：操作不断 abort 然后重试（operations continually abort and retry），每次都失败。
- **现实类比**：两个人面对面走在窄走廊里，同时向左让、又同时向右让，来回好几次谁也过不去——动作很多，但"让路"这件事毫无进展。
- **与死锁的区别**：死锁是"谁也不动"；活锁是"大家都在动但白动"。两者都使程序无法完成，属于正确性问题。

3. Starvation（饥饿）

- **定义**：系统整体在推进，但**某些进程始终得不到资源、毫无进展**的状态。幻灯片用交通图说明：黄车（左右方向）必须给绿车（上下方向）让路，绿车一辆辆通过，黄车一直停在原地。饥饿通常**不是永久状态**——绿车走完后黄车还能走。
- **现实类比**：食堂打饭窗口永远被同一批人插队，后面的同学一直吃不上饭；但窗口总会轮到他们（只是可能很晚）。

- **与公平性的关系**：饥饿是公平性问题而非正确性问题——程序最终能完成，只是某些线程被"饿"了很久。

4. Test-and-Set（测试并置位）

- **定义**：一条原子指令 `ts R0, mem[addr]`：把 `mem[addr]` 的旧值装入 `R0`；如果旧值为 0，则把 `mem[addr]` 置为 1。它同时完成"读旧值 + 条件写"且不可被打断，是构建自旋锁的最小原语。
- **现实类比**：食堂占座：看一眼座位是否空（读），如果空就立刻把书包甩上去占住（写）——"看一眼 + 占住"必须是瞬间完成的一个动作，否则两个人会同时坐下。
- **公式/图示**：

```
ts R0, mem[addr]   // R0 = mem[addr];  if (mem[addr] == 0) mem[addr] = 1
```

5. Compare-and-Swap（CAS，比较并交换）

- **定义**：原子地执行"若 `dst` 当前值等于 `EAX`，则把 `dst` 写成 `src` 并置标志位 ZF=1；否则把 `EAX` 更新为 `dst` 的当前值并置 ZF=0"。x86 上需加 `lock` 前缀才是原子的（`lock cmpxchg dst, src`）。它是几乎所有无锁算法的基础。
- **现实类比**：对暗号开门：先报出你记忆中的暗号（期望值），如果门锁里的暗号没变，门就开了（且换成新暗号）；如果暗号变了，说明别人动过锁，你记下新暗号再试。
- **公式/图示**（幻灯片原文语义）：

```
lock cmpxchg dst, src
if (dst == EAX) { ZF = 1; dst = src; }   // 比较成功：交换
else            { ZF = 0; EAX = dst; }   // 失败：把当前值读回 EAX
```

6. Fine-Grained Locking / Hand-over-Hand Locking（细粒度锁 / 手递手锁）

- **定义**：把一把"全局数据结构锁"拆成**每个节点一把锁**；遍历链表时，先锁住前驱节点，再锁住下一个节点，然后释放前驱的锁——像攀岩者手递手抓住下一个握点才松开上一个（幻灯片配图正是 American Ninja Warrior）。这样两个线程可以同时操作链表的不同区域，获得并行度。
- **现实类比**：山路上"手递手"接力护送：只有当前后两个路段都有人把守时才放行车辆；把守范围跟着车队移动，而不是整条山路只设一个关卡。
- **图示**（线程 0 删除 11、线程 1 删除 10 时各持两把相邻锁）：

```
        [T0][T0]                [T1][T1]
3 → 5 → 10 → 11 → 18     3 → 5 → 10 → 18
      (T0: prev,cur)          (T1: prev,cur)
```

7. Ticket Lock（取号锁）

- **定义**：维护 `next_ticket`（发号器）与 `now_serving`（当前叫号）两个计数器。获取锁 = 原子地 `atomic_increment(&next_ticket)` 取一个号，然后**纯读**地自旋等待 `now_serving == my_ticket`；释放锁 = `now_serving++`。获得锁的过程不再需要原子操作（只需要读），且严格 FIFO，天然公平。
- **现实类比**：银行/医院取号叫号：进门先取号，然后坐着等叫号屏变成自己的号；办完业务下一个号自动顶上。所有人按取号顺序被服务，不会有人插队。
- **特点**：每次释放锁只有一次失效（one invalidation per lock release），互连流量为 O(P)，远优于 test-and-set 族。

8. Blocking Algorithm（阻塞式算法）

- **定义**：一个算法允许某个线程**无限期地阻止其他线程完成对共享数据结构的操作**。典型例子：线程 0 拿到链表某节点的锁后被操作系统换出（或被 page fault 卡住、崩溃、极慢），其他线程就再也无法操作该数据结构——尽管线程 0 并没有在修改它。**只要用了锁，无论锁是自旋还是让出 CPU 实现，算法就是阻塞式的。**

- **现实类比**：一个人堵住唯一的门后去接电话，其他人只能在门外干等，哪怕他根本不在动那扇门。

9. Lock-Free（无锁）

- **定义**：非阻塞算法中，若**保证至少有一个线程（some thread）能推进**（systemwide progress，系统级进展），则称该算法是 lock-free。关键点：不允许"某线程恰好在不巧的时刻被抢占而导致整个系统停止进展"。注意：这个定义**不保证**任何单个线程不被饿死——可能某个线程永远失败重试，但系统整体一直在前进。
- **现实类比**：一扇双向弹簧门，多人同时推门：无论怎么抢，**总有人**能把门推开（可能同一人推开好多次，另一个人一直没成功——但"门在动"就是系统级进展）。
- **公式/图示**：lock-free 的无锁栈 push/pop 见代码示例 4；核心不变量："只要没有其他线程修改过栈顶，我的修改就可以成功落地"。

10. Single-Reader/Single-Writer Queue（单读单写队列）

- **定义**：**只允许一个生产者、一个消费者**同时访问的队列。因为 head 只被消费者写、tail 只被生产者写，两个线程**从不互相同步、从不等待对方**：队列空时 pop 直接返回 false，队满时 push 直接返回 false。前提是顺序一致性内存（或加 fence，或用 C++11 atomic）。
- **现实类比**：单车道隧道：只有一辆车能进、只有一辆车能出，出入口各自独立管理，进出的车不需要互相打招呼——只要各自看清"隧道里有没有位置/有没有车"。
- **图示**（有界环形缓冲）：

```
        head                    tail
         |                       |
         ▼                       ▼
data: [ ][ ][ 3 ][ 10 ][ ][ ]... (N 个槽位，环形)
       空: head == tail；满: tail == MOD_N(head - 1)
```

11. ABA Problem（ABA 问题）

- **定义**：CAS 只比较"值"，无法区分"值从头到尾没变过"与"值从 A 变到 B 又变回 A"。在无锁栈中，线程 0 读到 `old_top = A` 后被抢占；期间其他线程弹出 A、修改 A、再把 A 压回去、又压入 D；线程 0 恢复后 CAS 发现 top 仍等于 A 而成功，把 top 设成了 B，导致 D 被静默丢失、栈结构被破坏。注意幻灯片特别提醒：这里的 A、B、C、D 是**节点地址**，不是节点里存的值；且不要与 ABBA 问题混淆。
- **现实类比**：你确认"门是开着的"就走进去，却没注意到门在你确认之后**被关上又打开了**——你以为状态没变，其实世界已经转了一圈。
- **图示**：见代码示例 5 的完整时间线图。

12. Hazard Pointer（危险指针）

- **定义**：无锁数据结构中避免 use-after-free 的高级技巧：每个线程维护一个"当前正在访问、绝不能被释放"的指针（hazard pointer）；被弹出的节点不立即 `delete`，而是进入每线程的 retire list；当 retire list 超过阈值时，扫描其中所有节点，只有**没有任何线程的 hazard pointer 指向**的节点才真正释放。
- **现实类比**：工地上给正在施工的墙挂"施工中，请勿拆除"的警示牌：拆墙前先确认所有警示牌都没指着这面墙。
- **用途**：解决无锁栈中"另一个线程可能已经 free 掉我即将解引用的 old_top"的悬垂引用问题。

二、代码示例与详细解说（本讲重点）

示例 1：从 test-and-set 到 ticket lock——三种锁实现对比（C）

```
// ---- 1) test-and-set 自旋锁：每次尝试都发 BusRdX，流量巨大 ----
typedef int lock;
void Lock1(lock* l) {
    while (test_and_set(l) != 0);   // 一直尝试：原子"读+写1"
}
void Unlock1(lock* l) {
    *l = 0;                          // 直接写 0 释放
}

// ---- 2) test-and-test-and-set 锁：先自旋"读"，读到 0 才尝试原子获取 ----
void Lock2(lock* l) {
    while (1) {
        while (*l != 0);             // 纯读自旋：锁被持有时只读本地缓存
        if (test_and_set(*l) == 0)   // 锁释放了，才发一次原子尝试
            return;
    }
}
void Unlock2(lock* l) {
    *l = 0;
}

// ---- 3) ticket lock：取号 + 等叫号，FIFO 公平 ----
struct lock {
    int next_ticket;
    int now_serving;
};
void Lock3(lock* l) {
    int my_ticket = atomic_increment(&l->next_ticket); // 原子取号
    while (my_ticket != l->now_serving);               // 纯读等待叫号
}
void Unlock3(lock* l) {
    l->now_serving++;                                  // 叫下一个号
}
```

【代码做了什么？】 – `Lock1` 是最朴素的 test-and-set 锁：每次循环都执行原子"测试并置位"。幻灯片用 coherence 流量图展示了它的灾难：当 P1 持有锁时，P2、P3 等每个等待者每尝试一次就发一次 `BusRdX`（把锁变量所在 cache line 置为 1 并失效别人的副本），导致互连网络上无效化请求风暴。 – `Lock2` 先做一次**普通读**自旋（`while (*l != 0)`），只有当观察到锁被释放（读到 0）时才真正发一次 `test_and_set`。幻灯片分析：每个等待者**每次锁释放**只产生一次失效，共 O(P) 次失效；若所有处理器都把锁缓存了，则总流量是 $O(P^2)$。 – `Lock3` 把"抢锁"变成"取号 + 等号"：`atomic_increment` 是唯一需要原子性的操作；等号阶段是纯读（`my_ticket != l->now_serving` 不产生原子流量），释放时 `now_serving++` 产生**一次失效**，互连流量仅 O(P)，且保证先来先服务。

【并行机制解说】 – 三个版本都依赖 cache coherence 协议来传播锁状态（这正是幻灯片先复习 MSI 状态转移图的原因）：`test_and_set` 本质是"读-改-写"，必须以 `BusRdX` 形式独占 cache line 才能原子完成，因此每次尝试都会把其他处理器持有的副本失效。 – **对应概念**：本示例对应核心概念 4（test-and-set）、6（细粒度锁的"降低流量"动机）、7（ticket lock）。幻灯片给出的锁的**理想特征清单**是评价标准：低延迟（无竞争时快速获取）、低互连流量（高竞争时依次获取）、可扩展（流量随处理器数合理增长）、低存储开销、公平（按请求顺序获取，避免饥饿）。简单 test-and-set：低竞争下延迟低、但流量高、扩展性差、存储仅一个 int、无公平性条款；ticket lock 补上了公平性。

示例 2：hand-over-hand 细粒度锁——有序链表插入（C）

```
struct Node {
    int   value;
    Node* next;
    Lock* lock;        // 每节点一把锁
};

struct List {
    Node* head;        // 哨兵节点
    Lock* lock;        // 列表级锁：只用于"取第一个节点"的瞬间
};

void insert(List* list, int value) {
    Node* n = new Node;
    n->value = value;
    //  （幻灯片为简洁省略了"插入表头"的边界处理）

    Node* prev, *cur;
    lock(list->lock);
    prev = list->head;
    lock(prev->lock);          // 锁住第一个节点
    unlock(list->lock);        // 列表级锁立刻释放

    cur = prev->next;
    if (cur) lock(cur->lock);  // 锁住第二个节点（手递手第一步）

    while (cur) {
        if (cur->value > value)
            break;             // 找到插入位置
        Node* old_prev = prev;
        prev = cur;
        cur = cur->next;
        unlock(old_prev->lock); // 释放身后的锁
        if (cur) lock(cur->lock);// 锁住前面的新节点
    }
    n->next = cur;             // 在 prev 与 cur 之间插入
    prev->next = n;
    unlock(prev->lock);
    if (cur) unlock(cur->lock);
}
```

【代码做了什么？】 – 遍历从哨兵 `head` 开始，全程保证**手里最多握着两把相邻节点的锁**（`prev` 和 `cur`）。 – 每前进一步：先锁住下一个节点，再释放上一个节点的锁——"手递手"。 – 找到插入点后：`n->next = cur; prev->next = n;` 完成插入，最后释放手里两把锁。 – 注意 `list->lock` 只保护"从 head 出发"这一个瞬间（防止并发线程同时从表头开始遍历），一旦锁住第一个节点就立即释放。

【并行机制解说】 – **对应概念**：细粒度锁（概念 6）。它和示例 1 的"全局锁"对比：全局单锁把**所有**链表操作串行化（幻灯片：single global lock——简单正确但操作被串行化，限制并行性能）；细粒度锁让操作链表**不同区域**的线程并行推进。 – **为什么一定不会死锁（幻灯片留给学生的自检题）**：所有线程都**沿着同一个方向（表头→表尾）按一致的顺序**获取锁，且一次最多持有两把、总是"先获取更靠前的锁，再获取更靠后的锁"。因此资源依赖图不可能出现环，circular wait 条件不成立——立刻就能断定代码无死锁。 – **代价**：每步遍历都要取锁/放锁（额外指令，且遍历变成"带内存写"的操作）、每节点多一份锁的存储；幻灯片提示的折中方案：像选择任务粒度一样，**把链表分成若干段、每段一把锁**，用部分并行度换更低的锁开销。 – 幻灯片还留了一个挑战题：`insert()` 其实可以进一步优化——插入操作只修改 `prev->next`，并不需要修改 `cur`，因此可以不必持有 `cur` 的锁（delete 才需要两把锁）。

示例 3：单读单写有界队列——零同步的无锁队列（C）

```
#define N 1024                        // 队列容量（2 的幂）
#define MOD_N(x) ((x) & (N - 1))      // 环形下标取模

struct Queue {
    int data[N];
    int head;   // 队头：下一个要取出的元素位置（仅消费者写）
    int tail;   // 队尾：下一个空闲槽位（仅生产者写）
};

void init(Queue* q) { q->head = q->tail = 0; }

// 队列满时返回 false（tail 紧跟在 head 后面一格）
bool push(Queue* q, int value) {
    if (q->tail == MOD_N(q->head - 1))
        return false;
    q->data[q->tail] = value;
    q->tail = MOD_N(q->tail + 1);
    return true;
}

// 队列空时返回 false（head 追上 tail）
bool pop(Queue* q, int* value) {
    if (q->head != q->tail) {
        *value = q->data[q->head];
        q->head = MOD_N(q->head + 1);
        return true;
    }
    return false;
}
```

【代码做了什么？】 – `push` 先检查是否满：`tail == MOD_N(head - 1)`（环形缓冲里 tail 紧贴在 head 后面一格表示满）；不满则写数据、推进 `tail`。 – `pop` 检查是否空：`head != tail`；不空则读出 `data[head]`、推进 `head`。 – 全程**没有任何锁、没有原子操作、没有等待**：满/空时直接返回失败，由调用方决定重试或放弃。

【并行机制解说】 – **对应概念**：单读单写队列（概念 10）。为什么零同步也安全？因为 **head 只有一个写者（消费者）、tail 只有一个写者（生产者）**，两个线程从不写同一个变量；读对方的变量（生产者读 head 判断满、消费者读 tail 判断空）在顺序一致性（或正确 fence / C++11 atomic）下看到的是完整的最新值。这就是"单读单写"约束的价值：它把共享状态拆成两半，各归一方写，从而消除竞争。 – 幻灯片强调：这里假设顺序一致内存（或加适当 memory fences，或用 C++11 `atomic<>`）；现代弱一致性硬件上不能裸奔。这属于**无锁编程**（概念 9）的特例——两个线程互不阻塞对方。 – 幻灯片还给出**无界版本**（Dr. Dobbs 来源）：`head` 指向**队首元素之前**的节点，`tail` 指向最后加入的元素，生产者 push 时顺带用 `reclaim` 指针回收已经越过 `head` 的节点——**节点的分配与释放都由生产者线程完成**，这是它能保持无锁的关键。

示例 4：基于 std::atomic 的无锁栈 push/pop（C++11）

```
#include <atomic>

struct Node {
    Node* next;
    int   value;
};

class LockFreeStack {
public:
    void push(Node* n) {
        while (true) {
            Node* old_top = top.load(std::memory_order_relaxed);
            n->next = old_top;                 // 新节点指向当前栈顶
            if (top.compare_exchange_weak(old_top, n))
                return;                        // CAS 成功：栈顶没被改过
            // CAS 失败：有其他线程动过 top，重读再试
        }
    }

    Node* pop() {
        while (true) {
            Node* old_top = top.load(std::memory_order_relaxed);
            if (old_top == nullptr)
                return nullptr;                // 空栈
            Node* new_top = old_top->next;     // 预读下一个节点
            if (top.compare_exchange_weak(old_top, new_top))
                return old_top;                // CAS 成功：弹出 old_top
        }
    }

private:
    std::atomic<Node*> top{nullptr};
};
```

【代码做了什么？】 – `push(n)`：把新节点的 `next` 指向当前 `top`，然后 CAS 把 `top` 从 `old_top` 换成 `n`；若 CAS 失败说明读 `top` 之后有别的线程改了栈，循环重试。 – `pop()`：读 `top` 得 `old_top`（空栈返回 nullptr），预读 `old_top->next` 作为 `new_top`，CAS 把 `top` 换成 `new_top`；成功则返回弹出的节点。 – 主思想（幻灯片原话）：**只要没有其他线程修改过栈，这个线程的修改就可以进行**——CAS 就是"检查是否被修改过"的那一步。

【并行机制解说】 – **对应概念**：lock–free（概念 9）与 CAS（概念 5）。与细粒度锁的关键区别（幻灯片特别指出）：细粒度锁是"锁住数据结构的一部分"，而无锁实现**根本不对数据结构加锁**——线程通过 CAS 的返回值来确认自己的操作是否基于最新状态。 – 正确性论证：任何时候最多只有一个线程的 CAS 能成功，因此栈的"全局进展"有保证（失败者会重试，系统不会被某个被抢占的线程卡死）；但该定义不保证单个线程不饿死。 – 幻灯片给出的注意事项，这里必须诚实标注：`std::memory_order_relaxed` 在实际代码中通常不够，需要 acquire/release 语义或 fence 来保证 `n->next = old_top` 的可见性；此外本实现还**没有处理 ABA 问题与内存回收**（见示例 5 与核心概念 12）。可以用 `is_lock_free()` 检查当前平台上 `atomic<Node*>` 是否真的由硬件原子指令实现（否则可能退化为 mutex）。

示例 5：ABA 问题完整时间线演示（C++ 伪代码 + 注释）

```
// 演示 ABA 问题：CAS 只比较"值"，分不清"一直没变"与"变了又变回来"。
// 注意：A、B、C、D 是节点的【地址】，不是节点里的值！

// 初始栈：top → A → B → C
// 线程 0 开始 pop()：
Node* old_top = top;        // 线程 0 读到 old_top = A（地址）
Node* new_top = old_top->next;   // new_top = B
// 【此刻线程 0 被抢占！】

// 线程 1 执行 pop()：
//   CAS(&top, A, B) 成功  → top → B → C           （A 被弹出）
//   线程 1 修改节点 A：A->value = 42              （复用被弹出的节点！）
//   线程 1 执行 push(A)：
//     CAS(&top, B, A) 成功 → top → A → B → C      （A 又被压回去）
//   线程 1 执行 push(D)：
//     CAS(&top, A, D) 成功 → top → D → A → B → C

// 【线程 0 恢复执行】：
//   CAS(&top, A, B)  →  top 的当前值【又是 A】！CAS 成功！
//   → top → B → C    （节点 D 被静默丢失，栈被破坏！）
```

【代码做了什么？】 – 这是一段"叙述式"演示：把示例 4 的 `pop()` 拆开，逐行标注线程 0 被抢占期间线程 1 做了什么。关键在最后一步：线程 0 的 CAS 比较的期望值是地址 A，而 top 恰好又等于 A（因为 A 被弹出去又压回来了），于是 CAS 误判"没人动过栈"而成功。

【并行机制解说】 – **对应概念**：ABA 问题（概念 11）。它揭示 CAS 的语义盲区：**CAS 无法区分"对象没变"与"对象变了又变回原样"**。ABA 的名字来自值的变化轨迹 A → B → A。 – 幻灯片给出的两条解决路径： 1. **计数器方案**：给栈加一个 `pop_count`，每次 pop 都递增；用 **double compare-and-swap（DCAS）或 doubleword CAS** 同时比较 `(top, pop_count)` 两个值，只有两者都未变才算成功。x86 支持 `cmpxchg8b`（一次比较两个 32 位值）和 `cmpxchg16b`（两个 64 位值），把 `top` 和 `pop_count` 连续放置即可用一条指令实现。 2. **节点分配/复用策略**：精心设计分配器，保证"被弹出的节点地址不会这么快被重新压回栈顶"（例如不立即复用、延迟回收）。 – 幻灯片还补充了**另一个问题**：即使解决了 ABA，`pop()` 里 `old.top->next` 可能在解引用前已被其他线程 `delete`——即**引用已释放内存**。进阶解法就是 hazard pointer（概念 12）：弹出的节点先进 `retire` 列表，只有确认**没有任何线程的 hazard pointer** 指向它时才真正 `delete`。

三、关键要点

1. **死锁四条件缺一不可**：mutual exclusion、hold and wait、no preemption、circular wait。锁实现/使用中的死锁都可通过"破坏其中一条"来避免，例如细粒度锁中"所有线程按同一方向、一致顺序获取锁"直接消灭循环等待。
2. **锁的性能关键在于 coherence 流量**：test-and-set 每次尝试都发 BusRdX（流量灾难）；test-and-test-and-set 把每次锁释放的失效降到 O(P)、总流量 $O(P^2)$；ticket lock 每次释放仅一次失效（O(P) 流量）且天然公平——但公平性之外的理想特征还包括低延迟、低流量、可扩展、低存储。
3. **细粒度锁以复杂度换并行度**：hand-over-hand 让不同链表区域的操作并行，代价是每步的锁开销、每节点的存储开销和正确性难度；折中方案（分段锁）与"任务粒度选择"是同一个权衡。
4. **lock-free ≠ 无竞争**：lock-free 保证"系统级进展"（至少一个线程推进），但不保证单个线程不饿死；无锁设计并不消除竞争——高竞争下 CAS 会反复失败导致自旋重试（幻灯片 Summary 原话）。
5. **CAS 是"我操作期间别人动过没有"的探测器**：这正是下一讲事务内存的伏笔——事务内存把这个机制推广为"推测整个操作能成功，若被其他线程修改则 abort 重来"。

四、常见陷阱与注意事项

1. **把 ABA 误当成"不可能发生"或"只是理论问题"**：ABA 在节点被弹后又压回（地址复用）的真实场景中极易发生；幻灯片特意注明 A/B/C/D 是地址而非值，并提醒不要与 ABBA 问题混淆。
2. **无锁代码忘记内存回收/内存顺序**：只写 CAS 循环还不够——`old_top->next` 可能指向已释放内存（需要 hazard pointer 等方案），现代弱一致性硬件上还必须有 fence 或 C++11 acquire/release 语义；幻灯片明确"仍然需要 appropriate memory fences on modern relaxed consistency hardware"。
3. **CAS 循环在高竞争下"活锁"**：所有线程同时抢栈顶时 CAS 反复失败、不断重试，系统在"执行大量操作"但推进缓慢——这正是 livelock 的雏形（幻灯片对 livelock 的计算机系统例子就是"operations continually abort and retry"）。
4. **细粒度锁的正确性想当然**：要精确判断"哪些步骤必须互斥"（例如删除节点必须同时锁 prev 和 cur，插入只需锁 prev）；否则会出现幻灯片演示的两类损坏：两个 insert 同时算得相同 prev/cur 导致**一次插入丢失**，insert 与 delete 并发导致**插入节点指向已删除节点**。
5. **以为无锁一定更快**：幻灯片引用 Hunt 2011 的测量：无锁队列/链表的运行时间**以 pthread mutex 为基准归一化后可能高于 1**——在"只有你的程序使用这台机器"的典型优化场景（科学计算、图形、ML、数据分析）里，写得好、带锁的代码常常与无锁一样快甚至更快、而且简单得多；无锁的价值主要出现在大量线程、critical section 内可能发生 page fault/被抢占的场合（数据库、web server），因为锁在那里会引发 priority inversion、convoying、临界区内崩溃等问题。

五、思考题（带答案）

Q1：幻灯片问：在 test-and-set 锁的 coherence 流量图中，运行在 P1 上的线程**持有锁多长时间**？P1 的 cache 在哪些时刻**含有锁变量的有效副本**？ **A1**：P1 从它那次成功的 test-and-set（BusRdX 把线置 1 并获得锁）开始持有锁，直到它执行 `st mem[addr], #0` 释放锁。P1 的 cache 在成功获得锁的那次 BusRdX 之后到其他处理器发起 BusRdX 使它失效之前，含有有效副本；之后它的副本被置为 Invalid，直到它再次读/写锁变量（释放时重新获得独占）。这也解释了为什么 P1 释放锁时还要"等总线"——互连争用会延长锁转移时间。

Q2：为什么 hand-over-hand 链表代码"立刻就能断定无死锁"？如果把遍历方向改成"有的线程从表头往表尾、有的线程从表尾往表头"，还会无死锁吗？ **A2**：因为所有线程都按**表头→表尾的同一顺序**获取锁（先锁更靠前的节点），且一次最多持有两把、总是先获取下一把再释放上一把，资源依赖图不可能成环（circular wait 不成立）。若允许反向遍历，两个相向而行的线程可能各自持有一把"对方下一步需要"的锁，循环等待成立，就可能死锁——这正是"系统级锁顺序策略"（lock ordering）要解决的问题。

Q3：为什么 ticket lock 的"等号"阶段不需要原子操作？它相比 test-and-test-and-set 还多了什么好处、代价是什么？ **A3**：等号阶段只读 `now_serving`（读操作天然安全，多个线程可同时读）；唯一需要原子性的是取号时的 `atomic_increment(&next_ticket)`。好处：每次释放只产生一次失效（O(P) 流量）且 FIFO 公平、无饥饿。代价：比 test-and-test-and-set 多一个计数器（存储开销略增），且每个线程都要先取号——在竞争极低时多了一次原子自增的开销。

Lecture 17: Transactional Memory (Part I)（日期：Dec 02）

> **概述**：本讲把同步的抽象层次再往上抬一层：从机器级原子指令（test-and-set、fetch-and-op、CAS、LL/SC）和软件层原语（锁、屏障、无锁结构），提升到**事务内存（Transactional Memory, TM）**。先讲清 memory transaction 的语义（atomicity、isolation、serializability）以及 `atomic { }` 与 `lock/unlock` 在语义上的本质区别；然后讨论 TM 实现的两大设计问题——**data versioning policy（数据版本策略）**与**conflict detection policy（冲突检测策略）**；最后分别给出乐观（optimistic）与悲观（pessimistic）两种冲突检测的行为与权衡。本讲末尾的"实现"内容（STM/HTM 细节）将在第 18 讲继续展开。

一、核心概念与定义

1. Memory Transaction（内存事务）

- **定义**：一段**原子且隔离**的内存访问序列，灵感来自数据库事务（database transactions）。它有三个语义性质：
- **Atomicity（原子性，all or nothing）**：事务 commit（提交）时，事务内的所有内存写在**同一时刻**全部生效；事务 abort（中止）时，所有写都像从未发生过一样不可见。
- **Isolation（隔离性）**：在 commit 之前，**任何其他处理器都观察不到**本事务的写。
- **Serializability（可串行化）**：所有事务看起来是以**某个单一串行顺序**提交的；但语义**不保证**确切的提交顺序。
- **现实类比**：银行转账——"从 A 扣钱 + 给 B 加钱"必须整体生效或整体不生效；你不可能看到"钱已扣但没到账"的中间状态。
- **公式/图示**：

```
事务 T：读 X, Y, Z；写 A, X
其他处理器要么看到 T 的【全部】读写结果，要么【一个都看不到】——
就好像这些读写在同一瞬间发生（幻灯片：effectively all happen at the same time）
```

幻灯片总结：我们在一致性系统里为**单个地址**维护的那些性质，事务把它推广到**一组读写**上。

2. Declarative vs. Imperative Abstraction（声明式 vs. 命令式抽象）

- **定义**：声明式抽象只说明"**做什么**"（what），命令式抽象说明"**怎么做**"（how）。`atomic { ... }` 是声明式的：程序员声明"这段代码要原子执行"，**不指定**用锁、用无锁还是用其他机制；系统负责实现原子性。对比：命令式的做法是"获取这把锁、执行操作、释放锁"。
- **现实类比**：点外卖时你说"我要一份宫保鸡丁"（声明式），而不是"请打电话给某饭店、下单、付款、等配送"（命令式）——后者是你替店家把实现细节全包了。
- **图例**（幻灯片原例）：

```
声明式：执行这 1000 个相互独立的任务
命令式：spawn N 个工作线程；从一个共享任务队列取任务分配给线程
声明式：把这一组操作原子地执行
命令式：获取锁 → 执行操作 → 释放锁
```

3. `atomic { }` 与 `lock()/unlock()` 的语义差异

- **定义**：`atomic` 是对原子性的**高层声明，不规定实现方式**；`lock` 是**低层阻塞原语**，本身**不提供**原子性或隔离性（它只是互斥手段）。要点：锁**可以用来**实现 atomic block，但锁的用途**超出**原子性（如生产者–消费者同步、排队、条件等待）；因此**不能**把所有锁用法都替换成 atomic 区域。反过来，用 atomic 编程消除了很多 data race，但**仍可能犯 atomicity violation**——例如程序员把本该一个原子块完成的序列错误地拆成两个 atomic 块。
- **现实类比**：`atomic` 像"承诺书"（我承诺这段代码原子执行），`lock` 像"门闩"（把门锁上不让别人进）——承诺可以用门闩实现，但门闩还能用于别的场景（比如防止宠物跑出门），不能一看到门就以为必须承诺书。
- **反例（幻灯片）**：用 `synchronized` + 两个 flag 做线程间握手（`flagA = true; while (flagB == 0);`）——这种"等待"语义是锁才有的，原子块**不提供等待**，不能直接替换。
- **反例（atomicity violation，幻灯片）**：程序员把逻辑上原子的序列错误拆成两个 atomic 块，另一个线程就能在中间插入破坏性操作：

```
// 线程 1（错误写法：被拆成两个原子块）
atomic { ...; ptr = A; ... }      // 块 1：写指针
atomic { B = ptr->field; }        // 块 2：解引用指针
// 线程 2
atomic { ...; ptr = NULL; ... }   // 恰好在块 1 与块 2 之间执行 → 崩溃/错误
```

即使每个块自身原子，**块与块之间仍可被插入**，逻辑原子性照样被破坏——这是"atomic 也救不了程序员"的经典示例。

4. Optimistic Concurrency（乐观并发）

- **定义**：系统默认事务之间不会有真正的冲突，**只有在真正发生 contention（竞争）时才做串行化**。幻灯片：TM 系统采用乐观并发——只在出现**真实冲突**（read-write 或 write-write 冲突）时才需要保证串行化；没有冲突的事务完全并行执行。
- **现实类比**：多人同时编辑同一份在线文档的不同段落：系统假定大家改的是不同段落（乐观），只有两人确实改了同一段（冲突）时才需要协调（合并/回滚）。
- **对比**：悲观并发（锁）是"先拿锁再干活"，乐观并发是"先干活，提交时再检查有没有撞车"。

5. Read-Write Conflict / Write-Write Conflict（读写冲突 / 写写冲突）

- **定义**：**read-write conflict（R-W）**：事务 A 读了地址 X，而事务 B **未提交地**写了 X；**write-write conflict（W-W）**：事务 A 与 B 都处于 pending（未提交）状态且都写了 X。注意：**read-read 永远不冲突**——两个事务读同一个地址可以安全并行。
- **现实类比**：两个人同时改一份报表的同一个单元格（W-W），或一个人在看、另一个人在改同一个单元格（R-W）；两人只是同时"看"则毫无问题。
- **图例**（幻灯片树形例子，credit: Austen McDonald）：

```
        1
       / \
      2   3
     /     \
    4       5        目标：线程安全地同时修改节点 3 和 4

事务 A: READ 1,2,3; WRITE 3     事务 B: READ 1,2,4; WRITE 4
→ 没有 R-W 也没有 W-W 冲突（没人写对方读/写的数据）→ 可并行提交

事务 A: READ 1,2,3; WRITE 3     事务 B: READ 1,2,3; WRITE 3
→ 两者都写节点 3 → 冲突存在 → 两个事务必须串行化
```

- 幻灯片用这个例子对比细粒度锁：hand-over-hand 锁在更新节点 3 时**顺路锁住节点 1、2**（遍历路径上的所有节点），可能延误另一个对节点 4 的更新——**锁会阻碍本可并行的操作**（locking can prevent concurrency）；而事务只记录"实际读/写了哪些节点"，只要两个事务的读集/写集不冲突就互不干扰。

6. Read Set / Write Set（读集 / 写集）

- **定义**：系统为每个进行中的事务记录它访问过的地址集合：**read set** = 事务执行期间读过的地址，**write set** = 事务执行期间写过的地址。冲突检测正是基于"我的读/写集 与 别人的读/写集 是否有交集"。
- **现实类比**：每个人在超市购物时拿一个购物清单（写集）和试吃记录（读集）；结账（commit）时收银员核对有没有人和你买了同一件东西、或者有人试吃后又改了价格。

7. Data Versioning Policy（数据版本策略）

- **定义**：TM 系统如何管理**未提交（新）版本**与**已提交（旧）版本**两份数据。两种基本策略：
- **Eager versioning（undo-log based，基于撤销日志）**：**写内存时立即就地更新**，同时在 undo log 里记下旧值，以备 abort 时回滚。
- **Lazy versioning（write-buffer based，基于写缓冲）**：写操作先进入事务的 write buffer，**commit 时才真正更新内存**；abort 只需清空缓冲。
- **现实类比**：eager 像"先改账本、另记一本撤销账"（改得快，但万一要撤销得逐条回退）；lazy 像"先在便签上打草稿，定稿了才誊抄进账本"（撤销就是撕掉便签，但誊抄要花时间）。
- **权衡（幻灯片）**：eager——每次 store 都要记 undo（per-store overhead），**commit 快**（数据已在内存）、**abort 慢**、有**容错问题**（事务中途崩溃时内存里是半成品）；lazy——**abort 快**（清日志即可）、无容错问题、**commit 慢**（要刷缓冲）。哲学：eager 是"立刻写内存，赌事务不会 abort"；lazy 是"只在不得不写的时候才写内存"。

8. Pessimistic Conflict Detection（悲观冲突检测，又称 eager）

- **定义**：在**每次 load/store 执行时立即检查**是否与别的 pending 事务冲突。哲学（幻灯片原话）："我怀疑冲突随时可能发生，所以每次内存操作后都检查一次……反正迟早要回滚，不如现在就发现，避免浪费更多工作。"检测到冲突时，由 **contention manager（竞争管理器）** 决定**暂停（stall）**还是**中止（abort）**该事务。
- **现实类比**：开车时每过一个路口都停下来确认没有对向来车（哪怕大概率没有）——安全，但每个路口都要踩刹车。

9. Optimistic Conflict Detection（乐观冲突检测，又称 lazy/commit）

- **定义**：**只在事务尝试 commit 时**才检测冲突。哲学："先往最好的方向想，等提交时再集中处理冲突。"一旦提交事务与其他事务冲突，**提交方优先**，其他事务可能被 abort。
- **现实类比**：一路畅行到目的地才在终点检查有没有违章——大多数时候什么事都没有，撞上了（冲突）再处理。

10. Contention Manager（竞争管理器）

- **定义**：悲观检测中的仲裁组件：冲突发生时由它决定让谁 stall、让谁 abort、以及 abort 后何时重试。不同的策略服务于不同场景（幻灯片：various policies to handle common case fast）。幻灯片在悲观检测的第 4 个 case（双方反复 abort、毫无进展）下留问：**如何避免 livelock?** ——答案是 contention manager 的仲裁策略（如随机退避、按年龄优先、写者优先等）。
- **现实类比**：十字路口的交警（contention manager）：两车相持时由交警决定谁先走、谁倒车（abort）重来，避免两车永远互相让路（livelock）。

11. Failure Atomicity（失败原子性）

- **定义**：事务系统把"异常处理"也纳入原子性：**除了程序员显式管理的异常外，所有异常都导致事务 abort 并撤销内存更新**——因为事务要么整体提交要么整体不存在，所以"失败线程持有的锁丢失"这类问题不会发生（失败恢复 = abort + restart）。
- **现实类比**：网购下单流程中途断网：订单要么成功要么被系统整体回滚，绝不会出现"钱扣了、订单没生成"的中间态，也不需要你手动写"退款"补救代码。
- **对比（幻灯片）**：手动同步 + try/catch 的写法要求程序员**逐 case 提供 undo 代码**（"undo code 1"、"undo code 2"...），还要追踪哪些副作用对其他线程可见；事务把这一切交给系统：

```
// 手动版本：每个异常都要手写撤销逻辑
void transfer(A, B, amount) {
    synchronized(bank) {
        try {
            withdraw(A, amount);
            deposit(B, amount);
        }
        catch (exception1) { /* undo code 1 */ }
        catch (exception2) { /* undo code 2 */ }
        ...
    }
}
// 事务版本：系统处理所有（程序员未显式管理的）异常
void transfer(A, B, amount) {
    atomic {
        withdraw(A, amount);
        deposit(B, amount);
    }
}
```

注意事务版本的额外好处：**不存在"失败线程持有的锁丢失"**——因为事务根本没有持锁，失败就是 abort + 内存回滚。

12. Composability（可组合性）

- **定义**：把多个同步代码模块组合成更大的同步操作的能力。锁的组合需要**全系统范围的锁顺序策略**才能正确（否则 `transfer(A,B)` 与 `transfer(B,A)` 并发即死锁），这破坏软件模块化；事务**天然可组合**：程序员声明外层"transfer 原子执

行"，内层的 withdraw/deposit 若有自己的事务会被**外层事务吸收**（subsume），**最外层事务决定原子性边界**；系统对冲突的事务做串行化（如 transfer(A,B) 与 transfer(B,A)），对不冲突的事务保持并发（如 transfer(A,B) 与 transfer(C,D)）。

- **现实类比**：乐高积木：每个模块（withdraw、deposit）内部自己是完整的，拼成大结构（transfer）后整体依然是一个原子单元——而不是像锁那样"两个零件拼在一起需要额外胶水规则（锁顺序）"。

补充：动机回顾——HashMap 的三级演进（幻灯片主线例子）

幻灯片用 Java `HashMap` 串起整个动机链条，值得单独梳理：

方案	线程安全	编程难度	性能
裸 `HashMap`（get 直接遍历 bucket 链表）	**否**（需要同步时是坑）	低	好（无同步时零锁开销）
`synchronized` 粗粒度包装层	是	低	**差**：全局锁限制并发、扩展性差
细粒度（每 bucket 一把锁）	是	高	好：减少争用（但**不需要同步时也付出锁开销**）
`atomic { return m.get(key); }`	是	低（和粗粒度一样简单）	**取决于工作负载与 atomic 的实现**（幻灯片原话）

关键洞察（幻灯片）：细粒度锁"即使不需要同步也付出锁开销"，而事务是**乐观**的——`get` 几乎总是只读（read-read 不冲突），在无竞争时按普通读执行，几乎零开销。配合第 16 讲回顾的图表（balanced tree 与 hash table 上 fine locks 优于 coarse locks），事务的目标是在**简单性**与**并发度**之间同时拿高分。

补充：TM 实现的设计空间小结（本讲范围）

本讲只展开两大设计轴，第 18 讲再叠加具体系统实例：

```
                    数据版本策略（Data Versioning）
          Eager（undo-log，写内存立即生效）        Lazy（write-buffer，提交时才写内存）
冲突    悲观（每次访存检查）        早发现、可 stall；无前进保证       每次访问都要查/写缓冲，开销更高
检测    乐观（提交时检查）          提交方优先；有前进保证             最自然的组合：写缓冲 + 提交校验
```

（HTM 的 cache 位元版本管理、STM 的时间戳方案、TCC/LogTM 等具体实例在第 18 讲展开。）

二、代码示例与详细解说（本讲重点）

示例 1：`deposit`——锁版本 vs 事务版本（C 风格伪代码）

```
// ---- 版本 A：用锁保证原子性 ----
void deposit(Acct account, int amount) {
    lock(account.lock);                 // 先拿锁（悲观：先互斥再干活）
    int tmp = bank.get(account);        // 读
    tmp += amount;                      // 改
    bank.put(account, tmp);             // 写
    unlock(account.lock);
}

// ---- 版本 B：用事务保证原子性 ----
void deposit(Acct account, int amount) {
    atomic {                            // 声明式：只声明"这段要原子"，不说怎么实现
        int tmp = bank.get(account);    // 读
        tmp += amount;                  // 改
        bank.put(account, tmp);         // 写
    }
}
```

【代码做了什么？】 – 两个版本都实现同一个 `read-modify-write` 操作：读出余额、加金额、写回。版本 A 显式管理锁；版本 B 只声明 `atomic { }`，把同步的"如何做"完全交给系统（系统可以用锁实现 atomic，也可以用乐观并发实现）。 – `deposit` 需要原子性，是因为它是"读-改-写"三连：两个线程同时执行时，若没有原子性，会出现经典的 lost update（两个线程都读到旧值、各自加钱、后写覆盖先写）。

【并行机制解说】 – **对应概念**：memory transaction（概念 1）与声明式抽象（概念 2）。版本 B 的原子块语义是：提交时所有写一次性生效（atomicity）；提交前其他线程看不到（isolation）；两个并发 deposit 最终呈现某个串行顺序（serializability）。 – 幻灯片强调的**语义区别**（概念 3）：`atomic` 不承诺"怎么实现"——系统可以（幻灯片原话）"用锁实现 atomic { }"；而本讲讨论的实现采用**乐观并发**：只在真正出现 R-W 或 W-W 冲突时才做串行化。这正是事务与"锁住的临界区"的分水岭：锁是悲观地"先互斥后执行"，事务是乐观地"先执行、提交时再对账"。

示例 2：双链表 `PushLeft`——用 `atomic` 一行声明搞定（C）

```
typedef struct QNode {
    struct QNode *left, *right;
    int val;
} QNode;

// ---- 非线程安全版本 ----
void PushLeft(DQueue *q, int val) {
    QNode *qn = malloc(sizeof(QNode));
    qn->val = val;
    QNode *leftSentinel = q->left;       // 左哨兵
    QNode *oldLeftNode = leftSentinel->right;
    qn->left = leftSentinel;
    qn->right = oldLeftNode;
    leftSentinel->right = qn;            // 修改左哨兵的 right
    oldLeftNode->left = qn;              // 修改旧首节点的 left
}

// ---- 线程安全版本：整个操作包进 atomic ----
void PushLeft(DQueue *q, int val) {
    QNode *qn = malloc(sizeof(QNode));
    qn->val = val;
    atomic {
        QNode *leftSentinel = q->left;
        QNode *oldLeftNode = leftSentinel->right;
        qn->left = leftSentinel;
        qn->right = oldLeftNode;
        leftSentinel->right = qn;        // 两处指针写
        oldLeftNode->left = qn;          // 必须一起原子生效!
    }
}
```

【代码做了什么？】 – 在双链表头部插入一个新节点需要**同时**更新两个指针：`leftSentinel->right` 和 `oldLeftNode->left`。若这两步被并发线程打断，链表会出现**只挂了一半**的不一致状态（如：从左向右能遍历到 qn，从右向左却遍历不到）。 – 用锁实现需要精细决定锁哪些节点（细粒度锁的正确性难题，正是第 16 讲内容）；用 `atomic` 只需要把整个序列包起来。

【并行机制解说】 – **对应概念**：memory transaction（概念 1）。这里的"原子性"覆盖**多个不同地址的写**——这正是事务相对单地址原子指令的价值：CAS 只能原子地改一个地址，而事务把"改 leftSentinel->right + 改 oldLeftNode->left"这两个地址的写当作一个整体提交。 – 若两个线程同时 PushLeft：它们的写集（哨兵、旧首节点）重叠 → 检测到冲突 → 系统让其中一个 abort 重来，保证最终一致性；若两个线程操作**不同的**双链表（不同节点集合），无冲突，完全并行——这就是幻灯片说的"事务提供 automatic fine-grained concurrency"。

示例 3：`transfer`——锁的组合死锁 vs 事务的组合（C 伪代码）

```
// ---- 锁版本：组合出死锁 ----
void transfer(A, B, amount) {
    synchronized(A) {
        synchronized(B) {
            withdraw(A, amount);   // 先锁 A 再锁 B
            deposit(B, amount);
        }
    }
}
// 线程 0: transfer(A, B, 100)
// 线程 1: transfer(B, A, 200)
// → 线程 0 持 A 等 B，线程 1 持 B 等 A → DEADLOCK!

// ---- 事务版本：组合优雅 ----
void transfer(A, B, amount) {
    atomic {
        withdraw(A, amount);   // withdraw 内部若有原子块，被外层吸收
        deposit(B, amount);    // 最外层 atomic 定义原子性边界
    }
}
// 线程 0: transfer(A, B, 100) 与 线程 1: transfer(B, A, 200)
// → 系统检测到写集冲突，串行化这两个事务（而不是死锁）
// → transfer(A, B, 100) 与 transfer(C, D, 200) 无冲突，并行执行
```

【代码做了什么？】 – 锁版本：`transfer` 要保证"从 A 取钱给 B"整体原子，最直接的做法是同时锁 A 和 B。但两个转账方向相反的线程会互相持锁等待——教科书式死锁。幻灯片还展示另一个变体：两个线程各自锁 B 再锁 A（`transfer(B, A, amount)` 把锁顺序写成 `synchronized(B) { synchronized(A) }`）同样死锁。 – 事务版本：`transfer` 外层一个 `atomic`，内部的 `withdraw` / `deposit` 即使是独立模块（各自带锁或原子块），其原子性诉求会被外层事务**吸收**，最外层决定边界。

【并行机制解说】 – **对应概念**：composability（概念 12）。锁的组合需要"全系统锁顺序策略"，而策略往往要跨模块约定、破坏模块化；事务把"组合时的串行化决策"交给系统：冲突对串行化，无冲突对并行。幻灯片原话：**"Transactions compose gracefully (in theory)"**——程序员只需声明全局意图（transfer 原子执行），无需知道全局实现策略。 – 这正是 TM 的**生产力论点**（slides 承诺清单之一）：事务用接近粗粒度锁的简单性，获得接近细粒度锁的性能（见示例 4 的性能对比图），还能自动适配核心数（4 核最优的锁方案未必是 64 核最优的——performance portability）。

示例 4：悲观 vs 乐观冲突检测行为对比（伪代码时间线）

```
======== 悲观检测（eager）：每次 load/store 后立即检查 ========
Case 1（无冲突）：T0 rd A – wr B – wr C – commit     T1 rd A – commit   → 两者都成功
Case 2（提前发现）：T0 wr A ...（check 发现 T1 也在动 A）→ T0 stall 等待 → 之后 commit
Case 3（中止）：T0 rd A; T1 wr A → check 冲突 → T1（或 T0）abort → 重执行
Case 4（无进展）：T0 wr A / T1 wr A 反复 check 互相 abort restart → livelock!
                  （幻灯片提问：如何避免? → 需要 contention manager 仲裁）

======== 乐观检测（lazy/commit）：只在 commit 时检查 ========
Case 1（无冲突）：两个事务都跑到 commit → 检查通过 → 都成功
Case 2（提交方优先）：T1 先 commit（写 A）→ check 时发现 T0 读/写了 A → T0 abort 重来
Case 3/4：冲突事务被 abort 后 restart → 有 forward progress 保证
```

【代码做了什么？】 – 这是两段"行为时间线"伪代码，归纳幻灯片第 47、49 页的四种 case：悲观检测下，Case 2 能把 abort 提前变成 stall（省掉已做的无用功），Case 4 则可能因双方互踩而**毫无进展**（需要仲裁避免 livelock）；乐观检测下，提交方总是赢，被 abort 的事务 restart，系统有**前进保证（forward progress）**。

【并行机制解说】 – **对应概念**：pessimistic（概念 8）与 optimistic（概念 9）检测、contention manager（概念 10）。幻灯片给出的权衡表： – 悲观：**好**——冲突发现早（少撤销无用工作、部分 abort 变成 stall）；**坏**——无前进保证、某些 case 反而更多 abort、每次 load/store 都要检查（细粒度通信）、检测在关键路径上。 – 乐观：**好**——前进保证、批量（bulk）通信与批量冲突检测（提交时一次检查）；**坏**——冲突发现晚、仍有公平性问题（总是后提交者吃亏）。 – 幻灯片在悲观 Case 4 的注释：图示假设"激进（aggressive）的 contention manager：写者赢"，即谁先写谁占上风，其他事务 abort——这解释了为什么 Case 4 会反复

restart。 – **性能预告**：幻灯片给出"locks vs. transactions"对比图：在 balanced tree 与 HashMap 两个基准上，**TCC**（Stanford 的硬件事务内存系统，第 18 讲详述）不仅超过 coarse locks，也优于 fine locks——这支撑了"事务常常达到细粒度锁的性能"这一承诺。但注意这是理想化结果，真实收益取决于冲突率（见第四节陷阱 4）。

示例 5：HTM 风格伪代码预览——`xbegin` / `xend`（伪汇编）

```
; 硬件事务内存（HTM）风格的 begin/end 伪代码（第 18 讲详述）
    xbegin  fallback       ; 开始事务；若 abort，跳转到 fallback 地址执行
    ; ---- 事务体：普通 load/store 即可 ----
    ld   R1, [A]
    ld   R2, [B]
    st   [C], 5            ; 写入暂存（硬件在 cache 里维护写集）
    xend                    ; 提交：所有写一次性生效
    ret
fallback:
    ; abort 路径：例如退化为自旋锁保护的重试路径（Intel RTM 的典型做法）
    call  acquire_spinlock
    ...                    ; 用锁重做事务体
    call  release_spinlock
    ret
```

【代码做了什么？】 – `xbegin` 让硬件开始一个事务（保存寄存器检查点）；事务体内的普通 load/store 被硬件记录到读集/写集；`xend` 提交。若发生冲突（或其他原因），硬件跳转到 `fallback`。 – 本讲只做"预告"，细节在第 18 讲（HTM 的 cache 位元、coherence 冲突检测、Intel Haswell RTM 的 `xbegin` / `xend` / `xabort`）。

【并行机制解说】 – **对应概念**：memory transaction（概念 1）+ 乐观并发（概念 4）。事务把"同步"从软件抬进硬件：程序员只需写普通代码 + 声明边界，硬件通过 cache coherence 协议自动检测冲突、自动回滚。 – 幻灯片在讲完 `deposit` 的锁版本后专门设了一个 self-check：**`atomic { }` ≠ `lock() + unlock()`**——前者是声明，后者是底层原语；锁能实现原子块，但锁还能做原子性之外的事（如示例 3 的握手等待），所以**并非所有锁都能被原子块替换**。

三、关键要点

1. **事务 = 原子 + 隔离 + 可串行化**：提交时所有写一次生效；提交前无人可见；整体呈现某个串行提交顺序（但顺序本身不保证）。"对一个地址维护的性质"被推广到"对一组读写"。
2. **`atomic` 是声明，`lock` 是原语**：`atomic { }` 不规定实现；锁可以用于实现原子块，也可以用于原子性之外的目的（等待/握手/排队），因此不能把所有锁换成 atomic；而错误地把一个逻辑原子序列拆成两个 atomic 块会造成 atomicity violation。
3. **事务的目标是"鱼与熊掌兼得"**：像粗粒度锁一样好写（声明式），像细粒度锁一样快（自动读–读并发、细粒度并发、性能可移植性），外加失败原子性与可组合性。
4. **实现的两大设计轴**：数据版本策略（eager/undo-log vs lazy/write-buffer）× 冲突检测策略（悲观/每次访存检查 vs 乐观/提交时检查）；再叠加检测粒度，就构成 TM 设计空间。
5. **乐观检测有前进保证，悲观检测发现早**：悲观把部分 abort 变成 stall 但可能 livelock（需要 contention manager）；乐观"提交方优先"，牺牲公平性换取 forward progress。

四、常见陷阱与注意事项

1. **以为 `atomic` 就是"自动加锁"**：语义上它只是声明原子性；系统可以完全不互斥（乐观并发下两个无冲突事务并行执行）。同时要记住锁能做 atomic 做不了的事（如轮询等待 flag），"全换成 atomic"会写出错误程序（幻灯片 flagA/flagB 握手例子）。
2. **把原子块切碎**：本该一个原子序列被程序员误拆成两个 atomic 块（如"`ptr = A`"与"`B = ptr->field`"分开），另一个线程在中间把 `ptr` 置 NULL——atomicity violation。atomic 消除 data race，但**不消除程序员造成的原子性错误**。

3. **以为事务内没有冲突就万事大吉**：事务的正确性取决于 commit 时能否串行化；事务里的读必须是"一致的快照"——若事务读到了别人未提交的写，检测机制必须能抓住它（这正是 read set 验证的意义，第 18 讲展开）。
4. **用锁的思维去度量事务性能**：事务在无冲突时开销很低（乐观），但冲突率升高时 abort 重试会带来放大效应；性能好不好**取决于工作负载与 atomic 的实现**（幻灯片在事务化 HashMap 处原话：performance and scalability depend on the workload and implementation）。
5. **忽略"提交顺序不保证"**：serializability 只要求存在某个串行顺序，不代表按时间先后提交；依赖特定提交顺序（比如"先到先得"）的程序逻辑是不可移植的。

五、思考题（带答案）

Q1：为什么说"用锁实现原子块"可行，但"把锁替换成原子块"不可行？请各举一例。 **A1**：可行方向：`deposit` 的锁版本与 `atomic` 版本语义等价——锁是实现原子性的手段之一，系统完全可以用锁来落实 `atomic { }`。不可行方向：生产者–消费者/线程握手场景，如线程 1 在锁内 `flagA = true; while (flagB == 0);` 等待线程 2 置位 flagB——原子块提供的是"原子+隔离"，**不提供等待/阻塞语义**，替换后语义就变了；这类同步只能由锁等低层原语完成。

Q2：悲观检测的 Case 4（两个事务反复写同一地址、互相 abort）如何避免 livelock？乐观检测为什么天然不存在这个 Case? **A2**：悲观检测需要 contention manager 仲裁：例如随机退避（abort 后随机延迟再试）、优先级/年龄策略（老事务优先）、写者优先等，打破"双方同时 abort 又同时重试"的同步节奏。乐观检测天然避免：因为冲突只在 commit 时裁决且**提交方优先**——任何时刻至少有一个事务（先提交的那个）能成功完成，其余 abort 后重试，因此系统有前进保证（forward progress）；它付出的代价是公平性（晚提交者可能被反复 abort）。

Q3：给定两个并发事务：T1 读 x、写 y；T2 读 y、写 x。请问它们是否冲突？若在一个乐观检测系统里，可能发生什么？ **A3**：冲突：T1 写了 y 而 T2 读了 y（R–W 冲突，T2 读到了 T1 未提交的 y），T2 写了 x 而 T1 读了 x（R–W 冲突）。若两个都提交，读集/写集验证会失败。在乐观检测下，先到达 commit 的事务（比如 T1）成功提交；后提交的 T2 在验证时发现其读集（y）被 T1 写过、或写集（x）与 T1 的读集冲突而被 abort、重执行——重执行后读到 T1 的提交值，两个事务最终呈现 T1→T2 的串行顺序。

Lecture 18: Transactional Memory (Part II) + Course Wrap Up（日期：Dec 04）

> **概述**：本讲先完成事务内存（TM）的"实现篇"：介绍 TM 设计空间中已提出的代表性 STM 与 HTM 系统（TL2、OSTM、Intel STM、TCC、LTM/VTM、LogTM），深入 STM 的软件实现细节（事务描述符/事务记录、基于时间戳的 McRT 算法、软件 barrier 及其编译优化、STM 慢的原因），再进入 HTM：缓存中的 R/W 位元版本管理、基于 cache coherence 协议的冲突检测、Intel Haswell RTM 指令，以及把 TM 当作一致性机制的 TCC 例子。最后是**课程收尾**：总结 CS149 的核心议题（识别并行性、调度、通信、局部性）与学完之后的进阶方向（CS217、CS/EE 282、CS348K、科研机会），并以 Ask Me Anything 收场。思考题部分改为"关于课程整体"的反思题。

一、核心概念与定义

1. TM 实现设计空间（Data Versioning × Conflict Detection × Granularity）

- **定义**：所有 TM 实现都要回答两个问题（第 17 讲已述）：数据版本策略（eager/lazy）与冲突检测策略（pessimistic/optimistic），再加上检测粒度（object/word/cache-line）。幻灯片按此给出现有系统地图：

```
软件 TM:  Sun TL2        = lazy 版本 + optimistic 检测（读/写）
            MS OSTM       = lazy 版本 + optimistic(读)/pessimistic(写)
            Intel STM     = eager 版本 + optimistic(读)/pessimistic(写);
                            eager 版本 + pessimistic(读/写)
硬件 TM:  Stanford TCC   = lazy + optimistic
            MIT LTM / Intel VTM = lazy + pessimistic
            Wisconsin LogTM = eager + pessimistic（与常规 cache coherence 最容易结合）
```

- **要点（幻灯片原话）**：**最优设计仍是开放问题**（optimal design remains an open question），对 HW、SW、hybrid 三种形态答案可能各不相同。
- **现实类比**：同一门课程（TM）不同老师（系统）的教案：有的先写黑板再擦（eager），有的先在草稿纸上写（lazy）；有的每讲一句就查纪律（pessimistic），有的下课才点名（optimistic）——没有公认最好的教法。

2. STM Barrier（软件事务屏障 / 插桩代码）

- **定义**：编译器把 `atomic { }` 里的每个内存访问替换成一次 STM 运行时的函数调用（如 `tmRead` / `tmWr` / `tmTxnBegin` / `tmTxnCommit`），这些插桩代码（instrumentation）负责版本管理、读集/写集跟踪、提交等簿记。因为同一函数**可能在事务内外都被调用**，STM 需要**函数克隆（function cloning）或动态翻译（dynamic translation）**来生成事务内/事务外两个版本。
- **现实类比**：安检插桩：每个乘客过安检门（内存访问）都要被机器扫一遍（barrier 函数）；同一乘客平时走普通通道、安检时走专用通道——两条通道是同一人的两个版本。
- **图示**：

```
atomic { a.x = t1; a.y = t2; if (a.z == 0) { a.x = 0; a.z = t3; } }

    ↓ 编译器插桩（软件 barrier）

tmTxnBegin();
tmWr(&a.x, t1);
tmWr(&a.y, t2);
if (tmRd(&a.z) != 0) { tmWr(&a.x, 0); tmWr(&a.z, t3); }
tmTxnCommit();
```

3. Transaction Descriptor / Transaction Record（事务描述符 / 事务记录）

- **定义**：STM 的两类核心数据结构：
- **Transaction descriptor（每线程一个）**：记录事务状态，用于冲突检测、提交、中止；包含**读集、写集、undo log 或 write buffer**。
- **Transaction record（每个数据项一个）**：一个指针大小的记录，守护共享数据的**事务状态**；处于 **shared** 状态时用**版本号或共享读锁**（允许多个读者）；处于 **exclusive** 状态时用**指向属主事务的写锁**。幻灯片特别注明：这与硬件 cache coherence 的工作方式相同。
- **现实类比**：descriptor 像每个人的"办事档案"（办到哪一步、碰过哪些材料）；record 像每份文件上的"借阅状态牌"：多人可同时"只读"（shared），一旦有人要"独占修改"（exclusive）就挂牌写明是谁。

4. 冲突检测粒度（Object / Word / Cache-Line）与 False Conflict（假冲突）

- **定义**：以多大的数据单位做冲突检测：
- **Object 粒度**：映射开销低、暴露优化机会，但产生**假冲突（false conflict）**——两个事务碰同一对象的不同字段也被判冲突（例如 Txn1 写 `a.x`、`a.y`，Txn2 读 `a.z`，对象级检测认为它们冲突）。
- **Element/word（字段）粒度**：减少假冲突、提高并发，但时间和空间开销都增加。
- **Cache-line 粒度**：与硬件 TM 天然匹配、降低事务记录的存储开销，但程序员和编译器难以分析。
- **混合策略**：按类型混搭（例如数组用元素级、非数组用对象级）。

- **现实类比**：宿舍卫生检查按"房间"（object）还是按"床位"（word）评分：按房间查，一个人乱就拖累全屋（假冲突）；按床位查更精细但检查成本高。
- **图例**：

```
Txn1: a.x = …; a.y = …      Txn2: … = a.z …
对象级检测：两者都碰对象 a → 判冲突（假冲突！x/y 与 z 无关）
字段级检测：x,y 与 z 不同字段 → 无冲突，可并行
```

5. 基于时间戳的 STM 版本跟踪（McRT STM 风格）

- **定义**：以 Intel McRT STM 为例（eager versioning + optimistic reads + pessimistic writes）：
- **Global timestamp（全局时间戳）**：每次**写事务提交**时递增。
- **Local timestamp（每事务时间戳）**：该事务上次验证时读到的全局时间戳值。
- **32-bit transaction record**：**最低位（LS bit）**：0 表示被写者锁定，1 表示未锁定；**高位（MS bits）**：未锁定时存"最近一次提交的时间戳（版本号）"，锁定时存"属主事务指针"。
- **现实类比**：图书馆的"版本号 + 借出牌"：书的版本号记录最后一次修订（timestamp），借出牌（lock bit）记录谁正在改；读者确认"版本号没涨且没人借出"才放心读。
- **不变量（验证条件）**：数据未被锁定 **且** 数据版本 ⩽ 本地时间戳 → 说明"我读到的数据没有被更新提交过"，可以安全使用。

6. Strong vs. Weak Atomicity（强原子性 vs. 弱原子性）

- **定义**：STM 面临的内存模型问题：**strong atomicity** 要求**即使是非事务代码**访问共享数据，也必须与事务代码保持原子语义（即非事务访问也要参与冲突检测）；**weak atomicity** 只保证事务之间的原子性，非事务代码可以"绕过"检测。**在纯软件中提供 strong atomicity 代价高昂**——这是幻灯片列出的 STM 挑战之一，也是推动硬件支持（HTM）的重要动机。
- **现实类比**：strong 像"所有车（无论是不是校车）都要遵守校车停靠规则"；weak 像"只有校车之间互相避让，普通车可以随便穿行"。

7. HTM：缓存中的版本管理（R/W 位元）

- **定义**：硬件事务内存把**数据版本管理放进 cache**：要么把 write buffer（lazy）缓存在 cache，要么把 undo log（eager）缓存在 cache，并给 cache line 增加**元数据位**跟踪事务读集/写集：**R 位**（load 时置位，标记读集）、**W 位**（store 时置位，标记写集）。R/W 位可以按 word 或 cache-line 粒度设置，在 commit/abort 时**批量清零（gang-clear）**。注意：eager 版本策略下，每次写还需要**第二次 cache 写**来记录 undo log。
- **现实类比**：用便利贴（R/W 位）贴在书架格子上记录"这本书我正在看/正在改"，结账（commit）或放弃（abort）时把整片便利贴一次性撕掉。
- **图示**：



8. HTM：基于 coherence 协议的冲突检测

- **定义**：**coherence 请求检查 R/W 位来检测冲突**（幻灯片原文）：
- 观察到对 **W-word 的 shared 请求** → **read-write 冲突**（别人要读我写的字）；
- 观察到对 **R-word 的 exclusive（写意图）请求** → **write-read 冲突**（别人要写我读的字）；
- 观察到对 **W-word 的 exclusive 请求** → **write-write 冲突**（别人也要写我写的字）。
- 该机制对 snooping 与 directory 两种 coherence 协议都适用。

- **现实类比**：教室里的"占位牌"系统：别人来借阅（shared 请求）发现你正在写（W 位）——冲突；别人要划掉（exclusive 请求）你正在读（R 位）的笔记——冲突；两人都要在同一页上写（W 位）——冲突。

9. Register Checkpoint（寄存器检查点）

- **定义**：事务 begin 时**必须**保存处理器寄存器状态（register checkpoint），以便 abort 时恢复执行上下文（寄存器、状态等），配合缓存中写集的失效完成"回到事务起点"。这是 HTM 的 CPU 侧改动之一（许多 CPU 本身已具备该能力）；CPU 侧还需 **TM state registers**（记录事务状态、abort handler 指针等）。
- **现实类比**：游戏存档：开始打 BOSS（事务）前先存档；打不过（abort）就读档重来，装备和血量回到开打前。

10. TCC（Transactional Coherence and Consistency）

- **定义**：Stanford 提出的激进方案：**把 TM 当作一致性机制本身——所有事务、所有时间（all transactions all the time）**，每个处理器上每个内存操作都在事务中；成功提交的事务更新内存与系统中所有 cache。TCC 的假设：**lazy + optimistic**；每个执行步（execution step）在所有处理器上**至多一个 commit**；当一个事务导致另一个事务 abort 重执行时，允许前者的 commit 与后者的 begin **重叠**，以**最小化执行步数**。
- **现实类比**：把整台机器当成一个"记账本共享会话"：谁要改账本都得先声明"我要改这几页"（事务），改完一次性誊写（commit）并通知所有人；撞页（冲突）的就得重写。

11. Intel RTM（Restricted Transactional Memory，受限事务内存）

- **定义**：Intel Haswell 引入的硬件事务内存指令集：`xbegin`（参数为 abort 时的**回退地址 fallback address**，例如回退到带自旋锁的代码路径）、`xend`（提交）、`xabort`（显式中止）。实现上**在 L1 cache 跟踪读集与写集**，处理器保证事务内所有内存操作原子提交。但**处理器可能因很多原因自动 abort**（例如读集/写集所在 cache line 被逐出就会 abort），且**实现不保证进展**（所以才需要 fallback 地址）。Intel 优化手册第 12 章给出提高事务不 abort 概率的指南。
- **现实类比**：用便利贴记账（L1 里的 R/W 位）；便利贴被风吹掉（cache line 逐出）就得整笔重来——所以重要交易（高冲突、大读集）不要用便利贴，直接用正式账本（fallback 锁路径）。

12. 并行 + 硬件特化（Parallelism + Hardware Specialization）与课程核心议题

- **定义**：课程收尾的核心论断（幻灯片原话）：**在可预见的未来，获得更高性能计算硬件的主要途径 = 增加并行度 + 硬件特化（hardware specialization）的结合**。证据就是当代芯片：NVIDIA GPU（单个 SMM core：32-wide SIMD、每 SMM 2048 个 CUDA/thread、Tensor Cores）、Apple A11（异构 SoC：多核 CPU + 多核 GPU + 媒体 ASIC + AI 单元）、Intel Core i7（CPU + 集成 GPU 与媒体）、FPGA（可重构逻辑）、Google TPU 与 AWS Trainium（AI 加速器）。
- **现实类比**：赛道升级不是靠把一辆车改到极限（单核频率），而是"多车并行（并行度）+ 每种赛道配专用车（特化）"。
- 课程反复强调的四大议题：**识别并行性（或识别依赖）**；**高效调度任务**（① 负载均衡 ② 克服通信约束：带宽限制、延迟、同步）；**利用数据/计算局部性 = 高效管理状态**。这些议题在异构移动 SoC、单芯片多核 CPU、多核 GPU、CPU+GPU、机器集群、AI 加速器等各种规模与场景下反复出现。

二、代码示例与详细解说（本讲重点）

示例 1：STM 展开——`atomic { obj.f1 = 42; }` 需要哪几步？（C 伪代码）

```
// 给定：乐观读、悲观写、eager 版本策略的 STM
// 问：实现 atomic { obj.f1 = 42; } 需要哪些步骤?

atomic {
    obj.f1 = 42;
}
```

```
// 答（幻灯片给出）：
TxDescriptor* tx = GetTxDescriptor();      // 1. 取本线程的事务描述符
OpenForWriteTx(tx, obj);                   // 2. 悲观写：验证数据版本、加写锁、加入写集
                                          //      （若被他人锁定/版本过期 → 处理冲突/中止）
LogForUndoIntTx(tx, obj, offset);          // 3. eager 版本：把旧值记入 undo log
obj.f1 = 42;                               // 4. 就地写入（eager：写内存立即生效）
// 事务结束处还有 commit：递增全局时间戳、释放写锁并置新版本号
```

【代码做了什么？】 – 一个看似普通的单字段写，在 eager + pessimistic-write STM 下被展开为四步：取描述符 → 打开写（验证 + 加锁 + 记写集）→ 记 undo → 真正写。若中途 abort，undo log 把 `obj.f1` 恢复原值。

【并行机制解说】 – **对应概念**：STM barrier（概念 2）、事务描述符/记录（概念 3）、时间戳版本跟踪（概念 5）。注意"悲观写"意味着**写之前**必须确保数据未锁定且版本不旧于本地时间戳——写冲突在**发生前**就被拦截；而"乐观读"意味着读不立即验证，把验证推迟到 read-set validation / commit 阶段。这套组合正是 McRT STM 的取舍：写冲突稀有时开销小、读多写少时读路径轻快。

示例 2：HTM 事务——Intel RTM 风格 `xbegin`/`xend` 与 fallback（C + 伪指令）

```
#include <immintrin.h>   // Intel RTM intrinsics: _xbegin/_xend/_xabort

void update(SharedState* s) {
    // 尝试硬件事务
    unsigned status = _xbegin();          // 相当于 xbegin fallback_addr
    if (status == _XBEGIN_STARTED) {
        // ---- 事务体：普通代码，硬件在 L1 cache 维护读集/写集 ----
        s->x += 1;
        s->y = s->x * 2;
        _xend();                          // 提交：所有写原子生效
        return;
    }
    // ---- abort 路径（fallback）：用自旋锁重做（幻灯片建议的典型回退） ----
    // 注：事务可能因冲突、cache line 逐出、中断等任何原因 abort；
    //      RTM 不保证进展，fallback 必须存在
    spinlock_acquire(&s->lock);
    s->x += 1;
    s->y = s->x * 2;
    spinlock_release(&s->lock);
}
```

【代码做了什么？】 – `_xbegin()` 返回 `_XBEGIN_STARTED` 表示事务已开始；事务体内是普通读写；`_xend()` 提交。任何 abort 都会让 `_xbegin()` 返回一个非 STARTED 的状态码（区分冲突、逐出、显式 `_xabort` 等），执行流落到 fallback 分支。 – fallback 分支用一把自旋锁把同样的操作以互斥方式重做——这是幻灯片明说的典型用法（"fallback to code-path with a spin-lock"）。

【并行机制解说】 – **对应概念**：HTM 的缓存版本管理（概念 7）、coherence 冲突检测（概念 8）、register checkpoint（概念 9）、Intel RTM（概念 11）。硬件做的事：begin 时取寄存器检查点；每次 load 置 R 位、store 置 W 位；其他核的 coherence 请求命中 R/W 位即触发 abort（见概念 8 的三种冲突）；commit 时"gang-clear"R/W 位、把写集变成有效脏数据。幻灯片强调：**处理器可能因很多原因自动 abort**（例如读集/写集所在 cache line 被逐出），且 **RTM 不保证进展**——所以 fallback 地址不是可选项而是必需品。 – 幻灯片给出的 HTM 性能数据：比 STM 快 **2x–7x**；单线程时离顺序执行**只差 10%** 以内；随处理器数高效扩展——因为冲突检测与版本管理全部由硬件流水线完成，无软件 barrier 开销。

示例 3：TM vs 锁——从"程序员的视角"对比（伪代码）

```
// 视角 1：粗粒度锁（简单但串行化）
Object get_sync(Map m, Key k) {
    synchronized (m) { return m.get(k); }   // 整个 map 一把锁：安全、易写、扩展性差
}
// 视角 2：细粒度锁（并发好但难写、无需同步也付锁开销）
Object get_fine(Map m, Key k) {
    lock(m.bucket[hash(k)].lock);           // 每 bucket 一把锁
    return m.bucket[hash(k)].get(k);
    unlock(m.bucket[hash(k)].lock);
}
// 视角 3：事务（声明式：系统保证原子性）
Object get_tx(Map m, Key k) {
    atomic { return m.get(k); }             // 系统实现原子性；读-读并发自动获得
}
```

```
三种实现的取舍（幻灯片综合）：
  正确性成本：synchronized 最易写；细粒度最难（锁顺序、死锁）；
              atomic 与 synchronized 一样易写（声明式）
  并发潜力：synchronized 最低；细粒度与 atomic 高（读-读天然并发）
  开销特征：细粒度"即使不需要同步也付锁开销"；atomic 乐观执行，无冲突时几乎零开销
  可移植性：锁方案在 4 核最优未必在 64 核最优（performance portability）
```

【代码做了什么？】 – 三个版本都实现线程安全的 `HashMap.get`。事务版本只是把 `m.get(k)` 包进 `atomic { }`，语义上系统保证原子性；性能取决于工作负载与 `atomic` 的实现（幻灯片原话，第 17 讲已述）。

【并行机制解说】 – **对应概念**：TM 的承诺（第 17 讲）+ 本讲的设计空间（概念 1）。幻灯片数据：硬件 TM（TCC）在 balanced tree 与 HashMap 上都优于 coarse locks 与 fine locks。但**性能不是唯一的账**——生产力论点（幻灯片原话）：系统级事务支持能以**开发时间的 10%** 获得专家级细粒度锁编程**约 90% 的收益**（第 18 讲版本的数字）。这正是"事务"作为第 17 讲所定义的**更高层抽象**存在的意义：把同步的复杂度从程序员转移到系统。

示例 4：TCC trace——把 TM 当一致性机制（表格演示，据幻灯片整理）

```
处理器 P1          P2            P3
————————————————————————————————————————
Begin T1           Begin T2      Begin T4
Read  A (A:0)      Read  A (A:0) Read  E (E:0)
Write A ← 1        Write E ← 3   Write B ← 6
Write C ← 2                      Write C ← 7
Read  D (D:0)                    Read  F (F:0)
Commit T1  →       Commit T2  →  Commit T4  →
（随后 P1 开始 T3）  Read E (E:3)  （随后 Commit T3）
Write C ← 5
Read  A (A:1)
Write E ← 6
Commit T3
————————————————————————————————————————
幻灯片 trace 中提交顺序：T2 → T1 → T4 → T3
读集/写集随执行逐步累积；每步至多一个 commit；
若某事务因其他事务 commit 而 abort，其重执行可与提交重叠，以最小化执行步数
```

【代码做了什么？】 – 这是幻灯片第 44–48 页的 TCC 执行 trace 的简化整理：三个处理器并发执行事务，表格记录每个事务逐步累积的**读集**（如 `A:0` 表示读到 A 的旧值 0）与**写集**（如 `A:1` 表示把 A 写成 1）。提交按某个串行顺序发生（T2 → T1 → T4 → T3），读到的值反映之前提交者的结果（T3 读 A 得到 1，正是 T1 提交的值）。

【并行机制解说】 – **对应概念**：TCC（概念 10）+ HTM（概念 7/8）。TCC 把事务提升为**系统唯一的一致性机制**：所有内存操作都在事务中，成功提交的事务更新内存与全部 cache。幻灯片列出的假设决定其行为：lazy + optimistic；**每执行步至多一个 commit**；被 abort 的事务重执行可与提交方 commit 重叠。这个例子把"读集/写集跟踪 + 串行提交顺序 + 提交可见性"完整串起来，是理解 HTM 如何兑现第 17 讲三条语义（atomicity/isolation/serializability）的最佳直观样例。

三、关键要点

1. **TM 设计空间已"人满为患"但无定论**：eager/lazy × pessimistic/optimistic × object/word/cache-line 的组合几乎都被实现过（TL2、OSTM、Intel STM、TCC、LTM、VTM、LogTM），但**最优设计仍是开放问题**，且 HW、SW、hybrid 答案可能不同。
2. **STM 的代价在 barrier 与内存模型**：软件插桩带来 2–8x 每线程开销（单线程即比顺序执行慢 1.8–5.6x，大头在读 barrier 与 commit），还需要函数克隆，且纯软件提供 strong atomicity 成本高昂——这些正是硬件支持的动机。编译器优化（分解 barrier 暴露冗余）能把单线程开销压到无并发控制的 40% 以内、锁方案的 30% 以内。
3. **HTM 把版本管理与冲突检测"搬进" coherence 协议**：cache line 的 R/W 位记录读集/写集，shared/exclusive 请求命中 R/W 位即检测出 R–W / W–R / W–W 冲突；R/W 位 commit/abort 时批量清零；配合寄存器检查点完成回滚。性能：比 STM 快 2–7x，单线程距顺序执行 10% 以内。
4. **HTM 不是银弹**：Intel RTM 不保证进展（cache line 逐出等就会 abort），必须提供 fallback；TCC 这类"全事务"系统依赖 lazy+optimistic 与每步单 commit 的假设。事务是"提高同步抽象层次"的一种工具，不是取代一切锁的万能方案。
5. **课程主线回顾**：性能来自**并行度 + 硬件特化**；获得性能的关键能力是识别并行/依赖、高效调度（负载均衡、克服带宽/延迟/同步约束）、利用局部性管理状态——这些思想在 CPU、GPU、SoC、集群、AI 加速器上以不同形态反复出现；而"现代软件相对硬件峰值能力惊人地低效"，理解并行机器的原理是挖掘这份性能的前提。

四、常见陷阱与注意事项

1. **把 HTM 当"免费的原子性"**：RTM 事务可能因 cache line 逐出、中断、过大读写集等原因**无冲突地 abort**，且硬件不保证进展——没有 fallback（如自旋锁路径）的程序会挂死或反复失败；Intel 优化手册第 12 章的指南（控制事务大小、避免逐出等）是提高成功率的必修课。
2. **忽略 STM 的常数开销**：软件 barrier 使单线程就比顺序执行慢近 2–6 倍，读 barrier 与 commit 是主要开销源（大多数应用读多写少）；"事务免费"是幻觉，收益必须与冲突率、事务大小一起评估。
3. **以为对象级检测就够细**：对象/粗粒度会造成假冲突（Txn1 写 a.x、Txn2 读 a.z 被误判冲突），反而损失并发；但 word 级检测又抬高时间/空间开销——粒度选择本身就是工程权衡，幻灯片明说 cache-line 粒度"对程序员与编译器都难分析"。
4. **课程层面的老毛病**：只优化串行部分忽略 Amdahl 定律、只盯计算峰值忽略带宽与延迟约束、把负载均衡想当然（workload imbalance 静默拖垮扩展性）、在弱一致性/弱原子性模型上写想当然的同步代码——这些在第 16–18 讲的多线程与事务语境下依然成立。
5. **学完就停**：幻灯片强调课程结论——未来性能靠"并行 + 特化"；如果只记住 API 不掌握"识别并行、调度、通信、局部性"的分析框架，面对新的并行硬件（下一代的 GPU、加速器、异构 SoC）会无从下手。

五、思考题（带答案）（本讲为课程整体反思题）

Q1：课程结尾说"现代软件相对硬件峰值能力惊人地低效，大量性能被留在桌上"。结合课程内容，请列举至少三个"性能被留下"的典型原因，并说明对应的课程知识点。 **A1**：① 并行度没被利用：程序存在未识别的依赖/串行瓶颈（Amdahl 定律，识别并行性/依赖）；② 调度不当：任务粒度过粗导致负载不均衡、或通信/同步开销吞掉并行收益（work distribution、work stealing、通信约束：带宽限制与延迟）；③ 局部性差：数据访问模式导致 cache miss、内存带宽浪费（数据/计算局部性 = 高效管理状态，例如分块、向量化、减少数据移动）。机器越复杂（多核、异构、加速器），这份"被留下"的性能越大——这正是并行系统原理知识的价值所在。

Q2：请用课程框架（并行性识别、调度、通信、局部性）快速评估一个"新"系统：例如一个 64 核 CPU + 4 个 Tensor Core 加速器的异构芯片上跑 LLM 推理。你会关注哪些问题？ **A2**：并行性识别：哪些算子是 data-parallel 可向量化/可上 Tensor Core（矩阵乘），哪些是串行依赖（如 attention 的 softmax 归约、KV cache 顺序更新）；调度：任务如何切分到核与加速器、如何避免负载不均衡与同步瓶颈（每层一次 kernel launch 还是融合）；通信：权重与激活的带宽需求 vs 芯片互连带宽、数据移动次数（算子融合减少中间张量读写）、延迟隐藏（流水线/多 batch）；局部性：权重驻留（cache/片上内存复用）、K/V 复用、batch 内复用。这套"先看并行、再看调度、再看通信、再看局部性"的顺序正是课程每讲的通用分析套路。

Q3：学完 CS149 后，如果想继续深入，幻灯片推荐的三门课分别侧重什么？结合你自己的兴趣，你会选哪条路？ **A3**：① CS 217（Hardware Accelerators for Machine Learning，冬季，Kunle 授课）：面向 ML 的硬件加速器设计——延续课程"硬件特化"主题，从芯片/架构角度理解 TPU、Trainium 这类加速器；② CS/EE 282（Computer Systems Architecture）：计算机系统架构，深入处理器微架构、内存层次、一致性协议（本讲 HTM 的 coherence 冲突检测正是在这类课程里继续深入）；③ CS 348K（Visual Computing Systems，春季，Kayvon 授课）：面向图像/视频的高性能软硬件系统设计（光线追踪、视频分析、手机相机处理、NeRF/AI 图形、快速数据标注等）——把课程的"并行+特化+局部性"方法论应用到图形与视觉系统。选课取决于兴趣方向：硬件/架构选 217/282，系统与图形选 348K；也可以继续了解 Kayvon 实验室的研究机会（LLM agent 效率优化、并行调度编译器抽象、1M fps 世界模拟引擎、虚拟运动员模拟、AI play tester、CS149 assistant agent 等）。

核心术语表（Glossary）

> 按字母顺序排列，涵盖全部 18 讲中定义的关键术语。

- **`__syncthreads()`（块内屏障）**：block 内所有线程必须到达的屏障点，用于隔开 shared memory 的"写"与"读"阶段；只同步本 block。
- **ABA problem（ABA 问题）**：CAS 只比较值而无法区分"值未变"与"值变回原样"导致的错误，典型后果是无锁栈中栈顶地址 A→B→A 变化后 CAS 误成功而丢失节点。
- **Abstraction vs Implementation（抽象 vs 实现）**：程序"算什么"（semantics）与"在并行机上如何算"（scheduling：谁/何时/在哪个 ALU 或 lane 上）的区分；把二者混为一谈是并行编程最常见的困惑来源。
- **Acquire / Release（获取/释放语义）**：成对使用的单向内存序——release 保证其之前的操作在其之后对其他线程可见，acquire 保证其之后的操作能看到 release 之前发布的内容；锁、屏障等同步原语的内部分子。
- **Algorithm / Schedule 分离**：Halide 的核心设计——算法声明输出如何由输入计算（不规定迭代顺序），调度指定循环顺序、分块、向量化与并行化方式，二者独立可调。
- **All-to-All（全交换）**：每个节点把自己的数据分片分别发送给所有其他节点（如 rank 0 的 A0..A3 分别发给 rank 0..3）。
- **AllGather（全收集）**：每个节点把自己的分片广播汇总，最终所有节点都拥有完整数据。
- **AllReduce（全归约）**：所有节点对各自数据归约后得到相同完整结果；等价于 ReduceScatter + AllGather。
- **AMAT（Average Memory Access Time）**：平均访存时间 = Σ(访问频率 × 访问延迟)；多处理器下因一致性通信而增大（AMAT_Multiprocessor > AMAT_Uniprocessor）。
- **Amdahl's Law（Amdahl 定律）**：可并行部分占比 p、串行部分占比 1−p 时，加速比上限 `1 / ((1-p) + p/P)`，P→∞ 时收敛到 `1/(1-p)`——串行部分决定加速比天花板（本讲以三个课堂演示建立直觉，后续课程系统使用）。
- **Amdahl's Law（阿姆达尔定律）**：最大加速比受串行占比限制：设 S 为串行占比，则加速比 ⩽ 1/S。
- **Arithmetic intensity / roofline（运算强度 / 屋顶线模型）**：单位字节数据搬运对应的运算量；决定程序是 compute-bound 还是 bandwidth-bound，是评估加速器匹配度的基本工具（贯穿第 9–12 讲）。
- **Arithmetic Intensity（算术强度）**：单位数据搬运对应的计算量；如逐元素向量乘法每元素 12 字节只换 1 次 MUL（强度极低），是带宽杀手（本讲未用该词，但思想实验即其雏形）。
- **Artifactual Communication（人为通信）**：源于系统实现细节（cache line 粒度、容量有限等）的额外通信，可用局部性优化减少。
- **ASIC（Application-Specific Integrated Circuit，专用集成电路）**：为单一用途定制、不可编程的电路；性能/面积与性能/功率可分别比 CPU 核高约 1000 倍与 100 倍（FFT 案例），但设计/验证/流片成本高达数千万至数亿美元。
- **Assignment（分配）**：把任务分配给 worker（线程/实例/向量通道），目标是负载均衡与低通信成本，可静态或动态执行。
- **Asynchronous (nonblocking) execution（异步/非阻塞执行）**：在较早操作完成前就启动较晚操作，重叠访存、通信与计算，避免等待；需要软件+硬件异步指令与同步原语（或硬件乱序执行）。
- **Atomic Operation（原子操作）**：不可被其他线程并发观察到的中间状态的读-改-写操作（如 `atomicAdd`）。

- **Atomicity violation（原子性违例）**：程序员把逻辑上原子的序列错误拆成多个原子块，使其他线程可插入破坏性操作导致的错误。
- **Atomicity（原子性，all or nothing）**：事务提交时所有写一次性生效、中止时所有写如同从未发生。
- **Attention（注意力）**：transformer 核心块——`S = QKᵀ`（N×N 分数）、`P = softmax(S)`（逐行）、`O = PV`；朴素实现需 N^2 空间，长序列会内存爆炸。
- **Autoscheduler（自动调度器）**：把调度建模为序列化决策（每个节点的 compute_at 层级 + tile 尺寸），用贪婪/束搜索在调度空间中搜索，并用 MLP 代价模型（几十微秒/次）快速评估候选。
- **Bandwidth-Bound（带宽受限）**：程序执行时间由"需搬运的数据量/带宽"决定而非计算量决定的状态；数据并行方案多次遍历数据，容易带宽受限（"if you can avoid being bandwidth bound"）。
- **Bandwidth-Limited（带宽受限）**：处理器请求数据速率超过内存供给速率时的执行状态；稳态下核心利用率只取决于指令吞吐与内存吞吐之比，与延迟、未完成请求数无关。
- **Bank（DRAM 存储体）**：DRAM 芯片内可独立预充电/激活的存储分区；各 bank 共享数据引脚，但可流水操作（一个 bank 传输时另一个 bank 激活），实现高引脚利用率。
- **Barrier（屏障）**：让指定数量线程都到达后才能继续的同步原语，是表达依赖的保守方式，把计算划分为阶段。
- **Block Descriptor（块描述符）**：发生窃取时运行时为含 spawn 的代码块创建的数据结构，追踪该块已 spawn 与已完成的工作数，用于实现 sync。
- **Blocked Assignment（分块分配）**：实例 k 处理连续块 [k·count, (k+1)·count)；同一时刻各实例访问不连续地址，需昂贵的 gather 指令。
- **Blocking / Tiling（分块）**：把 GEMM 组织成 C 的子块计算，使 A、B 子块在缓存/shared memory 驻留期间被复用 BLOCKSIZE 次，显著提高算术强度；块须装得进存储层级（不是越大越好）。
- **Blocking algorithm（阻塞式算法）**：允许一个线程无限期阻止其他线程完成操作的算法，任何使用锁的算法（无论自旋或让出 CPU）都是阻塞式的。
- **Blocking Receive（阻塞接收）**：recv 在消息数据拷入本地址空间并发出确认后才返回的同步通信。
- **Blocking Send（阻塞发送）**：send 在收到接收方确认、消息已进入接收方地址空间后才返回的同步通信。
- **Bulk Launch（批量启动）**：一次 kernel launch 创建成千上万线程的机制，成本远低于逐个 `pthread_create`（无需每线程分配栈与 OS 控制块）。
- **Burst mode（突发传输模式）**：一条 DRAM 命令批量传输多个连续列的数据，摊销预充电/激活的固定开销。
- **BusRd / BusRdX / BusWB（总线事务）**：MSI 协议的三类总线消息——BusRd 获取共享副本、BusRdX 获取独占副本（写前必须发出，迫使他人失效）、BusWB 把脏行写回内存。
- **Cache Coherence（缓存一致性）**：对每个内存位置存在一个与所有处理器观察一致的假设串行顺序，且每处理器按程序序执行、读返回串行顺序中最后一次写的值；解决"多个私有缓存复制同一数据导致观察不一致"的问题。
- **Cache Hierarchy（缓存层级）**：L1 → L2 → L3 → DRAM 的多级实现；越近容量越小、延迟越低（Kaby Lake 示例：4 / 12 / 38 / ~248 周期）。
- **Cache Line（缓存行）**：cache 存取数据的粒度（如 4 字节/行）；装入一行会顺带预载相邻地址的数据。
- **Cache Miss（缓存缺失）**：目标地址不在 cache 中，必须从更低层级取数；包括 cold miss（首次访问）与 capacity miss（工作集超出容量、旧行被逐出后再访问）。
- **Cache（缓存）**：芯片上的存储，保存内存中一部分值的副本；只影响性能不影响程序输出；命中时访问远快于 DRAM。
- **Capacity Miss（容量缺失）**：因 cache 容量不足、数据在两次访问之间被逐出而导致的 cache miss。
- **Child Stealing（子任务窃取）**：先执行 continuation、把子任务入队的策略（run continuation first），广度优先、需 O(N) 空间存储已 spawn 工作。
- **cilk_spawn**：Cilk 关键字，调用函数但允许调用者与它异步并行继续执行。
- **cilk_sync**：Cilk 关键字，等待当前函数所有已 spawn 的调用完成；含 spawn 的函数末尾有隐式 sync。
- **Cilk（Cilk Plus）**：源自 MIT、现为开源标准（GCC/Intel ICC 支持）的 C++ 语言扩展，提供 fork–join 并行原语。

- **Coherent Execution（一致执行）**：同一段指令序列适用于大量数据元素的程序性质；是 SIMD 高效利用的必要条件（多核并行不需要）。
- **Combiner（合并函数）**：并行 fold 中把各子段结果合并成最终结果的二元函数。
- **Communication Overhead（通信开销）**：并行单元之间传递数据（如 partial sum）所花的时间；通信是限制最大加速比的首要因素（DEMO 1 结论）。
- **Compare-and-Swap（CAS，比较并交换）**：原子指令，若当前值等于期望值则写入新值并返回是否成功；构建无锁数据结构与高级同步的万能积木（`compare_exchange_strong`）。
- **Compare-and-swap（CAS，比较并交换）**：原子指令，若目标当前值等于期望值则写入新值并置成功标志、否则把当前值读回，是无锁算法的基础（x86 的 `lock cmpxchg`）。
- **Composability（可组合性）**：多个同步模块可安全组合成更大原子操作的性质，事务嵌套时最外层定义原子性边界。
- **Compute Bound（算力受限）**：执行时间由机器指令处理能力决定的状态，此时吞吐达到峰值算力。
- **Compute Mode（计算模式）**：NVIDIA Tesla（2007）提供的首个非图形专用 GPU 接口——分配 buffer、上传 kernel 二进制、`launch(myKernel, N)` 以 SPMD 方式运行 N 个实例。
- **Compute-communication overlap（计算-通信重叠）**：让通信（如 AllReduce）与计算（GEMM/权重加载）并行进行；RDU 上 AllReduce 走芯片间专用通路、不消耗 HBM 带宽，32 socket 时仍保持 70%+ 利用率。
- **Conflict detection（冲突检测）**：系统判定两个并发事务是否冲突以及何时判定的策略，分悲观（每次访存检查）与乐观（提交时检查）。
- **Contention manager（竞争管理器）**：冲突发生时仲裁谁暂停、谁中止、何时重试的策略组件，用于避免活锁等无进展局面。
- **Contention（争用）**：大量请求在短时间内涌向同一资源（热点）导致排队、整体操作时间变长的现象。
- **Continuation Stealing（延续窃取）**：先执行子任务、把 continuation 入队的策略（run child first），深度优先、空间有界、无窃取时执行顺序同串行。
- **Continuation（延续）**：cilk_spawn 之后调用者"剩余要执行的代码"，可被放入工作队列供其他线程窃取。
- **Convolutional Layer（卷积层）**：局部连接且同一层所有单元共享同一组参数（weights + bias）的神经网络层；滤波器可视为"模式检测器"，输出幅度为该滤波器对输入局部区域的响应。
- **Critical Section（临界区）**：需要互斥保护的、访问共享资源的代码段，其中的操作应整体"原子"执行。
- **Cross-Instance Operation（跨实例操作）**：gang 内实例间交换数据的标准库原语，如 reduce_add（求和）、reduce_min（取最小）、broadcast（广播某实例的值）、shift/rotate（把值沿实例编号平移/循环移动）。
- **Cross-Program-Instance Operation（跨实例操作）**：ISPC 标准库提供的实例间通信原语，如 `reduce_add`、`reduce_min`、`broadcast`、`rotate`。
- **Crossbar（交叉开关）**：将所有核心两两直连的片上互连结构（如 SUN Niagara 2），面积约等于一个核。
- **CSR（Compressed Sparse Row，压缩稀疏行）**：稀疏矩阵的存储格式——`values`（非零值）、`cols`（列号）、`row_starts`（每行起始下标）三个数组。
- **CUDA Thread（CUDA 线程）**：逻辑控制流单元，抽象上与 pthread 类似，但实现完全不同（硬件线程、无 OS 调度）；通过 `threadIdx` 等内置变量区分身份。
- **CUDA（Compute Unified Device Architecture）**：NVIDIA 于 2007 年随 Tesla 架构推出的"C 风格"GPU 编程语言与运行时，抽象贴近现代 GPU 硬件能力，设计目标为低抽象距离（low abstraction distance）。
- **cuDNN**：NVIDIA 提供的高性能 DNN 层库（卷积等关键层有多种算法可选）；其 backend 还能现场编译生成融合实现。
- **CUTLASS**：NVIDIA 的 CUDA 模板库，提供 shared-memory GEMM、warp 级 GEMM、块加载迭代器、张量归约等原语，用于编写自定义高性能 DNN 层。
- **CUTLASS / Cute-DSL**：NVIDIA 的 GEMM 模板库（CUTLASS）及其 Python 化表达（Cute-DSL），以 tiled tensor 与布局抽象实现 tensor core 高性能编程。
- **Data parallel patterns（数据并行模式）**：可组合的计算原语——MM、Map、Zip、Reduce、Gather、Scatter 等；程序=模式组合，编译器负责 Tiling/Parallelization/Metapipelining/Place&Route/Codegen。
- **Data Parallelism（DP，数据并行）**：多份模型副本并行处理不同 batch；通信模式为 ReduceScatter+AllGather 或 AllReduce。

- **Data Parallelism（数据并行）**：对大量数据元素执行同一序列操作的并行方式，典型表达为 `foreach`、`#pragma omp parallel for`、`map()`，强调"对每个元素独立地做同一件事"。
- **Data Race（数据竞争）**：多个线程无同步地并发访问同一内存位置且至少一个为写操作，导致结果不确定。
- **Data versioning policy（数据版本策略）**：TM 系统管理未提交（新）版本与已提交（旧）版本数据的方式，分 eager（undo-log）与 lazy（write-buffer）两种。
- **Data-Level Parallelism（DLP，数据级并行）**：同一序列的指令同时作用于大量不同数据；SIMD 利用的就是 DLP。
- **Data-Parallel Model（数据并行模型）**：把计算组织成对元素序列的操作（如对序列所有元素执行同一函数）；典型表达为 NumPy 的 `C = A + B`、`map()` 等。
- **Data-Value Invariant（数据值不变量 / 写串行化）**：一个 epoch 开始时某地址的值等于上一个写 epoch 结束时的值，保证写结果被串行化传播，是 coherence 的另一条不变量。
- **Dataflow execution（数据流执行）**：数据到达即计算、数据在计算单元间直接流动的执行方式，无指令流、无集中控制；AI 模型本身可看作数据流图（GEMM + Pool + SoftMax 等）。
- **DDR（Double Data Rate）**：每时钟传输两次数据的内存标准；如 DDR4 2400 = 64-bit × 1.2 GHz × 2 = 19.2 GB/s/通道，双通道 38.4 GB/s。
- **Deadlock（死锁）**：多线程相互等待对方释放资源/完成操作而永远无法继续的阻塞状态（如同步收发下全体先 send 后 recv）。
- **Declarative vs. Imperative abstraction（声明式 vs. 命令式抽象）**：前者只声明"做什么"（如 `atomic { }`），后者规定"怎么做"（如"拿锁-执行-放锁"）。
- **Decomposition / Assignment / Communication & Synchronization（分解 / 分配 / 通信与同步）**：并行思维三步骤——把问题拆成可安全并行的工作块、把工作分配给处理器、管理通信与同步使其不成为加速比瓶颈。
- **Decomposition（分解）**：把问题拆成可并行执行的子任务，关键挑战是识别依赖。
- **Dependency（依赖）**：一个计算必须等另一个计算完成后才能执行的关系；识别依赖是分解阶段的核心挑战。
- **Dequeue（双端队列）**：工作窃取中每个 worker 的工作队列实现，本地线程从底部 push/pop，窃取线程从顶部偷取。
- **Device（设备）**：执行 kernel 的 GPU 侧；device 代码即 kernel 与 `__device__` 辅助函数。
- **DGX SuperPOD**：NVIDIA 的模块化 AI 集群架构；1K GPU 集群 = 140 个 DGX A100 节点（1120 GPU）+ Lustre 存储 + Mellanox HDR 200 Gb/s InfiniBand 全胖树网络。
- **dim3 / threadIdx / blockIdx / blockDim**：CUDA 内置的多维索引机制——`threadIdx` 是线程在块内的坐标，`blockIdx` 是块在网格中的坐标，`blockDim` 是块维度；全局坐标由 `blockIdx * blockDim + threadIdx` 计算，最多三维。
- **DIMM（Dual Inline Memory Module）**：多个 DRAM 芯片组成的模块（如 8 芯片 × 8 bit = 64-bit 接口），对控制器表现为更高容量、更宽接口的内存，最小传输粒度 64 bit。
- **Directory-Based Coherence（目录式一致性）**：用集中目录记录每行在各缓存中的状态，通过点对点"按需告知"消息维护一致性（如 Intel Core i7 用 inclusive 的 L3 充当目录），避免广播以提升可扩展性。
- **Dirty Bit（脏位）**：缓存行中标记"该行已被修改、内存副本过期"的状态位；在写回一致性协议中，脏状态表示该缓存是行的唯一有效持有者（M 状态）。
- **Distributed Address Space（分布式地址空间）**：host 内存与 device global 内存是两个不同地址空间，数据搬运靠 `cudaMalloc` / `cudaMemcpy` 等原语（`cudaMemcpyHostToDevice` 等方向参数），host 不能直接解引用 device 指针。
- **Divergent Execution（分支发散）**：warp 内线程走了不同指令路径（如 `if` 两侧），硬件只能掩码执行、逐路径串行，性能受损；是 SIMT 执行的主要性能陷阱。
- **Divergent Execution（分歧执行）**：指令流缺乏一致性（如 SIMD 内 if/else 两路分支）；硬件顺序执行各分支并用 mask 掩蔽无用 ALU 输出，最坏只有 1/8（CPU）/1/32（GPU）峰值。
- **Domain-specific accelerator（领域专用加速器）**：在有限领域内可编程（通常用 DSL，如 DNN），能效约 20×（如 Google TPU），是"完全可编程"与"完全固定"之间的折中。
- **Domain-Specific Programming System**：围绕 DSL 构建的系统，通过提高抽象层次让程序员快速写出高性能、可移植的程序，并用领域知识自动选择算法与并行化策略。

- **Double Buffering（双缓冲）**：重叠数据搬运与计算的技术——同时持有"正在处理的数据"与"正在传输的数据"两份片上缓冲，上一块计算时下一块已在加载。
- **Double compare-and-swap（DCAS / doubleword CAS）**：同时比较交换两个相邻值（如栈顶与弹出计数）的原子操作，x86 用 `cmpxchg8b` / `cmpxchg16b` 实现，是 ABA 的计数器解法。
- **DRAM row buffer（DRAM 行缓冲）**：DRAM 阵列中存放当前激活行（约 2 Kbit）的缓冲；访问已激活行（row hit）可跳过 precharge 与 row activation，延迟大幅降低。
- **DRF（Data-Race-Free）**：无数据竞争程序；同步化的 DRF 程序在非 SC 系统上也得到 SC 结果（"SC for DRF"），这是语言内存模型承诺的基础。
- **DSL（Domain-Specific Language）**：针对特定领域、表达能力受限的编程语言，通常高层、声明式且确定性，用受限表达力换取系统级的自动优化能力。
- **DSL（Domain-Specific Language，领域专用语言）**：面向特定领域（如 DNN）的编程语言/抽象，限制可编程范围以换取易用性与性能；GPU AI kernel 场景的 DSL 包括 Mosaic GPU、Cute-DSL 等。
- **DSP（Digital Signal Processor，数字信号处理器）**：可编程但指令流控制更简单的处理器，用 SIMD/VLIW 等复杂指令摊销控制成本；例：Qualcomm Hexagon DSP 每周期执行 29 个 RISC 级操作。
- **Dual channel（双通道内存）**：两个内存控制器/通道并行工作，等效加宽总线、提升吞吐。
- **Dynamic Assignment（动态分配）**：运行时动态决定分配以保证负载均衡，用于任务代价或数量不可预测的场景（如共享计数器、work queue）。
- **Eager versioning（急切版本管理，undo-log based）**：写内存立即生效并记录撤销日志，commit 快、abort 慢、有中途崩溃容错问题。
- **Efficiency（效率）**：加速比除以处理器数，`efficiency = speedup / P = T(1) / (P·T(P))`，衡量硬件被有效利用的程度。
- **Embedded DSL（嵌入式 DSL）**：嵌入宿主语言（如 CUDA/C++）的库式 DSL，如 ThunderKittens——用模板类型封装布局、异步原语与协调模式，程序员写"布局声明 + producer/consumer 回调"。
- **Energy efficiency（能效）**：单位能量完成的运算量；由 Power = Ops/s × Joules/Op 可知，提高能效即提高每焦耳运算数或降低每运算能耗。
- **Execution Context（执行上下文）**：处理器为正在执行的指令流保存的状态（寄存器值 + PC），硬件多线程下每个硬件线程各有一份。
- **Expert Parallelism（EP，专家并行）**：Mixture-of-Experts 中把不同 expert 放在不同设备；通信模式为 All-to-All。
- **Explicit SIMD（显式 SIMD）**：向量化发生在编译期，二进制中可见向量指令（vloadps/vmulps 等）；来源包括 intrinsics、并行语言语义、编译器自动向量化。
- **Failure atomicity（失败原子性）**：异常（除程序员显式管理的）导致事务整体中止并撤销内存更新，失败恢复即 abort + restart。
- **False conflict（假冲突）**：检测粒度过粗（如对象级）时，两个访问不同字段的事务被误判冲突而损失并发。
- **False Sharing（伪共享）**：两个处理器写不同地址但这些地址落在同一条缓存行，导致缓存行乒乓的纯人为通信；与程序语义无关，修复方法是 padding/对齐使数据独占缓存行。
- **FAST ≠ EFFICIENT（快 ≠ 高效）**：程序跑得快不代表用好了硬件；判断标准是是否充分利用了机器提供的能力（如 10 处理器上 2x speedup 的效率只有 20%）。
- **Fat-tree（胖树拓扑）**：数据中心常用网络拓扑，逐层收敛带宽充足，支持自适应路由与网内计算卸载（如 SharpV2）。
- **Filter（过滤）**：删除序列中不满足谓词的元素，输出为输入的子序列。
- **Fine-grained locking（细粒度锁）**：把一把全局数据结构锁拆成多个小锁（如每节点一把）以提高操作并行度的同步策略。
- **FlashAttention / Fused Attention（融合注意力）**：把 softmax 分块计算（增量维护 running max/sum-of-exp）并融合 QK^T 与 PV 两个矩阵乘，从不物化 N^2 矩阵、O 块常驻缓存；代价是每并入新块需重缩放旧累加值（额外计算）。
- **Fold / Reduce（折叠 / 归约）**：用二元操作 `f :: (b,a) -> b` 把序列折叠成一个值（串行 fold 需初始值）；并行 fold 还需 combiner 函数 `comb :: (b,b) -> b`，若 f 本身满足结合律则无需 combiner，初始值必须是单位元。
- **foreach**：ISPC 关键语言构造，声明循环迭代可并行，由系统负责把迭代分配给 gang 内的 program instance。

- **foreach（ISPC 循环结构）**：声明循环迭代互相独立、由整个 gang 完成；迭代如何分配给实例由 ISPC 实现决定，让程序员几乎像写串行程序一样表达数据并行。
- **Fork–Join Parallelism（分叉–汇合并行）**：用"创建新控制流（fork）→ 等待其完成（join）"表达分治算法中独立工作的并行模式。
- **FPGA（Field Programmable Gate Array，现场可编程门阵列）**：逻辑块 + 可编程互连构成的芯片，介于 ASIC 与处理器之间；用 LUT（查找表）实现逻辑，用硬件描述语言（如 Verilog）编程；现代 FPGA 含硬化的 SRAM 块、DSP 块与 CPU（ARM/RISC-V）。
- **FR-FCFS（First-Ready First-Come-First-Serve）**：常见内存调度策略——优先服务当前开放行的请求（最大化行局部性），其余按 FIFO；控制器还会合并小请求为大连续请求。
- **Gang（线程组）**：一次 ISPC 调用 spawn 的全部程序实例的集合；由 SIMD 指令实现（实例数 = SIMD 宽度或其倍数），运行在一个核的一个线程内。
- **Gang（组）**：一次调用 ISPC 函数时同时执行的一组 program instance 的总称。
- **Gather（收集指令）**：一条 SIMD 指令从多个不连续内存地址取数（如 vgatherdps）；比 packed load 复杂且昂贵。
- **Gather（聚集）**：`output[i] = input[index[i]]`——按索引序列取值，可并行且无冲突；CPU 有 AVX2 gather 指令，GPU 硬件支持但比连续加载贵。
- **Gauss–Seidel（高斯–赛德尔迭代）**：逐点用邻居当前值更新网格的迭代求解算法，本课程以 2D grid solver 为其并行化案例。
- **GEMM（General Matrix Multiply）**：稠密矩阵乘 `C += A×B`，是全连接层、卷积层、transformer 注意力块的共同内核（"the kernel for…"）。
- **Ghost Cell（幽灵单元）**：消息传递模型中从远端线程地址空间复制到本地数组边缘、归他人所有的数据副本（如网格边界行）。
- **Global Memory（全局内存）**：device 的 DRAM 主存，所有线程（所有 block）可读写，容量大但延迟高、带宽宝贵。
- **GPGPU（General-Purpose computation on GPU）**：2002–2003 年把 GPU 当作数据并行机器使用的"hack"（如把 512×512 图像映射为数组、用 fragment shader 逐像素计算），为 CUDA 的诞生铺路。
- **Greedy Join Scheduling（贪心汇合调度）**：线程在 sync/空闲时立即尝试窃取而非空等，只有系统中无工作可偷才空闲的调度策略。
- **Grid（网格）**：一次 kernel launch 启动的全部线程块集合；块之间被假设无依赖、可按任意顺序调度到任意数量核上。
- **GroupByKey（按键分组）**：`Seq (key, T) -> Seq (key, Seq T)`，把相同 key 的元素聚合成"序列的序列"。
- **Halide**：嵌入 C++ 的图像处理 DSL，用 Func/无副作用表达式声明式描述"做什么"（算法），用调度原语描述"怎么做"（调度），由编译器生成平台特定代码。
- **Hand-over-hand locking（手递手锁）**：遍历链表时先锁下一个节点再释放上一个节点的细粒度锁模式，所有线程按一致方向获取锁从而无死锁。
- **Happens-Before（先于关系）**：事件之间的必须先后关系；若某个期望输出导致 happens-before 图出现环（事件必须先于自身），则该输出不可能。
- **Hardware accelerator（硬件加速器）**：为特定计算任务（如 DNN 推理、FFT、视频编码）定制的专用硬件，通过消除通用处理器的指令流开销与减少数据搬运获得更高能效（perf/watt）。
- **Hardware Multi-Threading（硬件多线程）**：在同一个核上交错执行多个线程以隐藏停顿——当前线程无法推进就执行另一线程的指令。
- **Hardware transactional memory（HTM，硬件事务内存）**：版本管理放缓存（R/W 位记录读集/写集）、冲突检测融入 cache coherence 协议的事务内存。
- **Hazard pointer（危险指针）**：每线程声明"我正在访问、不可释放"的指针，被弹出节点进入 retire 列表、确认无任何危险指针指向后才真正释放，用于解决无锁内存回收。
- **HBM（High-Bandwidth Memory，高带宽内存）**：3D 堆叠 DRAM 芯片、用硅通孔（TSV）连接、经硅中介层与处理器互连的高带宽内存，接口 1024-bit/stack；优势为更多带宽、高能效、小尺寸（H100：6×HBM3=6144-bit、3.2 TB/s、80 GB）。
- **Hierarchical Blocking（层次化分块）**：按 L2 → L1 → 寄存器多级分块，逐级匹配内存层级以最大化各级复用。

- **High Watermark（性能上限水印）**：通过程序修改实验（如全部访问 A[0]、删除锁、删除数学运算）建立的"最好能做到多快"的上界，用于判断性能受何限制。
- **Hillis–Steele Scan（朴素并行 scan）**：每步跨距翻倍的并行 scan，Work = O(N log N)、Span = O(log N)——Work 高于串行算法，但 SIMD 通道利用率高，适合 warp 内 32 元素扫描。
- **Host（主机）**：运行普通 C/C++ 串行代码的 CPU 侧；host 代码负责分配内存、拷贝数据、启动 kernel。
- **Hot Spot（热点）**：被过多并发请求集中的共享资源。
- **im2col / Explicit GEMM（显式 GEMM）**：把卷积展开为矩阵乘——为每个输出位置物化一行"输入窗口拉平"向量（带 0-padding），再与滤波器矩阵做 GEMM；代价是存储开销 O(N) 且 DRAM 流量放大 R×S 倍。
- **Implicit GEMM（隐式 GEMM）**：不物化完整卷积矩阵，只在 GPU 片上 shared memory 里按块构造子矩阵并用调优子块 GEMM（如 CUTLASS）计算——无额外片外存储、不增加 DRAM 流量。
- **Implicit SIMD（隐式 SIMD）**：编译器生成标量指令，但硬件总是同时运行 N 份程序实例，并在运行时把相同指令合并到 SIMD ALU（GPU 模式，宽度 8–32）。
- **Inherent Communication（固有通信）**：并行算法中必须发生的通信，是算法根本属性（如发送 ghost rows），可用好的 assignment 减少。
- **Instruction Pipeline（指令流水线）**：指令分 IF/D/EX/WB 等阶段重叠执行；单条指令延迟 4 周期但吞吐 1 条/周期；"每时钟一条指令"指的是吞吐而非延迟。
- **Instruction Stream（指令流）**：程序编译后得到的一串处理器指令；处理器逐条取指、译码、执行。
- **Instruction-Level Parallelism（ILP，指令级并行）**：单条指令流中互不依赖的指令可并行执行；超标量处理器在硬件上自动发现并利用它。
- **Intel RTM（Restricted Transactional Memory）**：Intel Haswell 的 HTM 指令集（xbegin/xend/xabort），在 L1 跟踪读写集、不保证进展、abort 需跳转 fallback 地址。
- **Interleaved Assignment（交错分配）**：实例 k 处理元素 k, k+8, …；同一时刻各实例访问连续地址，可用一条 packed vector load 实现。
- **Interleaved Multi-Threading（交错多线程）**：每个时钟从多个线程中选一个执行其一条指令（时间上交错）。
- **Interposer（硅中介层）**：承载 DRAM 堆与处理器的硅基板，作为高带宽互连介质。
- **Invalidation（失效）**：一致性协议中通知其他缓存"某行即将/已被独占修改，你的副本作废"的机制；失效后其他处理器再读会触发缓存缺失。
- **Isolation（隔离性）**：事务提交前，其他处理器观察不到该事务的任何写。
- **ISPC Task（ISPC 任务）**：用于实现多核并行的第二种抽象；task 函数 + `launch[count]`（启动若干任务实例，各得 taskIndex/taskCount）+ `sync`（等待全部完成）。
- **ISPC（Intel SPMD Program Compiler）**：Intel 的 SPMD 编译器，把一份"看起来像标量"的代码编译成含 SIMD 指令的目标文件，供 C/C++ 链接调用。
- **Kernel fusion（kernel 融合）**：把多个计算步骤融合为一个 kernel，中间结果留在片上，提高数据局部性、消除启动与同步开销；RDU 把一个 Llama 3.1 8B decoder 融合成一次 kernel 调用（每 token 3 次 vs GPU ~800 次）。
- **Kernel launch / synchronization overhead（kernel 启动与同步开销）**：每次启动 kernel 与 kernel 间同步的固定代价；融合与持久 kernel 可将其消除。
- **Kernel loop / persistent kernel（kernel 循环 / 持久 kernel）**：把整个 decoder（甚至多个 decoder）的迭代放入一个 kernel 执行，消除每次迭代的启动开销（RDU 每 token 3 次调用 vs GPU ~800 次）。
- **KernelBench**：含数百个 PyTorch kernel 的基准测试集，用于评测 LLM 智能体自动生成快速且正确 CUDA kernel 的能力。
- **Kernel（内核）**：用 `__global__` 修饰、在 GPU（device）上以 SPMD 方式执行的函数；一次 kernel launch（`kernel<<<grid, block>>>(args)`）批量启动大量 CUDA 线程执行同一份代码。
- **Latency Hiding（延迟隐藏）**：用其他线程的工作填满访存停顿；不改变延迟本身，只消除利用率损失；算术/访存比越高，需要的线程越少（3 算术+12 周期 load 需 5 线程，6 算术只需 3 线程）。
- **Latency（延迟）**：完成一次操作所需的时间（如从 SF 开车到 Stanford 0.5 小时、一次访存 ~248 周期）。

- **launch / sync（启动 / 同步）**：ISPC 任务并行的两个语句——launch 创建并启动任务实例，sync 等待所有任务结束（相当于多线程的 spawn + join）。
- **Layer Fusion（层融合）**：把相邻层（如 Conv → Scale/Bias → MaxPool）融合进同一 kernel，中间结果不落 DRAM——如 scale/bias 在 conv 每算出一个元素后立即执行、max pool 在每算完 2×2 区域后立即取最大值，DRAM 流量减少约一个数量级。
- **Lazy versioning（懒惰版本管理，write-buffer based）**：写先进入写缓冲、提交时才更新内存，abort 快、无容错问题、commit 慢。
- **Livelock（活锁）**：系统在执行大量操作但没有任何线程取得有意义进展的状态，典型场景是操作不断 abort 并重试。
- **LLM Agent Optimization Loop（LLM 智能体优化循环）**：LLM 生成 kernel → 执行/profile（正确性、耗时、SM/DRAM 利用率、L2 命中率）→ 反思瓶颈 → 修改代码的迭代闭环（"trial and error via reflection"）。
- **Load Imbalance（负载不均）**：工作未平均分配导致部分处理器闲置、部分仍在忙；并行完成时间由最慢的处理器决定（DEMO 2 结论）。
- **Lock-free stack（无锁栈）**：用 CAS 循环实现 push/pop 的栈，核心思想是"只要没有其他线程改过栈顶，本线程的修改就可进行"。
- **Lock-free（无锁）**：非阻塞算法中保证至少有一个线程能推进（系统级进展）的性质，不保证单个线程不被饿死。
- **Lock（锁）**：提供互斥的同步原语，保证临界区同时只有一个线程进入。
- **Loop Fusion（循环融合）**：把多个遍历同一数组的循环合并为一个，提高算术强度、减少重复数据加载。
- **Loop Tiling/Blocking（循环分块）**：重排计算顺序使小邻域数据在 cache 中存活到被复用、减少 capacity miss 的优化。
- **Low Precision（低精度）**：用 16-bit/8-bit（正向 4-bit、极端 1-bit）表示 DNN 权重与激活，降低存储与带宽需求、配合张量核提速。
- **LUT（Lookup Table，查找表）**：FPGA 中实现组合逻辑的可编程真值表；如 LUT6 是 6 输入 1 输出、可视为 64 元素表，40 输入 AND 可由 8 个 LUT6 级联实现（延迟 3）。
- **Mapping（映射）**：把线程/worker 映射到硬件执行单元，可由操作系统、编译器或硬件完成。
- **Map（映射）**：把无副作用的一元函数 `f :: a -> b` 应用到序列每个元素、产生等长输出序列的高阶函数（Haskell `map`、C++ `std::transform`、JAX `vmap`）；因元素互不影响可任意并行。
- **mbarrier（内存屏障）**：CUDA 中用于异步数据搬运完成通知与同步的轻量原语（arrive/wait 语义）。
- **Memory Access Latency（访存延迟）**：内存系统把数据提供给处理器所需的时间（如 100 周期 / 100 nsec）。
- **Memory Bandwidth（内存带宽）**：内存系统向处理器提供数据的速率（如 20 GB/s）；与延迟正交——加带宽不降延迟。
- **Memory Consistency（内存一致性模型）**：定义不同地址上读写操作对其他处理器可见的顺序（硬件与编译器允许的重排范围），是硬件/编译器与应用软件之间的契约。
- **Memory controller（内存控制器）**：连接 LLC 与 DRAM 的访存请求调度器；负责物理地址到 bank/row/column 的映射，并在吞吐/延迟/能耗三个冲突目标间调度几十至几百个未完成请求。
- **Memory fence / memory ordering（内存栅栏 / 内存顺序）**：在弱一致性硬件上保证访存顺序可见性的机制，无锁代码与单读单写队列都依赖它（或 C++11 `atomic<>` 的顺序语义）。
- **Memory Fence（内存屏障）**：阻止重排的指令——fence 之前的所有内存操作完成后，之后的操作才能开始；昂贵但能恢复任意排序保证（如 x86 的 lfence/sfence/mfence）。
- **Memory Hierarchy（内存层次结构）**：寄存器、各级 cache、本地内存、远端内存按"低延迟高带宽小容量 → 高延迟低带宽大容量"排列的存储体系。
- **Memory transaction（内存事务）**：一段原子且隔离的内存访问序列，满足原子性、隔离性、可串行化三条语义。
- **MESI（协议）**：MSI 增加 E（Exclusive Clean）状态——行未修改但只有本缓存持有，E→M 升级无需总线事务，消除"无共享也要付两次事务"的低效（MESI，不是 Messi！）。
- **Message Passing（消息传递模型）**：每个线程在私有地址空间中运行、仅通过 send/receive 消息交换数据的编程模型，是集群与超算的模型。

- **Metapipelining（元流水线）**：层级化粗粒度流水线（"流水线的流水线"），把嵌套循环转成流式流水、重叠多个循环迭代、级间双缓冲；可改变访问模式（如转置），在 fusion 失效时仍有效。
- **Micro-batch（微批）**：把 mini-batch 进一步拆成的小批样本，用于细粒度流水并行：forward/backward 在 micro-batch 间错位流水，消除 pipeline bubble。
- **MMA / FMA / DP4（矩阵乘加 / 融合乘加 / 四元点积指令）**：复合指令类型，用一条复杂指令摊销指令流处理开销；半精度场景下可编程性开销分别约 27% / 2000% / 500%。
- **MSI（协议）**：基于失效的写回一致性协议，缓存行三态 Invalid/Shared/Modified；只有 M 状态可本地静默写，写非独占行必须先广播 BusRdX。
- **Multi-Core Processor（多核处理器）**：把晶体管用于增加核数而非让单条指令流更快；每个核独立取指/译码、执行完全不同的指令流。
- **Mutual exclusion / Hold and wait / No preemption / Circular wait（互斥 / 持有并等待 / 不可抢占 / 循环等待）**：死锁成立的四个必要条件，缺一即不可能死锁。
- **Mutual Exclusion（互斥）**：保证同一时刻只有一个线程进入临界区访问共享资源的机制，通常用 lock 实现。
- **NKI（AWS Neuron Kernel Interface）**：AWS Trainium 加速器上编写自定义 kernel 的底层库（Assignment 4 使用）。
- **Non-Blocking（Asynchronous）Send/Receive（非阻塞异步收发）**：send/recv 立即返回句柄、由 checksend/checkrecv 查询完成状态的通信，允许通信与计算重叠。
- **NUMA（Non-Uniform Memory Access，非均匀内存访问）**：不同核心访问同一内存地址延迟/带宽不同的体系结构特性（如多插槽系统、Intel ring interconnect、Sun Niagara crossbar）。
- **Numerical formats：BF16 / BF8（数值格式）**：低精度浮点格式降低每运算能耗与面积；BF16（1+8+7）与 FP32 范围相同但精度低；BF8 有 E4M3（最大 448）与 E5M2（最大 57344）两种。
- **Optimistic conflict detection（乐观冲突检测）**：只在提交时检测冲突且提交方优先，有前进保证但发现晚、有公平性问题。
- **Orchestration（编排）**：组织通信结构、加入同步、组织数据布局、调度任务，目标是降低通信/同步成本并保持局部性。
- **Out-of-Order Execution（乱序执行）**：不按程序顺序执行指令、但保证输出与顺序执行一致的调度方式；"respect program order"即结果必须与按序执行相同。
- **Overhead（开销）**：为获得并行性/均衡/低通信而付出的额外工作（调度、同步、簿记等），串行程序中不存在。
- **Parallel Computer（并行计算机）**：一组相互协作（cooperate）以快速求解问题的处理单元（processing elements）的集合——既要"多"，更要"协作"。
- **Parallel Slack（并行松弛）**：独立工作总量与机器并行执行能力的比值，实践中约 8 为宜。
- **Parallelism + hardware specialization（并行 + 硬件特化）**：课程结论——未来高性能硬件的来源是增加并行度与专用硬件（GPU、TPU、媒体 ASIC 等）的结合。
- **PCU / PMU / AGCU**：Pattern Compute Unit（模式计算单元，systolic+SIMD）、Pattern Memory Unit（模式存储单元，0.5 MB 高带宽）、Address Generator and Coalescing Unit（地址生成与合并单元，片外内存/IO 门户）。
- **Performance Counter（性能计数器）**：CPU 中统计指令数、时钟、cache miss、内存字节数等事件的寄存器（可用 PCM/VTune/PAPI 读取）。
- **Persistent Threads（常驻线程编程风格）**：程序员启动恰好填满 GPU 的 block 数，在 while 循环里用 `atomicInc` 自行领取工作的风格——依赖底层硬件参数（如 V100 的 `BLOCKS_PER_CHIP`），绕过硬件的线程块调度器。
- **Pessimistic conflict detection（悲观冲突检测）**：每次 load/store 后立即检查冲突，由 contention manager 决定暂停或中止，发现早但无前进保证。
- **PE（Processing Element，处理单元）**：脉动阵列中的基本计算单元，内部通常为"乘法器 + 加法器 + 局部寄存器"，每周期完成一次乘加并把数据传给邻居。
- **Pipeline bubble（流水气泡）**：流水并行中设备处于空闲的时间段（如流水线填充/排空期）；micro-batch 流水与合理 schedule 可显著缩小。
- **Pipeline Parallelism（PP，流水并行）**：按层切分模型，不同层在不同设备上，数据流水流过；通信模式为 Send-Receive。
- **Pipelining（流水线）**：把处理拆成多级、各级重叠执行以隐藏延迟/增加通信计算重叠的技术（硬件层面亦指指令/内存流水）。

- **Pooling（池化）**：在局部区域（如 2×2）取最大值（max pool）或平均值（avg pool）的下采样层，减小空间分辨率（如 W×H → W/2×H/2）。
- **Power Wall（功耗墙）**：动态功耗 ∝ 容性负载 × 电压^2 × 频率，静态功耗来自漏电；功耗=发热，限制了时钟频率无限提升，是单核性能停止增长的原因之一。
- **Prefetching（预取）**：硬件动态分析访存模式、预测未来地址并提前装入 cache 以减少停顿；预测错误会浪费带宽、污染 cache。
- **Prefix Sum（前缀和）**：当 ⊕ 为加法时的 scan，即 `[a0, a0+a1, a0+a1+a2, ...]`。
- **Private Memory（私有内存）**：kernel 内每线程独占的地址空间（通常即寄存器），其他线程不可见。
- **Producer–consumer pipeline（生产者–消费者流水线）**：把 kernel 组织为"生产者异步加载 tile → 消费者（tensor core）计算 → 写回"的流水，级间用多级缓冲与 mbarrier 协调，使计算与访存重叠。
- **Program Instance（程序实例）**：一次 ISPC 函数调用中并行执行的函数副本；每个实例有自己的一份局部（varying）变量。
- **Program Order（程序序）**：单个线程程序中 load/store 的原始执行顺序；SC 要求每线程操作按程序序提交。
- **programCount**：gang 中同时执行的程序实例个数（uniform 值）。
- **programIndex**：当前程序实例在 gang 中的编号（varying 值，每个实例不同）。
- **Programmability overhead（可编程性开销）**：现代处理器执行一条指令所需取指/译码/冒险检查/寄存器堆访问等环节消耗的能耗与时间；H.264 能耗分解显示功能单元占比很小，开销主要来自控制与数据通路。
- **PSO（Partial Store Order）**：进一步放松 W→W，允许写缓冲内重排写（如 `A=1; flag=1` 可能被其他处理器看到反序）。
- **R bit / W bit（读位 / 写位）**：cache line 上标记该行属于当前事务读集/写集的元数据位，commit/abort 时批量清零。
- **RDU（Reconfigurable Dataflow Unit）**：SambaNova SN40L 的数据流处理单元，含 1040 个 PCU+PMU、638 TFLOPS (bf16)、520 MB 片上 SRAM；PCU 做 systolic+SIMD 计算，PMU 做高带宽寻址存储，AGCU 访问片外内存。
- **Read set / Write set（读集 / 写集）**：事务执行期间读过的地址集合与写过的地址集合，冲突检测基于二者与别的事务集合的交集。
- **Read–write conflict（读写冲突）**：事务 A 读了地址 X，而事务 B 未提交地写了 X。
- **Reconfigurable Dataflow Architecture（RDA，可重构数据流架构）**：把计算按空间布局在芯片上的可配置架构（如 Plasticine、SambaNova SN40L），无指令 ⇒ 无取指/译码开销，极端异步。
- **Red–Black Coloring（红黑着色）**：把网格按棋盘格染色、先并行更新所有红格再更新所有黑格的 Gauss–Seidel 重排方法，使同色格互不依赖。
- **ReduceScatter（归约–散播）**：先做归约，再把结果按 rank 分片分发，每个节点得到最终结果的一部分。
- **Register checkpoint（寄存器检查点）**：事务 begin 时保存的寄存器状态，abort 时据此恢复执行上下文。
- **Relaxed Consistency（放松一致性）**：允许违反部分内存操作排序约束以换取性能（隐藏内存延迟）的模型族，如 TSO、PC、PSO、WO、RC，区别在于放弃哪几条排序。
- **ReLU（Rectified Linear Unit）**：`f(x) = max(0, x)` 的逐元素激活函数，常用于卷积后引入非线性。
- **Ring Interconnect（环形互连）**：Intel 处理器（Sandy Bridge 起）的片上互连结构，四环分别承载 request/snoop/ack/data 消息。
- **Roofline Model（屋顶模型）**：以算术强度为横轴、可达吞吐为纵轴的性能分析模型，水平区为计算受限、对角区为带宽受限。
- **Row hit / row miss（行命中 / 行缺失）**：访问是否落在当前已激活行；row miss 需 Precharge + Activate + Column Access 全流程（各约 10 ns），row hit 仅需列选+传输。
- **SC for DRF（无数据竞争即顺序一致性）**：C11/C++11 与 Java 5 的承诺——DRF 程序获得顺序一致性，编译器负责插入必要同步；有 race 则无任何保证。
- **Scale up / Scale out（纵向扩展 / 横向扩展）**：单节点做强（更快互连、更多内存）vs 标准节点堆集群（网络互连）；AI 数据中心两者并用。
- **Scan（扫描）**：给定结合律操作 ⊕，输出每个位置"从开头到该位置"的累计结果；inclusive scan 含当前元素，exclusive scan 不含（首元素为单位元 I）。

- **Scatter（散开）**：`output[index[i]] = input[i]`——按索引序列写入目标位置，可能冲突（多个 i 写同一位置），可用原子操作、排序+分段 scan 或 sort+gather 实现。
- **Scheduling Primitive（调度原语）**：描述 N 维域迭代方式的高层指令，如 tile（分块）、vectorize（SIMD 向量化）、parallel（多线程）、compute_at（指定中间量计算层级）。
- **Segmented Scan（分段扫描）**：scan 的推广，对输入序列的连续分区各自独立做 scan（如 `[[1,2],[6],[1,2,3,4]]` 的 exclusive 分段 scan 为 `[[0,1],[0],[0,1,3,6]]`）。
- **Semi-Static Assignment（半静态分配）**：周期性 profile 执行并重新调整、两次调整之间保持静态的分配，适合变化缓慢的自适应网格/粒子模拟。
- **Sequence / Context Parallelism（SP/CP，序列/上下文并行）**：按序列或上下文长度切分激活（长序列场景）；CP 常用于超长上下文的 attention 计算。
- **Sequence（序列）**：有序元素集合（C++ `Sequence<T>`、Scala `List[T]`、Pandas Dataframe、PyTorch/JAX Tensor）；与数组不同，只能通过特定操作访问元素，不能直接按下标访问。
- **Sequential Consistency（顺序一致性，SC）**：Lamport 1976 提出——所有内存操作存在一个全局串行顺序（如同操作单一内存），且每线程保持程序序；维持 W→R、R→R、R→W、W→W 全部四种排序。
- **Serializability（可串行化）**：所有事务看起来以某个单一串行顺序提交，但语义不保证确切提交顺序。
- **Shared Address Space（共享地址空间模型）**：线程通过读写共享内存变量进行通信（通信隐式于 load/store），并需用锁、屏障等同步原语协调访问的编程模型。
- **Shared Memory（共享内存，`__shared__`）**：按 block 分配、块内所有线程可读写的片上高速存储，用于把 DRAM 中复用的数据暂存片上、节省带宽。
- **SIMD Utilization（SIMD 利用率）**：SIMD 通道实际忙碌的比例；不规则负载（如行长度不同的稀疏矩阵按行并行）会导致 lane 空转，需用分段 scan 等方式转为规则并行。
- **SIMD（Single Instruction, Multiple Data，单指令多数据）**：一条指令广播给多个 ALU，各 ALU 对不同数据同时执行；取指/译码成本被摊薄（如 AVX2 的 8-wide float）。
- **SIMT（Single Instruction, Multiple Thread）**：GPU 的执行模式：硬件线程流只有标量指令，硬件检测到多个线程执行同一指令时用 SIMD ALU 同时执行，分歧线程被 mask。
- **Simultaneous Multi-Threading（SMT，同时多线程）**：每个时钟从多个线程同时选指令放到不同 ALU 上执行；Intel Hyper-Threading（超线程，每核 2 线程）是其代表。
- **Single-reader/single-writer queue（单读单写队列）**：只允许一个生产者与一个消费者访问的队列，因 head/tail 各由一方写而完全无需同步，空/满时直接返回失败。
- **SM（Streaming Multiprocessor，流式多处理器）**：GPU 上的一个多线程 SIMD 核；V100 有 80 个 SM，每 SM 含 4 个 sub-core、256 KB 寄存器堆、128 KB shared+L1，最多驻留 64 个 warp。
- **Snooping（总线嗅探）**：基于广播的一致性方案——缓存控制器监听总线上所有一致性相关操作并按协议响应，同时响应本地 LD/ST 与互连广播两个方向的事件。
- **Softmax**：`softmax(x)_i = e^{x_i − m(x)} / Σ_j e^{x_j − m(x)}`（m 为行最大值，保证数值稳定）的归一化函数，把分数向量变为概率分布。
- **Software transactional memory（STM，软件事务内存）**：由编译器插桩 + 运行时（锁、时间戳、数据拷贝等）实现版本管理与冲突检测的事务内存，无硬件支持。
- **Sort（排序）**：按 key 对整个序列排序；数据并行实现常与 scan/gather 组合构建直方图、粒子网格等结构。
- **Span（跨度）**：并行算法最长串行依赖链的长度（关键路径），决定理想并行执行时间。
- **Spatial execution（空间执行）**：通过在芯片上布局计算单元与数据通路来"调度"计算，而非逐条指令执行；数据流架构的核心执行方式。
- **Spatial Locality（空间局部性）**：装入一条 cache line 顺带预载同行相邻地址的数据，使后续不同地址的访问命中。
- **Specialization / fixed-function（专用化 / 固定功能）**：把"程序控制"变成"固定数据通路"的硬件设计思路，能效收益来自省掉取指、译码、调度等控制开销。

- **Speculative execution（推测执行）**：事务系统"推测操作能成功完成，若被其他线程修改则中止重来"的执行方式，是 CAS 思路向多地址操作的推广。
- **Speedup（加速比）**：使用 P 个处理器相对 1 个处理器的执行时间缩减倍数，`speedup(P) = T(1) / T(P)`。
- **Spinlock（自旋锁）**：等待者通过原子 test-and-set 循环忙等抢锁的锁实现；低竞争时延迟低，高竞争时产生大量一致性通信与 CPU 浪费。
- **SPMD（Single Program, Multiple Data）**：只定义一个函数，但并行运行该函数的多个实例，各实例处理不同数据。
- **Stall（停顿）**：后续指令依赖未完成的指令（典型如访存）时处理器无法推进的状态；访存是停顿的主要来源。
- **Start-Flag Representation（起始标志表示）**：用与数据平行的 flag 序列（1 表示分段起点）表示嵌套序列的紧凑编码。
- **Starvation（饥饿）**：系统整体在推进但某些进程始终得不到资源的状态，通常不是永久状态，本质是公平性问题而非正确性问题。
- **Static Assignment（静态分配）**：不依赖运行时动态行为的分配；只要工作量与 worker 数已知即可预先确定。
- **STM barrier（软件事务屏障 / 插桩）**：编译器为事务内每个内存访问插入的 STM 运行时调用（tmRead/tmWr 等），负责簿记并需要函数克隆或动态翻译。
- **Streaming dataflow（流式数据流）**：数据以流水方式流经各计算/存储单元的执行模式，常与 kernel fusion、pipelining 配合，避免中间结果落回 off-chip 内存。
- **Strong atomicity / Weak atomicity（强原子性 / 弱原子性）**：是否要求非事务代码的访问也参与事务原子性语义；纯软件提供强原子性代价高昂。
- **Sub-core（子核）**：SM 内的一个执行分区，含独立取指/译码、warp selector 与一组 SIMD 功能单元（V100 每 sub-core 16 个 fp32 ALU），可在 16 个 warp 间交织调度。
- **Super-Linear Speedup（超线性加速比）**：加速比超过处理器数的现象，通常因工作集变小后装进 cache（或从磁盘换页变为装入内存）所致。
- **Superscalar Execution（超标量执行）**：处理器自动找出指令流中的独立指令，在多个执行单元（ALU）上并行执行（如每时钟 2 条），由乱序控制逻辑调度。
- **SWMR Invariant（单写多读不变量）**：任意时刻每个地址要么处于"单写者 epoch"（只有一个处理器可写），要么处于"只读 epoch"（多个处理器只读），是 coherence 的两条不变量之一。
- **Systolic array（脉动阵列）**：大量同构处理单元（PE）组成的阵列，数据像脉搏一样在相邻 PE 间逐周期流动（数据驱动/wavefront）；权重驻留 PE、输入与部分和沿阵列传播，通信只发生在相邻 PE 之间。
- **Tag（消息标识）**：send/receive 中可选的消息标识符，用于区分不同消息。
- **Task Granularity（任务粒度）**：单个任务包含的工作量；细粒度利于均衡但同步开销高，粗粒度反之。
- **Task Parallelism（任务并行）**：把问题分解为相互独立（或带依赖）的子任务、由调度器分配给 worker 执行的并行方式，强调"不同的工作单元"。
- **TCC（Transactional Coherence and Consistency）**：把事务当作一致性机制本身（所有事务所有时间）的 HTM 方案，lazy + optimistic，每执行步至多一个提交。
- **tcgen05**：B100 上编程 tensor core 的指令族（alloc 分配 TMEM、mma batch + commit 启动异步 MMA、fence 排序回收），标志"Not your father's CUDA"。
- **Temporal Locality（时间局部性）**：反复访问同一地址导致命中。
- **Tensor Core（张量核）**：SM 内专为矩阵乘设计的 SIMD 单元（如低精度 FP16 4×4×4 乘加），提供远超普通 fp32 ALU 的吞吐；需要大量并行工作（大 batch/大矩阵）才能喂饱。
- **Tensor core（张量核）**：GPU 上的专用矩阵乘加单元，一条 MMA 指令完成整个矩阵块乘加（如 8×4 × 4×8，fp16 输入、fp32 累加）；H100 的 tensor core 提供 989 TFLOPS fp16，占芯片算力绝大部分。
- **Tensor Parallelism（TP，张量并行）**：按 hidden 维切分权重/激活，多卡协作计算同一层；通信模式为 ReduceScatter+AllGather 或 AllReduce。
- **Test-and-Set（测试并置位）**：原子指令，读旧值并置位，返回旧值；`std::atomic_flag::test_and_set`、x86 `lock xchg` 等是其实现。

- **Test-and-set（测试并置位）**：原子指令，读回内存旧值并在旧值为 0 时置 1，是构建自旋锁的最小硬件原语。
- **Test-and-test-and-set lock（测试-测试并置位锁）**：先普通读自旋、观察到锁释放后才执行一次原子 test-and-set 的锁，大幅降低 coherence 流量（每次释放 O(P) 次失效）。
- **Test-and-Test-and-Set（TTAS）**：先普通读测试锁状态、空闲才执行 test-and-set 的改进自旋锁，把每轮测试的写降为仅锁释放时一次失效。
- **Thread Block Scheduler（线程块调度器）**：GPU 上的硬件工作调度器，把 block 按动态调度策略映射到 SM，需满足线程上下文数与 shared memory 等资源约束；block 完成即释放资源。
- **Thread Block（线程块）**：若干 CUDA 线程组成的组，块内线程并发运行、可经 shared memory 通信并用 `__syncthreads()` 同步；一个 block 整体驻留在同一 SM 上。
- **Thread-Level Parallelism（TLP，线程级并行）**：多个线程（指令流）同时在多个核上执行；软件通过线程 API（如 std::thread）向硬件暴露。
- **Throughput / Bandwidth（吞吐 / 带宽）**：单位时间完成的操作数或提供的数据量（如高速公路每小时车辆数、内存 GB/s）。
- **Throughput Computing（吞吐计算）**：以提升系统总吞吐为目标的设计哲学——可能延长单个线程的完成时间以换取多线程整体吞吐。
- **Thunderkittens**：CUDA 瓦片（tile）编程原语库——异步瓦片加载/存储、高级内存布局支持，面向高级开发者编写分块 kernel（含 FlashAttention 示例）。
- **ThunderKittens（TK）**：Embedded CUDA DSL template library，设计三原则：16×16 tile 为基本数据类型、处处异步、高层 GPU 协调模式（producer-consumer）。
- **Ticket lock（取号锁）**：通过"取号 + 等叫号"实现 FIFO 公平的锁，取号需原子自增、等号阶段只需普通读，每次释放仅一次失效。
- **Ticket Lock（票锁）**：取号（fetch_add 自己的票号）+ 等叫号（只读 now_serving）的锁；等待只读无失效、每释放仅一条失效（O(P) 通信），且 FIFO 公平。
- **Tiled tensor / tiled programming（分块张量 / 分块编程）**：以 16×16、32×32 等形状的 tensor tile 为编程与搬运单位（如 CUTLASS、Triton、Thunderkittens），目标是 GEMM 最大 TFLOPS 与低指令开销。
- **TMA（Tensor Memory Accelerator，张量内存加速器）**：H100 起的专用数据搬运单元，用拷贝描述符描述区域、单线程发起、硬件生成地址并异步搬运张量分块，配合 mbarrier 同步。
- **TMEM（Tensor Memory）**：B100 引入的专供 tensor core 数据存放的片上存储；配合 tcgen05 系列指令，单个线程即可执行 MMA（不再有 warp 概念）。
- **torch.compile**：PyTorch 的编译器，自动对关键 DNN 操作做融合与调度。
- **TPU（Tensor Processing Unit，张量处理单元）**：Google 的领域专用 DNN 加速器，核心是脉动阵列执行 matrix_multiply/convolve，指令集极小（read/write host memory、read weights、matrix_multiply、activate）。
- **Transaction descriptor（事务描述符）**：每线程一份的事务状态结构，含读集、写集、undo log 或 write buffer。
- **Transaction record（事务记录）**：每个数据项关联的指针大小记录，shared 态用版本号/共享读锁、exclusive 态用指向属主事务的写锁。
- **Transactional memory（TM，事务内存）**：以数据库事务为灵感、把一段内存访问序列声明为原子且隔离执行的同步抽象，`atomic { }` 即其编程形态。
- **Triton**：面向 GPU 的 DSL——支持把数据"块"加载进 shared memory 并对其做数据并行操作（自带两层分块的矩阵乘参考实现）。
- **True Sharing（真共享）**：多个处理器访问同一地址、通信由程序语义必然引起的共享；与伪共享相对。
- **TSO（Total Store Order）**：写缓冲引入的模型——只放松 W→R（自己的读可越过自己的写），W→W 仍保持；x86 使用一种未完全定义的 TSO。
- **TSV（Through-Silicon-Via，硅通孔）**：穿过 DRAM 芯片的垂直互连通孔，为 3D 堆叠提供高并行度的层间连接；堆叠底层为"逻辑层"（内存控制器）。
- **uniform（类型修饰符）**：变量在所有程序实例中值相同、只有一份存储；使用它纯属优化，与正确性无关。

- **Uniform（统一类型）**：ISPC 类型修饰符，表示所有 program instance 取值相同；使用它纯粹是优化，不影响正确性。
- **Varying（可变类型）**：ISPC 默认类型，每个 program instance 各有一份独立的取值。
- **varying（类型修饰符）**：变量在每个程序实例中各有一份（ISPC 默认），对应 SIMD 向量中的一个 lane；varying 值不能赋给 uniform 变量。
- **Victim（受害者）**：工作窃取中被空闲线程选中、被偷取工作的线程。
- **VLIW（Very-Long Instruction Word，超长指令字）**：一条指令同时指定多个不同运算（与 SIMD 的"一条指令对多数据做同运算"相对），由编译器静态安排并行。
- **Warp Selector（warp 选择器）**：sub-core 中每时钟从可运行 warp 里选一个取指执行的硬件单元。
- **Warpgroup / PTX**：Warpgroup 是 128 个连续线程的组合（跨 4 个 warp）；PTX（Parallel Thread Execution）是 NVIDIA 的虚拟指令集架构。
- **Warp（线程束）**：32 个连续 CUDA 线程组成的一组，是 GPU 硬件调度/执行的基本单位；不属于 CUDA 编程模型，而是 NVIDIA GPU 的重要实现细节（block 内 0–31 为 warp 0，32–63 为 warp 1……）。
- **Work Queue（工作队列）**：存放待执行任务的列表，worker 从中取任务、产生新任务时再推入。
- **Work Stealing（工作窃取）**：每个 worker 有本地队列，空闲时从随机选中的 victim 线程队列偷取工作的动态负载均衡调度。
- **Work-Efficient Scan（工作高效 scan，Blelloch 算法）**：up-sweep（自底向上建树）+ down-sweep（自顶向下修正）两阶段，Work = O(N)、Span = O(log N)，但指令数与常数因子较大。
- **Workload Balance（负载均衡）**：让所有处理器在程序执行期间持续计算且同时完成各自工作的目标状态。
- **Work（工作量）**：并行算法执行的总操作数；与 Span 一起评价并行算法。
- **Write Buffer（写缓冲）**：处理器把写暂存其中即可继续执行后续指令的硬件缓冲，用于隐藏写延迟；是现代 x86/ARM/RISC-V 处理器的标配，也是 TSO 行为的来源。
- **Write-Back Cache（写回缓存）**：写只改缓存行并置 dirty bit，行被换出时才写回内存；吸收写流量，但 dirty 意味着独占所有权，需要更复杂的一致性协议。
- **Write-Through Cache（写直达缓存）**：每次写立即穿透到内存，实现简单但带宽需求极高（每个写都占用内存带宽）。
- **Write-write conflict（写写冲突）**：两个事务都处于未提交状态且都写了地址 X。

附录：资料与作业说明

已获取的公开资料清单

所有资料均下载自课程公开网站，保存在本工作目录 `cs149_fall25/` 下：

资料	位置	说明
课程主页 / 课程信息页 / 讲座索引页	`pages/*.html`	主页含完整日程表、作业链接；课程信息页含评分、政策、FAQ
全部 18 讲幻灯片 PDF	`slides/*.pdf`	与讲座页面上链接一致
幻灯片全文提取文本	`raw/*.txt`	每讲一文件，含页码标记
书面作业 1–4 PDF 与文本	`pdfs/written_asst*.pdf`、`raw/written_asst*.txt`	全部公开可下载
编程作业 README	`pdfs/asst*_README.md`	来自 GitHub 公开仓库 stanford-cs149/asst1…asst5-kernels
结构化数据记录	`lectures_data.json`	每讲的编号、日期、主题、材料链接、文本路径

资料	位置	说明
最终学习笔记	`cs149_fall2025_learning_notes.md`	本文档

讲座幻灯片 PDF 链接（公开）

讲次	标题	幻灯片 PDF
1	Why Parallelism? Why Efficiency?	https://gfxcourses.stanford.edu/cs149/fall25content/media/efficiency/01_efficiency_hyF1AJq.pdf
2	A Modern Multi-Core Processor (Part I)	https://gfxcourses.stanford.edu/cs149/fall25content/media/multicore1/02_basicarch.pdf
3	Multi-Core Architecture (Part II) + ISPC	https://gfxcourses.stanford.edu/cs149/fall25content/media/multicore2/03_multicore2-ispc_WueDBzT.pdf
4	Parallelizing Code: An Example Thought Process	https://gfxcourses.stanford.edu/cs149/fall25content/media/thoughtprocess/04_progbasics.pdf
5	Program Optimization 1: Work Distribution and Scheduling	https://gfxcourses.stanford.edu/cs149/fall25content/media/perfopt1/05_progperf1.pdf
6	Program Optimization 2: Locality and Communication	https://gfxcourses.stanford.edu/cs149/fall25content/media/perfopt2/06_progperf2.pdf
7	GPU Architecture and CUDA Programming	https://gfxcourses.stanford.edu/cs149/fall25content/media/gpuarch/07_gpuarch.pdf
8	Data-Parallel Thinking	https://gfxcourses.stanford.edu/cs149/fall25content/media/dataparallel/08_dataparallel.pdf
9	Efficiently Evaluating DNNs on GPUs	https://gfxcourses.stanford.edu/cs149/fall25content/media/dnninference/09_dnneval.pdf
10	Hardware Specialization	https://gfxcourses.stanford.edu/cs149/fall25content/media/accelerators/10_Specialized.pdf
11	Programming Systems for Specialized Hardware	https://gfxcourses.stanford.edu/cs149/fall25content/media/proghardware/11_SpecializedHardwareProgramming.pdf

讲次	标题	幻灯片 PDF
12	Mapping AI Applications to the Datacenter Computer	https://gfxcourses.stanford.edu/cs149/fall25content/media/aidatacenter/12_AI_DatacenterMapping.pdf
13	Domain-Specific Programming Systems and AI-Driven Performance Optimization	https://gfxcourses.stanford.edu/cs149/fall25content/media/aiperfoptimization/13_autooptimize.pdf
14	Cache Coherence	https://gfxcourses.stanford.edu/cs149/fall25content/media/cachecoherence/14_coherence.pdf
15	Implementing Synchronization + Memory Consistency	https://gfxcourses.stanford.edu/cs149/fall25content/media/sync_consistency/15_consistency.pdf
16	Fine-Grained Locking and Lock-Free Programming	https://gfxcourses.stanford.edu/cs149/fall25content/media/finegrainedsync/16_finegrainedlock.pdf
17	Transactional Memory (Part I)	https://gfxcourses.stanford.edu/cs149/fall25content/media/transactions/17_transactionalmem.pdf
18	Transactional Memory (Part II) + AMA	https://gfxcourses.stanford.edu/cs149/fall25content/media/wrapup/18_transactionalmem_A4wu1Q8.pdf

无法访问的内容（已记录，未下载）

- **2025 年讲座视频**：课程主页明确说明 "We cannot distribute lecture videos to the public this year"。替代资源：2023 年版公开播放列表 https://www.youtube.com/playlist?list=PLoROMvodv4rMp7MTFr4hQsDEcX7Bx6Odp
- **Ed Discussion 论坛 / Canvas 内容**：需校内账号登录，非公开。
- **作业内部文件**（如测试用例、手写提纲图片、AWS 配置说明等 GitHub 仓库内非 README 文件）：编程作业主体（README）公开，其余 starter code 需注册学生身份使用。

作业说明汇总

- **Programming Assignment 1**（10 月 6 日截止，100 分 + 6 分附加）：在 myth 机器的四核 Intel Core i7（4.2GHz、AVX2、每核 2 硬件线程）上，分析 SIMD（AVX2 8 宽单精度）与多核并行、超线程对程序性能的影响；练习测量与推理并行程序性能。编程量小，分析为主。对应第 1–4 讲（处理器架构、ISPC、并行化思路）。
- **Programming Assignment 2**（10 月 16 日截止，100 分）：从零实现一个 C++ 任务执行库：先支持批量数据并行任务启动（类似 ISPC task launch），再扩展为支持带依赖的任务图调度；使用线程池、互斥锁、条件变量。对应第 5 讲（工作分配与调度）与第 15 讲（同步原语）。

- **Programming Assignment 3**（10 月 30 日截止，100 分）：在 AWS GPU 上用 CUDA 编写并行圆形渲染器；重点是可并行构建的数据结构设计。对应第 7–8 讲（CUDA、数据并行思维）。
- **Programming Assignment 4**（11 月 13 日截止，100 分）：在 AWS Trainium2 加速器上编写并优化 kernel：第 1 部分向量加法与矩阵转置，第 2 部分融合卷积+最大池化层；体会循环分块（blocking）与循环融合（fusion）的局部性价值。对应第 9–11 讲（DNN 评估、硬件特化、专用硬件编程）。
- **Programming Assignment 5**（12 月 4 日截止，100 分，**不可用晚交天数**）：开放课题，从若干 AI kernel 中选择，在 H100 GPU 上用 CUDA / Triton / TileLang 优化，允许使用 LLM 辅助，有班级排行榜；成绩依据优化工作日志评定。对应第 13 讲（AI 驱动性能优化）及全课程性能工程能力。
- **Written Assignment 1**（10 月 9 日）：Hardware Basics——多核/SIMD 峰值吞吐计算、处理器基础。
- **Written Assignment 2**（10 月 21 日）：To Fuse or Not to Fuse——循环融合与并行化分析。
- **Written Assignment 3**（11 月 6 日）：Improving Locality on Specialized Hardware——图像模糊两遍算法的局部性优化与专用硬件设计。
- **Written Assignment 4**（12 月 3 日）：MSI Coherence Protocol Warmup——MSI 状态机、CAS 与一致性交互分析。

> 注：以上全部作业描述均已公开获取；不存在"作业描述未公开"的情况。若个别内部文件（如测试用例）需要课程账号，本文档已在"无法访问的内容"一节说明。